



*data***POWER**®

Data Dictionary

—

Database Schema Manual

Release 9.0 and Higher



DOCUMENTS



CONTENTS

85-059702-O3

Notice

The software programs described in this document are copyrighted and are confidential information and proprietary products of PDF Solutions, Inc. This Manual is furnished to authorized users of PDF Solutions products, and is intended solely to facilitate the use of PDF Solutions tools and products, as specified in written agreements. No part of this publication may be reproduced, stored in a retrieval system, translated, transcribed, or transmitted, in any form, or by any means without prior explicit written permission from PDF Solutions, Inc.

Trademarks

dataPOWER is a trademark of PDF Solutions, Inc. All other brand and product names mentioned herein are the trademarks and registered trademarks of PDF Solutions, Inc. or their respective owners.

Printed in the United States of America.

Copyright© 1991-2011 PDF Solutions, Inc. All Rights Reserved.

PDF Solutions, Inc.

333 West San Carlos Street, Suite 700 ~ San Jose, CA 95110 USA

Tel: (408) 280-7900 ~ Fax: (408) 280-7915

email: info@pdf.com (general), doc@pdf.com (documentation),

dpsupport@pdf.com (support)

web: www.pdf.com

CONTENTS

1.0	Introduction	15
1.1	Document Charter.....	15
1.2	Supporting Documents	15
1.3	Conventions and Usage	26
1.4	List of General Tables:.....	28
1.5	List of <i>bit</i> MAP Tables:	29
1.6	List of pdCV Tables:.....	30
1.7	List of Characterization Tables (Shmoo/Robustness):	31
1.8	List of Central Schema Tables:.....	31
2.0	Oracle Database — More Than 1,000 Parameters	32
3.0	Log Tables	33
3.1	OP_LOG	34
3.2	WF_LOG	39
3.3	LEH_LOG	40
3.4	WEH_LOG	42
3.5	META_LOG	43
4.0	PROGRAM and Related Tables	44
4.1	Data Model Description.....	44
4.2	Tester Summary Sheet Storage	46
4.3	PROGRAM.....	48
4.4	PROGRAM_GROUP	56
4.5	PROG_DATATYPE.....	57
4.6	PROG2PROC	58
4.7	PROG2FAM	58
4.8	PROG2LOT	59
4.9	PROG2EQUIP	59
4.10	PROG2TECH	60
4.11	PROG2STAGE	60
4.12	PROG2SRCLOT	61
4.13	PROG2CUST	61
4.14	PROG2PROD	62
4.15	PROG2STEP	62
4.16	PROG2PKG.....	63
4.17	PROG_REV	63
4.18	DEF.....	65
4.19	LIM... ..	70
4.20	LIM_LOG.....	71
4.21	RES... ..	72
4.22	WAF.....	74
4.23	LOT.....	76
4.24	STYPE	78
4.25	BIN_LOG	79
4.26	HIST_BIN.....	82
4.27	HIST_BIN_EXTRAS	83

4.28	HIST_BIN_ZEROS	84
4.29	CONDITION	85
4.30	SUM_COND	86
5.0	Program Class and Configurable Statistics	87
5.1	PROG_CLASS	87
6.0	Product, Lot, Wafer and Related Tables	91
6.1	PRODUCT	91
6.2	PROD2LOT	94
6.3	PROD2SRCLLOT	94
6.4	PROD2WMAP	95
6.5	WMAP_CONFIG	96
6.6	ORG_WMAP	98
6.7	WMAPS	100
6.8	RETICLE	101
6.9	RETICLE_SET	102
6.10	RET2DIE	103
6.11	PKG	103
6.12	PKG2LOT	104
6.13	PKG2SRCLLOT	104
6.14	FAMILY	105
6.15	LOT	106
6.16	UPDATE_STATS	110
6.17	DF_UPSTAT	112
6.18	WAFER	114
6.19	WAFER_CLASS	117
6.20	WFCLS2WF	118
6.21	WAFER_TYPE	119
6.22	WAFER_DISTANCE	120
6.23	WAFER2LOT	121
6.24	WMAP2DIE	121
6.25	NWMAP2DIE	122
6.26	WF_ALIAS	123
6.27	VENDOR	124
6.28	CUSTOMER	125
6.29	LOT_CLASS	126
6.30	LTCLS2LOT	126
6.31	LOT2CUST	127
6.32	LW_CLASS_LOG	128
6.33	TAGS	129
6.34	TAG_LOG	130
6.35	RULE_INFO	131
7.0	Process Related Tables	133
7.1	PROC_STEP	133
7.2	TECHNOLOGY	136
7.3	TECH_STAGE	136
7.4	PROCESS	137
7.5	PROC2LOT	138
7.6	PROC2SRCLLOT	138

7.7	FLOW_STAGE	139
7.8	RECIPE.....	139
7.9	RCP2STEP	140
7.10	TECH2FAB	140
7.11	TOOL_OPTION.....	141
7.12	STEP_ORDER_SET	142
7.13	STEP_ORDER_DETAILS	143
7.14	META_DEF.....	144
7.15	META_IDX_DEF.....	144
7.16	MSTEP2PSTEP.....	146
8.0	Zonal Bin Summary Tables	147
8.1	Z_WMAP_CONFIG.....	147
8.2	Z_UPDATE_LOT.....	148
8.3	Z_UPDATE_WAF	149
8.4	Z_CONFIG	149
8.5	Z_CNT.....	150
8.6	Z_YLD.....	151
8.7	Z_STATS	152
8.8	Z_LOG.....	154
8.9	ZONES	155
8.10	ZONE_NUM	156
8.11	ZONE_DEF	157
9.0	dpWEB Table	158
9.1	WEB_RPT_CFG	158
10.0	defectMAP Tables	159
10.1	DF_LOG	159
10.2	DF_GALIMG	161
10.3	DF_TEST.....	162
10.4	DFL2TEST	163
10.5	DEFECT	164
10.6	DF_TYPES.....	166
10.7	DF_METHOD	167
10.8	DF_CLASS.....	168
10.9	DEF2CLASS	169
10.10	DF_DIE.....	169
10.11	DF_ALNIM	170
10.12	DF_IMAGE	171
10.13	DF_SUM.....	172
10.14	DF_IMGSRC.....	173
10.15	DF_RECIPE	174
10.16	DF_SAMPLE	175
10.17	DF_TOLERANCE	177
10.18	DRCP2PDLYR.....	178
10.19	DF_CONTRIB.....	178
10.20	DF_EXCLUDE.....	179
10.21	DF_LYRINFO	180
10.22	DF_LOCK	181
10.23	DF_BIN	181

10.24	DF_CONFIG	182
10.25	DF_EXPORT	183
10.26	DF_EXPORT_DEF	184
10.27	DF_TRIGGER.....	184
11.0	dbTOOLS Tables.....	186
11.1	DBTOOLS_MERGE_FAM.....	186
11.2	DBTOOLS_MERGE_PROC.....	186
11.3	DBTOOLS_MERGE_PROD	187
11.4	DBTOOLS_MERGE_TECH.....	187
12.0	Various Tables.....	188
12.1	EQUIPMENT	188
12.2	EQUIP_TYPE.....	190
12.3	EQUIP_CLASS	191
12.4	PEOPLE.....	192
12.5	FAB.....	194
12.6	DEPARTMENT	195
12.7	QUERIES.....	195
12.8	DBINFO.....	196
12.9	PAR_GRP	197
12.10	GROUP_TYPE.....	198
12.11	REPORTS	199
12.12	ARCHIVE_LOG	200
12.13	ARCHIVE_PLOG	201
12.14	PAR2PARGRP	202
12.15	RFX.....	202
12.16	DP_FUNC.....	203
12.17	DP_LOG	204
12.18	LEH_CONFIG.....	205
12.19	WEH_CONFIG	207
12.20	DATA_SET.....	208
12.21	BM_ARGS	211
12.22	DP_TEMP_RUN	211
12.23	DP_TEMP_RUN_KEYS.....	212
12.24	DP_TRIGGER.....	213
12.25	DP_TRIGGER_RUNS	214
12.26	DP_TRIGGER_RUNS_ARRAY.....	214
12.27	DPF_GROUP	215
12.28	DPF_SUBGROUP.....	215
12.29	DPF_HIER.....	216
12.30	DIR_PATH_DEF	216
12.31	STACKING_COLORS.....	217
12.32	RAW_PATH.....	218
12.33	RAW_FILE.....	219
12.34	CLS_METHOD.....	219
12.35	DATABASES.....	220
12.36	ERROR_MESSAGE	221
12.37	FILE2MEMCONFIG.....	221
12.38	FMT_FILE.....	222
12.39	META_TYPES	222

12.40	STAGE2STEP	223
12.41	STAGING_DIRS	223
12.42	STRING HOLDER.....	224
13.0	Characterization Tables — Shmoo and Robustness	225
13.1	CHAR_LOG	226
13.2	DATA_TYPE	231
13.3	EXPERIMENT_STEP.....	231
13.4	FAB_SKEW	232
13.5	FAB_STEPPING	232
13.6	PARAM_INFO	233
13.7	CRITICAL_PARAM	234
13.8	SHMOO_CLASS	235
13.9	FTEST.....	236
13.10	PAT_LIST	236
13.11	PATTERN	237
13.12	COND_VALUE	238
13.13	TESTER_PIN_DEF.....	238
13.14	TESTER_PKG_TYPE.....	239
13.15	PAT_MODE.....	239
13.16	PAT_TYPE.....	240
13.17	PAT_STATUS	240
13.18	PAT2LIST	241
13.19	ROB_FAIL	242
13.20	ROBUST.....	244
13.21	LINE_DIST	246
13.22	SHM_STATS	247
13.23	SIGN_PARETO	248
13.24	PIN_PARETO.....	249
13.25	HEADER_INFO.....	250
13.26	SHMOO.....	251
13.27	LINE... ..	252
13.28	FAIL.....	254
14.0	Lot Genealogy Tables.....	256
14.1	LOT_GEN	256
14.2	LOT_GEVENT.....	257
14.3	WAF2LOTGEN	258
14.4	FAB_TYPE.....	259
14.5	LOT_TYPE.....	259
14.6	LOT2TYPE.....	260
14.7	FAB2PROD	260
14.8	LOT2LOTGEN.....	261
15.0	bitMAP Tables.....	262
15.1	MEM_META_CLASS	262
15.2	MEM_CLASS	263
15.3	MEM_MCLASS_COLL	266
15.4	MEM_BIT_PATT	267
15.5	MEM_PATT_LOC.....	267
15.6	MEM_CLS2CLS	268

15.7	MEM_MCLIB	269
15.8	MEM_MCL2MC	270
15.9	MEM_MCGRP	270
15.10	MEM_MCGRPLIB	271
15.11	MEM_MC2MCGRP	272
15.12	MEM_CLASS_GROUP	272
15.13	MEM_LIB2PROD	273
15.14	MEM_PI	274
15.15	MEM_ROWS	275
15.16	MEM_BOX	276
15.17	MEM_CUSTOM	277
15.18	MEM_COLS	278
15.19	MEM_MISS	279
15.20	MEM_SPOT	280
15.21	MEM_CROSS	281
15.22	MEM_CROSS_BITS	282
15.23	MEM_FOG	283
15.24	MEM_MULTI_ROW	284
15.25	MEM_MULTI_ROW_BITS	285
15.26	MEM_MULTI_COL	286
15.27	MEM_MULTI_COL_BITS	287
15.28	MEM_CELL_CLASS	288
15.29	MEM_CELL_BLK	289
15.30	MEM_BLK_CFG	290
15.31	MEM_CONFIG	293
15.32	MEM_DEF2MEM	296
15.33	MEM_UPSTAT	297
15.34	MEM_LAYER2PAT	299
15.35	MEM_MEMLOG2LAYER	299
15.36	MEM_MEMORY	300
15.37	MEM_LOG	301
15.38	MEM_GALIMG	303
15.39	MEM_DIE	304
15.40	MEM_DEFAULT_TEST	304
15.41	MEM_BOP_ROWS	305
15.42	MEM_BOP_MULTI_ROW	306
15.43	MEM_BOP_COLS	307
15.44	MEM_BOP_MULTI_COL	308
15.45	MEM_BOP_BOX	309
15.46	MEM_BOP_MISS	310
15.47	MEM_BOP_SPOT	311
15.48	MEM_BOP_FOG	312
15.49	MEM_BOP_CROSS	313
15.50	MEM_BOP_CUSTOM	314
15.51	MEM_BOP_CONFIG	315
15.52	MEM_BOP_TESTS	316
15.53	MEM_BOP2PAT	316
15.54	MEM_BOP_UPSTAT	317
15.55	MEM_FILE_SETUP	318
15.56	MEM_FILE_CONFIG	319

15.57	MEM_WAFER	319
15.58	MEM_RAW_FILE_DETAILS	320
15.59	MEM_RAW_FILE_DETAILS_TMP	321
15.60	MEM_WF_RF	322
15.61	MEM_META_RF	323
15.62	MEM_FILE2SETUP	323
15.63	MEM_RUN_TEMPLATE	324
15.64	MEM_RUN_SCHEDULE	325
15.65	MEM_RUN_BOP	326
15.66	MEM_SCRAMBLE	327
15.67	MEM_TEST	327
16.0	CV Tables	329
16.1	CV_TEST_PROG	329
16.2	CV_TEST_PLAN	329
16.3	CV_CONFIG	330
16.4	CV_MINOR_CONFIG	335
16.5	CV_LOT2CFG	336
16.6	CV_CELL	337
16.7	CV_CELL_TYPE	338
16.8	CV_CELL2AMOD	338
16.9	CV_CELL_I	339
16.10	CV_CELLI2IDEF	341
16.11	CV_CELLI_BLOCK_NUM	342
16.12	CV_CELLI_BLOCK_DEF	343
16.13	CV_CELLI_ZONES	344
16.14	CV_CELLI_ZONE_NUM	345
16.15	CV_CATTR	345
16.16	CV_CATTR_V	346
16.17	CV_MATTR	346
16.18	CV_MATTR_V	347
16.19	CV_CIATTR	347
16.20	CV_CIATTR_V	348
16.21	CV_MEAS	349
16.22	CV_MEAS2PIN	350
16.23	CV_LIMITS	350
16.24	CV_LIMITS_V	351
16.25	CV_LIM2CELL	352
16.26	CV_AI	352
16.27	CV_AI2AI	353
16.28	CV_AI_MAP	354
16.29	CV_AI_MAPC	354
16.30	CV_AI_MOD	355
16.31	CV_LIM2AI	356
16.32	CV_PR_MOD	357
16.33	CV_MAP2IN	358
16.34	CV_MAP2OUT	358
16.35	CV_UNQCI	359
16.36	CV_AATTR	360
16.37	CV_AATTR_V	360

16.38	CV_DOE.....	361
16.39	CV_DOE_VAL	362
16.40	CV_DOE2CELL.....	362
16.41	CV_MODEL.....	363
16.42	CV_MODEL_DEF	364
16.43	CV_FAIL_MODE	365
16.44	CV_FAIL_LAYER	366
16.45	CV_FINE_FTYPE.....	366
16.46	CV_FRN	367
16.47	CV_FMD_TYPE	368
16.48	CV_FMD2MDL	369
16.49	CV_FMD2PAR.....	370
16.50	CV_FMD2LSPLT.....	371
16.51	CV_FRM_PARAM.....	372
16.52	CV_FRM_PARM	372
16.53	CV_FRM_MOD.....	373
16.54	CV_FRM_CLUST_DET.....	374
16.55	CV_FRM_COV	375
16.56	CV_FRM_DEF.....	376
16.57	CV_FRM_DIAG_ATTR	376
16.58	CV_FRM_DIAG_V.....	377
16.59	CV_FRM_OUT_SUM	378
16.60	CV_FRM_RUN.....	379
16.61	CV_FRM_RUN_DET	381
16.62	CV_EXPORT.....	382
16.63	CV_MODULE2CELL.....	383
16.64	CV_FRM_OUT_DATA_<model_key>.....	384
16.65	CV_FRM_USC_<model_key>	385
16.66	CV_MODULE2CATTR.....	386
16.67	CV_COEF_LABEL.....	386
16.68	CV_USC_DEF	387
16.69	CV_CA	388
16.70	CV_CA_VAL.....	389
16.71	CV_P_VAL.....	389
16.72	CV_CA_NV.....	390
16.73	CV_BIN.....	391
16.74	CV_STEP_ORDER.....	391
16.75	CV_DEFECT.....	393
16.76	CV_DEF2TYPE_<lot_key>	395
16.77	CV_EDEF2IDEF	396
16.78	CV_EDEF2IDEF_ALL	397
16.79	CV_IDEF2IDEF	398
16.80	CV_UPSTAT	399
16.81	CV_SUM_ZONE.....	400
16.82	CV_PRIO.....	401
16.83	CV_WF_CELLL.....	402
16.84	CV_WF2MICFG	402
16.85	CV_FLTR	403
16.86	CV_FLTR_INFO	404
16.87	CV_ADL.....	405

16.88	CV_ADL_DEF	406
16.89	CV_PIN_RESULTS	407
16.90	CV_PIN_SUM.....	408
16.91	CV_DIE_RESULTS	409
16.92	CV_GATTR.....	410
16.93	CV_GATTR_V	410
16.94	CV_FORM	411
16.95	CV_FORM2PARAM.....	411
16.96	CV_FORM2LSPLT	412
16.97	CV_PAR2MEAS	413
16.98	CV_FUNCTION.....	413
16.99	CV_FUNC2FORM.....	414
16.100	CV_FUNC2INP.....	415
16.101	CV_E_IDEF2IDEF.....	415
16.102	CV_E2I_INHERIT	416
16.103	CV_LOG.....	417
16.104	CV_INSP_SET	418
16.105	CV_SO_SET.....	418
16.106	CV_INSP_V	419
16.107	CV_MIC_DEF.....	420
16.108	CV_PAD_GROUP.....	421
16.109	CV_PADGRP_SUM.....	421
16.110	CV_MEAS2PADGRP	422
16.111	CV_PIN2PADGRP	423
16.112	CV_CATEGORY	423
16.113	CV_UFORM.....	424
16.114	CV_UFORM_DEF	424
16.115	CV_FILTER_UFORM	425
16.116	CV_UFORM2FILTER	426
16.117	CV_UPARAM	426
16.118	CV_UPARAM_DEF	428
16.119	CV_UCONSTANT	429
16.120	CV_UOPERATION.....	429
16.121	CV_FRS_SETUP	430
16.122	CV_FRS_RUN	431
16.123	CV_FRS_INP_SPLITS	432
16.124	CV_CELL_POSITION.....	433
16.125	CV_FRM_STEPWREG_COEF	433
16.126	CV_FRM_STEPWREG_OUT.....	434
17.0	binMAP tables.....	436
17.1	BIN_COLORS.....	436
17.2	BIN_GAL	436
18.0	SPLIT Tables.....	438
18.1	SPLIT.....	438
18.2	LOT_SPLIT	438
18.3	WAFER_SPLIT	439
19.0	Event Tables	440
19.1	EVENT_LOG	440

19.2	EV_...	441
19.3	EVENT_CONFIG	442
20.0	Tool-to-Tool Tables	443
20.1	T2T_REPORT	443
20.2	T2T_SUM.....	444
20.3	T2T_TEST.....	446
20.4	T2T_TOOL.....	447
21.0	Data Mining Tables	448
21.1	CT_JOB	448
21.2	CT_TREE	449
21.3	CT_RULE_SET.....	450
21.4	CT_CLASS.....	450
21.5	CT_CONFIG	451
21.6	CT_MODIFY.....	453
22.0	PCM Wafer Sort Alignment	454
22.1	SITE_MAP	454
22.2	SITE2DIE	454
23.0	Multi-Bin Tables.....	456
23.1	MB_UPSTAT	456
23.2	MB_SET	457
23.3	MB_SET2FORMULA	458
23.4	MB_FORMULA.....	458
23.5	MB_RULE.....	459
23.6	MB_PROGRULE	460
23.7	MB_BIN	460
24.0	Spatial Signature Analysis Tables	461
24.1	SSA_DIE_...	462
24.2	SSA_ZONE_\$.	462
24.3	SSA_CLUSTER_\$.	463
24.4	SSA_RUN.....	464
24.5	SSA_RUN_DET.....	467
24.6	SSA_RUN2ZONE	468
24.7	SSA_DEF	468
24.8	SSA_DEF_DET.....	469
24.9	SSA_IMAGE.....	470
24.10	SSA_CLUSTER	471
24.11	SSA_CLUSTER2WFCLS	472
24.12	SSA_WF2CLUSTER	472
24.13	SSA_WAFER_DISTANCE	473
24.14	SSA_OPER_HIST.....	474
24.15	SSA_OPER_TYPES.....	475
24.16	SSA_OPER.....	476
24.17	SSA_YLD_RULE	476
24.18	SSA_YLD_RULE_DEF.....	477
	Central Schema — Overview	479
25.0	Central Schema or General Schema — File Storage Tables	480
25.1	DP_FOLDER.....	480

25.2	DP_FILE.....	481
25.3	DPF2FIELD.....	482
25.4	DPFILE2FILE	483
25.5	DPF_FIELD.....	483
26.0	Central Schema — dPMonitor Tables.....	484
26.1	DPMON_HOTLIST	484
26.2	DPMON_APP.....	485
26.3	DPMON_APP_SERVER	486
26.4	DPMON_TASK.....	487
26.5	DPMON_USER_INFO	488
26.6	DPMON_USER_ATTRIB.....	488
26.7	DPMON_REPORT.....	489
26.8	DPMON_DASH_COMP.....	490
26.9	DPMON_ANNOT	491
26.10	DPMON_SCHEDULE_LOG.....	492
26.11	AUTHORITIES.....	492
26.12	ACL_OBJECT_IDENTITY	493
26.13	ACL_PERMISSION.....	493
27.0	Central Schema — Quartz Tables	494
27.1	QRTZ_JOB_DETAILS.....	494
27.2	QRTZ_JOB_LISTENERS.....	495
27.3	QRTZ_TRIGGERS.....	495
27.4	QRTZ_SIMPLE_TRIGGERS	497
27.5	QRTZ_CRON_TRIGGERS	497
27.6	QRTZ_BLOB_TRIGGERS.....	498
27.7	QRTZ_TRIGGER_LISTENERS.....	499
27.8	QRTZ_CALENDARS	499
27.9	QRTZ_PAUSED_TRIGGER_GRP.....	500
27.10	QRTZ_FIRED_TRIGGERS	500
27.11	QRTZ_SCHEDULER_STATE.....	501
27.12	QRTZ_LOCKS.....	502
28.0	Central Schema — Load Balancing Tables.....	503
28.1	GENERAL_CONFIG.....	503
28.2	MACHINES	504
28.3	MACHINE_STAT	505
28.4	MACHINE_STAT_ARCHIVE.....	506
28.5	NODES	507
28.6	NODE_STAT	509
28.7	NODE_STAT_ARCHIVE	509
28.8	SERVERS	511
28.9	DB_ENGINES.....	511
28.10	MAP_SRV2DB.....	512
28.11	SCHEDULING_DECISION	513
28.12	SCHEDULING_DETAILS	514
29.0	Glossary.....	517
30.0	Index.....	521

1.0 Introduction

1.1 Document Charter

This document describes the data model that comprises the *dataPOWER*/*dataBASE* package.

This document includes:

- Data flow information:
 - *dataBASE* tables and relationships.
 - What conventions / standardizations exist at the data sources.
- Data dictionary information:
 - What are legal values for some data fields which need standardization across the implementation scope of *dataPOWER*.
 - What usage is made of the data, both in the interactive *dataPOWER* tool and in regular reports.
 - Default values for those data fields that have defaults.
- Definitions of terms used ([“Glossary,”](#) page 517).

1.2 Supporting Documents

This document is one part of the complete documentation set that should be accessible to *dataPOWER* administrators and end-users. The following documents comprise the list of supporting documentation for the *dataPOWER* system:

General ***dataPOWER OnLine Documentation Library*** — The *dataPOWER* library of on-line documents, including all *dataPOWER* reference documents listed here. This library is delivered in Adobe Acrobat™ PDF™ format.

Release Notes — dataPOWER 8.x — Database administrator and *dataPOWER* user notes for *dataPOWER* release 8.0 and higher.

End-Users ***dataPOWER End-Users Manual*** — Reference manual for the core *dataPOWER* product, including:

- ***dataPOWER Environment*** — working with *dataPOWER* worksheets, manipulating data, *dataPOWER* menus and palettes
- ***Database Retrieval*** — standard *dataBASE* data retrieval window functionality, including Raw, Statistics, Multi-Program, Metrology and Equipment History
- ***dataPOWER Charts*** — statistical analysis and chart generation tools, including: ANOVA, Histograms, Boxplots, Product Sensitivity Analysis, Regression, Reject Oriented Analysis
- ***dataPOWER Reports*** — *dataPOWER* analysis reports, including: Custom Summary Report, Extreme Value Reports, Temperature Characterization
- ***Yield Optimization Tools*** — *yieldOPTIMIZER* Database Trend Analysis tools, including Yield Report and Zonal Report

- **Journal** — tools that enable you to create presentation quality documents from *dataPOWER*, in PowerPoint or Microsoft Word format.

dataPOWER Mapping Tools Users Manual — The reference manual for all *dataPOWER* mapping tools:

- **defectMAP** — defect analysis and defect/wafer mapping
- **waferMAP/binMAP** — parametric wafer and bin mapping
- **bitMAP** — bit failure wafer-based mapping

dataPOWER Data Mining Tools Users Manual —

- **Classification Trees** — used for the development of classification trees intended to suggest possible reasons for the existence of groups, clusters, or “classes” of events.
- **Self Organizing Maps** — the SOM tool provides a visual overview of high dimensional data sets to help you explore the data and identify groups within it.
- **FSS and FSDA** — Wafer Signature Analysis Reports. The FSS (Fail Signature Summary) tool is designed for faster, larger analyses of lots for predetermined wafer signatures. The FSDA (Fail Signature Detection and Analysis) tool is designed for more detailed analysis of lots, using algorithms that automatically identify and sort significant wafer signatures.

Administrators

dataPOWER Products Installation and Configuration Guide — Informix or Oracle Database — The administrators guide to the installation and configuration manual of *dataPOWER* and related software products, for installations using the Informix or Oracle database engine.

dataBASE Administrator Utilities Manual — The reference manual for *dataBASE* Administrators, defining usage of tools for database configuration, data exporting, lot statistics updating, batch scheduling and administration. All administration operations windows, such as Program, Lot, Parameter, Product, Zonal Operations, Tagging, database parameter scaling, database privileges, *dataFLOW*, etc. are also documented.

dataPOWER Data Dictionary — The schema document for the data model and data tables that make up the *dataPOWER* database.

Data Readers – dataPOWER — The reference manual for the *dataPOWER* database and worksheet reader configurations, providing both an end-user’s overview and a functional description of the formatting and configuration for individual readers (ASCII, Metrology, *bitMAP* reader, etc.).

Training

dataPOWER Training and Tutorial Guide — Intended for first-time users and users who want to learn new areas of the *dataPOWER* tool set. The modules in this guide have been constructed to introduce database retrieval techniques, data manipulation, and the most widely used *dataPOWER* analysis charts and tools. These procedures are used with the *dataPOWER* training database and a pre-generated set of example worksheets.

dataPOWER Administrator Training — Intended for *dataPOWER* administrators and DBAs. Provides hands-on training for most *dataPOWER* administrator tools, including database operations windows, data readers, data retrieval, license management, database schema, database platform and data export. This manual is primarily intended for in-class use, but can also serve as a stand-alone training tool.

Developers

HyperScript Programming Manual – dataPOWER Application Programming Interface — The developer’s guide to the *dataPOWER* API environment and statistics engine. This manual is intended for programmers or “power users” who will be developing or customizing system features.

HyperScript Programming Manual – Commands — The developer’s guide to the syntax and definition of each HyperScript command.

HyperScript Programming Manual – Functions — The developer’s guide to the components of the HyperScript functions that are used in *dataPOWER*.

Third-Party Documentation

Informix

IBM Informix Dynamic Server 9.x-II.x documentation CDs — The database administrator’s reference guide to the Informix database environment.

Oracle

Web-based Documentation Downloads — Database, developer, and applications documentation:
<http://otn.oracle.com/documentation/index.html>

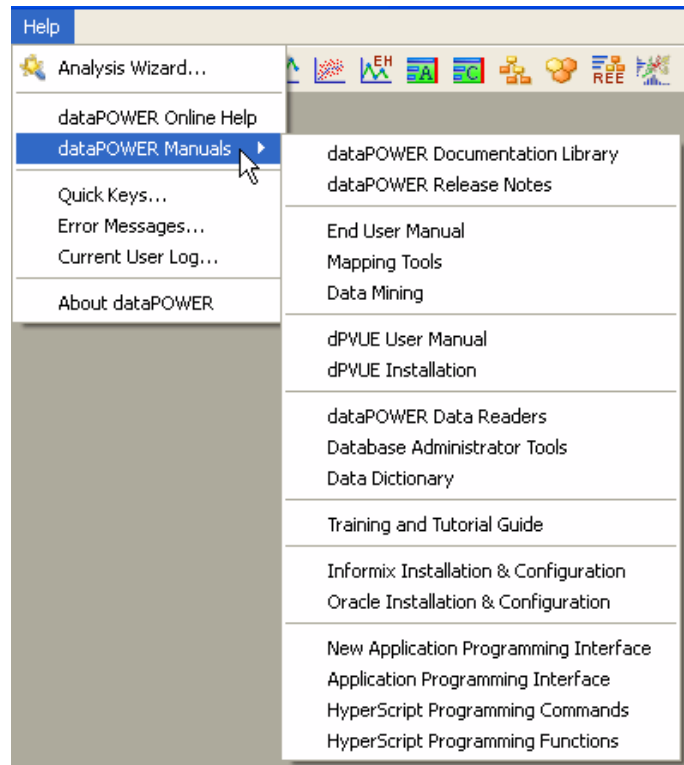
FLEXIm

FLEXIm End User Manual — the user manual for *FLEXIm*, the licence manager software used with *dataPOWER* products.

Accessing On-line Manuals From the Interface

All *dataPOWER* manuals are accessible via the **Help > dataPOWER Manuals** menu functions. Accessing any manual name will open the Adobe Acrobat PDF version of the manual.

The document library needs to be installed in the `...\\dp\\dpdocs` directory, which should take place during the software installation process.

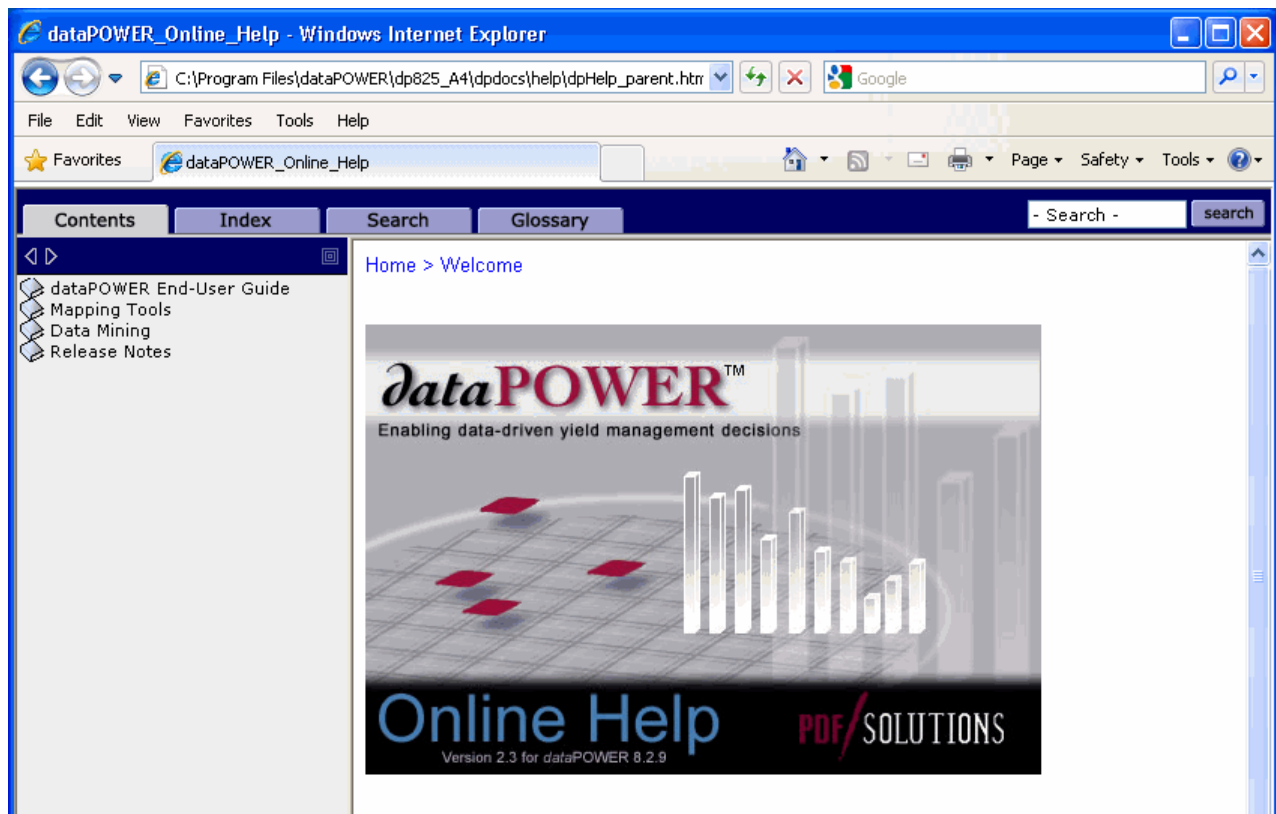


CONTEXT SENSITIVE HTML HELP SYSTEM

The *dataPOWER* context-sensitive HTML-based Help system works with your default browser and is launched in two ways: from the menu or from most of the windows and dialogs in *dataPOWER*.

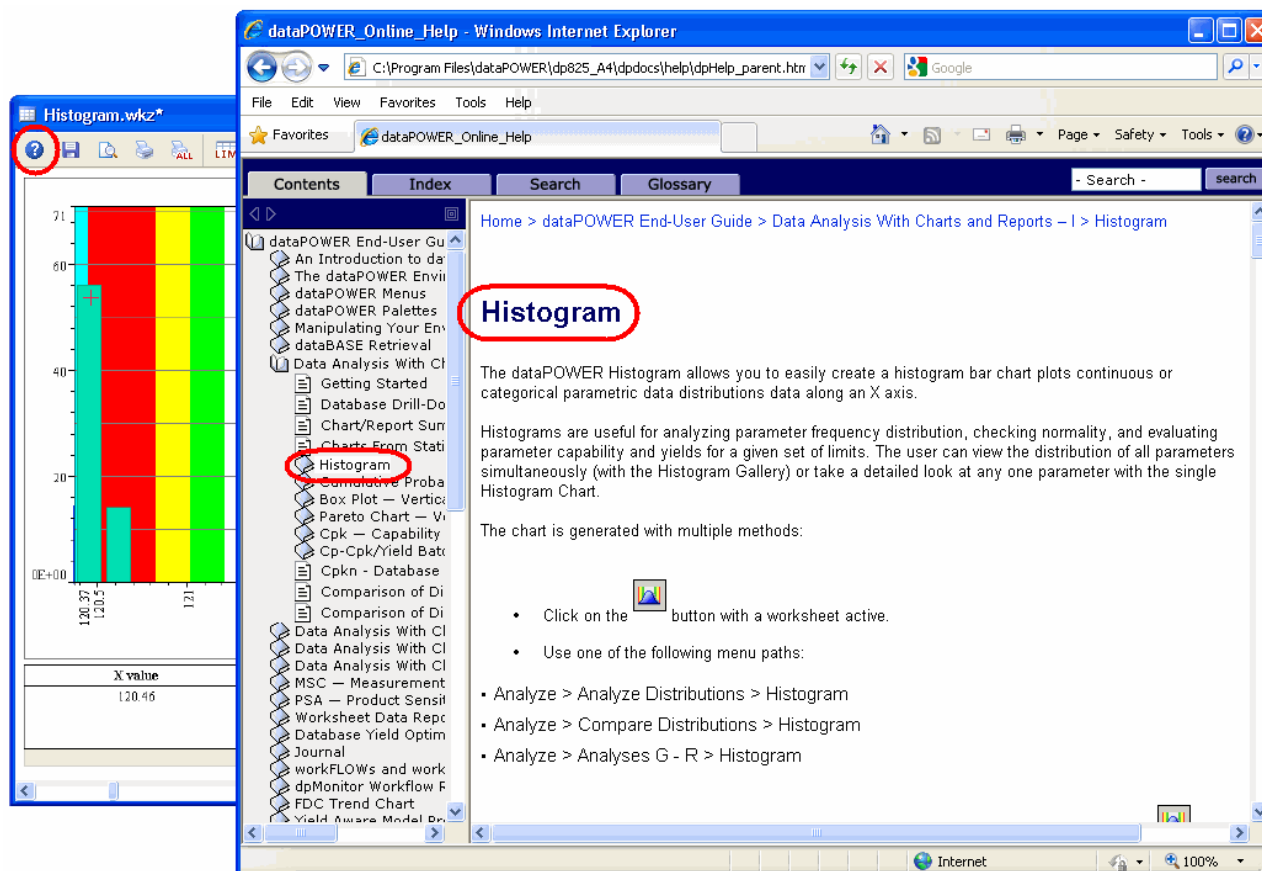
- From the menu, select the **Help > dataPOWER Online Help** menu function

to access the top-level Help GUI.



- From most windows and dialogs, you can click on the  button in the chart or tool you are using and the documentation topic for the current interface/

chart will be opened in a new HTML window.



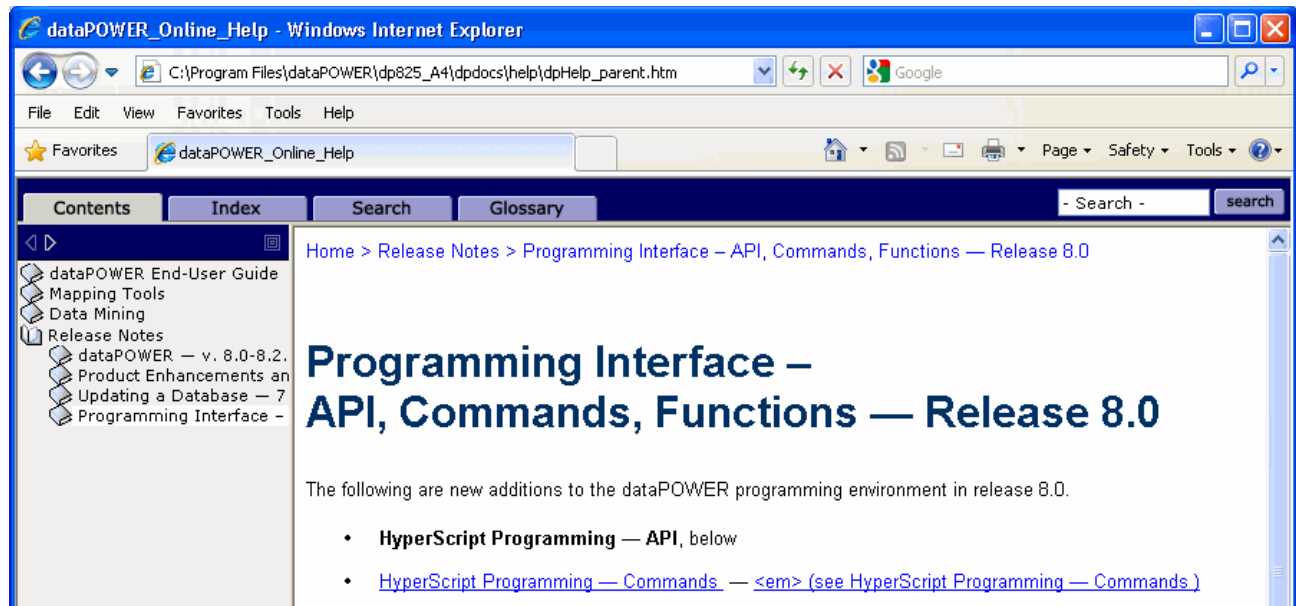
Not every manual in the *dataPOWER* documentation library are included. The documents published in this on-line Help system are:

- *dataPOWER* End-User Manual
- Mapping Tools End-User Manual
- Data Mining User Manual
- *dataPOWER* Release Notes

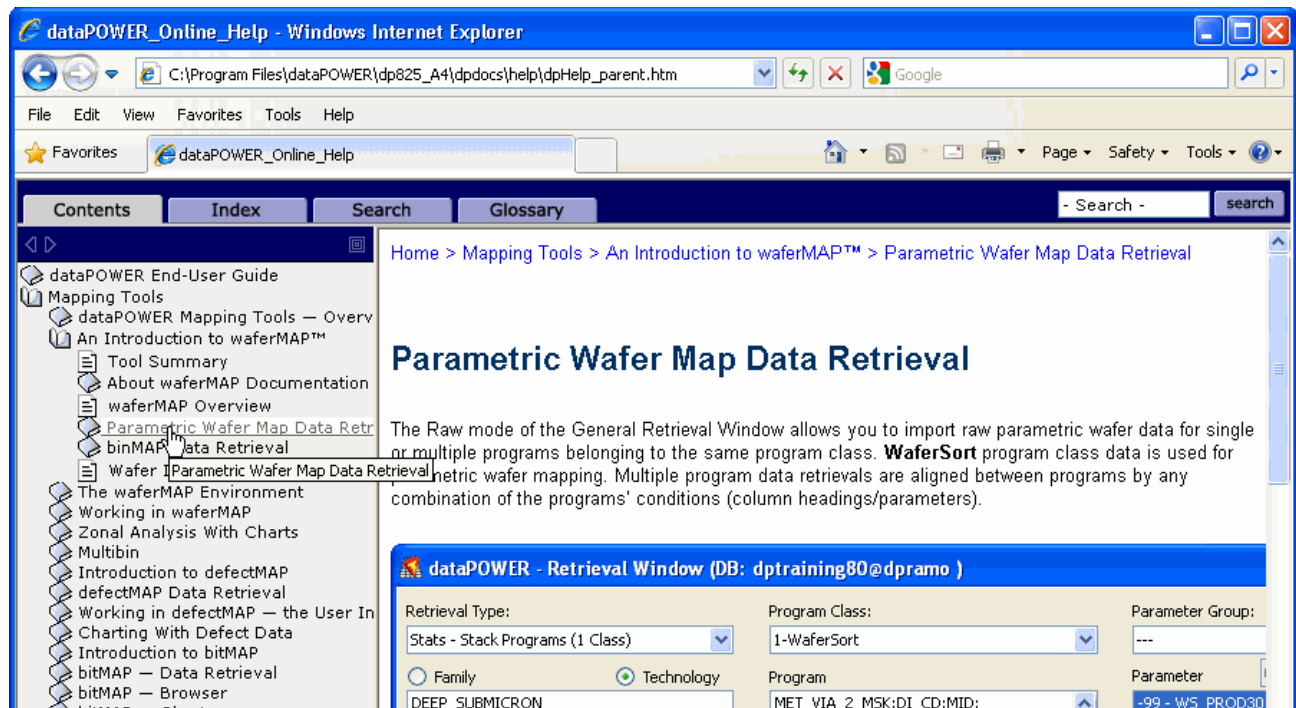
The system is compatible with any current web browser and includes intelligent full-text search and a keyword index.

The full *dataPOWER* documentation set, including all “back-end” documentation is also published in Acrobat PDF form, as described previously. This includes the manuals published in the Help system format. These documents are launched from the Help > *dataPOWER* Manuals submenu.

Using Help Once a Help window is called, either from the GUI or from the menu, there are a number of tools available, including a full table of contents, intelligent weighted search, and full indexing.



Contents The **Contents** panel contains a nested, linked headings list for all books included in the Help system.



Index The **Index** panel provides an alphabetized listing of all indexed terms in the source document.

Search The **Search** panel lets you search the text of the entire Help system for a topic. The Help system performs an AND search for the words you type in the query box. Enter one or more words separated by spaces.

dataPOWER_Online_Help - Windows Internet Explorer

C:\Program Files\dataPOWER\dp825_A4\dpdocs\help\dpHelp_parent.htm

File Edit View Favorites Tools Help

dataPOWER_Online_Help

Contents Index **Search** Glossary

whiskers search

☒ Highlight search results

Search results per page 10

Box_Plot_Vertical

Boxplot Chart - Horizontal

Boxplot Chart -

unselected on the Chart Configuration **Setup** panel ([Setup - Single Parameter Per Chart](#)), the legend and bar colors will reflect the different subgroups.

Boxplot Values — Quartiles and Whiskers

- The height of the box represents the 1st through 3rd quartile for the data distribution on the Y axis.
- Above and below the box are "whiskers" that represent the stretch of the tail of the distribution on either side of the box. The length of the top whisker is Q3 to the maximum observed value — to a maximum length of $1.5 \times \text{IQR}$.

("IQR" or Inter-quartile range = $Q3 - Q1$ (the "length of the box")). The length of the bottom whisker is Q1 to the minimum observed non-outlier value — to a maximum length of $1.5 \times \text{IQR}$.

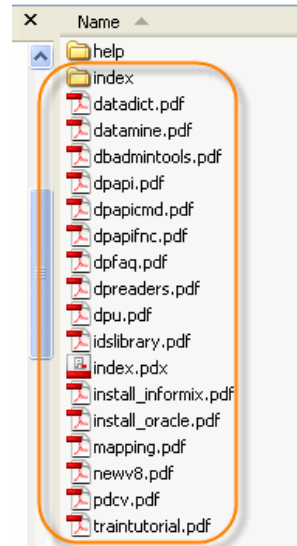
The search results are ranked based on frequency and position. If the title of a Help topic contains the search term, the topic will appear at the top of the results list.

ON-LINE PDF DOCUMENTATION AND GLOBAL DOCUMENT INDEXING INSTALLATION

Delete Old On-Line Documentation

If you already have older Acrobat documents installed for a previous *dataPOWER* software release, you must delete or move the entire contents of the obsolete directories and subdirectories before installing the new documentation set. If you are replacing existing files, this documentation directory should be set up as the **...\\PDF\\dp\\dpdocs** directory in your *dataPOWER* installation directory.

Delete all of the **....pdf** files, the **index.pdx** file and the **index** directory. Do not delete the **help** directory; it is for the separate HTML Help system.



Installing New On-Line Documentation

After deleting any older versions of *dataPOWER* Docs Online:

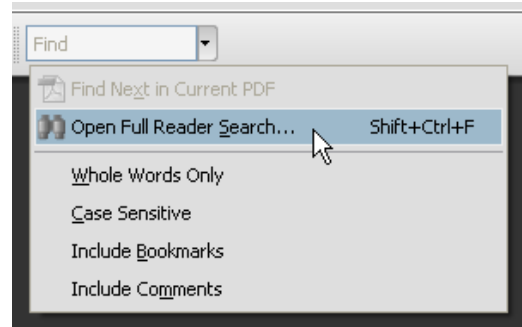
1. Copy the entire contents of the **Docs** directory from your installation package or your documentation update package to an appropriate location on your workstation or server. If you install locally, you need to copy the entire directory contents into the **...\\PDF\\dp\\dpdocs** directory. When the document set is updated for new releases, the contents of this directory always needs to be deleted and replaced with the newest version of document set.
2. If not already installed, install Acrobat Reader. (Version 9.0 or higher is preferred, and the following instructions will apply to version 9. But earlier versions will work in much the same way as described here and the files will be compatible.) You can download the Acrobat software from <http://get.adobe.com/uk/reader/>.
3. Open the **idslibrary.pdf** file for a top-level menu of all manuals, linking to all of the documents in the set.

Setting Up the dataPOWER Search Library Index

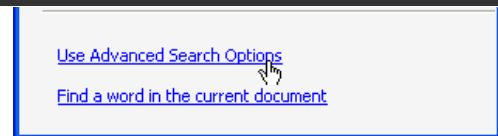
You can search the entire *dataPOWER* document library with Adobe Acrobat Reader, by adding the *dataPOWER* **index.pdx** index file in your Reader set-up. This section demonstrates how to select the **index.pdx** index in Acrobat Reader version 9x. This operation only needs to be performed once for each client PC to update your system so it will search the new document set.

1. Open Acrobat Reader.

2. From the **Find** field's drop-down menu, select the **Open Full Reader Search** options.



3. At the bottom of the Search panel, select the **Use Advanced Search Options** function.



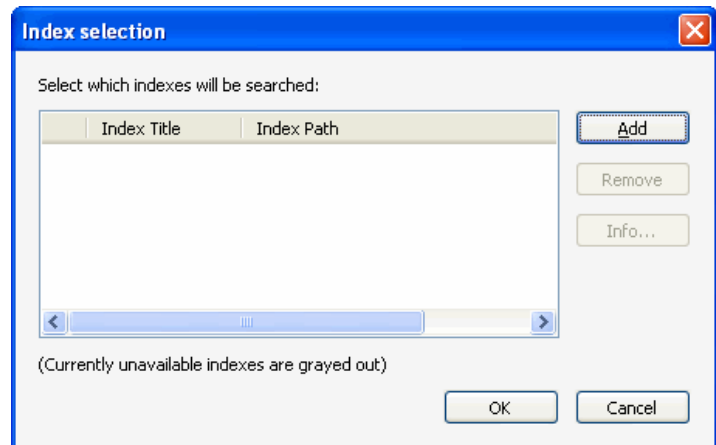
4. Choose **Select Index...** from the **Look In:** drop-down list.



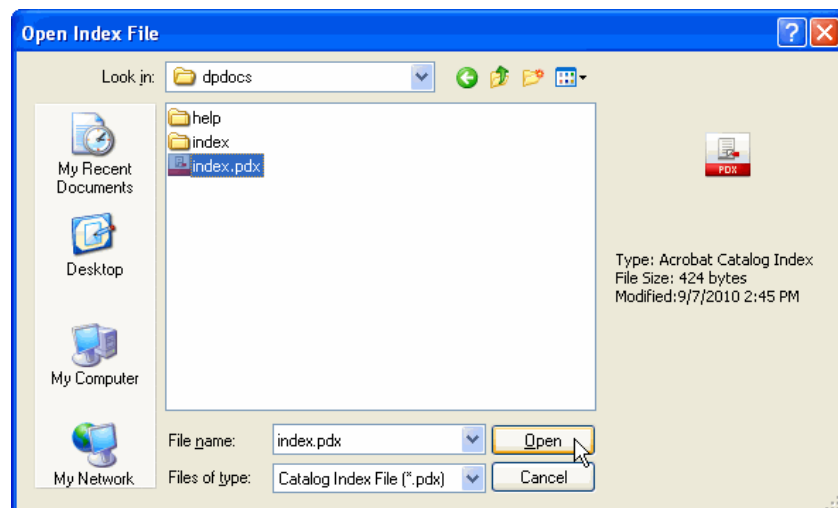
The system will display the Index Selection dialog.

5. Click on the **Add** button.

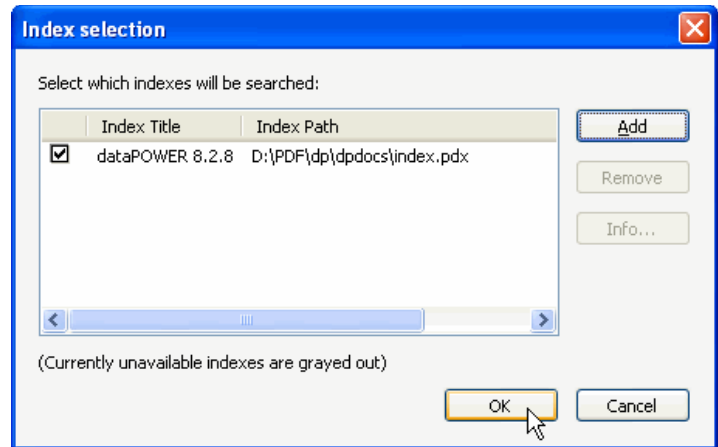
The system will display the Open Index File dialog.



6. Browse to **...\\PDF\\dp\\dpdocs** directory or where your PDF documents are located and select the **index.pdx** file. Then click on **Open** in the dialog.



The system will display the Index selection dialog. Make sure the *dataPOWER* index is selected. (Any other Acrobat search indexes that you want available can be added in the same way.)



7. Click on the **OK** button.

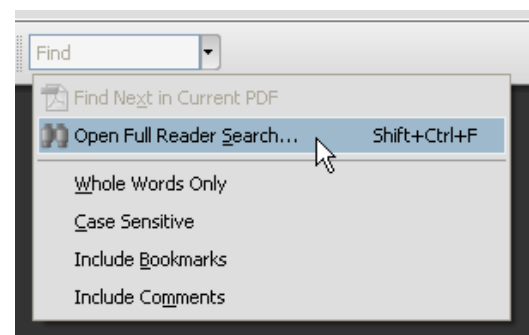
The *dataPOWER* index is selected and made available for future searches by selecting **Currently Selected Indexes** from the **Look In:** drop-down list in the advanced search options. You can now perform a search on any term or string in the *dataPOWER* documentation set. (Example below.)

You will only need to perform this operation once on each computer that the documentation set is installed on.

Using Acrobat to Search Multiple Documents

If you only need to search only your currently opened document, simply open the file and use the **Find** field in the Acrobat tool bar. But if you need to search the entire *dataPOWER* document set, you'll use the advanced search options. With an up-to-date *dataPOWER* Acrobat search index installed (as described above), you can search the entire *dataPOWER* library and find a term or text string in any *dataPOWER* document.

1. From the **Find** field's drop-down menu, select the **Open Full Reader Search** options.

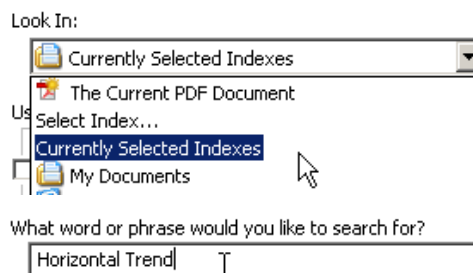


2. At the bottom of the Search panel, select the **Use Advanced Search Options** function.

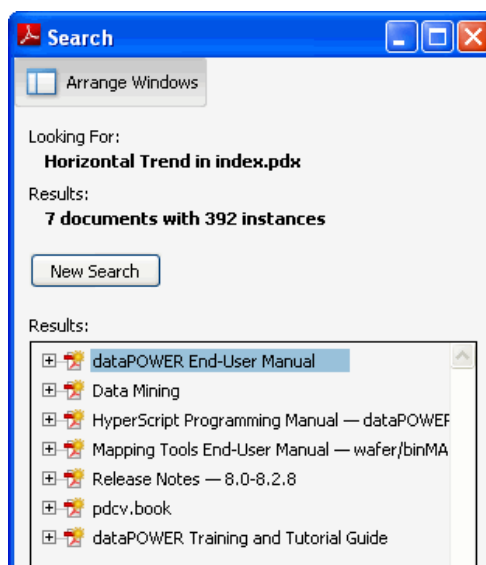
[Use Advanced Search Options](#)
[Find a word in the current document](#)

3. Choose **Currently Selected Indexes** from the **Look In:** drop-down list.

If you have installed the *dataPOWER* search index as previously described, you will be able to search all documents in the set.



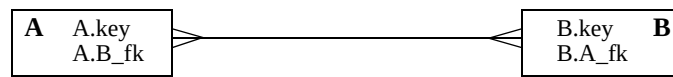
4. Enter a word or text string in the text field and click on the **Search** button. (If the *dataPOWER* index.pdx file is not installed, as described in the previous section, the **Search** button will be disabled.)
5. The system will search the documents in the *dataPOWER* OnLine Documentation Library for your text. The **Results** list box displays the documents where your text string was found. Expand the entry for any manual to see all the results that match your search string. Click on any entry to navigate to that selection in the manual.
6. Click on **New Search** to initiate a search for another term or text string.



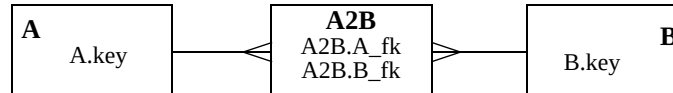
1.3 Conventions and Usage

The following conventions exist throughout the document:

- Variables (typically lower case) and table names (typically upper case) are printed in **sans serif** font.
- For online document users:
 - Any term appearing in blue text is hyperlinked to a related section, e.g. [Conventions and Usage](#).
 - Any term appearing in pink text is hyperlinked to its glossary definition, e.g. [program](#).
- **n** → **1** and **1** → **n** links between tables are described/listed at the “n” and “1” side of the link.
- **n** → **m** links between tables A and B are implemented in the data model by means of a hidden table (A2B) between them. The hidden table will be listed in references to the link.

FIGURE 1.Internal data model for $n \rightarrow m$ links:

is implemented as:



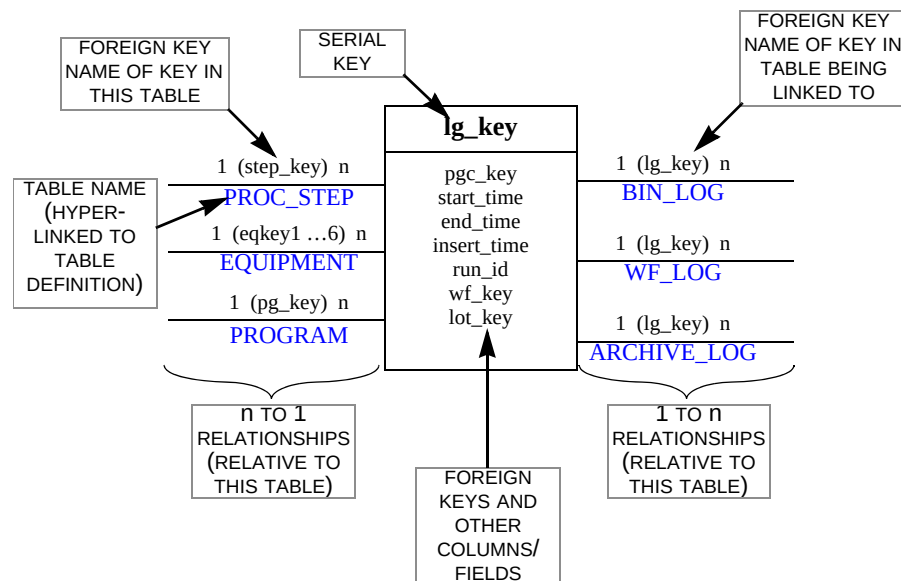
Thus, the foreign keys are ‘missing’ in the graphical representation of the data model.

- Foreign keys in all tables are described as such in this document, but are not declared as foreign keys in the actual database. All processes that insert data in the database maintain the integrity of the database without the use of foreign keys.

Table Description

Throughout the document, tables are represented by block diagrams which represent the table’s indexes, table-to-table relationships, foreign keys, and other columns and fields.

TABLE DIAGRAM



1.4 List of General Tables:

(HyperText links to table definitions.)

ARCHIVE_LOG	DP_TRIGGER	ORG_WMAP	SSA_RUN_DET
ARCHIVE_PLOG	DP_TRIGGER_RUNS	PAR2PARGRP	SSA_WF2CLUSTER
BIN_COLORS	DP_TRIGGER_RUNS_ARR	PAR_GRP	SSA_WAFER_DISTANCE
BIN_GAL	AY	PAT2LIST	SSA_ZONE_...
BIN_LOG	DPF_GROUP	PEOPLE	STACKING_COLORS
BM_ARGS	DPF_HIER	PKG	STAGE2STEP
CLS_METHOD	DPF_SUBGROUP	PKG2LOT	STAGING_DIRS
CONDITION	DRCP2PDLR	PKG2SRCLLOT	STEP_ORDER_SET
CT_CLASS	EQUIPMENT	PROC2LOT	STEP_ORDER_DETAILS
CT_CONFIG	EQUIP_CLASS	PROC2SRCLLOT	STRING HOLDER
CT_JOB	EQUIP_TYPE	PROC_STEP	STYPE
CT_MODIFY	ERROR_MESSAGE	PROCESS	SUM_COND
CT_RULE_SET	EV_...	PROD2LOT	T2T_REPORT
CT_TREE	EVENT_CONFIG	PROD2SRCLLOT	T2T_SUM
CUSTOMER	EVENT_LOG	PROD2WMAP	T2T_TEST
DATA_SET	FAB2PROD	PRODUCT	T2T_TOOL
DATABASES	FAB	PROG2CUST	TAGS
DBINFO	FAB_TYPE	PROG2EQUIP	TAG_LOG
DBTOOLS_MERGE_FAM	FAMILY	PROG2FAM	TECHNOLOGY
DBTOOLS_MERGE_PROC	FILE2MEMCONFIG	PROG2LOT	TECH_STAGE
DBTOOLS_MERGE_PROD	FLOW_STAGE	PROG2PKG	UPDATE_STATS
DBTOOLS_MERGE_TECH	FMT_FILE	PROG2PROC	VENDOR
DEF...	GROUP_TYPE	PROG2PROD	WAFER2LOT
DEPARTMENT	HIST_BIN	PROG2SRCLLOT	WAF2LOTGEN
DEF2CLASS	HIST_BIN_EXTRAS	PROG2STAGE	WAFER
DEFECT	HIST_BIN_ZEROS	PROG2STEP	WAFER_CLASS
DF_ALNIM	LEH_CONFIG	PROG2TECH	WAFER_DISTANCE
DF_BIN	LEH_LOG	PROGRAM	WAFER_SPLIT
DF_CLASS	LIM...	PROGRAM_GROUP	WAFER_TYPE
DF_CONFIG	LIM_LOG	PROG_CLASS	WAF...
DF_CONTRIB	LOT	PROG_DATATYPE	WEB_RPT_CFG
DF_DIE	LOT...	PROG_REV	WEH_CONFIG
DF_EXCLUDE	LOT2CUST	QUERIES	WEH_LOG
DF_EXPORT	LOT2LOTGEN	RAW_FILE	WF_ALIAS
DF_EXPORT_DEF	LOT2TYPE	RAW_PATH	WF_LOG
DF_GALIMG	LOT_CLASS	RCP2STEP	WFCLS2WF
DF_IMAGE	LOT_GEN	RECIPE	WMAP2DIE
DF_IMGSRC	LOT_GEVENT	RES...	WMAPS
DF_LOCK	LOT_SPLIT	REPORTS	WMAP_CONFIG
DF_LOG	LOT_TYPE	RET2DIE	Z_CNT
DF_LYRINFO	LTCLS2LOT	RETICLE	Z_CONFIG
DF_METHOD	LW_CLASS_LOG	RETICLE_SET	Z_LOG
DF_RECIPE	MB_BIN	RFX	ZONES
DF_SAMPLE	MB_FORMULA	RULE_INFO	ZONE_DEF
DF_SUM	MB_PROGRULE	SITE2DIE	ZONE_NUM
DF_TEST	MB_RULE	SITE_MAP	Z_STATS
DF_TOLERANCE	MB_SET	SPLIT	Z_UPDATE_LOT
DF_TRIGGER	MB_SET2FORMULA	SSA_CLUSTER	Z_UPDATE_WAF
DF_TYPES	MB_UPSTAT	SSA_CLUSTER_...	Z_WMAP_CONFIG
DF_UPSTAT	META_DEF	SSA_CLUSTER2WFCLS	Z_YLD
DFL2TEST	META_IDX_DEF	SSA_DEF	
DIR_PATH_DEF	META_LOG	SSA_DEF_DET	
DP_FUNC	META_TYPES	SSA_DIE_...	
DP_LOG	MSTEP2PSTEP	SSA_IMAGE	
DP_TEMP_RUN	NWMAP2DIE	SSA_RUN	
DP_TEMP_RUN_KEYS	OP_LOG	SSA_RUN2ZONE	

1.5 List of *bit*MAP Tables:

(HyperText links to table definitions.)

[MEM_BIT_PATT](#)
[MEM_BLK_CFG](#)
[MEM_BOP2PAT](#)
[MEM_BOP_BOX](#)
[MEM_BOP_COLS](#)
[MEM_BOP_CONFIG](#)
[MEM_BOP_CROSS](#)
[MEM_BOP_CUSTOM](#)
[MEM_BOP_FOG](#)
[MEM_BOP_MISS](#)
[MEM_BOP_MULTI_COL](#)
[MEM_BOP_MULTI_ROW](#)
[MEM_BOP_ROWS](#)
[MEM_BOP_SPOT](#)
[MEM_BOP_TESTS](#)
[MEM_BOP_UPSTAT](#)
[MEM_BOX](#)
[MEM_CELL_BLK](#)
[MEM_CELL_CLASS](#)
[MEM_CLASS](#)
[MEM_CLASS_GROUP](#)
[MEM_CLS2CLS](#)

[MEM_COLS](#)
[MEM_CONFIG](#)
[MEM_CROSS](#)
[MEM_CROSS_BITS](#)
[MEM_CUSTOM](#)
[MEM_DEF2MEM](#)
[MEM_DEFAULT_TEST](#)
[MEM_DIE](#)
[MEM_FILE2SETUP](#)
[MEM_FILE_CONFIG](#)
[MEM_FILE_SETUP](#)
[MEM_FOG](#)
[MEM_GALIMG](#)
[MEM_LAYER2PAT](#)
[MEM_LIB2PROD](#)
[MEM_LOG](#)
[MEM_MC2MCGRP](#)
[MEM_MCGRP](#)
[MEM_MCGRPLIB](#)
[MEM_MCLASS_COLL](#)
[MEM_MCL2MC](#)
[MEM_MCLIB](#)

[MEM_MEMLOG2LAYER](#)
[MEM_MEMORY](#)
[MEM_META_CLASS](#)
[MEM_META_RF](#)
[MEM_MISS](#)
[MEM_MULTI_COL](#)
[MEM_MULTI_COL_BITS](#)
[MEM_MULTI_ROW](#)
[MEM_MULTI_ROW_BITS](#)
[MEM_PATT_LOC](#)
[MEM_PI](#)
[MEM_RAW_FILE_DETAILS](#)
[MEM_RAW_FILE_DETAILS_TMP](#)
[MEM_ROWS](#)
[MEM_RUN_TEMPLATE](#)
[MEM_SPOT](#)
[MEM_UPSTAT](#)
[MEM_WAFER](#)
[MEM_WF_RF](#)

1.6 List of pdCV Tables:

(HyperText links to table definitions.)

CV_AATTR	CV_FILTER_UFORM	CV_MATTR
CV_AATTR_V	CV_FINE_FTYPE	CV_MATTR_V
CV_ADL	CV_FLTR	CV_MEAS
CV_ADL_DEF	CV_FLTR_INFO	CV_MEAS2PADGRP
CV_AI	CV_FMD_TYPE	CV_MEAS2PIN
CV_AI2AI	CV_FMD2LSPLT	CV_MIC_DEF
CV_AI_MAP	CV_FMD2MDL	CV_MINOR_CONFIG
CV_AI_MAPC	CV_FMD2PAR	CV_MODEL
CV_AI_MOD	CV_FORM	CV_MODEL_DEF
CV_BIN	CV_FORM2PARAM	CV_MODULE2CATTR
CV_CA	CV_FORM2LSPLT	CV_MODULE2CELL
CV_CA_NV	CV_FRM_CLUST_DET	CV_FRM_OUT_DATA_<model_key>
CV_CA_VAL	CV_FRM_COV	CV_FRM_USC_<model_key>
CV_CATEGORY	CV_FRM_DEF	CV_PAD_GROUP
CV_CATTR	CV_FRM_DIAG_ATTR	CV_PADGRP_SUM
CV_CATTR_V	CV_FRM_DIAG_V	CV_PAR2MEAS
CV_CELL	CV_FRM_MOD	CV_PIN2PADGRP
CV_CELL2AMOD	CV_FRM_OUT_SUM	CV_PIN_RESULTS
CV_CELL_I	CV_FRM_PARAM	CV_PIN_SUM
CV_CELLI2IDEF	CV_FRM_PARM	CV_PRIO
CV_CELLI_BLOCK_DEF	CV_FRM_RUN	CV_P_VAL
CV_CELLI_BLOCK_NUM	CV_FRM_RUN_DET	CV_PR_MOD
CV_CELLI_ZONES	CV_FRN	CV_STEP_ORDER
CV_CELLI_ZONE_NUM	CV_FRS_INP_SPLITS	CV_SO_SET
CV_CELL_TYPE	CV_FRS_RUN	CV_SUM_ZONE
CV_CIATTR	CV_FRS_SETUP	CV_TEST_PLAN
CV_CIATTR_V	CV_FUNCTION	CV_TEST_PROG
CV_COEF_LABEL	CV_FUNC2FORM	CV_UCONSTANT
CV_CONFIG	CV_FUNC2INP	CV_UFORM
CV_DEF2TYPE_<lot_key>	CV_GATTR	CV_UFORM2FILTER
CV_DEFECT	CV_GATTR_V	CV_UFORM_DEF
CV_DIE_RESULTS	CV_IDEF2IDEF	CV_UNQCI
CV_DOE	CV_INSP_SET	CV_UOPERATION
CV_DOE2CELL	CV_INSP_V	CV_UPARAM
CV_DOE_VAL	CV_LIMITS	CV_UPARAM_DEF
CV_E_IDEF2IDEF	CV_LIMITS_V	CV_UPSTAT
CV_E2I_INHERIT	CV_LIM2AI	CV_USC_DEF
CV_EDEF2IDEF	CV_LIM2CELL	CV_WF_CELLI
CV_EDEF2IDEF_ALL	CV_LOG	CV_WF2MICFG
CV_EXPORT	CV_LOT2CFG	
CV_FAIL_LAYER	CV_MAP2IN	
CV_FAIL_MODE	CV_MAP2OUT	

1.7 List of Characterization Tables (Shmoo/Robustness):

CHAR_LOG	HEADER_INFO	PIN_PARETO
COND_VALUE	LINE...	ROB_FAIL
CRITICAL_PARAM	LINE_DIST	ROBUST
DATA_TYPE	PARAM_INFO	SHMOO...
EXPERIMENT_STEP	PAT_LIST	SHMOO_CLASS
FAB_SKEW	PAT_MODE	SHM_STATS
FAB_STEPPING	PAT_STATUS	SIGN_PARETO
FAIL...	PAT_TYPE	TESTER_PIN_DEF
FTEST	PATTERN	TESTER_PKG_TYPE

1.8 List of Central Schema Tables:

(HyperText links to table definitions.)

ACL_OBJECT_IDENTITY	DPMON_REPORT	QRTZ_CALEDARS
ACL_PERMISSION	DPMON_SCHEDULE_LOG	QRTZ_CRON_TRIGGERS
AUTHORITIES	DPMON_TASK	QRTZ FIRED TRIGGERS
DB_ENGINES	DPMON_USER_ATTRIB	QRTZ_JOB_DETAILS
DPF2FIELD	DPMON_USER_INFO	QRTZ_JOB_LISTENERS
DPFILE2FILE	GENERAL_CONFIG	QRTZ_LOCKS
DPF_FIELD	MACHINES	QRTZ_PAUSED_TRIGGER_GRP
DP_FILE	MACHINE_STAT	QRTZ_SIMPLE_TRIGGERS
DP_FOLDER	MACHINE_STAT_ARCHIVE	QRTZ_TRIGGERS
DPMON_ANNOT	MAP_SRV2DB	QRTZ_TRIGGER_LISTENERS
DPMON_APP	NODES	SCHEDULING_DECISION
DPMON_APP_SERVER	NODE_STAT	SCHEDULING_DETAILS
DPMON_DASH_COMP	NODE_STAT_ARCHIVE	SERVERS
DPMON_HOTLIST	QRTZ_BLOB_TRIGGERS	

2.0 Oracle Database — More Than 1,000 Parameters

Server Side Changes

dataPOWER installations on the Oracle database platform handle programs with more than 1,000 columns by creating additional versions of the RES..., WAF..., LIM..., and LOT... dynamic database tables, when necessary.

The dynamic tables on the Oracle platform limit the t1-tn test index fields to t[900-indexes] (*n*, as defined in **test_index** in the DEF... table.) When a program has more than 900 parameters, the RES..._1, WAF..._1, LIM..._1, and LOT..._1 dynamic database tables will be created to handle the additional parameters — 901-1,800. If there are more than 1,800 parameters, the RES..._2, WAF..._2, LIM..._2, and LOT..._2 dynamic database tables will be created to handle parameters 1,801-2,700, and so on.

Updating Old Databases to 6.5 and Later

Because the Oracle dynamic tables that were limited to 999 parameters are now limited to 900, if an old database has between 901 and 999 parameters, running DbUpdate on the old database will automatically create the RES..._1, WAF..._1, LIM..._1, and LOT..._1 dynamic database tables to handle the additional parameters.

3.0 Log Tables

Abstract Log tables ([EVENT_LOG](#), [OP_LOG](#), [WF_LOG](#), [TAG_LOG](#), [LEH_LOG](#), [Z_LOG](#), [MEM_LOG](#), [DF_LOG](#) and [BIN_LOG](#)) keep track of ongoing operations at the data sources. They form the heart of the ‘living’ part of the database. The [BIN_LOG](#), [TAG_LOG](#), [DF_LOG](#), [MEM_LOG](#), [Z_LOG](#), [EVENT_LOG](#) tables are not described in this section; however they are described in the following sections [“BIN_LOG,”](#) page 79, [“TAG_LOG,”](#) page 130, [“DF_LOG,”](#) page 159, [“MEM_LOG,”](#) page 301, [“Z_LOG,”](#) page 154 and [“EVENT_LOG,”](#) page 440.

Along the manufacturing process of a final [product](#), it is subjected to multiple process steps ([PROC_STEP](#)). A process step may either be a [PCM](#), [Esort](#) or F/T [test program](#), in which case the data insertion results in an entry in [OP_LOG](#) (and potentially several entries in [WF_LOG](#)) and when binning information is available in entries in [BIN_LOG](#). Equipment and/or lot maintenance are handled as events in [EVENT_LOG](#) and linked to what is affected by the event process step, [lot](#), wafer, etc. A defect inspection step results in entries in the [OP_LOG](#) and [DF_LOG](#) tables. A *bit*MAP inspection step results in entries in the [MEM_LOG](#) table.

3.1 OP_LOG

Abstract Each **raw data** file processed by a data **reader** results in an entry in this table. The exceptions are the Metrology, LEH, Bitmap and Events readers. For Metrology, one data file may result in many entries in OP_LOG, one per program/wafer combination. For LEH, one data file may result in many entries in OP_LOG, one per lot. For defect, one data file may result in many entries in OP_LOG, one for each wafer/step combination.

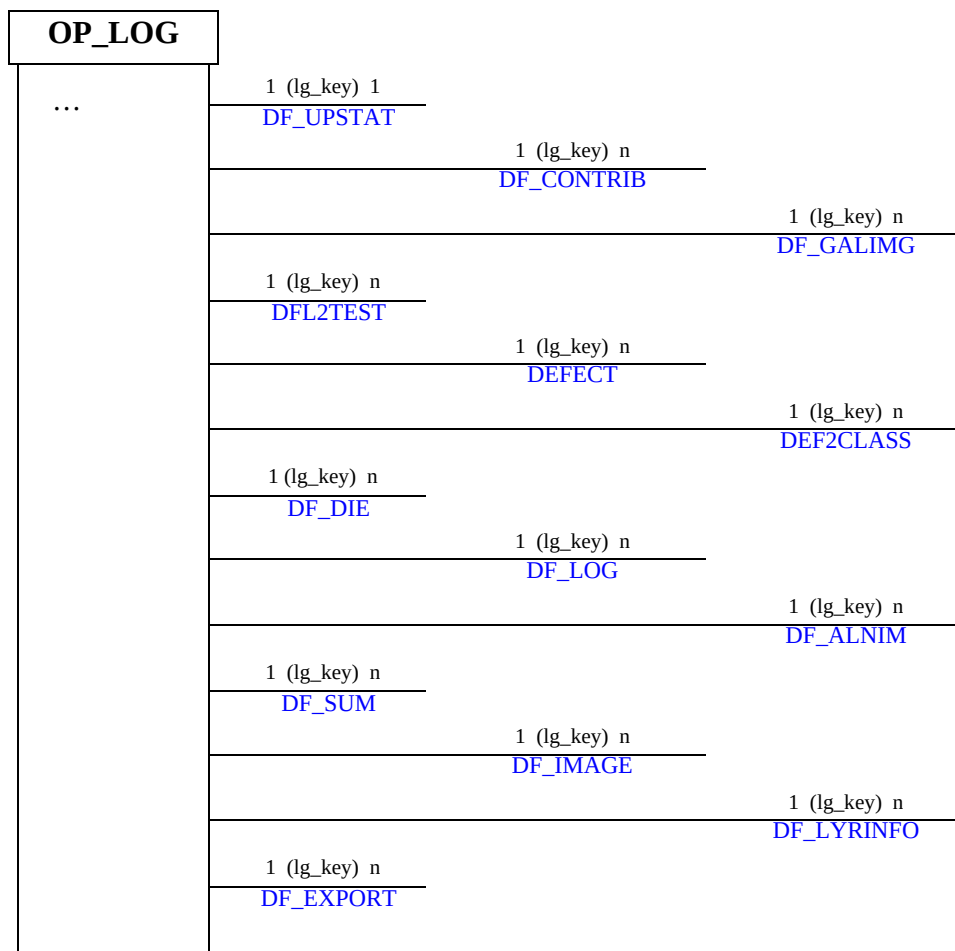
Table

OP_LOG		
	lg_key	
1 (step_key) n PROC_STEP	pgc_key	1 (lg_key) n BIN_LOG
1 (eqkey1..6) n EQUIPMENT	start_time	
	end_time	1 (lg_key) n
	insert_time	ARCHIVE_LOG
1 (pg_key) n PROGRAM	run_id	
	wf_key	1 (lg_key) n
	lot_key	WF_LOG
1 (lot_key) n LOT	src_lot	
	pg_key	1 (lg_key) n
	step_key	LOT...
1 (src_lot) n LOT	stage_key	
	rcp_key	1 (lg_key) n
1 (pd_key) n PRODUCT	pd_key	SSA_RUN_DET
	em_key	
1 (em_key) n PEOPLE	eqkey1	1 (lg_key) n
	eqkey2	WAF...
	eqkey3	
	eqkey4	1 (lg_key) n
	eqkey5	RES...
1 (pgc_key) n PROG_CLASS	eqkey6	
	ni	1 (lg_key) n
	row_cnt	MEM_DEF2MEM
1 (stage_key) n TECH_STAGE	pr_key	
	monitor_flag	1 (lg_key) n
	rework_flag	MEM_UPSTAT
1 (wf_key) n WAFER	test_mode	
		1 (lg_key) n
1 (rcp_key) n RECIPE		TAG_LOG
		1 (lg_key) n
1 (pr_key) n PROCESS		MEM_MEMLOG2LAYER
		1 (lg_key) n
		Z_UPDATE_WAF
		1 (lg_key) n
		Z_LOG
		1 (lg_key) n
		MEM_LOG
		1 (lg_key) n
		BIN_GAL
		Links to CV Related Tables ▶
		Links to Defect Related Tables ▶

Links to CV Related Tables

OP_LOG	
...	1 (a_lg_key) 1 CV_IDEF2IDEF
	1 (s_lg_key) n CV_EDEF2IDEF
	1 (lg_key) n CV_UPSTAT
	1 (lg_key) n CV_ADL
	1 (lg_key) n CV_PIN_RESULTS
	1 (lg_key) n CV_DEFECT
	1 (s_lg_key) 1 CV_EDEF2IDEF_ALL
	1 (lg_key) n CV_DIE_RESULTS
	1 (lg_key) n CV_CELLI2IDEF
	1 (a_lg_key) 1 CV_EDEF2IDEF_ALL
	1 (a_lg_key) n CV_E_IDEF2IDEF
	1 (s_lg_key) n CV_E_IDEF2IDEF
	1 (a_lg_key) 1 CV_E2I_INHERIT
	1 (s_lg_key) n CV_E2I_INHERIT
	1 (s_lg_key) n CV_LOG
	1 (lg_key) n CV_PIN_SUM
	1 (s_lg_key) n CV_PADGRP_SUM
	1 (s_lg_key) n CV_DEF2TYPE_<lot_key>

Links to Defect Related Tables



Serial Key

lg_key

A unique key is assigned to each entry in OP_LOG. It is stored automatically.

Foreign Keys / Link Description

step_key

Identifies which process step lead to the entry in OP_LOG. Links into PROC_STEP.

eqkey1...eqkey6

A given process step which leads to a data file is executed employing various kinds of **equipment**. Each type of equipment may exist in multiple instantiations. Which one is used is controlled via the links into EQUIPMENT.

pg_key

A raw data file is produced by a particular program which defines its exact format and content. Programs are maintained in PROGRAM.

lot_key

Links into LOT to allow lot tracking over multiple process steps.



Front



Contents



Index

pd_key A raw data file is related to a product which has an entry in PRODUCT.

em_key This link identifies the responsible operator (maintained in PEOPLE) for a given raw data file resulting in a OP_LOG entry.



Note that a single operator must take responsibility for the entire raw data file production. Shift changes during a process step are not handled.

pgc_key Program class key. Defines a specific grouping of programs. Can be set as a command line argument to the readers when program is new.

wf_key A link to the WAFER. This link is used with the metrology data only.

src_lot A link to the LOT table, where the lot is the source (parent) lot. Used for aligning data from several child-lots that have the same source lot.

stage_key Links into TECH_STAGE, which has information about the current stage of the process.

rcp_key Links into RECIPE, where the recipe name resides.

pr_key A raw data file is related to a product which has an entry in PRODUCT.

Other Fields in OP_LOG

run_id Run identifier. Reserved for future use.

start_time, end_time Start / end time of process step leading to entry in OP_LOG.

insert_time Time of data insertion into the database, defaults to the current time.

test_mode Used to identify the consolidation action to be taken. The following are the acceptable values for the summary consolidation operation (all other values are ignored):

Value	Description
p/P	Production (First pass) or Norm Test
o/O	Reject Retest (recycle) or Retest opens/shorts
r/R	Complete Lot Rework
l/L	Low yield Lot Request of reject
e/E	Engineering

Value	Description
a/A	Quality Acceptance
s/S	Stop on fail

monitor_flag Flags monitor wafers or devices, such as when testing golden devices. Legal values are ‘Y’ (Yes) and ‘N’ (No) (N is the default). Used by the Metrology reader to differentiate between monitor (offline) and inline data.

rework_flag Indicates whether data is reworked or not. Rework action is determined by the **rework_action** field in the PROGRAM table. Legal values are:

Value	Description
0	no rework
1	latest reworked data
n	oldest rework data

When data is at wafer level (non-metrology) the **rework_flag** in WF_LOG is used, while the **rework_flag** in OP_LOG is set to 0 or 1 only.

ni Boxplot criteria factor. Used in the outlier filtering of the summaries.

Value	Description
0	no outlier filtering
-1	outlier filtering using database outlier limits
x	boxplot criteria factor, where x is a floating-point value

row_cnt Number of rows in the RES... table.

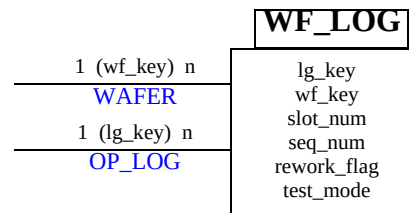
For program class 4 (bin map) **row_cnt** is the number of columns (die on the wafer) for that OP_LOG entry.

For Defect program class (14), the **row_cnt** is the number of defects in the given log entry.

3.2 WF_LOG

Abstract A raw data file processed by a data reader results in n entries in this table, where n is the number of wafers in the file (**WF_LOG** is only used in those cases where wafer ID is defined). **WF_LOG** is not used by the Metrology, bitMAP, Events, Defect or LEH readers.

Table



Foreign Keys / Link Description

lg_key Foreign key into OP_LOG.

wf_key Foreign key into WAFER.

Other Fields in WF_LOG

slot_num Slot number. Reserved for future use

seq_num Sequential wafer number. Reserved for future use

rework_flag Indicates whether data is reworked or not. Rework action is determined by the **rework_action** field in the PROGRAM table. Legal values are:

Value	Description
0	no rework
1	latest reworked data
n	oldest rework data

test_mode Not in use in the current release.

3.3 LEH_LOG

Abstract Log table for LEH data.

Table

	LEH_LOG
1 (pg_key) n	pg_key pd_key pr_key lot_key src_lot test_index test_name step_key stage_key start_time end_time eq_key rcp_key em_key rework_flag
PROGRAM	
1 (pd_key) n	
PRODUCT	
1 (pr_key) n	
PROCESS	
1 (lot_key) n	
LOT	
1 (src_lot) n	
LOT	
1 (step_key) n	
PROC_STEP	
1 (stage_key) n	
TECH_STAGE	
1 (eq_key) n	
EQUIPMENT	
1 (rcp_key) n	
RECIPE	
1 (em_key) n	
PEOPLE	

Serial Key None.

Foreign Keys / Link Description

pg_key	Reference to PROGRAM table.
pd_key	Reference to PRODUCT table.
pr_key	Reference to PROCESS table.
lot_key	Reference to LOT table.
src_lot	Reference to LOT table.
step_key	Reference to PROC_STEP table.
stage_key	Reference to TECH_STAGE table.
eq_key	Reference to EQUIPMENT table.
rcp_key	Reference to RECIPE table.
em_key	Reference to PEOPLE table.
test_index	Test index from DEF... table.
test_name	Test name (cond0 from DEF... table).

Other Fields in LEH_LOG



Front



Contents



Index

start_time	Track-in.
end_time	Track-out.
rework_flag	Indicates whether data is reworked or not.

3.4 WEH_LOG

Abstract Log table for WEH data.

Table

	WEH_LOG
1 (pg_key) n PROGRAM	pg_key*
1 (pd_key) n PRODUCT	pd_key*
1 (pr_key) n PROCESS	pr_key*
1 (lot_key) n LOT	lot_key*
1 (src_lot) n LOT	src_lot*
1 (step_key) n PROC_STEP	test_index*
1 (stage_key) n TECH_STAGE	test_name*
1 (eq_key) n EQUIPMENT	step_key*
1 (rcp_key) n RECIPE	stage_key*
1 (em_key) n PEOPLE	start_time*
	end_time*
	eq_key*
	em_key*
	rework_flag*

Serial Key None.

Foreign Keys / Link Description

pg_key	Reference to PROGRAM table.
pd_key	Reference to PRODUCT table.
pr_key	Reference to PROCESS table.
lot_key	Reference to LOT table.
src_lot	Reference to LOT table.
step_key	Reference to PROC_STEP table.
stage_key	Reference to TECH_STAGE table.
eq_key	Reference to EQUIPMENT table.
rcp_key	Reference to RECIPE table.
em_key	Reference to PEOPLE table.

Other Fields in WEH_LOG

test_index	Test index from DEF... table.
test_name	Test name (cond0 from DEF... table).



Front



Contents



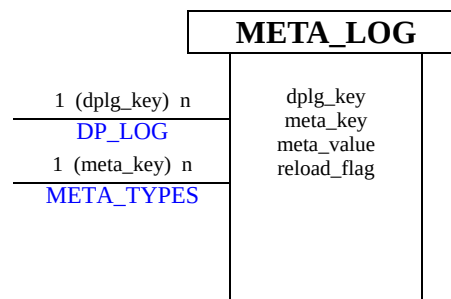
Index

start_time	Track-in.
end_time	Track-out.
rework_flag	Indicates whether data is reworked or not.

3.5 META_LOG

Abstract This table stores the META_KEY from the META_TYPES table and also stores the meta value. For example, if META_KEY represents "LOT", the meta value will be something like "ABCD1.0". Meta values are limited to 128 characters.

Table



Serial Key None.

Foreign Keys / Link Description

dplg_key	Foreign key from the DP_LOG table.
meta_key	Foreign key from the META_TYPES table.

Other Fields in META_LOG

meta_value
reload_flag

4.0 PROGRAM and Related Tables

Abstract The term “**program**” is central in the design of *dataPOWER*. Each type of a program generates an entry in the **PROGRAM** table and instantiates a collection of five tables: **RES...**, **WAF...**, **LOT...**, **LIM...** and **DEF...** where the dots (‘...’) indicate that the table names are postfixed by **pg_key**. This concept is obviously highly denormalized for performance reasons.

Additional tables such as **BIN_LOG**, **HIST_BIN** and **CONDITION** are linked to **PROGRAM** to store binning and condition information.

Definition of a “program” as a record in this table is as follows: **A program defines a set of parametric tests which is stable over time and uses a constant set of test limits.**

Each program is associated with a fixed set of *dataPOWER* indexes and conditions that can not be changed and is automatically generated by the data reader, according to a given format file.

4.1 Data Model Description

Since this part of the data model is highly denormalized (for performance reasons — the bulk of the data is stored in the **RES...** tables) and is closely linked to the spreadsheet structure of the analysis front end, it is worthwhile to further explain the setup of the tables involved.

In essence, the **RES...** table corresponds to the **data** view in the *dataPOWER* analysis spreadsheet, whereas the **sum** (summary) view corresponds to the information stored in the **WAF...**, and **LOT...** tables. The **lim** view is a representation of the **LIM...** table. This is further explained in the following figures:

FIGURE 2. data view in the *dataPOWER* analysis spreadsheet

				Test_Name Pin Unit	Test_a 1 mV	Test_a 2 mV	Test_b 1 μV	...
				Lot	Wafer	Device	← (tot. 50 columns) →	
lg_key	i0.....	 i2		t1 t50				
	A	1	1					
	A	1	2					
	A	2	1	— — —	— — —	— — —	— — —	— — —
	A	2	2					
	B	1	1	— — —	— — —	— — —	— — —	— — —
	B	1	2					
	B	1	3					
RES... table								

RES... table

In the above example, the **RES...** table has three indexes (**i0 ... im-1** with $m=3$): **Lot**, **Wafer** and **Device**. They all happen to be key indexes; the combination of all

three uniquely identifies a row. Additional non-key indexes can be added optionally. Furthermore, an index **lg_key** is automatically added to link data to a particular record in **OP_LOG**.

The table also contains 50 columns **t1 ... tn** with $n=50$, corresponding to the 50 tests executed on each device. These columns are headed by three **conditions**: **Test_Name**, **Pin** and **Unit**. The first two are key conditions, as only a combination of the two uniquely identify a column. **Unit** has only a descriptive purpose and is therefore not considered a key condition. Details about indexes are stored in **DEF...** and details of conditions in **CONDITION**.

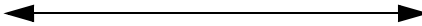
The corresponding summary sheet is depicted below:

FIGURE 3. summary view in the *dataPOWER* analysis spread sheet

Test #	Parameter	Unit	Mean	Std.Dev.	Count	Min.	Max
	Test_a_1	mV					
	Test_a_2	mV					
	Test_b_1	μA					

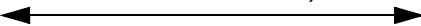
This statistical summary information is calculated at spread sheet load time. And it resembles the information stored in **LOT...** (on a per lot bases) and **WAF...** (on a per wafer basis). Consequently, for example, the **LOT...** table has the following layout (which is in essence the transposed summary sheet as displayed above). The database tables also have more statistical information in the form of q1, q2, q3, sum, sum square, ...etc.

FIGURE 4. LOT... table layout

lg_key	lot_key	stat_type	t1 t50
1	A	avg	(tot. 50 columns corresponding to the columns in RES...) 
1	A	std	
1	A	cnt	
2	A	:	
3	B	avg	

Similarly to the summary sheet information, test limits are stored in the **LIM...** table (depicted below) which is in essence a transformation of the '**lim**' view in the spread sheet.

FIGURE 5. LIM... table layout

sbin_num	limit_name	t1 t50
1	LSL	(tot. 50 columns corresponding to the columns in RES...) 
1	USL	
1	LPL	
	:	
2	LSL	

Thus, for all pass bins, a complete set of limits can be stored for all tests **t1 ... tm** ($m=50$). However, no limit history is stored.

Given the above table layout, it becomes clear that a new record entered into **PROGRAM** - which causes the instantiation of a set of the previously described tables - should describe a **raw data** source, which over time will:

- Always produce the same set of tests (**t1 ... tm**). (Or at least a reasonably big subset. Missing tests are stored as empty fields and thus add to the storage inefficiency.)
- Always use the same test limits, as they are stored on a per program basis. This must be kept in mind when the program name **PPID** is generated, which causes a new entry in **PROGRAM**. The **PPID** could encode such items as:
 - generic device type name
 - ROM code version
 - **test stage**
 - test program revision and update level

A given entry **PROGRAM** can be either of **prog_type** *fixed*, *semi_dynamic*, *dynamic* or *update* rows. In the static case (default), the format of previously generated tables **RES...**, etc. will never change. Thus, if for some reason, a future **datalog** file contains additional tests, they will be neglected. Semi_Dynamic is similar to static, but in addition the data readers automatically fill in fields in tables pointed to by **OP_LOG**, if available in the datalog files. Dynamic programs, in contrast, will automatically expand the tables to handle the additional tests. This however, is a computationally expensive operation.



*Note that such strategies as datalog on fail lead to very inefficient data storage, as a new record is generated in **RES...** for each failing device with only one non-empty field, namely for the failing test.*

4.2 Tester Summary Sheet Storage

Some test systems produce both datalog and **tester summary** sheet data, which in an aggregated stage belongs to the **WAF...** and **LOT...** tables. But this is not feasible for the following reasons:

- Summary sheets typically have a different resolution than datalog files. E.g., leakage currents are measured on all pins, thus causing multiple entries (columns) in **RES...**, but result in only one entry in the tester summary sheet.
- New releases of test programs very often have different test limits, thus must cause a new entry in **PROGRAM** and consequently instantiate a new set of tables (including **LIM...**). Only rarely do new releases contain additional tests.

As a result, tester summary sheet information is typically stored as independent entries in **PROGRAM**. Corresponding **PPID** names will end with **..._SUM**. Since summary sheet formats do not typically change between test program updates, **PPID** names do need to encode test program version and update level. For summaries, the **prog_type** field in the **PROGRAM** table should be set to Dynamic.

The resulting **sum** (summary) view when loaded into the *dataPOWER* analysis front end is then as follows:

Parameter	Units	mean	std.dev.	count	min	max
Lot	index					
Wafer	index					
TP_vers.	index					
Tested	pcs	1010	5	5	1000	1020
Passed	pcs	690	2	5	680	700
Tst_a	pcs	135	7	5	120	150
Tst_b	pcs	70	12	5	50	90
Tst_c	pcs	60	10	5	40	80
Tst_d	pcs	55	20	5	20	90

The shaded information is stored on a run, wafer, and lot basis, respectively, in **WAF...**, **LOT...** (in a transposed format).

This layout allows immediate meaningful use of box plots (yields per lot with distribution over the wafers), histograms (yield distribution), trend charts, etc.

4.3 PROGRAM

Abstract Base data related to raw data files (produced by a program) are stored in this table.

Table

PROGRAM		
	pg_key	
1 (eqt_key) n	pg_grp_key meta_key identifier_key eqt_key pgc_key wmap_key ppid prog_desc bins tests indexes conditions sampling_method sampling_rate status accept_data res_aging_days stats_aging_days prog_typ rework_action wafer_index site_index diex_index diey_index bin_index bad_yield scrap_yield tag_action prog_wrap die_index mdl_index wfg_index	1 (pg_key) n
EQUIP_TYPE		MEM_UPSTAT
1 (wmap_key) n		1 (pg_key) n
WMAP_CONFIG		SITE2DIE
1 (pgc_key) n		1 (pg_key) n
PROG_CLASS		DF_BIN
(using PROG2PROD) m (pg_key) n		1 (pg_key) n
PRODUCT		BIN_COLORS
(using PROG2CUST) m (pg_key) n		1 (pg_key) n
CUSTOMER		BIN_GAL
(using PAR2PARGRP) m (pg_key) n		1 (pg_key) n
PAR_GRP		T2T_SUM
(using PROG2STEP) m (pg_key) n		1 (pg_key) n
PROC_STEP		EVENT_CONFIG
(using PROG2PROC) m (pg_key) n		1 (pg_key) n
PROCESS		MB_PROGRULE
(using PROG2FAM) m (pg_key) n		1 (pg_key) n
FAMILY		EQUIP_TYPE
(using PROG2LOT) m (pg_key) n		1 (pg_key) n
LOT		LEH_CONFIG
(using PROG2EQUIP) m (pg_key) n		1 (pg_key) n
EQUIPMENT		WEH_CONFIG
(using PROG2TECH) m (pg_key) n		1 (pg_key) n
TECHNOLOGY		PAR2PARGRP
(using PROG2STAGE) m (pg_key) n		1 (pg_key) n
TECH_STAGE		ARCHIVE_LOG
(using PROG2SRCLLOT) m (pg_key) n		1 (pg_key) n
LOT		DF_EXPORT
1 (pg_grp_key) n		1 (pg_key) n
PROGRAM_GROUP		MEM_BOP_CONFIG
1 (meta_key) n		1 (pg_key) n
META_DEF		MEM_BOP_UPSTAT
1 (identifier_key) n		
		Links to Program Related Tables
		Links to Characterization Tables
		Links to Product/Lot/Wafer Tables
		Links to Zonal Bin Summary Tables
		Links to CV Tables
		Links to SSA Tables
		Links to Log Tables

Links to Program
Related Tables

PROGRAM	
...	1 (pg_key) n PROG_REV
	1 (pg_key) n PROG2PKG
	1 (pg_key) n CONDITION
	1 (pg_key) n BIN_LOG
	1 (pg_key) n HIST_BIN
	1 (pg_key) n HIST_BIN_EXTRAS
	1 (pg_key) n HIST_BIN_ZEROS
	1 (pg_key) n SUM_COND
	1 (pg_key) n PROG_DATATYPE
	1 (pg_key) n PROG2CUST
	1 (pg_key) n PROG2PROD
	1 (pg_key) n PROG2STEP

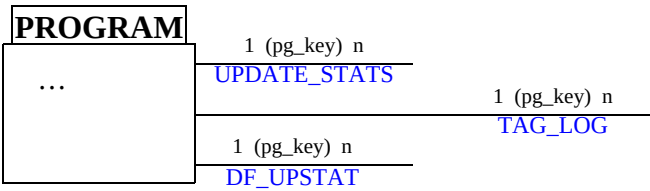
Links to Zonal Bin
Summary Tables

PROGRAM	
...	1 (pg_key) n Z_UPDATE_LOT
	1 (pg_key) n Z_CONFIG
	1 (pg_key) n Z_UPDATE_WAF
	1 (pg_key) n Z_LOG
	1 (pg_key) n ZONES

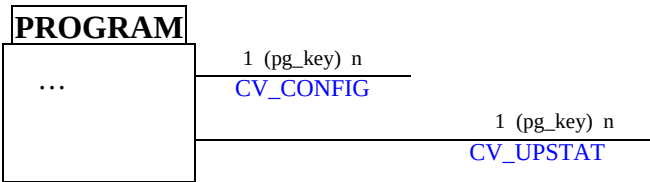
Links to
Characterization
Tables

PROGRAM	
...	1 (pg_key) n CHAR_LOG
	1 (pg_key) n FTEST
	1 (char_key) n FAIL...

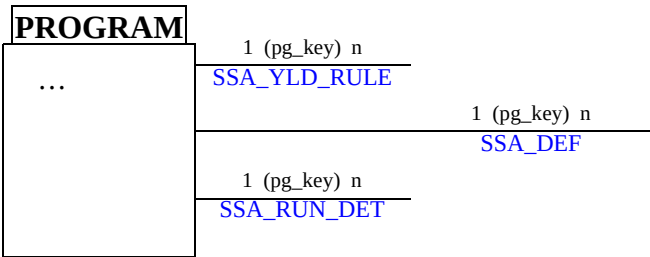
Links to Product/
Lot/Wafer Tables



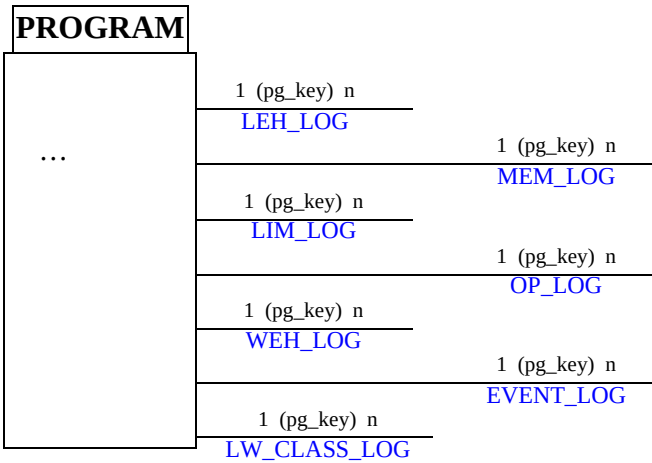
Links to CV Tables



Links to SSA Tables



Links to Log Tables



Serial Key	pg_key	uniquely identifies an entry in PROGRAM. It is used to postfix table names of RES..., WAF..., LOT..., LIM... and DEF.... pg_key is automatically generated out of the program name PPID .
Foreign Keys / Link Description	eqt_key	Link into EQUIP_TYPE.
	wmap_key	Link into WMAP_CONFIG.
	pg_grp_key	Foreign key from the PROGRAM_GROUP table. If pr_grp_key is not NULL, program belongs to a given program group.
	meta_key	Foreign key from META_TYPES table.
Other Fields in PROGRAM	identifier_key	For fab/metrology programs, used to identify the step of the program. For WEH and LEH programs, used to identify the process of the program. For ASCII programs, used to identify the product of the program.
	PPID	Production Program ID. This is more than the actual program name as, based on the PPID, a unique pg_key is calculated, which in turn causes instantiations of RES... and related tables. (See “ Data Model Description ,” page 44).
		A typical value for PPID (assuming a family test program for multiple subtypes, ROM code alternatives and test stages) is a concatenation: PPID = TP_name + revision (+ sub_type) (+ rom_code) + test_stage
	pgc_key	Program class key. Defines a specific grouping of programs. Can be set as a command line argument to the readers when program is new.
	prog_desc	Descriptive text per program.
	bins	Number of bins produced by the program. This is used to interpret data in BIN_LOG .
	tests	Number of indexes plus tests executed by the program, thus defining the value of n in the fields t1 ... tn-indexes in RES..., WAF..., LOT..., LIM... Mandatory ; automatically generated upon record insertion, by the data readers. For Events, this number does not include the hidden indexes of the program. It only includes the parameters and the indexes stored in RES...

indexes Number of indexes used by the program, thus defining the value of **m** in the fields i0 ... **im-1** in RES....
Mandatory; automatically generated upon record insertion, by the data readers.

For Events, this number does not include the hidden indexes of the program. It only includes the indexes stored in RES...

conditions Number of conditions used by the program (see table CONDITION). **Mandatory**; automatically generated upon record insertion, by the data readers.

sampling_method Defines reader sampling strategies. Available strategies include:

Strategy	sample_flag
All	1
First n Devices	2
Last n Devices	3
Every nth Device	4
Random sample of n devices	5
bin = n only	6
bin <= n only	7
Bin >= n only	8
Bin != n only	9
Pass Bins	10
Fail Bins	11

The value of n is stored in **sampling_rate** below.
sample_flag defaults to 1 (collect all data).

sampling_rate Value of n as used in the sampling strategy defined with **sample_method**. **Mandatory** if **sample_flag** > 1.

status Indicates whether a program is normalized and/or aligned with other data types.

accept_data If not set, data for the described program will not be accepted by the data reader and will be left in the polling directory. This is typically the case before a program has been appropriately configured by the DBA. A value of 1 allows acceptance of data. Default is 0. Controllable by the **-db_accept** option of data readers. This field is not used by the Metrology reader.

res_aging_days Used by backup and delete procedures to decide when (at what age) program related raw data (RES...) shall be backed up or deleted.

stats_aging_days Used by backup and delete procedures to decide when (at what age) program related summary data (LOT... and WAF...) shall be deleted.

prog_type Defines whether or not dynamic expansion of RES... and related tables is allowed. Can be set as a command line argument to the readers when program is new.

Allowed options are:

Value	Description
1	semi_dynamic
2	fixed
3	dynamic
4	update row

Default is 1 for static RES... and related tables.

rework_action Action to be performed when rework is detected.

Legal values are:

Value	Description
1	no action (append)
2	append and mark
3	overwrite
4	load no data

wafer_index Index number (starting with 0) in RES... that has the Wafer ID.

site_index Index number (starting with 0) in RES... that has site information, e.g. for PCM programs.

diex_index Index number (starting with 0) in RES... that has **die_x** information. (**track-in date** index for LEH reader.)

For Events, it holds the number of hidden indexes for the program.

diey_index Index number (starting with 0) in RES... that has **die_y** information. (**track-out date** index for LEH reader.)

For Events, it holds the number of normalized indexes for the program.

bin_index Index number (starting with 0) in RES... that has Binning information. (**lot** index for LEH reader.)

tag_action Used to enable and disable the automatic tagging of wafers and lots. The tagging is based on the calculated yield and the **bad_yield** and **scrap_yield** values. Legal values are:

Value	Description
1	tagging is disabled
2	tagging is enabled

The default is **2**.

For *bitMAP* programs, this field determines the amount of *bitMAP* raw data inserted into the database:

Value	Description
0	no control
1	control using max fail
2	control using incident count
3	control using both fail count and incident count

The default is **0**.

For Events, there is no automatic tagging. This column specifies whether manual tagging is enabled in this program.

Value	Description
1	manual tagging is allowed
2	manual tagging is not allowed

bad_yield *User-defined value from 0 to 100. It is used to tag the wafers and lots as “bad” when the yield is less than or equal to this value. When the value is less than 0, it is ignored and considered undefined.

For Defect programs, this column holds the maximum defect density before a wafer is tagged as “High Defect Density” (**tag_flag**=5).

For *bitMAP* programs, this column defines the maximum number of failing bits allowed per log entry. This value will be ignored if **tag_action** = 0. The default is **-1**.

scrap_yield *User-defined value from 0 to 100. It is used to tag the wafers and lots as “scrap” when the yield is less than or equal to this value. When the value is less than 0, it is ignored and considered undefined.

For Defect programs, this column holds the maximum number of defects before a wafer is tagged as “High Defect Count” (tag_flag = 6)

For bitMAP programs, this column defines the maximum number of incidents allowed per log entry. This value will be ignored if tag_action = 0.

The default is -1.



When both **bad_yield and **scrap_yield** are defined, the value of **scrap_yield** must be less than the value of **bad_yield**.*

prog_wrap A flag indicating if a program is wrapped. If it is wrapped, the value is set to the maximum number of columns. If not it is set to -1.

die_index For future use. Die traceability — die level. Default = 1.

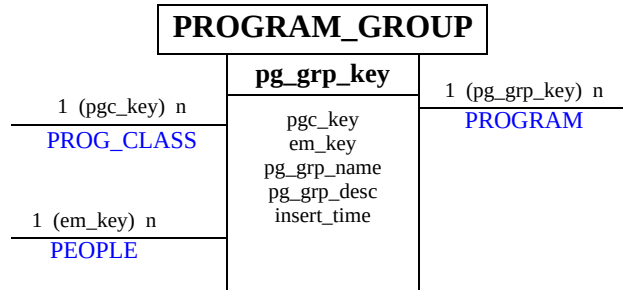
mdl_index For future use. Die traceability — module level. Default = 1.

wfg_index For future use. Die traceability — wafer group level. Default = 1.

4.4 PROGRAM_GROUP

Abstract This table stores program groups.

Table



Serial Key

pg_grp_key A unique key is assigned to each program group.

Foreign Keys / Link Description

em_key Foreign key from PEOPLE table.

pgc_key Foreign key from PROG_CLASS.

Other Fields in PROGRAM_GROUP

pg_grp_name Program group name

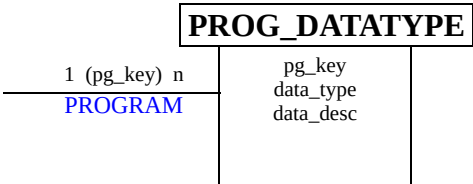
pg_grp_desc Program group description.

insert_time Time of insertion into the database.

4.5 PROG_DATATYPE

Abstract This table is used by the LEH (Lot Equipment History) dataBASE reader.

Table



Serial Key None.

Foreign Keys / Link Description **pg_key** The key of PROGRAM which the LEH data belongs to.

Other Fields in PROG_DATATYPE **data_type** Data types must be one of the following: TI, TO, TL, PE, OP, RP, LT, WN, RF, LO, or PR.

- TI — Track In
- TO — Track Out
- TL — Tooling
- PE — Processing Equipment
- OP — Operator
- RP — Recipe
- LT — Current Lotid
- WN — Wafer Number (current slot)
- RF — Rework
- LO — Location
- PR — Process

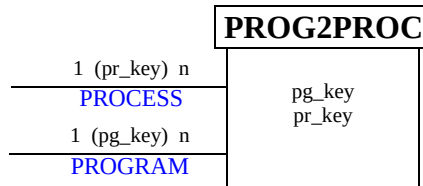
The only hard-coded data types are TI for track-in and TO for track-out.

data_desc The data description is user-defined in the reader's format file.

4.6 PROG2PROC

Abstract An m2n table holding all the available combinations of pr_keys and pg_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key A foreign key from the PROGRAM table.

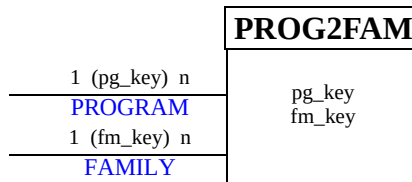
pr_key A foreign key from the PROCESS table.

Other Fields in PROG2PROC None.

4.7 PROG2FAM

Abstract An m2n table holding all the available combinations of fm_keys and pg_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key A foreign key from the PROGRAM table.

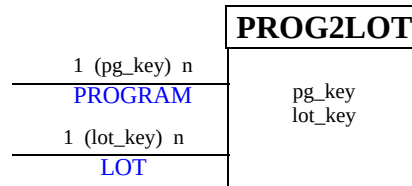
fm_key A foreign key from the FAMILY table.

Other Fields in PROG2FAM None.

4.8 PROG2LOT

Abstract An m2n table holding all the available combinations of lot_keys and pg_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key A foreign key from the PROGRAM table.

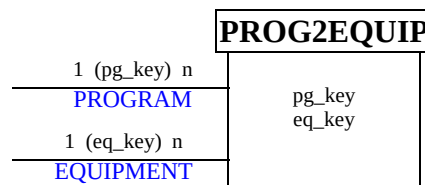
lot_key A foreign key from the LOT table.

Other Fields in PROG2LOT None.

4.9 PROG2EQUIP

Abstract An m2n table holding all the available combinations of eq_keys and pg_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key A foreign key from the PROGRAM table.

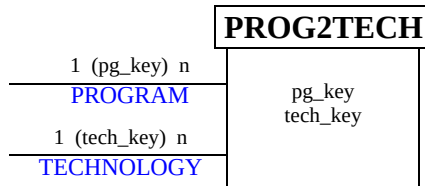
eq_key A foreign key from the EQUIPMENT table.

Other Fields in PROG2EQUIP None.

4.10 PROG2TECH

Abstract An m2n table holding all the available combinations of tech_keys and pg_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key A foreign key from the PROGRAM table.

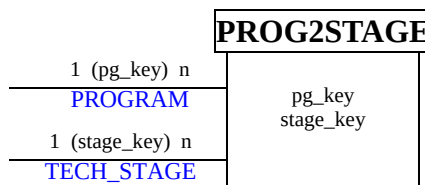
tech_key A foreign key from the TECHNOLOGY table.

Other Fields in PROG2TECH None.

4.11 PROG2STAGE

Abstract An m2n table holding all the available combinations of stage_keys and pg_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key A foreign key from the PROGRAM table.

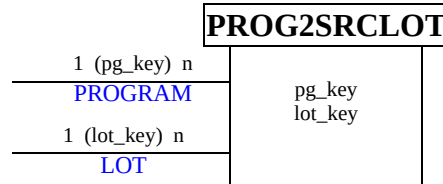
stage_key A foreign key from the TECH_STAGE table.

Other Fields in PROG2STAGE None.

4.12 PROG2SRCLOT

Abstract An m2n table holding all the available combinations of lot_keys and pg_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key A foreign key from the PROGRAM table.

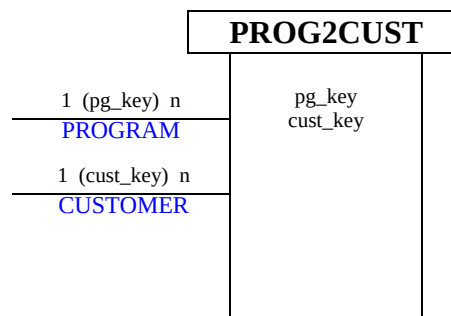
lot_key Foreign key from LOT table. Identifies the source lot.

Other Fields in PROG2SRCLOT None.

4.13 PROG2CUST

Abstract This table contains information about the relationship between programs and customers.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key Foreign key from the PROGRAM table.

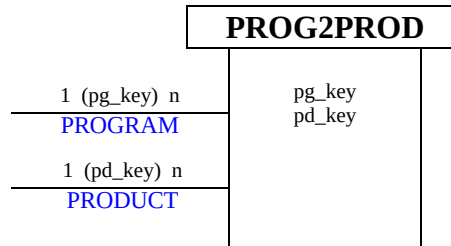
cust_key Foreign key from the CUSTOMER table.

Other Fields in PROG2CUST None.

4.14 PROG2PROD

Abstract This table contains information about the relationship between programs and products.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key Foreign key from the PROGRAM table.

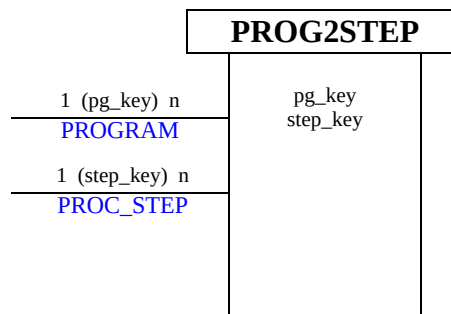
pd_key Foreign key from the PRODUCT table.

Other Fields in PROG2PROD None.

4.15 PROG2STEP

Abstract This table contains information about the relationship between programs and process steps.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key Foreign key from the PROGRAM table.

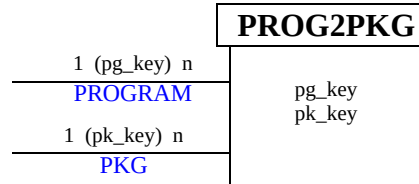
step_key Foreign key from the PROC_STEP table.

Other Fields in PROG2STEP None.

4.16 PROG2PKG

Abstract An m2n table holding all available combinations of pg_keys and pk_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key A foreign key from the PROGRAM table.

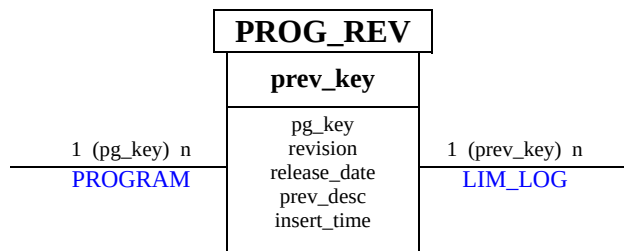
pk_key A foreign key from the PKG table.

Other Fields in PROG2PKG None.

4.17 PROG_REV

Abstract Program revision information are stored in this table.

Table



Serial Key

prev_key Automatically generated, unique key for each program revision, based on revision and pg_key.

Foreign Keys / Link Description

pg_key From the PROGRAM table. Relates a program to a program revision.

Other Fields in PROG_REV

revision Describes the revision of the program.

release_date Program revision release date.

prev_desc Descriptive text per program revision.

insert_time The current date and time of insertion into the database.
Date/time/year to the second. Defaults to current year to
the second.

4.18 DEF...

Abstract Contains descriptive data as per parameter collected by a given **program**. The table contains one record per test executed (parameter collected) by the program. Insofar as the table defines the contents of **RES...**, **WAF...**, **LOT...**, and **LIM...**. The “...” in the table name is replaced by the pg_key value from PROGRAM.

Table This table is related to a specific entry in **PROGRAM** in that its name is postfixed by **pg_key**.

DEF...
test_index
hist_mean
hist_stdev
hist_count
hist_q1, q2, q3
hist_yield
target_value
lower_bound
upper_bound
pts_below
c1...c20
pts_above
limits_type
fail_bin
precision
test_type
dist_type
store_flag
scale_factor
filter_flag
hilim
lolim
cond0...n

Serial Key None.

Foreign Keys / Link Description None.

Other Fields in DEF... **test_index** tests (indexes and parameters) are numbered (indexed) from 1 to m+n, corresponding to the m index and n test columns **i0 ... im-1** and **t1 ... tn** respectively in RES....



Note that many of the fields described below only make sense for test columns, not for indexes.

- hist_mean** Historical mean value of test described in current record. **Generated by the reader.**
- hist_stdev** Historical standard deviation of test described in current record. **Generated by the reader.**
- hist_count** Number of entries used to calculate **hist_mean** and **hist_count**. **Generated by the reader.**

hist_q1,q2,q3	Historical quartiles of test described in current record. Generated by the reader.
hist_yield	Yield loss in percent of test described in current record. Generated by the reader.
target_value	Target value of test described in current record.
lower_bound	Calculated from the historical q's as the lower limit for the historical distribution. Generated by the reader. For Defect program class it is used to determine the lower bound of the defect size bin when c2 = -1.
upper_bound	Calculated from the historical q's as the upper limit for the historical distribution. Generated by the reader. For Defect program class it is used to determine the upper bound of the defect size bin when c2 = -1.
pts_below	Points below the lower_bound. Generated by the reader.
c1-c20	Historical distribution. Generated by the reader.

For *binMAP* class data, **c20** is used to mark die as:

Value	Description
0	good die
1	bad die
2	ugly die

For Events class data, **c1** and **c5** are used according to the following:

Column	Description
c1	1, Normalized index 0, Non normalized index
c5	index sequence in the format file when the program is first created

For Defect program class, **c1** and **c2** are used to determine the summary type according to the following:

Summary Type	c1	c2
Adder defect class count for dfc_key = c1	0<c1	0
Defective die count for dfc_key = c1	0<c1	1
Die that are clean of defects with dfc_key = c1 and passed	0<c1	2
Count of die that are clean of defects with dfc_key = c1 and failed	0<c1	3
Count of die that are defective with defects of class dfc_key = c1 and passed	0<c1	4
Count of die that are defective with defects of class dfc_key = c1 and failed	0<c1	5
Count of adder defects with class dfc_key = c1 in failed die	0<c1	6
size bin using x_size	-1	-1
size bin using y_size	-2	-1
size bin using df_area	-3	-1
size bin using dsize	-4	-1
index	0	0

pts_above Points above the upper_bound. **Generated by the reader.**

limits_type Defines type of limits for test. **Generated by the reader.**

Allowed Values:

Value	Description
N	no limits
U	upper limit only
L	lower limit only
B	both upper and lower limits

fail_bin The fail bin into which the device is classified if failing this test. Defaults to 32.

For binMAP class data, this field is set to **1** in the index section of the **DEF...** table (**test_index** <= indexes) when the **BinTypeIndex()** function is called in the dbascii reader.

precision Default display precision in analysis spreadsheet. Defaults to 3. Reserved for future use.

test_type Type of test.

For parameter (**test_index** > **indexes**) data types, the allowed values are:

Value	Description
1	float
2	integer
3	string
4	small integer
5	character

dist_type For statistical analysis of parameter distributions. Currently only normal distribution is supported. Defaults to 1 (Normal). The index description rows (first n rows, where n is the number of indexes) use **dist_type** to indicate whether the index is KEY or NonKey.

store_flag If set, parameters related to this test are inserted into the database, otherwise they are not. (Not implemented as of version 3.0) Available options include:

Value	Description
1	None
2	Results and Summary (Default)

scale_factor Scale Factor used for Parametric test results.

filter_flag Defines whether hilim and lolim are used for filtering. Available options include:

Value	Description
1	None, No filtering
2	Filter data based on hilim and lolim (described below)

Options are set from within the *dataPOWER* graphical interface. The default is 2.

The index description rows (the first n rows where n is the number of indexes) use **filter_flag** to indicate the data-type of the index (e.g. 57-Integer, 59-Floating, 61-String, ...etc.).

hilim, lolim

High / low acceptance limit. Data points above / below this limit are not inserted into the database. These can be entered numbers in *dataPOWER* or calculated using historical sigmas or quartiles from DEF....

hilim	lolim
1e+19	-1e+19
mean + nsigma	mean - nsigma
Q3+2*(Q3 - Q1) (user-defined value)	Q1-2*(Q3 - Q1) (user-defined value)

lolim defaults to -1e+19 and **hilim** defaults to +1e+19.

The index description rows (the first n rows, where n is the number of indexes) use **hilim** to indicate the size (in bytes) of the data stored in that index).



That this acceptance limit is similar in effect to the outlier limits in statistical terms, but the effect is irreversible and rejected values will never be inserted into the database.

cond0 ... condn

Condition values (e.g. Test names, Test numbers, units, etc.).

For example, for a condition named “**Test Name**,” a legal value could be “**Leakage Test**,” for a condition named “**Pin**,” a value would be “**15**” and for a condition named “**Unit**,” a value would be “**mA**.”

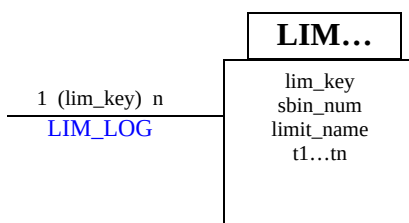
4.19 LIM...

Abstract Contains limits sets as per parameter collected by a given **program**. The table contains one record per limit set (outlier, production, spec, what-if) and a pass **bin**. The “...” in the table name is replaced by the **pg_key** value from **PROGRAM**.

This table is used by the following program classes:

- Wafer_Sort(1)
- Final_Test(2)
- PCM(5)
- Wafer_Sort_No_Bin(6)
- Metrology(10)

Table This table is related to a specific entry in **PROGRAM** because its name is postfixed by **pg_key**.



Serial Key None.

Foreign Keys / Link Description

lim_key From the LIM_LOG table. This table has information about the limit sets.

Other Fields in LIM...

t1 ... tn Limits related to tests described by test index 1 to n (as defined by **test_index** in DEF...).

sbin_num Pass **bin** number. Allows for separate sets of limits to be maintained for each pass bin. Reserved for future use.

limit_name Four sets of limits are stored per parameter: Outlier limits, production limits, spec limits and what-if limits. The legal values of **limit_name** are:

Value (lower bound)	Value (upper bound)	Description
LSL	USL	Lower / upper spec limits
LPL	UPL	Lower / upper production limits
LOL	UOL	Lower / upper outlier limits
LWL	UWL	Lower / upper what-if limits

4.20 LIM_LOG

Abstract The LIM_LOG table is for limit sets. This table holds information about each set of limits in the LIM... table.

Table

LIM_LOG		
lim_key		
1 (pg_key) n PROGRAM	pg_key prev_key meta_key lim_date lim_name lim_type insert_time def_flag meta_type owner	1 (lim_key) n LIM...
1 (prev_key) n PROG_REV		1 (lim_key) n CV_LIM2AI
1 (meta_key) n META_TYPES		
m (meta_key) n LOT , PRODUCT , PKG ...		

Serial Key

lim_key Primary key (lim_key). Constraint pk_lim_log.

Foreign Keys / Link Description

pg_key Foreign key from PROGRAM table.

prev_key Foreign key from PROG_REV table.

meta_key Foreign key from META_TYPES table.

Other Fields in LIM_LOG

lim_date Time stamp of the limit set.

lim_name Limit set name.

lim_type Specifies limit type. ("D--", "DMR"...)

insert_time The insertion time for the limit set. Date/time/year to the second, default is current year to the second.

def_flag Flag to specify default limit set.

meta_type Specifies the meta data that is associated with this limit set (LOT, SRC, PRD...)

owner The name of the owner of the limits set.

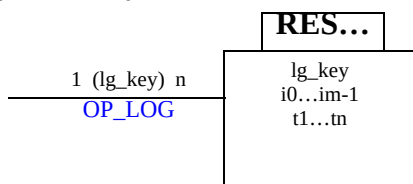
4.21 RES...

Abstract Contains **raw data** (results) as collected by a given **program**. The table contains one record per set of key indexes defined by the data **reader**. The “...” in the table name is replaced by the pg_key value from **PROGRAM**.

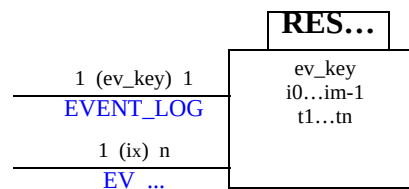
This table is used by the following program classes:

- Wafer_Sort(1)
- Final_Test(2)
- Summary(3)
- Bin_Map(4)
- PCM(5)
- Wafer_Sort_No_Bin(6)
- MultiBin(7)
- Metrology(10)
- DefectSum(11)
- FinalSummary(12)
- LEH(13)
- LotEvent(15)
- EquipEvent(16)
- bit(17)
- WEH(19)
- FabSite(23)
- FabWaf(24)
- FabLot(25)

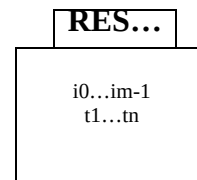
Table This table is related to a specific entry in **PROGRAM** because its name is postfixed by **pg_key**.



RES... table related to the above listed program classes, except LEH program class (13), LotEvent(15) and EquipEvent (16)



RES... table related to LotEvent(15) and EquipEvent (16)



RES... table related to LEH program class (13)

Serial Key None.

Foreign Keys / Link
Description

lg_key Foreign key into OP_LOG. **lg_key** is a predefined compulsory key index.

ev_key Foreign key into EVENT_LOG. **ev_key** is a predefined compulsory key index.

i0 ... im-1 Indexes into the table as defined by the (configuration of the) data reader. Typically, m is a small number and **i0 ... im-1** include device number, wafer number, etc.



For LotEvent program(15) and EquipEvent program (16), RES... can have one or more links to EV_... tables depending on the choice of the user. Whenever the user normalizes an index (i's) in RES..., an EV_... table is created for that index and a link to RES... is established. The 'x' in (ix) at the link represents the number of the index at RES...

Other Fields in
RES...

t1 ... tn Parametric data related to the test described by test indexes 1 to n (as defined in **test_index** in DEF...).

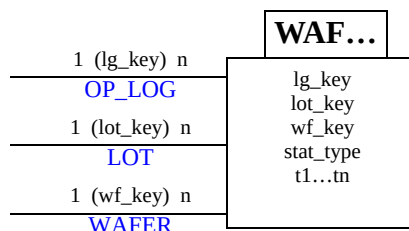
4.22 WAF...

Abstract Contains data summaries as calculated by the data **reader** or UpStat on an aggregate wafer level. The “...” in the table name is replaced by the pg_key value from **PROGRAM**.

WAF... table is used by the following program classes:

- Wafer_Sort(1)
- PCM(5)
- Wafer_Sort_No_Bin(6)
- WaferStats(8)
- Metrology(10)
- Defect(14)

Table This table is related to a specific entry in **PROGRAM** because its name is postfixed by **pg_key**.



Serial Key None.

Foreign Keys / Link Description

lg_key Foreign key into OP_LOG. **lg_key** is a predefined compulsory key index.

lot_key Foreign key into LOT.

wf_key Foreign key into WAFER.

Other Fields in WAF...

stat_type Type of statistics stored in current record. Legal values for **stat_type** are:

Value	Description
avg	Average
std	Standard deviation
cnt	Device count
min	minimum value
max	maximum value
q1	first quartile value
q2	second quartile value (median)
q3	third quartile value
sum	sum
ss	sum of squares

Value	Description
nlo	number of lower outliers
nuo	number of upper outliers
lol	points below lower outlier limit
uol	points above upper outlier limit
p1	value of 1%
p5	value of 5%
p10	value of 10%
p90	value of 90%
p95	value of 95%
p99	value of 99%

t1 ... tn Statistical data related to test described by test index 1 to n (as defined in **test_index** in DEF...).

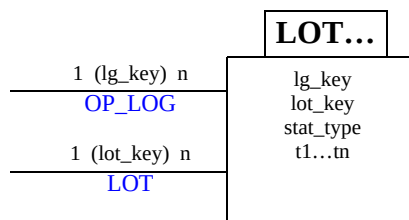
4.23 LOT...

Abstract Contains data summaries as calculated by the data reader or UpStat on an aggregate lot level. The “...” in the table name is replaced by the pg_key value from **PROGRAM**.

LOT... table is used by the following program classes:

- Wafer_Sort(1)
- Final_Tes(2)
- PCM(5)
- Wafer_Sort_No_Bin(6)
- LotStats(9)
- Metrology(10)
- Defect(14)

Table This table is related to a specific entry in **PROGRAM** because its name is postfixed by **pg_key**.



Serial Key None.

Foreign Keys / Link Description

lg_key Foreign key into OP_LOG. **lg_key** is a predefined compulsory key index.

lot_key Foreign key into LOT.

Other Fields in LOT...

stat_type Type of statistics stored in current record. Legal values for **stat_type** are:

Value	Description
avg	Average
std	Standard deviation
cnt	Device count
min	minimum value
max	maximum value
q1	first quartile value
q2	second quartile value (median)
q3	third quartile value
sum	sum

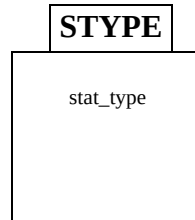
Value	Description
ss	sum of squares
nlo	number of lower outliers
nuo	number of upper outliers
lol	points below lower outlier limit
uol	points above upper outlier limit
p1	value of 1%
p5	value of 5%
p10	value of 10%
p90	value of 90%
p95	value of 95%
p99	value of 99%

t1 ... tn Statistical data related to test described by test index 1 to n (as defined in **test_index** in DEF...).

4.24 STYPE

Abstract This table is only used by the FAB reader. The WAF... and LOT... views are created by joining the underlying tables WWAF... and LLOT... with STYPE.

Table



Serial Key None.

Foreign Keys / Link Description None.

Other Fields in STYPE

stat_type The hard-coded types of statistics. Legal values for **stat_type** are:

Value	Description
avg	Average
std	Standard deviation
cnt	Device count
min	minimum value
max	maximum value
q1	first quartile value
q2	second quartile value (median)
q3	third quartile value
sum	sum
ss	sum of squares
nlo	number of lower outliers
nuo	number of upper outliers
lol	points below lower outlier limit
uol	points above upper outlier limit
p1	value of 1%
p5	value of 5%
p10	value of 10%
p90	value of 90%
p95	value of 95%
p99	value of 99%

4.25 BIN_LOG

Abstract The tester **bin** summary (consisting of not less than 32 and not more than 950 bins) is stored in **BIN_LOG** for quick reference.

This functionality is only available for programs which generate entries in **OP_LOG**.

Table

BIN_LOG	
1 (pg_key) n PROGRAM	pg_key log_level lot_key wf_key lg_key
1 (lot_key) n LOT	y1 y2 y3 y4 y5 y6
1 (wf_key) n WAFER	y7 y8 y9 y10
1 (lg_key) n OP_LOG	m d0 yield yr yield_pgdw yr_pgdw m_pgdw d0_pgdw auf total_tested total_failed b1...b950 b998 b999

Serial Key None.

Foreign Keys / Link Description

- pg_key** Relates a tester summary to a PROGRAM. Similarly, HIST_BIN is linked to PROGRAM. This way, we can maintain **bin** descriptions in **bin_name** of HIST_BIN.
- lot_key** Relates a tester summary to a LOT.
- wf_key** Relates a tester summary to a WAFER.
- lg_key** Relates a tester summary to a logging entry in OP_LOG.

Other Fields in BIN_LOG

- log_level** Identifies level at which summary was obtained with values such as **lot** and **wafer**. Depending on its value, **wf_key** and **lot_key** may or may not be used.

y1...y10* y1 - one die
 y2 - 1x2 matrix
 y3 - 2x1 matrix
 y4 - 2x2 matrix
 y5 - 4x4 matrix
 y6 - y10 - for future use
 Used to calculate **m** and **d0**.*

m Cluster factor — multiplicative factor used to scale yield to the exponential curve fit.*

d0 Defect density.*

*For a definition of m and d0, refer to “*Specification for Calculating Cluster Factor and Defect Density (m and d0)*” in the dataPOWER Data Readers manual.

yield Yield based on total tested.

yr Random Yield based on total tested.

yield_pgdw Yield based on PGDW (theoretical number of dice that fit on the wafer).

yr_pgdw Random Yield based on PGDW.

m_pgdw Cluster factor based on PGDW.

d0_pgdw Defect density based on PGDW.

auf number tested divided by PGDW (For wafer stats, this is based on exact number tested; for lot stats it is based on average number tested divided by the number of wafers in the lot.).

$$\text{auf} = \frac{\# \text{ tested}}{\text{PGDW}}$$

total_tested Total number of tested devices.

total_failed Total number of failed devices.



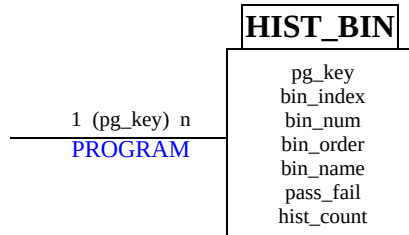
*Note that we do not distinguish at this level between multiple pass bins. However, their individual contents are stored in **b1** ... **b100**.*

- b1 ...bmax*** bin counters 1 ... 950*. Whether a bin is a pass or fail bin can be obtained by interpreting **pass_fail** in HIST_BIN.
- * The maximum number of bins value (not to exceed 950) is specified by the administrator when creating the database, and is stored as the **max_bins** value in the DBINFO table ([“DBINFO,”](#) page 196).
- b998, b999** Always added to BIN_LOG as the last columns.
b998 used for left-over (>*max_bins**) fail bins;
b999 used for left-over (>*max_bins**) pass bins.
- * The maximum number of bins value (not to exceed 950) specified by the administrator when creating the database, and stored as the **max_bins** value in the DBINFO table ([“DBINFO,”](#) page 196).

4.26 HIST_BIN

Abstract This table saves a historical **tester summary bin** count per **PROGRAM**.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key Relates historical tester summary to a PROGRAM.

Other Fields in HIST_BIN

bin_index Ties each record to bins defined in BIN_LOG, plus the left-over bins (998, 999) defined in BIN_LOG, if found.

bin_num Bin number (in tester summary).

bin_order The order in which bins were assigned to the die.

bin_name Descriptive text on bin.

pass_fail Flag indicating if bin is a pass or fail bin.

Allowed values:

Value	Description
P	pass bin
F	fail bin

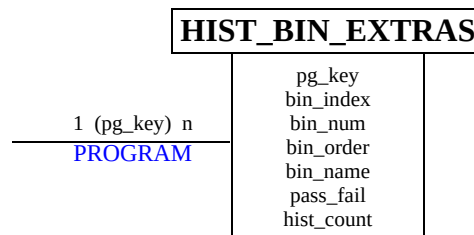
hist_count Historical count of occurrences of **bin_num**.

4.27 HIST_BIN_EXTRAS

Abstract This table saves a historical **tester summary bin** count per **PROGRAM**, for programs where there are more than the user-defined maximum number of bins* in the data file.

* The maximum number of bins value (not to exceed 950) is specified by the administrator when creating the database, and is stored as the **max_bins** value in the DBINFO table (“**DBINFO**,” page 196).

Table



Serial Key None.

Foreign Keys / Link Description

pg_key Relates historical tester summary to a PROGRAM.

Other Fields in HIST_BIN_EXTRAS

bin_index A continuation of the number of bin_index in HIST_BIN (**max_bins** +1, **max_bins** +2, ..., etc.).

bin_num Bin number (in tester summary).

bin_order The order in which bins were assigned to the die.

bin_name Descriptive text on bin.

pass_fail Flag indicating if bin is a pass or fail bin.

Allowed values:

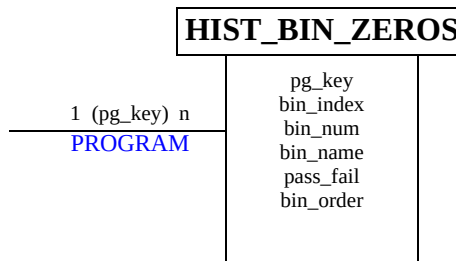
Value	Description
P	pass bin
F	fail bin

hist_count Historical count of occurrences of **bin_num**.

4.28 HIST_BIN_ZEROS

Abstract This table saves historical tester summary bin information, per program, for programs where there are no bin failures in the database.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key Relates historical tester summary to a PROGRAM.

Other Fields in HIST_BIN_ZEROS

bin_index Not used; set to -1.

bin_num Bin number (in tester summary).

bin_name Descriptive text on bin.

pass_fail Flag indicating if bin is a pass or fail bin.

Allowed values:

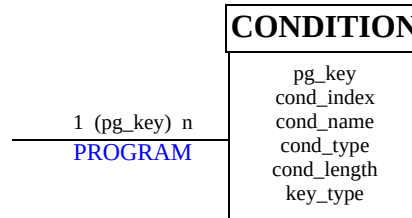
Value	Description
P	pass bin
F	fail bin

bin_order The order in which bins were assigned to the die.

4.29 CONDITION

Abstract Describes conditions on a per program basis.

Table



Serial Key None.

Foreign Keys / Link Descriptions

pg_key Relates record in CONDITION to a PROGRAM.

Other Fields in CONDITION

cond_index Condition number as referenced in [DEF...](#) in the fields **cond0 ... condn**. Legal values for **cond_index** are between 0 and *n*.

cond_name Name of condition as defined by the data reader, such as 'Test Name', 'Pin', 'Unit', etc.

cond_type Condition data type, allowed values.

Code	Type
50	Character
51	unsigned character
52	unsigned short integer
53	unsigned integer
54	unsigned long
55	one-byte integer
56	Short Integer
57	Integer
58	Long Integer
59	Float
60	Double
61	String

cond_length Condition value length in bytes.

key_type The class of condition as specified in the reader's format file.

Value	Description
0	key condition (KeyCond)
1	non-key condition (Cond)
2	limit condition (LimCond)

4.30 SUM_COND

Abstract Stores the value of the arguments passed to the dbascii reader's function, **DbBinSum()**. There will be one entry per program.

Table

SUM_COND	
1 (pg_key) 1	pg_key
PROGRAM	arg1
	arg2
	arg3
	arg4
	arg5

Serial Key None.

Foreign Keys / Link Descriptions

pg_key Relates the SUM_COND to the PROGRAM table.

Other Fields in CONDITION

- arg1** Argument 1 — condition name that has the bin numbers.
- arg2** Argument 2 — condition name that is used to identify parameters that have binning information.
- arg3** Argument 3 — the value of the condition specified in argument 2 that identifies a column with binning information.
- arg4** Argument 4 — The 4th argument has only two valid values, 'C' and 'P'. 'C' is used to indicate that the bin summaries are counts; 'P' indicates that the bin summaries are in percentages.
- arg5** Argument 5 — this argument is only used when the 4th argument has value 'P' to indicate the index name that has the total counts.

5.0 Program Class and Configurable Statistics

5.1 PROG_CLASS

Abstract Defines a specific grouping of programs. Can be set as a command line argument to the **reader** when a **program** is new. The first 30 program classes are reserved for the use of PDF Solutions only. (See **pgc_name**, below.)

Table

PROG_CLASS	
pgc_key	
pgc_name	1 (pgc_key) n
res_flag	PROGRAM
waf_flag	
lot_flag	
lim_flag	
bin_flag	1 (pgc_key) n
zstats_flag	PROGRAM_GROUP
cnt_flag	
nlo_flag	
nuo_flag	
min_flag	1 (pgc_key) n
max_flag	OP_LOG
q1_flag	
q2_flag	
q3_flag	
mean_flag	MEM_LOG
stdev_flag	
sum_flag	
sums_flag	
p1_flag	1 (pgc_key) n
p5_flag	EVENT_LOG
p10_flag	
p90_flag	
p95_flag	
p99_flag	1 (pgc_key) n
lol_flag	DIR_PATH_DEF
uol_flag	
hide_flag	
zcnt_flag	
fcf_flag	1 (pgc_key) n
fcs_flag	SSA_RUN
fco_flag	
fcw_flag	
yldp_flag	
ylds_flag	1 (pgc_key) n
yldo_flag	SSA_DEF
yldw_flag	
	1 (pgc_key) n
	PROD2WMAP

Serial Key **pgc_key** Uniquely identifies an entry in **PROG_CLASS**. Automatically generated out of the program class name, **pgc_name**.

Other Fields in PROG_CLASS

pgc_name

Program class name. You may add more custom classes starting from pgc_key=31. The following is a list of the reserved classes:

pgc_key	pgc_name
1	Wafer_Sort
2	Final_Test
3	Summary
4	Bin_Map
5	PCM
6	Wafer_Sort_No_Bin
7	MultiBin
8	WaferStats
9	LotStats
10	Metrology
11	DefectSum
12	FinalSummary
13	LEH
14	Defect
15	LotEvent
16	EquipEvent
17	Bit
18	BitFinalTest
19	Reserved_19
20	Reserved_20
21	shmoo data-log
22	robust-ness data-log
23	Reserved_23
24	Reserved_24
25	Reserved_25
26	CV_DATA
27	SCV_PARAM
28	SCV_YLD_MOD
29	SCV_WF_FR_MOD
30	SCV_LOT_FR_MOD

res_flag RES... table flag.

lot_flag LOT... table flag.

waf_flag WAF... table flag.

lim_flag LIM... table flag.

bin_flag BIN_LOG and HIST_BIN table flags.

zstats_flag	Enables the calculation of additional zonal statistics.
cnt_flag	Number of observations flag.
nlo_flag	Number of lower outliers flag.
nuo_flag	Number of upper outliers flag.
min_flag	Minimum value of retrieved data flag.
max_flag	Maximum value of retrieved data flag.
q1_flag	1st quartile point flag. For Defect program class (14), if set to zero, no size bin summaries are computed by UpStat.
q2_flag	Median flag. For Defect program class (14), if set to zero, no class related summaries are computed by UpStat.
q3_flag	3rd quartile flag. For Defect program class (14), if set to zero, no bin related summaries are computed by UpStat.
mean_flag	Mean flag.
stdev_flag	Standard deviation flag.
sum_flag	Sum flag.
sums_flag	Sum of squares flag.
p1_flag	1st percentile flag.
p5_flag	5% percentile flag.

For the defect program class (pgc_key = 14), the following settings apply:

p5_flag	meaning
-1	"Adder defect" gallery image in database
-2	"All defect" gallery image in database
-3	Both defect gallery images in database

p10_flag	10% percentile flag.
p90_flag	90% percentile flag.
p95_flag	95% percentile flag.

p99_flag	99% percentile flag.						
lol_flag	Lower outlier limits flag.						
uol_flag	Upper outlier limits flag.						
hide_flag	Hide/show the program class name in the retrieval window. Default is 0 . Legal values:						
<table> <tr> <th>Value</th><th>Description</th></tr> <tr> <td>0</td><td>show</td></tr> <tr> <td>1</td><td>hide</td></tr> </table>		Value	Description	0	show	1	hide
Value	Description						
0	show						
1	hide						
zcnt_flag	Enables the calculation of zonal yield/count.						
fcp_flag	Enables the calculation of fail count to production limits production flag.						
fcs_flag	Enables the calculation of fail count to specification limits.						
fco_flag	Enables the calculation of fail count to outlier limits.						
fcw_flag	Enables the calculation of fail count to what-if limits.						
yldp_flag	Enables the calculation of yield to production limits.						
ylds_flag	Enables the calculation of yield to specification limits.						
yldo_flag	Enables the calculation of yield to outlier limits.						
yldw_flag	Enables the calculation of yield to what-if flag limits.						

6.0 Product, Lot, Wafer and Related Tables

Abstract These tables describe a product, which, in the context of *dataPOWER* is defined as a unique die type.

6.1 PRODUCT

Abstract This table describes a **product**, which is basically a die type plus links to various package variants. A **n** → **1** link into RETICLE_SET also supports different marketing variants of the same die type if so desired.

Table

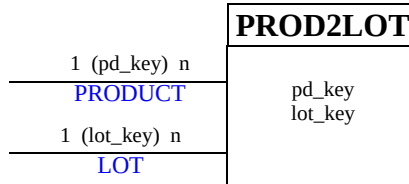
PRODUCT	
pd_key	
1 (rs_key) n RETICLE_SET	pd_name pd_desc code release rs_key die_count test_engr prod_engr apps_engr designer user1 user2 wmap_key
1 (wmap_key) n WMAP_CONFIG	1 (pd_key) n OP_LOG
1 (test_engr) n PEOPLE	1 (pd_key) n DF_EXCLUDE
1 (prod_engr) n PEOPLE	1 (pd_key) n DF_CONFIG
1 (apps_engr) n PEOPLE	1 (pd_key) n EVENT_LOG
1 (designer) n PEOPLE	1 (pd_key) n DRCP2PDLR
1 (user1) n PEOPLE	1 (pd_key) n LOT
1 (user2) n PEOPLE	m (pd_key) n FAMILY (using PROD2FAM)
m (pd_key) n LOT (using PROD2LOT)	1 (pd_key) n CHAR_LOG
m (pd_key) n LOT (using PROD2SRCLLOT)	m (pd_key) n PKG (using PKG2PROD)
	m (pd_key) n PROGRAM (using PROG2PROD)
	1 (pd_key) n LEH_LOG
	1 (pd_key) n WEH_LOG
	1 (src_prod) n DBTOOLS_MERGE_PROD
	1 (pd_key) n MEM_CONFIG
	1 (dest_prod) n DBTOOLS_MERGE_PROD
	1 (pd_key) n MEM_FILE_CONFIG
	1 (pd_key) n MEM_LIB2PROD
	1 (pd_key) n MEM_UPSTAT
	1 (pd_key) n MEM_BOP_UPSTAT
	1 (pd_key) n MEM_LAYER2PAT
	1 (pd_key) n CV_UPSTAT
	1 (pd_key) n LOT_GEN
	1 (pd_key) n FAB2PROD
	1 (pd_key) n MEM_LOG
	1 (pd_key) n CV_SO_SET
	1 (pd_key) n MEM_DEFAULT_TEST
	1 (pd_key) n MEM_BOP_CONFIG
	1 (pd_key) n PROG2PROD

Serial Key	pd_key	Automatically generated, unique key for each product, based on pd_name .
	rs_key	Each product is related to a unique RETICLE_SET . The fact that this link is n → 1 allows maintenance of multiple products per reticle set - such as marketing variants.
	wmap_key	Link into wafer layout definition tables WMAP_CONFIG (and indirectly WMAPI).
	test_engr	Links into PEOPLE to identify responsible test engineer per product.
	prod_engr	Links into PEOPLE to identify responsible product engineer per product.
	apps_engr	Links into PEOPLE to identify responsible applications engineer per product.
	designer	Links into PEOPLE to identify responsible designer per product.
	user1	Links into PEOPLE to identify one user of the product.
	user2	Links into PEOPLE to identify a second user of the per product.
	 Note that a product does not directly relate to a DEPARTMENT or FAB other than through PEOPLE. (As an example, user2 can be used as a mandatory link while the other fields are optional).	
Foreign Keys / Link Description	(PKG)	This link supports the fact that each product (die type) may be packaged in different packages.
	(FAMILY)	This link groups products into families. The fact that there is an n-> m link allows flexible grouping of products into multiple families for different selection requirements.
	pd_name	Product name as used in selection windows.
	pd_desc	Descriptive text per product, up to 128 characters.
	code	Product code.
	release	Release date of a the product.
Other Fields in PRODUCT	die_count	Number of die on wafer. This field is only valid until Esort .

6.2 PROD2LOT

Abstract An m2n table holding all the available combinations of lot_keys and pd_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pd_key A foreign key from the PRODUCT table.

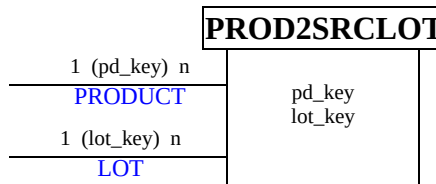
lot_key A foreign key from the LOT table.

Other Fields in PROD2LOT None.

6.3 PROD2SRCLLOT

Abstract An m2n table holding all the available combinations of lot_keys and pd_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pd_key A foreign key from the PRODUCT table.

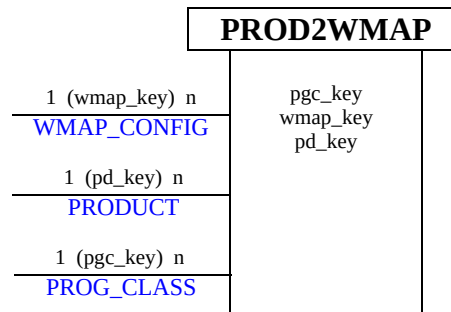
lot_key A foreign key from the LOT table.

Other Fields in PROD2SRCLLOT None.

6.4 PROD2WMAP

Abstract This table contains information about the relationship between products and wafermaps.

Table



Serial Key None.

Foreign Keys / Link Description

wmap_key Foreign key from the WMAP_CONFIG table.

pd_key Foreign key from the PRODUCT table.

pgc_key Foreign key from the PROG_CLASS table.

Other Fields in PROD2WMAP

None.

6.5 WMAP_CONFIG

Abstract This table describes the overall wafer configuration. Each die is further described with its exact location in WMAPS.

Table

WMAP_CONFIG	
wmap_key	
wmap_name	1 (wmap_key) n
wf_size	PRODUCT
wf_units	1 (wmap_key) n
flat	WMAPS
flat_type	
die_wd	1 (wmap_key) n
die_ht	RET2DIE
center_x	
center_y	1 (wmap_key) n
pos_x	PROGRAM
pos_y	
fld_rows	1 (wmap_key) n
fld_cols	Z_WMAP_CONFIG
row_offset	
col_offset	1 (wmap_key) 1
std_cfg	ORG_WMAP
die_x_offset	
die_y_offset	1 (wmap_key) 1
	ORG_WMAP
	1 (wmap_key) 1
	SITE2DIE
	1 (wmap_key) 1
	MEM_CONFIG
	1 (wmap_key) n
	CV_CONFIG
	1 (wmap_key) n
	CV_ADL
	1 (wmap_key) n
	DF_EXPORT
	1 (wmap_key) n
	WMAP2DIE
	1 (wmap_key) n
	RET2DIE

Serial Key **wmap_key** Automatically generated, unique key per wafer map configuration; based on **wmap_name**.

Foreign Keys / Link Description None.

Other Fields in WMAP_CONFIG **wmap_name** Name of wafer map configuration, typically program name, when generated by readers.

wf_size Wafer diameter.

wf_units Units used for **wf_size**, **die_wd**, **die_ht** **center_x** and **center_y**. No practical use is made of the field **wf_units**. However, only one unit is allowed per wafer type.

flat Can be one of the following to indicate location of flat (or notch) side of wafer.

Value	Description
R	right
L	left
T	top
B	bottom

Defaults to **R**.

flat_type Set to one of the following to indicate the type of wafer positioning feature — flat or notch.

Value	Description
F	flat
N	notch

Defaults to **F**.

die_wd, die_ht die size on wafer in units described in **wf_units**.

center_x, center_y Offset of die 0 from center of wafer.

pos_x, pos_y Direction of positive X and positive Y.

pos_x

Value	Description
L	left
R	right

Default is **R**.

pos_y

Value	Description
U	up
D	down

Default is **U**.

fld_rows Number of rows in the reticle set (mask).

fld_cols Number of columns in the reticle set (mask).

row_offset	Number of rows offset from the first mask on the wafer.
col_offset	Number of columns offset from the first mask on the wafer.
std_cfg	A value other than N indicates that alignment between binmap and defect data has not been performed. Default is N . Defect reader will fill with D , binmap reader will fill with B , DbTools will fill with U .
die_x_offset	Let DieOrigin be the X coordinate of the lower left corner of die (0,0) in sample coordinate system, and let SampleCenterLocationX be the X center of the sample (wafer) in the sample coordinate system, then center_x = DieOrigin - SampleCenterLocationX .
die_y_offset	Let DieOrigin be the Y coordinate of the lower left corner of die (0,0) in sample coordinate system, and let SampleCenterLocationY be the Y center of the sample (wafer) in the sample coordinate system, then center_y = DieOrigin - SampleCenterLocationY .

6.6 ORG_WMAP

Abstract Holds information pertaining to the original wafer map configuration before it was altered to provide global alignment.

Table

ORG_WMAP	
wmap_key	1 (wmap_key) 1 WMAP_CONFIG
flat	
die_wd	
die_ht	
center_x	
center_y	
pos_x	
pos_y	
center_x_estim	
center_y_estim	

Serial Key None.

Foreign Keys / Link Description **wmap_key** From WMAP_CONFIG table. Identifies wafer map configuration.

**Other Fields in
ORG_WMAP**

flat Can be one of the following to indicate location of flat (or notch) side of wafer.

Value	Description
R	right
L	left
T	top
B	bottom

Defaults to **R**.

die_wd, die_ht die size on wafer in units described in **wf_units**.

center_x, center_y Offset of die 0 from center of wafer.

pos_x, pos_y Direction of positive X and positive Y.

pos_x

Value	Description
L	left
R	right

Default is **R**.

pos_y

Value	Description
U	up
D	down

Default is **U**.

center_x_estim The estimated value of the wafer center in the X direction, before alignment was performed by DbTools.

center_y_estim The estimated value of the wafer center in the Y direction, before alignment was performed by DbTools.

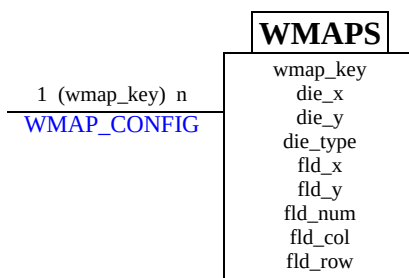
6.7 WMAPS

Abstract Describes each die on a wafer map configuration. Global parameters are stored in **WMAP_CONFIG**.

Each die has an x/y (column/row) location on the wafer, which is also part of a stepper field. Relative to this stepper field, a die also has a column/row location.

A stepper field, in turn, has an x/y (column/row) location on the wafer, which corresponds to a site address.

Table



Serial Key None.

Foreign Keys / Link Description **wmap_key** Index into WMAPS. This is the same as the index to WMAP_CONFIG. An **n** → **1** relation is formed by the fact that each record in WMAP_CONFIG has one record per die on wafer in WMAPS.

Other Fields in WMAPS

die_x, die_y Die location on the wafer.

die_type Die type is either good (normal device), bad (such as a device which overlaps with a **PCM** location) or ugly (such as a partial die at the wafer edge).

fld_x, fld_y Stepper field location on wafer.

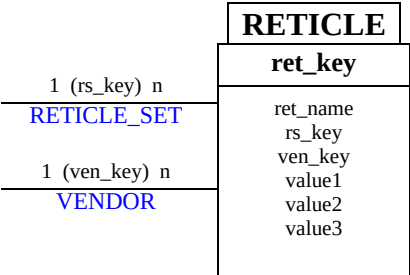
fld_num Stepper field number (site). **fld_num** is a function of **fld_x** and **fld_y**.

fld_col, fld_row Die location within the stepper field.

6.8 RETICLE

Abstract Describes a single reticle (or mask) which is part of a RETICLE_SET.

Table



Serial Key	ret_key	Automatically generated key, unique per reticle. Based on ret_name.
Foreign Keys / Link Description	rs_key	Relates a reticle to a full reticle set.
	ven_key	Link into VENDOR to support tracing of reticle vendors.
Other Fields in RETICLE	ret_name	Name of reticle.
	value1, 2, 3	Measurements stored per reticle. Example: Critical Dimensions.

6.9 RETICLE_SET

Abstract Describes a reticle (or mask) set as used to produce a die type or **PRODUCT**. Individual reticles are described in **RETICLE**.

Table

RETICLE_SET	
rs_key	1 (rs_key) n
rs_name	PRODUCT
rs_desc	1 (rs_key) n
shrink	LOT
fld_cols	1 (rs_key) n
fld_rows	RETICLE

Serial Key **rs_key** Automatically generated, unique key per reticle set. Based on **rs_name**.

Foreign Keys / Link Description None.

Other Fields in RETICLE_SET

rs_name Name of reticle set.

rs_desc Descriptive text per reticle set

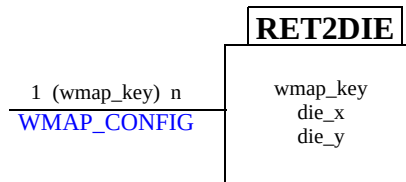
shrink Shrink scale from reticle to die.

fld_cols, fld_rows Reticle field size in number of die.

6.10 RET2DIE

Abstract This table is used by the *waferMAP* bricking function to identify the placement of the reticle field on the wafer map.

Table



Foreign Keys / Link Description

wmap_key Foreign key from WMAP_CONFIG.

Other Fields in RET2DIE

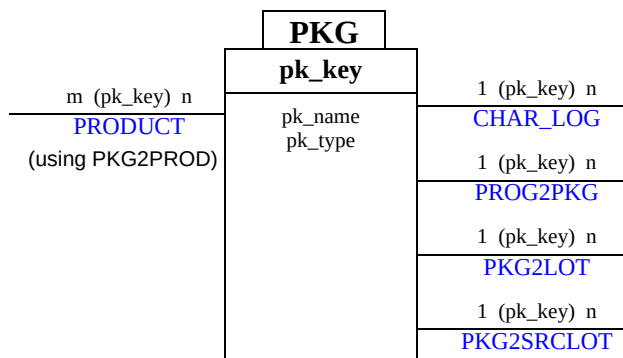
die_x The die_x location of the upper-left reticle field.

die_y The die_y location of the upper-left reticle field.

6.11 PKG

Abstract Describes a package.

Table



Serial Key

pk_key Automatically generated, unique package key. Based on **pk_name**.

Foreign Keys / Link Description

(PKG2PROD) Supports usage of a given package by multiple products.

Other Fields in PKG

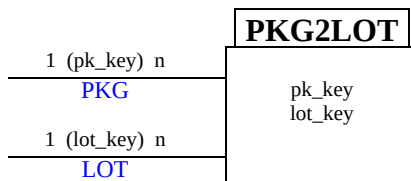
pk_name Name of package.

pk_type Package type.

6.12 PKG2LOT

Abstract An m2n table holding all the available combinations of pk_keys and lot_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pk_key The key to the related package info. A foreign key from the PKG table.

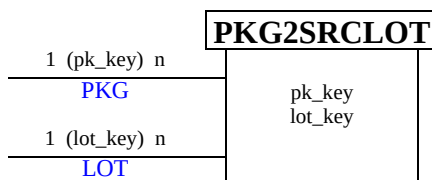
lot_key A foreign key from the LOT table. The key to the related lot information.

Other Fields in PKG2LOT None.

6.13 PKG2SRCLLOT

Abstract An m2n table holding all the available combinations of pk_keys and lot_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pk_key A foreign key from the PKG table.

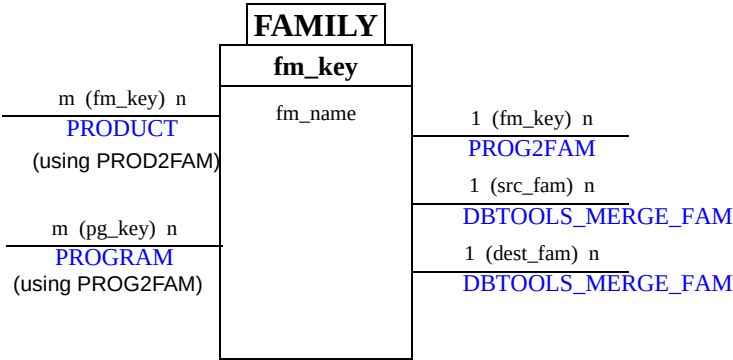
lot_key A foreign key from LOT. Identifies the source lot.

Other Fields in PKG2SRCLLOT None.

6.14 FAMILY

Abstract Describes a family of products.

Table



Serial Key fm_key Automatically generated, unique family key. Based on fm_name.

Foreign Keys / Link Description (PROD2FAM) Supports multiple products related to one family and multiple families related to one product.

Other Fields in FAMILY fm_name Family name.

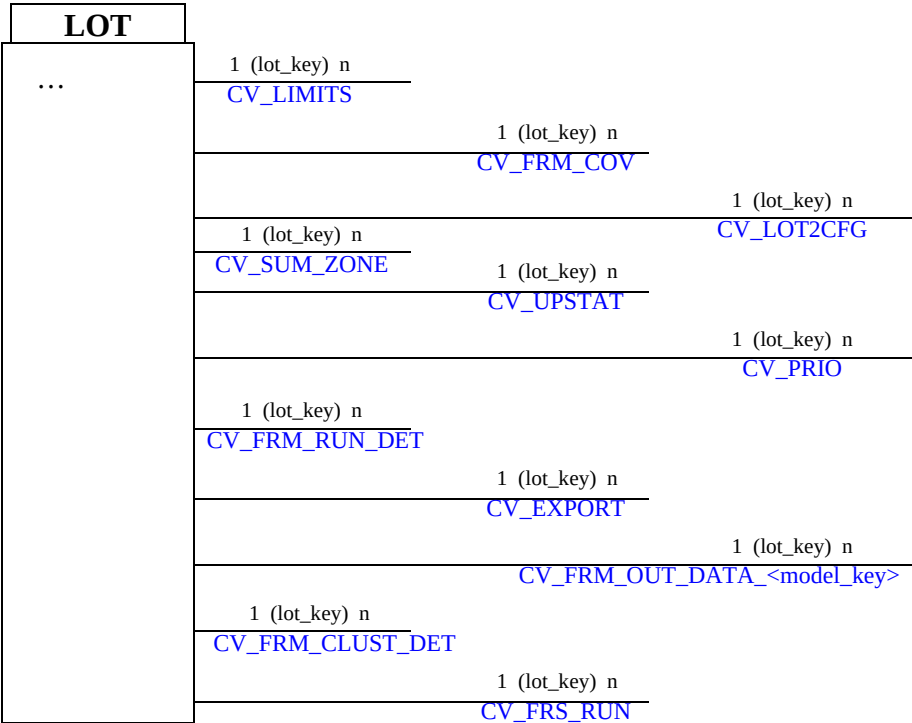
6.15 LOT

Abstract Describes a **lot**.

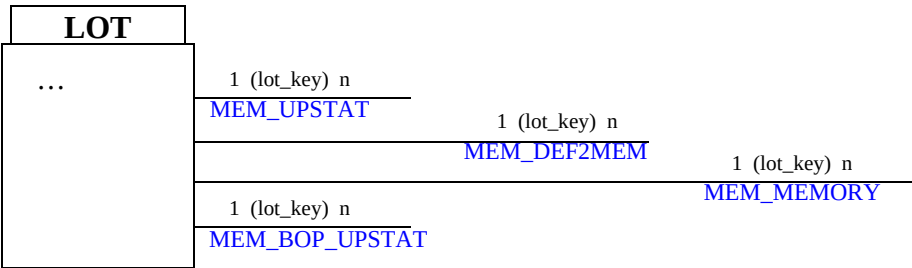
Table

LOT	
lot_key	
1 (fab_key) n FAB	1 (lot_key) n WAFER
1 (lc_key) n LOT_CLASS	1 (lot_key) n UPDATE_STATS
1 (rs_key) n RETICLE_SET	1 (lot_key) n WAF...
1 (from_lot) 1 LOT	1 (lot_key) n DF_UPSTAT
1 (to_lot) 1 LOT	1 (lot_key) n Z_UPDATE_LOT
1 (src_lot) 1 LOT	m (lot_key) n WAFER (using WAFER2LOT)
m (lot_key) n CUSTOMER (using LOT2CUST)	1 (lot_key) n DF_SUM
m (pg_key) n PROGRAM (using PROG2LOT)	1 (lot_key) n LOT_SPLIT
m (pg_key) n PROGRAM (using PROG2SRCLOT)	1 (lot_key) n PROG2SRCLOT
	1 (lot_key) n WAFER_SPLIT
	1 (lot_key) n PKG2LOT
	1 (lot_key) n LOT_GEN
	1 (lot_key) n PKG2SRCLOT
	1 (lot_key) n WAF2LOTGEN
	1 (lot_key) n MB_UPSTAT
	1 (lot_key) n LTCLS2LOT
	1 (lot_key) n SSA_RUN_DET
	1 (lot_key) n LOT2CUST
	Links to CV Tables ►
	Links to bitMAP Tables ►
	Links to ...LOG Tables ►

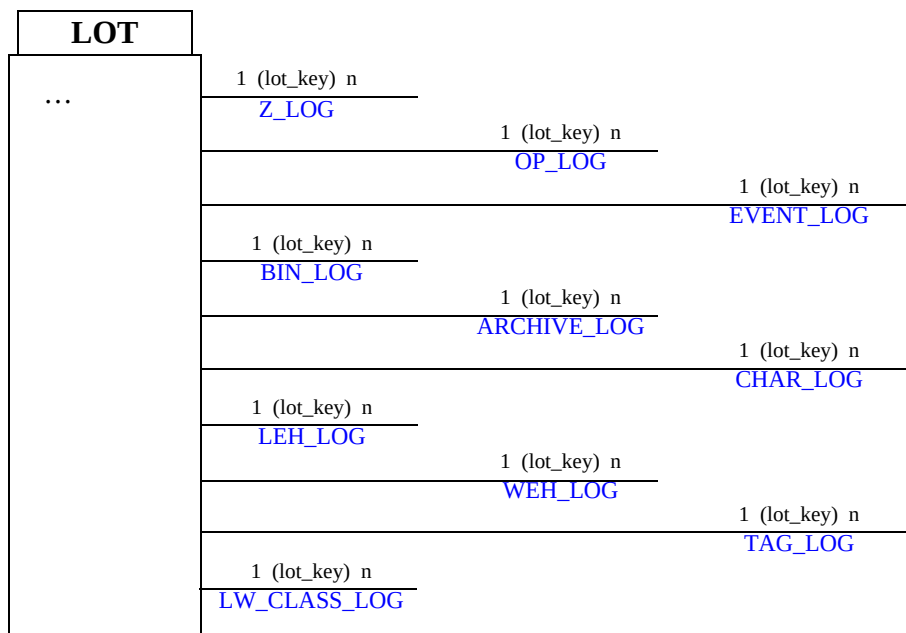
Links to CV Tables



Links to bitMAP Tables



Links to ...LOG Tables



Serial Key

lot_key Automatically generated, unique lot key. Based on **lot_id**.

Foreign Keys / Link Description

lc_key Foreign key from LOT_CLASS.

fab_key Foreign key from FAB.

rs_key Foreign key from RETICLE_SET.



*Note that this link is redundant as the reticle set can be retrieved by selecting the **PRODUCT** and then the respective **RETICLE_SET**. It is here for fast access only.*

to_lot, from_lot **lot splitting** and **lot merging** - as controlled in the Lot operations window.

src_lot The source (parent) lot, before splitting.

Other Fields in LOT

lot_id Lot ID, unique.

part_cnt Number of parts in LOT.

is_wafer Indicates whether a lot is referring to wafers or packaged products. Defaults to 'Y'.

start_time Lot start time.

merge_time Lot merging time.

tag_flag contains the tagging state of the lot. The legal values are:

Value	Description
0	normal
1	no action
2	bad
3	scrap
4	experimental

The default is **0**.

6.16 UPDATE_STATS

Abstract When a data **reader** (dbascii, stdf3, ...etc.) processes data for a lot which already exists in the database, it will not recalculate the lot statistics (LOT...). Instead, it inserts a row in UPDATE_STATS to indicate which lots require updating. The UpStat utility recalculates the lot statistics and updates the appropriate LOT... table.

Table

UPDATE_STATS	
1 (lot_key) n LOT	pg_key lot_key wf_key outliers error_code err_msg flag
1 (pg_key) n PROGRAM	
1 (wf_key) n WAFER	

Serial Key None.

Foreign Keys / Link Description

pg_key The key of the PROGRAM to which the lot belongs.

lot_key The key to the LOT that requires statistics recalculation.

wf_key The key to the WAFER that requires statistics recalculation. At lot level, it is set to **-1**.

Other Fields in UPDATE_STATS

outliers The outliers value or flag used in calculating the statistics. It could be one of the following:

Value	Description
0.0	do not filter outliers
-1.0	use outlier limits values stored in the database's LIM... table
>0.001	use this value in the calculation of the outliers limits

error_code The following is a list of possible values:

Value	Description
0	SUCCESS
1	WARNING
2	NO_MORE_DATA
3	ERROR
4	W_EMPTY_TABLE
5	MEM_ERROR
6	FETCH_ERROR
7	NULL_FIELD_ERR

Value	Description
8	PREP_ERROR
9	ALLOC_DESC_ERROR
10	DECLARE_CUR_ERROR
11	OPEN_CUR_ERROR
12	DESCRIBE_ERROR
13	GET_DESC_ERROR
14	EXEC_ERROR
15	COUNT_ERROR
16	INVALID_PARAM
17	SET_DESC_ERROR
18	DEL_ERROR
19	NO_ARGS
20	CONNECT_ERROR
21	INSERT_ERROR
22	SELECT_ERROR
23	NOT_FOUND
24	W_EMPTY_QUERY
25	UPDATE_ERROR
26	COUNT_0
27	STATS_ERROR

err_msg A descriptive text of the error message including Informix specific error code.

flag Used to describe the requested action by the UpStat utility. The legal values are:

Value	Description
1	update wafer-level statistics (future)
2	update lot-level statistics
3	update source-lot-level statistics
4	update wafer-level binning (BIN_LOG table)
5	update lot-level binning (BIN_LOG table)
6	update source-lot-level binning (BIN_LOG table)

6.17 DF_UPSTAT

Abstract When the Defect **reader** inserts raw defect data into the database, it does not compute wafer and lot level statistics. Instead it inserts a row in the DF_UPSTAT table. Furthermore, when the dbascii (ASCII) **reader** inserts binmap data for a bin program aligned with defect, a row for each wafer inserted into this table. The UpStat utility calculates the necessary statistics and inserts them in the DF_SUM, WAF... and LOT... tables.

Table

DF_UPSTAT	
1 (lot_key) n	pg_key
LOT	lot_key
1 (pg_key) n	wf_key
PROGRAM	lg_key
1 (wf_key) n	step_key
WAFER	error_code
1 (lg_key) n	err_msg
OP_LOG	flag
1 (step_key) n	
PROC_STEP	

Serial Key None.

Foreign Keys / Link Description

pg_key	The key of the PROGRAM to which the lot belongs.
lot_key	The key to the LOT that requires defect statistics calculation.
wf_key	The key to the WAFER that requires defect statistics calculation. At lot level, it is set to NULL .
lg_key	A foreign key to the OP_LOG table indicating the defect log entry that requires statistics calculation.
step_key	A foreign key to the PROC_STEP table indicating the key to the layer (step) that requires calculation. It is set to NULL , if flag = 8 .

Other Fields in UPDATE_STATS

error_code	An internal code specifying the calculation step that caused problems.
err_msg	A descriptive text of the error message including Informix specific error code.

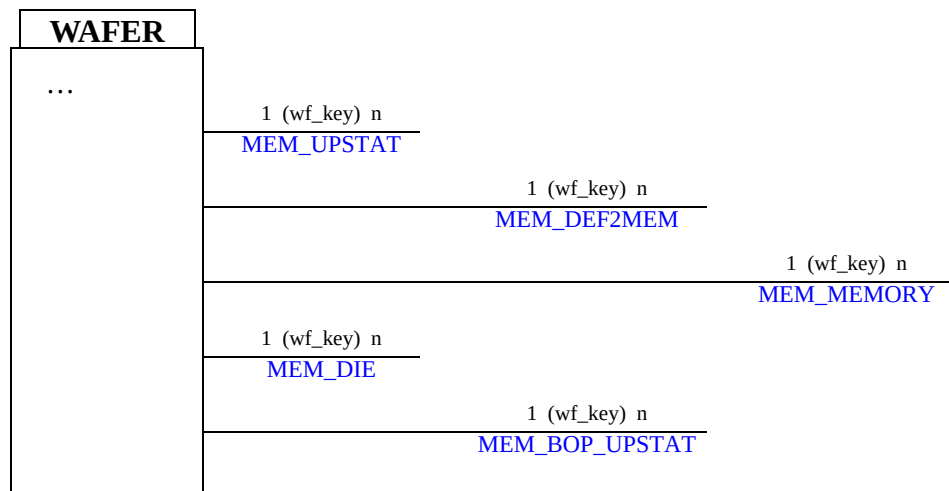
flag Used to describe the requested action by the UpStat utility. The legal values are:

Value	Inserted By	Description
7	Defect Reader	Calculate summaries for the given wafer-layer
8	Dbascii Reader	Calculate bin related summaries for the given wafer
9	Defect Reader	Unpatterned layer in DB. Do not calculate summaries until patterned layer for the same wafer is inserted into database
10	UpStat Utility	Calculate summaries for the given lot_layer
11	UpStat Utility	Calculate bin related summaries for the given lot_layer

Links to CV Tables

WAFER	
...	1 (wf_key) n SSA_OPER
	1 (wf_key) n SSA_ZONE_\$...
	1 (wf_key) n SSA_RUN_DET
	1 (wf_key) n SSA_WF2CLUSTER
	1 (wf_key) n SSA_WAFER_DISTANC
	1 (wf_key) n CV_EXPORT
	1 (wf_key) n CV_STEP_ORDER
	1 (wf_key) n CV_EDEF2IDEF
	1 (wf_key) n CV_EDEF2IDEF_ALL
	1 (wf_key) n CV_CELLI2IDEF
	1 (wf_key) n CV_SUM_ZONE
	1 (wf_key) n CV_FRN
	1 (wf_key) n CV_WF_CELLI
	1 (wf_key) n CV_ADL
	1 (wf_key) n CV_PIN_RESULTS
	1 (wf_key) n CV_DIE_RESULTS
	1 (wf_key) n CV_FRM_RUN_DET
	1 (wf_key) n CV_FRM_OUT_DATA_<model_key>
	1 (wf_key) n CV_UPSTAT
	1 (wf_key) n CV_E_IDEF2IDEF
	1 (wf_key) n CV_E2I_INHERIT
	1 (wf_key) n CV_FRM_CLUST_DET
	1 (wf_key) n CV_WF2MICFG
	1 (wf_key) n CV_PIN_SUM
	1 (wf_key) n CV_PADGRP_SUM
	1 (wf_key) n CV_FRS_RUN
	1 (wf_key) n CV_FRM_USC_<model_key>
	1 (wf_key) n CV_DEF2TYPE_<lot_key>

Links to *bitMAP* Tables



Serial Key

wf_key Automatically generated, unique wafer key. Based on **wf_id**.

Foreign Keys / Link Description

lot_key Foreign key from LOT.

wft_key Foreign key from WAFER_TYPE.

Other Fields in WAFER

wf_id Wafer ID. (The combination of wafer ID and lot key is unique.)

wf_num Wafer number.

tag_flag contains the tagging state of the wafer. The legal values are:

Value	Description
0	normal
1	no action
2	bad
3	scrap
4	experimental

The default is **0**.

wf_desc Description of the wafer.

6.19 WAFER_CLASS

Abstract Distinguishes different classes of wafers such as production vs. engineering.

Table

WAFER_CLASS	
wf_cls_key	1 (wf_cls_key) n
wf_cls_name	WFCLS2WF
wf_cls_desc	
cls_comment	1 (wf_cls_key) n
wf_cls_order	SSA_CLUSTER2WFCLS
wf_cls_image	
	1 (cls_key) n
	LW_CLASS_LOG
	1 (oper_hist_key) n
	SSA_OPER_HIST
	1 (wf_cls_key) n
	SSA_YLD_RULE
	1 (wf_cls_key) n
	LOT2CUST
	1 (cls_key) n
	LW_CLASS_LOG

Serial Key **wf_cls_key** Automatically generated key based on **wf_cls_name** and **method_key**.

Foreign Keys / Link Description None.

Other Fields in WAFER_CLASS

wf_cls_name Wafer class name.

wf_cls_desc Wafer class description.

cls_comment Wafer class comment.

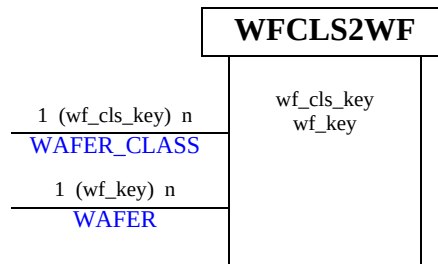
wf_cls_order Wafer class order.

wf_cls_image Wafer class image.

6.20 WFCLS2WF

Abstract An m to n table holding all the available combinations **wf_cls_key** and **wf_key**.

Table



Serial Key None.

Foreign Keys / Link Description **wf_cls_key** Foreign key from WAFER_CLASS.

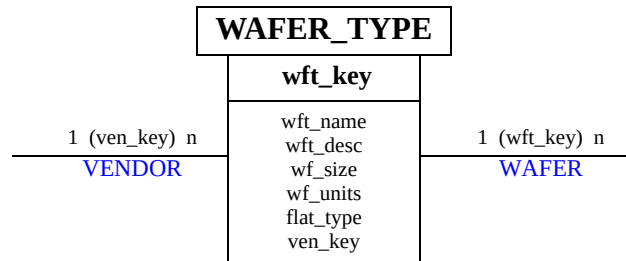
wf_key Foreign key from WAFER.

Other Fields in WFCLS2WF None.

6.21 WAFER_TYPE

Abstract Wafer types are vendor specific. Wafer types include epi wafers, etc. So, for example, the database can support lots with wafer 1 ...10 being of a different type than wafer 11 ...25.

Table



Serial Key	wft_key	Automatically generated key based on wft_name .
Foreign Keys / Link Description	ven_key	Foreign key from VENDOR for identification of the vendor of the wafer material.
Other Fields in WAFER_TYPE	wft_name	Wafer type name.
	wft_desc	Descriptive text of wafer type.
	wf_size	Wafer diameter.
	wf_units	Units used for wf_size . This is an attribute only and no practical use is made of the unit for calculation purposes.
	flat_type	is either flat or notch. This is redundant to flat_type in WMAP_CONFIG.

6.22 WAFER_DISTANCE

Abstract

Table

WAFER_DISTANCE	
1 (wf_key) n	wf_key lib_wf_key wf_cls_key lwc_lg_key distance
WAFER	
1 (lib_wf_key) n	
WAFER	
1 (wf_cls_key) n	
WAFER_CLASS	
1 (lwc_lg_key) n	
LW_CLASS_LOG	

Serial Key None.

Foreign Keys / Link Description

wf_key Foreign key to WAFER.

lib_wf_key Foreign key from WAFER. Defines which wafer is being classified.

wf_cls_key Foreign key to WAFER_CLASS.

lwc_lg_key Foreign key to LW_CLASS_LOG.

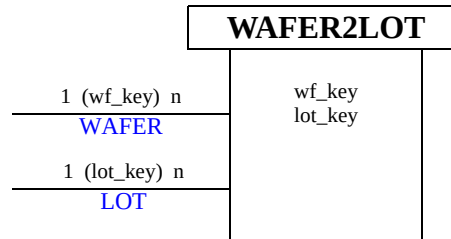
Other Fields in WAFER_DISTANCE

distance The distance between the class and the wafer.

6.23 WAFER2LOT

Abstract This table contains information about the relationship between wafers and lots.

Table



Serial Key None.

Foreign Keys / Link Description

wf_key Foreign key to WAFER.

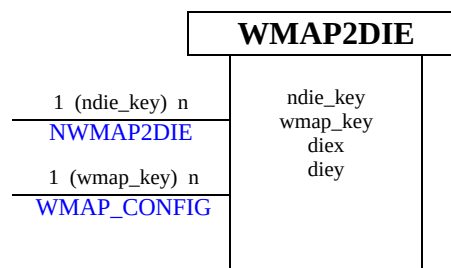
lot_key Foreign key to LOT.

Other Fields in WAFER2LOT None.

6.24 WMAP2DIE

Abstract This table contains information about the relationship between wafermaps and die.

Table



Serial Key None.

Foreign Keys / Link Description

ndie_key Foreign key from the NWMAP2DIE table.

wmap_key Foreign key from the WMAP_CONFIG table.

Other Fields in WMAP2DIE

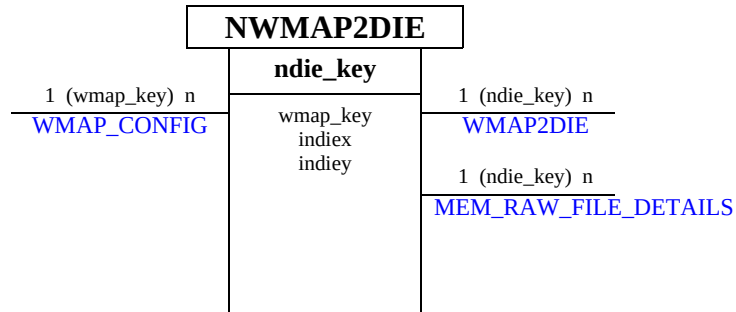
diex X index of die.

diey Y index of die.

6.25 NWMAP2DIE

Abstract This table contains die coordinates (**indiex** and **indiey**) of a die identified by NDIEID in the (normalized/native) wafermap referenced by WMAPID.

Table



Serial Key

ndie_key A unique key assigned to each entry in NWMAP2DIE. It is stored automatically.

Foreign Keys / Link Description

wmap_key Foreign key from the WMAP_CONFIG table.

Other Fields in NWMAP2DIE

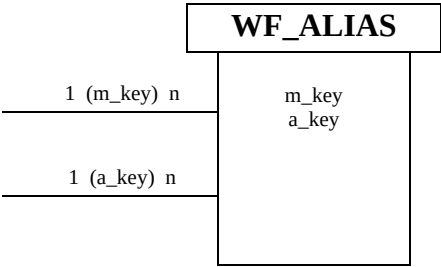
indiex Normalized die X coordinate.

indiey Normalized die Y coordinate.

6.26 WF_ALIAS

Abstract

Table



Serial Key	None.
Foreign Keys / Link Description	m_key . a_key .
Other Fields in WF_ALIAS	None.

6.27 VENDOR

Abstract Vendor related information.

Table

VENDOR	
ven_key	
ven_name	1 (ven_key) n
address1	WAFER_TYPE
address2	
address3	
postal_code	1 (ven_key) n
city	RETICLE
state	
country	
contact	
email	1 (ven_key) n
fax	EQUIP_TYPE
phone	

Serial Key **ven_key** Automatically generated key based on **ven_name**.

Foreign Keys / Link Description None.

Other Fields in VENDOR

ven_name Vendor name.

address1,2,3 Vendor addresses, up to three lines.

postal_code As relates to the address.

city As relates to the address.

state As relates to the address.

country As relates to the address.

contact Contact person at vendor.

e-mail e-mail address.

fax Fax number.

phone Telephone number.

6.28 CUSTOMER

Abstract The structure of CUSTOMER and VENDOR is identical although relations to other tables are different.

Table

CUSTOMER	
cust_key	
cust_name	m (cust_key) n
address1	PROGRAM
address2	(using PROG2CUST)
address3	
postal_code	m (cust_key) n
city	LOT
state	
country	(using LOT2CUST)
contact	
email	m (cust_key) n
fax	PROG2CUST
phone	
	1 (cust_key) n
	LOT2CUST

Serial Key

cust_key Automatically generated key based on **cust_name**

Foreign Keys / Link Description

None.

Other Fields in CUSTOMER

cust_name Customer name.

address1,2,3 Customer addresses, up to three lines.

postal_code As relates to the address.

city As relates to the address.

state As relates to the address.

country As relates to the address.

contact Contact person at customer.

e-mail e-mail address.

fax Fax number.

phone Telephone number.

6.29 LOT_CLASS

Abstract Distinguishes different classes of lots such as production vs. engineering.

Table

LOT_CLASS	
lot_cls_key	
lot_cls_name lot_cls_desc cls_comment lot_cls_order lot_cls_image	1 (lot_cls_key) n LTCLS2LOT

Serial Key **lot_cls_key** Automatically generated key based on **lc_name** and **method_key**.

Foreign Keys / Link Description None.

Other Fields in LOT_CLASS

lot_cls_name	Lot class name.
lot_cls_desc	Lot class description.
cls_comment	Lot class comment.
lot_cls_order	Lot class order.
lot_cls_image	Lot class image.

6.30 LTCLS2LOT

Abstract An m to n table holding all the available combinations **lot_cls_key** and **lot_key**.

Table

LTCLS2LOT	
1 (lot_cls_key) n LOT_CLASS	lot_cls_key lot_key
1 (lot_key) n LOT	

Serial Key None.

Foreign Keys / Link Description **lot_cls_key** Foreign key from LOT_CLS table.

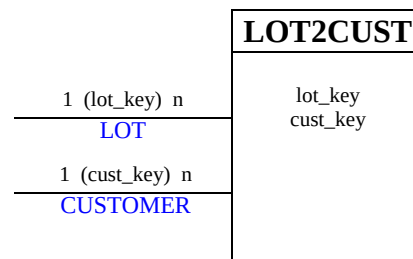
lot_key Foreign key from LOT table.

**Other Fields in
LTCLS2LOT** None.

6.31 LOT2CUST

Abstract This table contains information about the relationship between lots and customers.

Table



Serial Key None.

Foreign Keys / Link Description

lot_key Foreign key from LOT table.

cust_key Foreign key from the CUSTOMER table.

**Other Fields in
LOT2CUST** None.

6.32 LW_CLASS_LOG

Abstract This table is used to store meta-data associated with the spatial signature classification of wafers and lots.

Table

LW_CLASS_LOG		
1 (log_key) n (dynamic)	lwc_lg_key	1 (lwc_lg_key) n
1 (pg_key) n PROGRAM	log_key pg_key lot_key wf_key cls_key class_level insert_time owner class_comment	SSA_WAFER_DISTANCE
1 (lot_key) n LOT		1 (lwc_lg_key) n RULE_INFO
1 (wf_key) n WAFER		
1 (cls_key) n WAFER_CLASS or LOT_CLASS		

Serial Key

lwc_lg_key Serial key for LW_CLASS_LOG.

Foreign Keys / Link Description

log_key Dynamic foreign key that can link to any log table, primarily OP_LOG.

pg_key Foreign key from PROGRAM.

lot_key Foreign key from LOT.

wf_key Foreign key from WAFER.

cls_key Dynamic foreign key that inks to WAFER_CLASS when class_level = W, or Foreign key from LOT_CLASS when class level = L.

Other Fields in LW_CLASS_LOG

class_level Indicates class level (lot or wafer).

insert_time The date and time the classification was added to the database.

owner The name of the user who classified the lot or wafer.

class_comment The free-form comment field allowing the user to provide more details regarding the lot or wafer classification.

6.33 TAGS

Abstract Holds the description of the **tag_flag**s stored in the **LOT**, **WAFER**, and **TAG_LOG** tables.

Table

TAGS
tag_flag tag_name

Serial Key None.

Foreign Keys / Link Description None.

Other Fields in TAGS

tag_flag The tag flag value used in other database tables.

tag_name A description of the tag flag values. The legal values are:

Value	Description
0	normal
1	no action
2	bad
3	scrap
4	experimental
5	high defect density
6	high defect count
7	high defect density and high defect count
8-19	reserved for PDFS use

6.34 TAG_LOG

Abstract Holds information relevant to the latest tagging attempt of wafers, lots, and source lots.

Table

TAG_LOG	
1 (lg_key) n	lg_key
OP_LOG	pg_key
1 (pg_key) n	lot_key
PROGRAM	wf_key
1 (wf_key) n	tag_flag
WAFER	bad_yield
1 (lot_key) n	scrap_yield
LOT	tag_level
	comments

Serial Key None.

Foreign Keys / Link Description

lg_key Relates a tester summary to a logging entry in OP_LOG.

pg_key The key of the PROGRAM to which the lot belongs.

lot_key Foreign key from LOT.

wf_key The key to the WAFER that requires statistics recalculation. At lot level, it is set to **-1**.

Other Fields in LOT_CLASS

tag_flag contains the tagging state. The legal values are:

Value	Description
0	normal
1	no action
2	bad
3	scrap
4	experimental

bad_yield *User-defined value from 0 to 100. It is used to tag the wafers and lots as “bad” when the yield is less than or equal to this value. When the value is less than 0, it is ignored and considered undefined. The default is **-1**.

scrap_yield *User-defined value from 0 to 100. It is used to tag the wafers and lots as “scrap” when the yield is less than or equal to this value. When the value is less than 0, it is ignored and considered undefined. The default is **-1**.



When both **bad_yield and **scrap_yield** are defined, the value of **scrap_yield** must be less than the value of **bad_yield**.*

tag_level Tagging level. The legal values are:

Value	Description
'W'	wafer-level tagging
'L'	lot-level tagging
'S'	source-lot-level tagging

comment User-defined string.

6.35 RULE_INFO

Abstract This table is used to store the lot level rule classification results of a dpSSA Monitor Run. The rule configuration is specified in the dpSSA Monitor Control Sheet.

Table

RULE_INFO	
1 (lwc_lg_key) n LW_CLASS_LOG	lwc_lg_key rule_name rule_level total_wfrs fail_wfrs rule_type operator rule_value action result

Serial Key None.

Foreign Keys / Link Description

lwc_lg_key Foreign key from LW_CLASS_LOG. Log entry that classified the wafer

Other Fields in RULE_INFO

rule_name Name of the rule as it appears in the dpSSA Monitor Control Sheet.

rule_level Indicates the level of the rule (lot-level or wafer-level).

total_wfrs If rule_level = Lot (L), total_wfrs = the # of wafers in the lot that have been classified during dpSSA Monitor.

fail_wfrs The number of wafers failing the rule_name.

rule_type There are two rule_types available:

Value	Description
N	indicates rule is based on the absolute number of wafers in the lot
%	indicates rule is based on a fraction of wafers in the lot

operator The mathematical operator that was used in the evaluation of the rule.

rule_value The value (integer for rule_type = N, float for rule_type = %) that was used in the rule evaluation.

action The action to be taken based on the dpSSA Monitor result. The possible values are:

Value
Hold
Warn
Pass

result The result of the rule evaluation: either Pass or Fail.

7.0 Process Related Tables

Abstract These tables describe a **Fab** process.

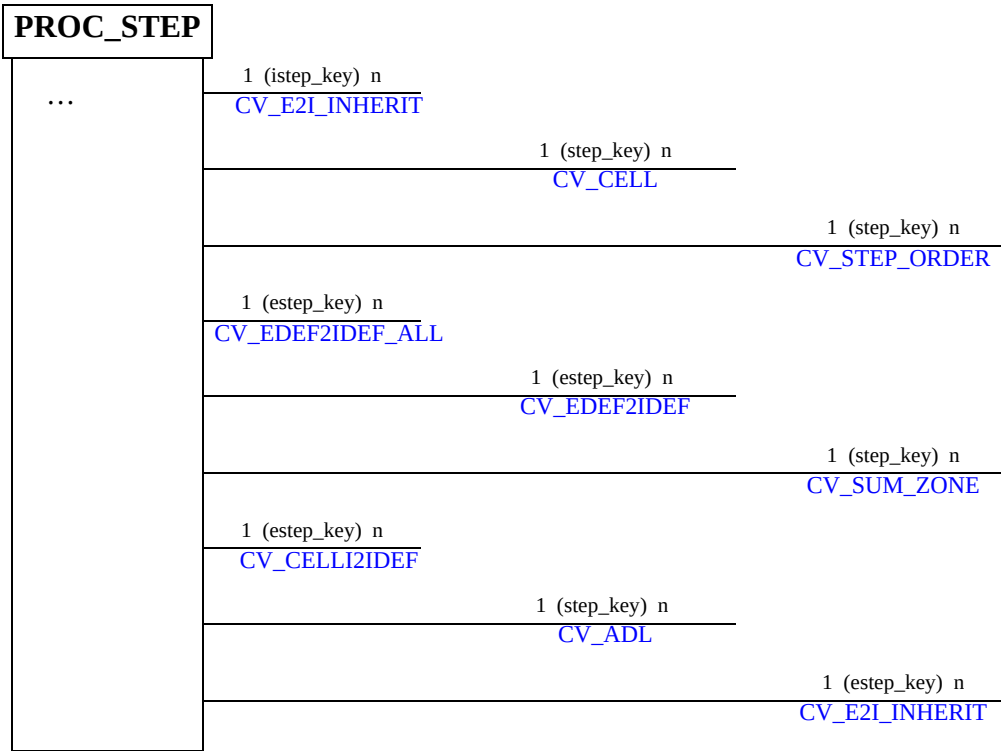
7.1 PROC_STEP

Abstract It may be useful to distinguish different data kinds produced in one step by an artificial process step: e.g., if at **Esort** testing, both a **datalog** and a **tester summary** sheet file are produced, which are handled as separate programs, then we may distinguish two steps: **esort_dlog** and **esort_summary**. In this way, we can support the data selection process because we are narrowing down to step related programs only. Also, distinguishing engineering vs. production data might be usefully done at a **PROC_STEP** level.

Table

PROC_STEP	
step_key	
step_name	1 (step_key) n
step_num	OP_LOG
step_desc	1 (step_key) n
eqt_key	EVENT_LOG
1 (eqt_key) n EQUIP_TYPE	1 (step_key) n
	LEH_LOG
m (step_key) n PROGRAM (using PROG2STEP)	1 (step_key) n
	WEH_LOG
	1 (step_key) n
	DF_SUM
m (step_key) n TECH_STAGE (using STAGE2STEP)	1 (step_key) n
	MEM_LOG
	1 (step_key) n
	LOT_SPLIT
m (step_key) n RECIPE (using RCP2STEP)	1 (step_key) n
	DF_CONTRIB
	1 (step_key) n
	DRCP2PDLYR
	1 (loo_step) n
	DF_CONTRIB
	1 (step_key) n
	PROG2STEP
	1 (step_key) n
	DF_EXCLUDE
	1 (step_key) n
	MEM_DEF2MEM
	1 (step_key) n
	MEM_UPSTAT
	1 (step_key) n
	MEM_LAYER2PAT
	1 (step_key) n
	DF_UPSTAT
	1 (step_key) n
	WAF2LOTGEN
	1 (step_key) n
	DF_EXPORT
	1 (step_key) n
	STEP_ORDER_DETAILS
	1 (mstep_key) n
	MSTEP2PSTEP
	1 (pstep_key) n
	MSTEP2PSTEP
	1 (step_key) n
	STAGE2STEP
Links to CV Related Tables	

Links to CV Related Tables



Serial Key

step_key

A process may consist of multiple steps which are described in PROC_STEP. It is automatically generated and it is based on **step_name**.

Foreign Keys / Link Description

eqt_key

Link to EQUIP_TYPE.

Other Fields in PROC_STEP

step_num

Process step number.

step_name

Process step name.

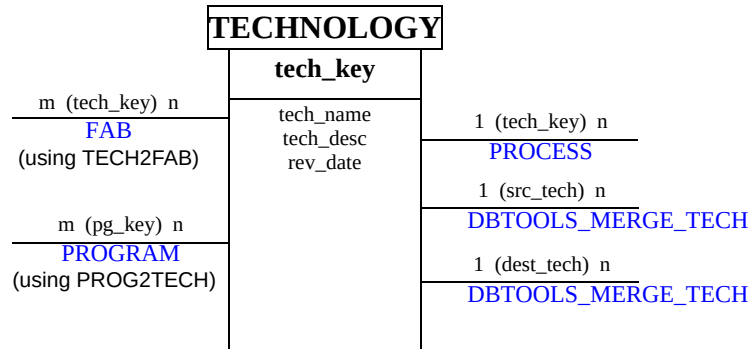
step_desc

Process step description.

7.2 TECHNOLOGY

Abstract Technology related information.

Table



Serial Key **tech_key** Automatically generated key based on **tech_name**.

Foreign Keys / Link Description None.

Other Fields in TECHNOLOGY

tech_name Technology name.

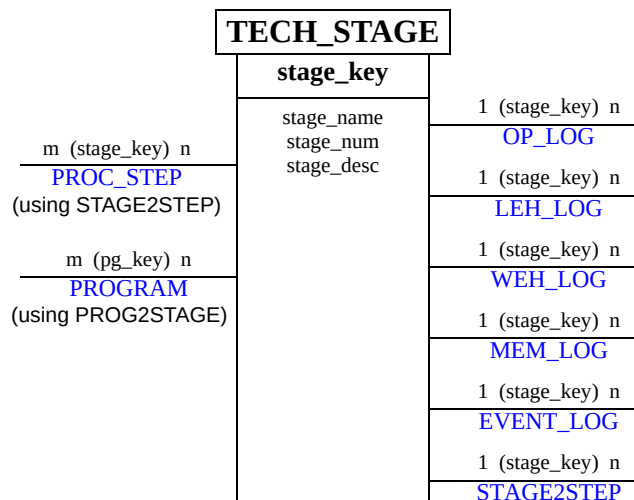
tech_desc Technology Description.

rev_date Revision date.

7.3 TECH_STAGE

Abstract Technology stage related information.

Table



Serial Key **stage_key** Automatically generated key based on **stage_name**.



Foreign Keys / Link Description None.

Other Fields in TECH_STAGE

stage_name Stage name.

stage_num Stage number.

stage_desc Stage description.

7.4 PROCESS

Table

PROCESS		
	pr_key	
1 (tech_key) n	pr_name pr_desc rev_date tech_key	1 (pr_key) n
TECHNOLOGY		OP_LOG
m (pr_key) n		1 (pr_key) n
PROGRAM (using PROG2PROC)		LEH_LOG
m (pr_key) n		1 (pr_key) n
LOT (using PROC2LOT)		WEH_LOG
m (pr_key) n		1 (pr_key) n
LOT (using PROC2SRCLLOT)		MEM_LOG
		1 (pr_key) n
		EVENT_LOG
		1 (pr_key) n
		DBTOOLS_MERGE_PROC
		1 (pr_key) n
		DBTOOLS_MERGE_PROC
		1 (pr_key) n
		MSTEP2PSTEP
		1 (pr_key) n
		STEP_ORDER_SET

Serial Key

pr_key Automatically generated key based on **pr_name**.

Foreign Keys / Link Description

tech_key Links PROCESS to TECHNOLOGY.

Other Fields in PROCESS

pr_name Process name.

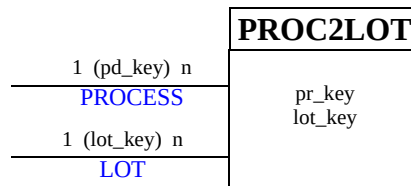
pr_desc Process description.

rev_date Revision date.

7.5 PROC2LOT

Abstract An m2n table holding all the available combinations of pr_keys and lot_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pr_key A foreign key from the PROCESS table.

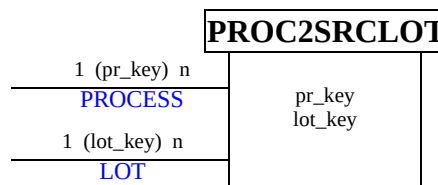
lot_key A foreign key from the LOT table.

Other Fields in PROC2LOT None.

7.6 PROC2SRCLOT

Abstract An m2n table holding all the available combinations of pr_keys and lot_keys.

Table



Serial Key None.

Foreign Keys / Link Description

pr_key A foreign key from the PROCESS table.

lot_key A foreign key from the LOT table.

Other Fields in PROC2SRCLOT None.

7.7 FLOW_STAGE

Abstract Process flow related information.

Table

FLOW_STAGE
flow_key
flow_name flow_num flow_desc

Serial Key **flow_key** Automatically generated key based on **flow_name**.

Other Fields in FLOW_STAGE

flow_name Flow Stage name.

flow_num Flow Stage number.

flow_desc Flow stage description.

7.8 RECIPE

Abstract This table holds information about the **recipe** used for processing the wafers.

Table

	RECIPE	
	rcp_key	1 (rcp_key) n
m (pr_key) n PROC_STEP (using RCP2STEP)	rcp_name	OP_LOG
		1 (rcp_key) n LEH_LOG
		1 (rcp_key) n WEH_LOG

Serial Key **rcp_key** Automatically generated based on **rcp_name**.

Foreign Keys / Link Description None.

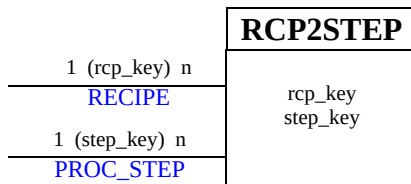
Other Fields in RECIPE

rcp_name The name of the recipe.

7.9 RCP2STEP

Abstract An m2n table holding all the available combinations of rcp_keys and step_keys.

Table



Serial Key None.

Foreign Keys / Link Description

rcp_key A foreign key from the RECIPE table.

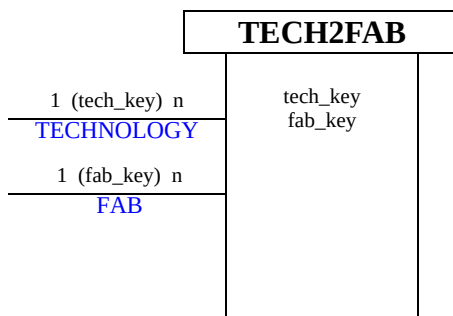
step_key A foreign key from the PROC_STEP table.

Other Fields in RCP2STEP None.

7.10 TECH2FAB

Abstract This table contains information about the relationship between technology information and fabs.

Table



Serial Key None.

Foreign Keys / Link Description

tech_key Foreign key from the TECHNOLOGY table.

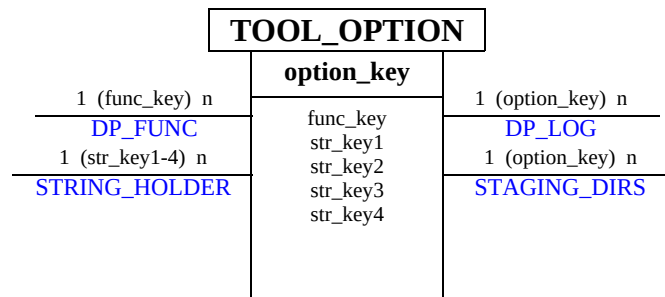
fab_key Foreign key from the FAB table.

Other Fields in TECH2FAB None.

7.11 TOOL_OPTION

Abstract This table stores the keys of the options string used for DpLoad and for every DpLoad configuration line. It also stores parallel options, if used, in this table. It also stores -99 in **func_key**, since parallel options are not for a specific reader or tool. In this table we store keys to strings that are stored in the STRING_HOLDER table. The options string can be up to 1,020 characters.

Table



Serial Key	option_key	A unique key assigned to each entry in TOOL_OPTION. It is stored automatically.
Foreign Keys / Link Description	func_key	A foreign key from the DP_FUNC table.
	str_key1-4	A foreign key from the STRING_HOLDER table.
Other Fields in TOOL_OPTION	None.	

7.12 STEP_ORDER_SET

Abstract This table is used to maintain information about step order at a process level and fab level. STEP_ORDER_SET is a grouping (Fab or Process) and the STEP_ORDER_DETAILS table ([pg. 143](#)) maintains the step orders for every group.

Table

STEP_ORDER_SET		
1 (meta_key) n META_DEF	step_order_key meta_key identifier_key em_key step_order_name insert_time def_flag set_desc	1 (step_order_key) n STEP_ORDER_DETAILS
(using pr_key) 1 (identifier_key) n PROCESS		
1 (em_key) n PEOPLE		

Serial Key **step_order_key** A unique key is assigned to each entry in STEP_ORDER_SET. It is generated and stored automatically.

Foreign Keys / Link Description

meta_key Foreign key from META_DEF table.

identifier_key From PROCESS table, uses **pr_key** for now. Set to -1 as default.

em_key Reference to PEOPLE table.

Other Fields in STEP_ORDER_SET

step_order_name Step order set name.

insert_time Time of insertion into the database.

def_flag Flag indicating if this is “default.” Legal values are **Y** and **N**.

set_desc Description of the step set.

7.14 META_DEF

Abstract This is table holds unique level IDs (e.g. product level, process level, family level etc....).

Table

META_DEF	
meta_key	1 (meta_key) n
meta_name meta_desc	PROGRAM 1 (meta_key) n STEP_ORDER_SET

Serial Key **meta_key** Serial key.

Foreign Keys / Link Description None.

Other Fields in META_DEF
meta_name
meta_desc

7.15 META_IDX_DEF

Abstract

Table

META_IDX_DEF	
idx_key	1 (idx_key) n
pg_key idx_order idx_name idx_type	PROGRAM 1 (pg_key) n CV Tables

Serial Key **idx_key** A unique key assigned to each entry in META_IDX_DEF. It is stored automatically.

Foreign Keys / Link Description **pg_key** Foreign key from the PROGRAM table.

Other Fields in META_IDX_DEF
idx_order
idx_name

idx_type

7.16 MSTEP2PSTEP

Abstract This table is a mapping table between measurement and production steps (MSTEP and PSTEP). This table stores, for a given production step, its corresponding measurement step. Information is stored per program, thus pr_key as a reference to PROGRAM table.

Table

MSTEP2PSTEP	
1 (pr_key) n	pr_key
PROCESS	mstep_key
1 (mstep_key) n	pstep_key
PROC_STEP	
1 (pstep_key) n	
PROC_STEP	

Serial Key None.

Foreign Keys / Link Description

pr_key	A foreign key from the PROCESS table.
mstep_key	Foreign key from the PROC_STEP table.
pstep_key	Foreign key from the PROC_STEP table.

Other Fields in MSTEP2PSTEP None.

8.0 Zonal Bin Summary Tables

Abstract The tables in this section are used by the *binMAP* zonal utility, which is part of the UpStat utility.

8.1 Z_WMAP_CONFIG

Abstract This table is an extension to the [WMAP_CONFIG](#) table. Z_WMAP_CONFIG contains additional information required by the zonal tools.

Table

Z_WMAP_CONFIG		
	1 (wmap_key) n	wmap_key quad_offset radial_offset radials circles nrow cols area_flag accept_percent
	WMAP_CONFIG	

Serial Key None.

Foreign Keys / Link Description **wmap_key** The key of the WMAP_CONFIG entry which this table extends.

Other Fields in Z_WMAP_CONFIG

quad_offset Used to change the default starting quadrant.

radial_offset Used to change the default starting radial zone.

radials The number of radial zones to be used in zonal charting.

circles The number of circular zones to be used in zonal charting.

nrow The number of row zones to be used in zonal charting.

cols The number of column zones to be used in zonal charting.

area_flag For circular zones, defines whether zonal boundaries will be of equal distance between circles or arranged for equal area within each circular zone.

area_flag value	result
0	equal distance between circles
1	equal area for each circular zone

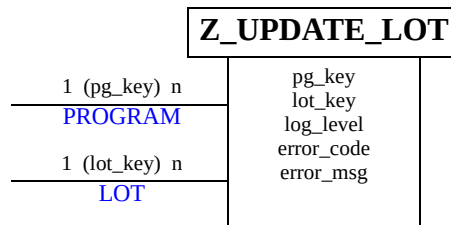
accept_percent Minimum percentage of tested die per wafer (of total number of die on the wafer) to enable analysis. If the minimum is not met, then that wafer is ignored.

Example: Total die is 2000; tested die on the wafer is 500. If **accept_percent** is >25 then this wafer is ignored; if the number of tested die is 501, it is accepted.

8.2 Z_UPDATE_LOT

Abstract This table contains lot-level or source lot-level entries to be processed by the zonal utility. Once the entries are successfully processed, those entries are removed from this table. Otherwise, the entries remain in the table and the error code of the error messages are set.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key Foreign key from the PROGRAM table. The key of the program to which the lot belongs.

lot_key Foreign key from the LOT table. The key of the lot to be processed.

Other Fields in Z_UPDATE_LOT

log_level L = lot-level; S = source lot-level.

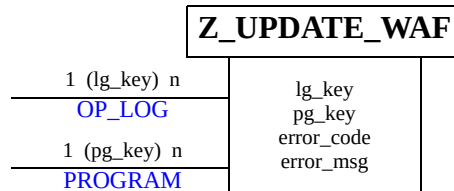
error_code Set to the appropriate error number when the processing of the lot fails.

error_msg Set to the appropriate error message when the processing of the lot fails.

8.3 Z_UPDATE_WAF

Abstract This table contains wafer-level entries to be processed by the zonal utility. Once the entries are successfully processed, those entries are removed from this table. Otherwise, the entries remain in the table and the error code of the error messages are set.

Table



Serial Key None.

Foreign Keys / Link Description

lg_key Foreign key from the OP_LOG table. The key of the op_log to which the wafer belongs.

pg_key Foreign key from the PROGRAM table. The key of the program to which the wafer belongs.

Other Fields in Z_UPDATE_WAF

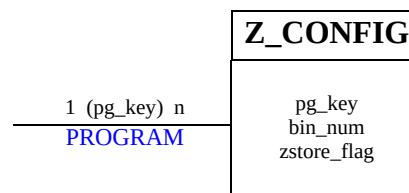
error_code Set to the appropriate error number when the processing of the wafer fails.

error_msg Set to the appropriate error message when the processing of the wafer fails.

8.4 Z_CONFIG

Abstract This table contains the zonal configuration information, per program.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key Foreign key from the PROGRAM table. The key of the program to which the configuration applies.

Other Fields in Z_CONFIG

bin_num The bin number as defined in [HIST_BIN](#). When -1 is used, all pass bins are used.

zstore_flag

value	description
1	calculate all statistics and yield/count
2	calculate yield/count without calculating other statistics
3	calculate statistics without calculating yield/count

8.5 Z_CNT

Abstract Contains the die count per bin in each zone.

Table

Z_CNT	
1 (z_key) n	z_key
Z_LOG	bin_num
	q1... q4
	c1... c16
	r1... r24
	s1... s100
	row1... row24
	col1... col24

Serial Key None.

Foreign Keys / Link Description

z_key Foreign key from the Z_LOG table. The key of the z_log to which the entry belongs.

Other Fields in Z_CNT

bin_num The bin number as defined in [HIST_BIN](#). When -1 is used, all pass bins are used.

q1... q4 The die count in each quadrant.

c1... c16 The die count in each circular zone.

r1... r24 The die count in each radial zone.

s1... s100 The die count in each stepper field zone.

row1... row24 The die count in each row.

col1... col24 The die count in each column.

8.6 Z_YLD

Abstract Contains the yield per bin in each zone.

Table

Z_YLD		
1 (z_key) n		z_key bin_num q1... q4 c1... c16 r1... r24 s1... s100 row1... row24 col1... col24
	Z_LOG	

Serial Key None.

Foreign Keys / Link Description

z_key	Foreign key from the Z_LOG table. The key of the z_log to which the entry belongs.
--------------	--

Other Fields in Z_YLD

bin_num	The bin number as defined in HIST_BIN . When -1 is used, all pass bins are used.
----------------	--

q1... q4	The yield in each quadrant.
-----------------	-----------------------------

c1... c16	The yield in each circular zone.
------------------	----------------------------------

r1... r24	The yield in each radial zone.
------------------	--------------------------------

s1... s100	The yield in each stepper field zone.
-------------------	---------------------------------------

row1... row24	The yield in each row.
----------------------	------------------------

col1... col24	The yield in each column.
----------------------	---------------------------

8.7 Z_STATS

Abstract Contains additional statistics for each zone.

Table

Z_STATS	
1 (z_key) n	z_key
Z_LOG	bin_num
	q_chisq
	q_chisqc
	q_df
	q_status
	c_chisq
	c_chisqc
	c_df
	c_status
	s_chisq
	s_chisqc
	s_df
	s_status
	r_chisq
	r_chisqc
	r_df
	r_status
	row_chisq
	row_chisqc
	row_df
	row_status
	col_chisq
	col_chisqc
	col_df
	col_status

Serial Key None.

Foreign Keys / Link Description

z_key Foreign key from the Z_LOG table. The key of the z_log to which the entry belongs.

Other Fields in Z_STATS

bin_num The bin number as defined in [HIST_BIN](#). When -1 is used, all pass bins are used.

q_chisq The chi-square for the quadrant zones.

q_chisqc The critical chi-square for the quadrant zones.

q_df The degrees of freedom for the quadrant zones.

q_status The status for the quadrant zones.

c_chisq The chi-square for the circular zones.

c_chisqc The critical chi-square for the circular zones.

c_df The degrees of freedom for the circular zones.

c_status The status for the circular zones.

s_chisq	The chi-square for the stepper field zones.
s_chisqc	The critical chi-square for the stepper field zones.
s_df	The degrees of freedom for the stepper field zones.
s_status	The status for the stepper field zones.
r_chisq	The chi-square for the radial zones.
r_chisqc	The critical chi-square for the radial zones.
r_df	The degrees of freedom for the radial zones.
r_status	The status for the radial zones.
row_chisq	The chi-square for the rows.
row_chisqc	The critical chi-square for the rows.
row_df	The degrees of freedom for the rows.
row_status	The status for the rows.
col_chisq	The chi-square for the columns.
col_chisqc	The critical chi-square for the columns.
col_df	The degrees of freedom for the columns.
col_status	The status for the columns.

**Formulas for
Statistics**

chi-square

$$Chisq = \sum_{i=0}^k \frac{(Fei - Foi)^2}{Fei}, Fei = P \cdot Ni$$

$$P = \frac{\sum_{i=0}^k Foi}{\sum_{i=0}^k Ni}$$

chi-square

= Value from Chisq with arguments
(confidence level, Df).

Df = Degrees of Freedom = count of zones - 1, If the zone has Fe < 5, group it with the adjacent one.

$$\text{status_value} = \frac{\text{Chisq}}{\text{Chisqc}}$$

Status	Description
<0.5 :	“Very Good”
<0.8 :	“Good”
<1 :	“Marginal Good”
<1.5 :	“Marginal High”
<2.5 :	“High”
<5 :	“Very High”
>5 :	“Extremely High”

8.8 Z_LOG

Abstract Links the zonal data to the programs, lots, and wafers.

Table

Z_LOG		
	z_key	
1 (pg_key) n PROGRAM	pg_key lg_key lot_key wf_key log_level die_cnt wf_cnt	1 (z_key) n Z_CNT
1 (lg_key) n OP_LOG		1 (z_key) n Z_YLD
1 (lot_key) n LOT		1 (z_key) n Z_STATS
1 (wf_key) n WAFER		

Serial Key

z_key A unique key is assigned to each entry in Z_LOG. It is generated and stored automatically.

Foreign Keys / Link Description

pg_key Foreign key from the PROGRAM table. The key to the program to which the zonal data belongs.

lg_key Foreign key from the OP_LOG table. The key to the op_log to which the zonal data belongs.

lot_key Foreign key from the LOT table. The key to the lot to which the zonal data belongs.

wf_key Foreign key from the WAFER table. The key to the wafer to which the zonal data belongs. Filled only when **log_level = W**.

Other Fields in Z_LOG

log_level

W	wafer level
L	lot level
S	source lot level

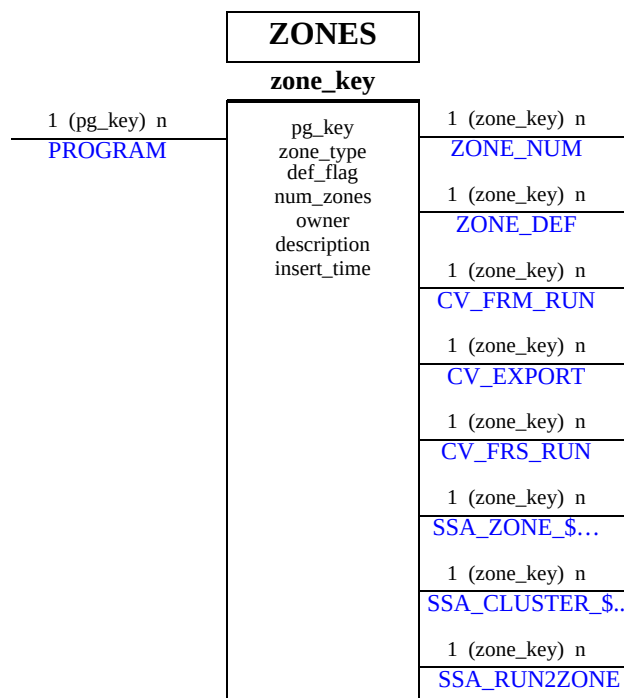
die_cnt The total number of die per wafer when **log_level = W**;
the total number of die per lot when **log_level = L**;
the total number of die per source_lot when **log_level = S**

wf_cnt The total number of wafers per lot or source_lot.

8.9 ZONES

Abstract Holds the type, owner and related program of the overall zone definition.

Table



Serial Key

zone_key Serial key generated based on zone type, owner and program.

Foreign Keys / Link Description

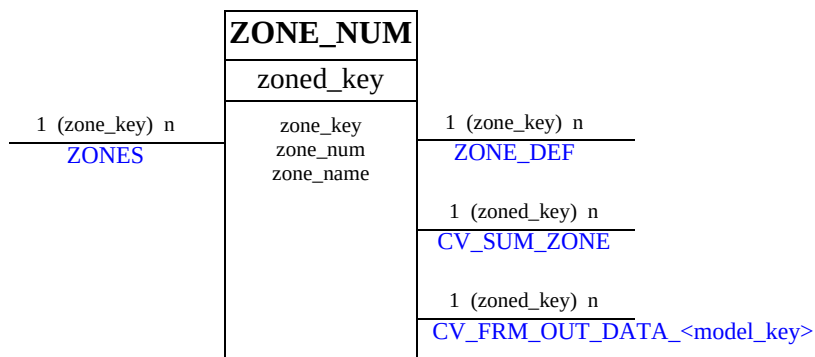
pg_key Foreign key from PROGRAM table.

Other Fields in ZONES

zone_type	Name of the zone definition.
def_flag	Name of the default zone definition.
num_zones	Number of zone areas in the zone definition.
owner	Name of the person who created the zone definition.
description	A field to hold any comments or notes a user has on the zone definition.
insert_time	Time that the user created the zone definition.

8.10 ZONE_NUM

Abstract Table that holds the definition of each zone area within a zone definition.

Table**Serial Key**

zoned_key Serial key generated based on zone_key, and zone_num.

Foreign Keys / Link Description

zone_key Foreign key from ZONES table.

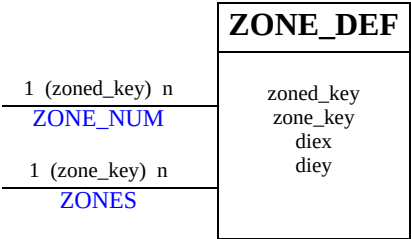
Other Fields in ZONE_NUM

zone_num	Mandatory number for zone area within the zone definition.
zone_name	Optional name for zone area within the zone definition.

8.11 ZONE_DEF

Abstract Table that holds details of what die belong to which zone.

Table



Serial Key None.

Foreign Keys / Link Description	zoned_key	From ZONE_NUM table.
	zone_key	From ZONES table.

Other Fields in ZONE_DEF	diex	X index of the die.
	diey	Y index of the die.

9.0 *dp*WEB Table

Abstract The tables in this section are used by the *dataPOWER* web-based analysis tool, *dp*WEB. There is one table associated with the *dp*WEB.

9.1 WEB_RPT_CFG

Abstract This table information is used to store user-specific report information required to generate WEB reports.

Table

WEB_RPT_CFG
owner
rpt_num
header
footer
num_charts
ww_day
ww_offset
chart_type

Serial Key None.

Foreign Keys / Link Description None.

Other Fields in WEB_RPT_CFG

owner	The user name. The combination of owner and rpt_num must be unique.
rpt_num	The value representing the <i>dp</i> WEB report number (e.g. rpt_num = 3 is the zonal report).
header	The report header section.
footer	The report footer section.
num_charts	The number of charts per page.
ww_day	The starting day of the work-week, as defined by the user.
ww_offset	The number of days to be used as an offset to the starting day of a work-week.
chart_type	The value representing the default chart type (e.g. chart_type = 1 is the line-graph).

10.0 defectMAP Tables

These tables describe the schema that holds the raw defect data and the related defect summaries.

10.1 DF_LOG

Abstract Provides information about the wafer chosen for defect inspection. Every record in a defect file adds a row in this table.

Table

DF_LOG	
lg_key	1 (lg_key) n OP_LOG
insp_or	
tol_maj	
tol_min	
tol_or	
seq_num	
proc_flag	
sum_flag	
slot_num	
ptrn_flag	
x_shift	
y_shift	
img_cnt	

Serial Key None.

Foreign Keys / Link Description

lg_key From OP_LOG table. Identifies the log entry.

Other Fields in DF_LOG

insp_or Orientation of flat (or notch) side of wafer at the time the wafer was inspected. Permitted values:

Value	Description
R	right
L	left
T	top
B	bottom

tol_maj Two defects are considered “common” if both their centers are within a certain elliptic area. The major axis of this ellipse is given by **tol_maj**.

tol_min Two defects are considered “common” if both their centers are within a certain elliptic area. The minor axis of this ellipse is given by **tol_min**.

tol_or Two defects are considered “common” if both their centers are within a certain elliptic area. The clockwise

orientation of the ellipse from the positive X-axis is given by **tol_or**.

seq_num A serial number that indicates the order of the layers within a wafer. It is computed by reader taking into account the date the results were measured.

proc_flag A flag indicating whether a layer has been processed by the update_stats operation to find defect type. The following table indicates the possible values and their meanings:

proc_flag value	meaning
0	needs full df_type computations
1	already fully processed to find df_type
>1	only "repeater" computation of df_type complete

sum_flag A flag indicating whether summaries for a layer had been computed by the update_stats operation. The following table indicates the possible values and their meanings:

sum_flag value	meaning
0	needs full summary computations
1	full summaries at wafer-layer layer computed
2	only missing Binmap summaries

slot_num The slot in which the wafer was held.

ptrn_flag A flag to determine whether the wafer was patterned at that particular layer. The following table indicates the possible values and their meanings:

ptrn_flag value	meaning
P	Wafer is patterned
U	Wafer is unpatterned

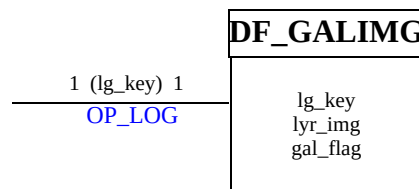
x_shift	Indicates how much the die indexes have been shifted in the X-direction to force die (0,0) to contain the wafer center.
y_shift	Indicates how much the die indexes have been shifted in the Y-direction to force die (0,0) to contain the wafer center.
img_cnt	Indicates the number of images stored with defects, and whether the images are stored as pointers.

img_cnt value	meaning
0	No images stored
>0	Number of defects with images stored inside the database
<0	Number of defects with images stored as pointers

10.2 DF_GALIMG

Abstract This table holds the images used in the *defectMAP* gallery view.

Table



Serial Key None.

Foreign Keys / Link Description

lg_key From OP_LOG table. Identifies the log entry.

Other Fields in DF_GALIMG

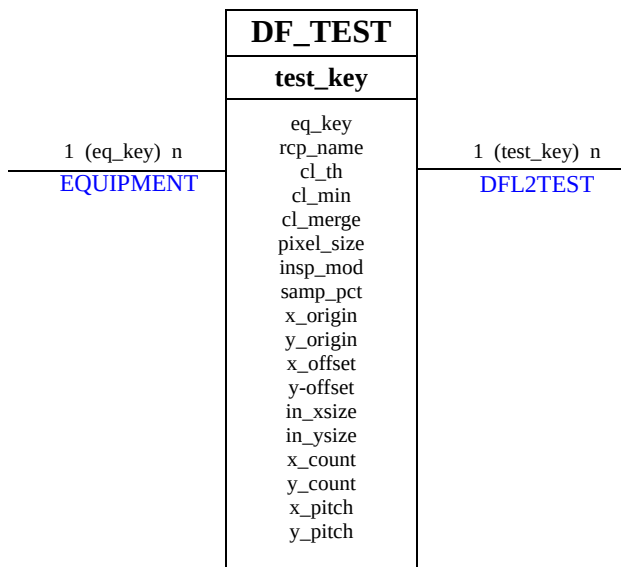
lyr_img Holds the blob image of the layer.

gal_flag Indicates whether an image is “A” (adder thumbnail), or “L” (all defects thumbnail).

10.3 DF_TEST

Abstract (Currently not implemented) Provides information about the inspection test.

Table



Serial Key	test_key	Automatically generated.
Foreign Keys / Link Description	eq_key	From EQUIPMENT Table. The key for the equipment that performed the inspection test.
Other Fields In DF_TEST	rcp_name	The name of the test recipe.
	cl_th	The initial tolerance used to determine if defects are members of the same cluster.
	cl_min	Minimum number of defects in a cluster.
	cl_merge	The maximum separation between clusters.
	pixel_size	The pixel size used during the run.
	insp_mod	For inspectors which have more than one operating mode (an integer).
	samp_pct	For inspectors which allow the user to sub-sample. Used to indicate the percentage of the selected area for this test.
	x_origin	X coordinate of the origin of an inspected area on the sample (wafer). Inspected area records are relative to this point.

y_origin	X coordinate of the origin of an inspected area on the sample (wafer). Inspected area records are relative to this point.
x_offset	X offset to lower left corner of inspected area.
y_offset	Y offset to lower left corner of inspected area.
in_xsize	(real) X size of inspected area.
in_ysize	Y size of inspected area.
x_count	Repeat count of area in x (range 1 to n).
y_count	Repeat count of area in y (range 1 to n).
x_pitch	X pitch of the array of inspected areas.
y_pitch	Y pitch of the array of inspected areas.

10.4 DFL2TEST

Abstract Holds defect summary at layer level, as provided in the data file.

Table

DFL2TEST	
1 (lg_key) n	lg_key
OP_LOG	test_key
1 (test_key) n	test_num
DF_TEST	def_cnt
	insp_area
	dd
	insp_die
	def_die

Serial Key None.

Foreign Keys / Link Description

lg_key From OP_LOG Table.

test_key From DF_TEST Table (not yet implemented)

Other Fields in DFL2TEST

test_num Indicates the test number in which the test was found.

def_cnt The number of defects in the record. Integer.

insp_area The area inspected in the test.

dd Defect density.

insp_die The number of inspected die. Integer.

def_die The number of defective die (def_die <= insp_die). Integer.

10.5 DEFECT

Abstract Provides details related to individual defects uncovered in a specific test for a certain wafer layer. Each test will insert *n* rows in this table.

Table

DEFECT		
	df_key	1 (df_key) n
	lg_key	DEF2CLASS
1 (lg_key) n	df_index	1 (df_key) n
OP_LOG	diex	DF_IMAGE
	diey	
	xAbs	1 (df_key) n
	yAbs	DF_EXPORT_DEF
1 (df_type) n	x_size	
DF_TYPES	y_size	1 (df_key) n
	df_area	CV_ADL_DEF
	df_size	
	intensity	1 (df_key) n
	clusters	
	test_num	1 (df_key) n
	df_type	CV_DEF2TYPE_<lot_key>
	loo	
	df_info	

Serial Key

df_key Automatically generated, unique key for each defect, based on **lg_key**, **df_index**.

Foreign Keys / Link Description

lg_key From OP_LOG table.
Identifies an inspection of a wafer layer.

Other Fields in DEFECT

df_index Defect ID within a wafer-layer. Integer.

diex X index of the die.

The die index is shifted such that die(0,0) contains the wafer center. The first wafer entered into the DB of a certain product determines a reference orientation for that product. Any other wafer of the same product is rotated so that it has the same orientation.

diey Y index of the die.

The die index is shifted such that die(0,0) contains the wafer center. The first wafer entered into the DB of a certain product determines a reference orientation for that product. Any other wafer of the same product is rotated so that it has the same orientation.

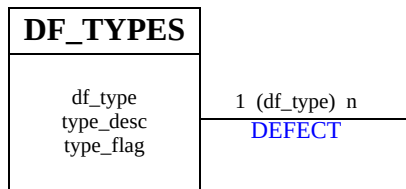
xAbs	X position of the defect from the wafer center.
yAbs	Y position of the defect from the wafer center.
x_size	X size of the defect in microns.
y_size	Y size of the defect in microns.
df_area	Area of the defect in microns.
df_size	A linear measure of defect.
Intensity	Used by Tencor binary formats as a measure of defect.
clusters	Indicates membership in a particular cluster. All defects within a cluster will share this number.
test_num	Indicates the test number in which the test was found.
df_type	default 'A'. Determined in an UPDATE_STATS operation, valid values found in DF_TYPES table.
loo	Determined in an UPDATE_STATS operation.
	Layer of origin. The lg_key of the layer of origin will be stored here. When inserted by the reader, the current lg_key is stored.
df_info	Flag to indicate info stored in DB about defect.

df_info	meaning
N	nothing
I	has image
C	is classified
Z	has image and class

10.6 DF_TYPES

Abstract Acts as a look up table for the defect types.

Table



Serial Key None.

Other Fields in DF_TYPES

df_type A character indicating the defect type.

type_desc The defect type description.

The valid values are given in the following table:

df_type (case sensitive)	type_desc	type_flag I - standard inline defect program C - cv inline defect program E - electrical defect program	Description
O	Adder-Repeater-Missing	I	Defects which are both adders and repeaters, but also missing from the next layer.
R	Adder - Repeater	I	Defects which appear at the same reticle XY coordinates on multiple reticles (repeaters) and are also adders.
M	Adder - Missing	I	Defects which are adders, but also missing from the next layer.
A	Adder	I	A new defect that was not in any of the previous layers.
T	Common - Repeater - Missing	I	Defects which appear at the same reticle XY coordinates on multiple reticles (repeaters) and also on all layers at the same wafer XY coordinates (common).
S	Common - Repeater	I	Defects which appear at the same reticle XY coordinates on multiple reticles (repeaters) and also on all layers at the same wafer XY coordinates (common).
N	Common - Missing	I	Defects which are common, but also missing from the next layer.
C	Common	I	Defects that were detected in one of the previous layers.
K	Common - In Fail Mask	I	Defect that were detected in one of the previous layers and fall into the failed Etest bounding box.

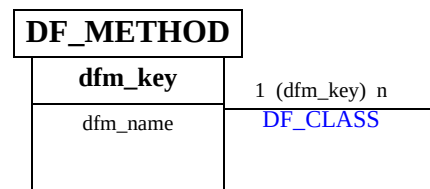
df_type (case sensitive)	type_desc	type_flag I - standard inline defect program C - cv inline defect program E - electrical defect program	Description
Z	Common - In Pass Mask	I	Defects that were detected in one of the previous layers and fall into the passed Etest bounding box.
c	Visual	E	Etest defects that have inline defects overlaid with them.
a	Non-Visual	E	Etest defect that don't have inline defects overlaid with them.
b	Non-Visual - No Inline	E	Etest defect that don't have inline defects overlaid with them because the inline layers are not inspected and the cv_cell_i.insp_flag is Y .
x	Non-Visual - Not Inline Inspected	E	Etest defect that don't have inline defects overlaid with them because the inline layers are not inspected and the cv_cell_i.insp_flag is N .

type_flag Indicates what type of defect program the df_type is used for:
 I — standard inline defect program
 C — cv inline defect program
 E — electrical defect program

10.7 DF_METHOD

Abstract Holds the defect classification methods.

Table



Serial Key

dfm_key based on **dfm_name**.

Other Fields in DF_METHOD

dfm_name Name of defect classification method.

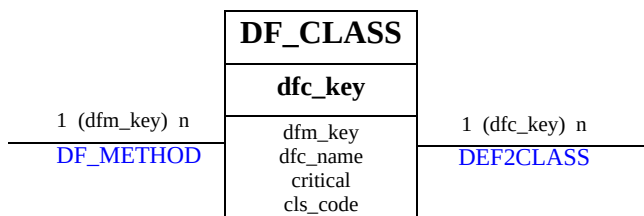
These groupings will be predefined. The following table holds the names and keys of the predefined classification groupings.

dfm_key	dfm_name
1	manual
2	roughbin
3	finebin
4	mansemclass
5	autosemclass
6	microsigclass
7	macrosigclass
8	adl

10.8 DF_CLASS

Abstract Holds the defect class names determined by the inspector.

Table



Serial Key

dfc_key Automatically generated based on **dfc_name**.

Foreign Keys / Link Description

dfm_key From DF_METHOD table

Other Fields in DF_CLASS

dfc_name Classification name for the defect.

critical A flag indicating that the given class should be included in the defect class summaries. Valid values are “N” (No), and “Y” (Yes).

cls_code Indicates if class was added “BY NAME” or “BY CODE”. Valid values are:

cls_code	meaning
0<	By code
-2	By name
-1	Unknown

10.9 DEF2CLASS

Abstract An m to n table that assigns multiple classes to each defect.

Table

DEF2CLASS	
1 (lg_key) n	lg_key df_key dfc_key
OP_LOG	
1 (df_key) n	
DEFECT	
1 (dfc_key) n	
DF_CLASS	

Serial Key None.

Foreign Keys / Link Description

lg_key From OP_LOG table

df_key From DEFECT table

dfc_key From the DF_CLASS table

10.10 DF_DIE

Abstract Provides information about the die inspected for the given defect test. Each entry in the DF_LOG table results in *n* rows in this table. The **res_aging_days** field in the PROGRAM table determines how often this table is deleted or backed up.

Table

DF_DIE	
1 (lg_key) n	lg_key diex diey die_type def_cnt
OP_LOG	

Serial Key None.

Foreign Keys / Link Description

lg_key From OP_LOG table.
Identifies an inspection of a wafer layer.

Other Fields in DF_DIE

diex The X-index of a die sampled in the current test.

diey The Y-index of a die sampled in the current test.

die_type Indicates if the die had been inspected. Valid values are **R** indicating a removed die and **V** indicating a valid inspected die.

def_cnt Integer computed from database.
The number of defects in a die.

10.11 DF_ALNIM

Abstract (Currently not implemented.)

Table

DF_ALNIM	
1 (lg_key) n	lg_key
OP_LOG	x_pt
	y_pt
	x_img
	y_img
	img_index

Serial Key None.

Foreign Keys / Link Description

lg_key From OP_LOG table.
Identifies an inspection of a wafer layer.

Other Fields in DF_ALNIM

x_pt X coordinate of the alignment point in the sample coordinate system.

y_pt Y coordinate of the alignment point in the sample coordinate system.

x_img X coordinate of the origin of the alignment mark image coordinate system in the sample coordinate system.

y_img Y coordinate of the origin of the alignment mark image coordinate system in the sample coordinate system.

img_index The number of the image in the TIFF file containing the alignment image.

10.12 DF_IMAGE

Abstract Holds information about the images related to individual defects. Each defect entry in the DEFECT table might result in *n* rows in this table.

Table

DF_IMAGE	
1 (lg_key) n	lg_key
OP_LOG	df_key
	ims_key
1 (df_key) n	img_index
DEFECT	df_index
	tiff_ptr
	def_img
	img_type
	img_dir
	img_com

Serial Key None.

Foreign Keys / Link Description

lg_key From OP_LOG table. Identifies an inspection of a wafer layer.

df_key From DEFECT table. Uniquely identifies a defect.

ims_key Holds the image source key (from DF_IMGSRC).

Other Fields in DF_IMAGE

img_index The number of the image in the image file containing the defect image.

df_index Defect ID within a wafer/layer.

tiff_ptr Name of image file that contains defect image.
varchar(255)

def_img Defect image blob.

img_type Holds the image type: “T” for TIFF, “G” for GIF, and “J” for JPEG.

img_dir Holds the image directory path. This function is needed for the storage and managing of image pointers.

img_com Holds comments for a specific image.

10.13 DF_SUM

Abstract Holds summary information at wafer, lot, and source lot levels.

Table

DF_SUM	
1 (lg_key) n OP_LOG	lg_key
1 (step_key) n PROC_STEP	step_key lot_key wf_key pg_key log_level
1 (lot_key) n LOT	tot_adr_cnt insp_area fail_die_cnt insp_die_cnt def_die_cnt fail_adr_cnt
1 (wf_key) n WAFER	fail_clean_cnt pass_clean_cnt pass_def_cnt cont_sig_lvl
1 (pg_key) n PROGRAM	

Serial Key None.

Foreign Keys / Link Description

lg_key	From OP_LOG table.
step_key	From PROC_STEP table.
lot_key	From LOT table. Identifies the lot.
wf_key	From WAFER table. Identifies the wafer.
pg_key	From PROGRAM table.

Other Fields in DF_SUM

log_level Permitted values are:

W	Wafer
L	Lot
S	SrcLot

tot_adr_cnt	Computed by Update_Stats Total number of adder defects.
insp_area	From data file. Total inspected area.
fail_die_cnt	Computed by Update_Stats. Total number of failed die given that the die had been inspected by a defect inspection tool and was defective with adder defects.

insp_die_cnt	From data file. Total number of die that had been inspected by a defect inspection tool.
def_die_cnt	Computed by Update_Stats. Total number of die defective with adder defects.
fail_adr_cnt	Computed by Update_Stats. Total number of adder defects that fell in failed die.
fail_clean_cnt	Computed by Update_Stats. Total number of die that failed but had no defects (clean).
pass_clean_cnt	Computed by Update_Stats. Total number of die that passed and had no defects (clean).
pass_def_cnt	Computed by Update_Stats. Total number of die that passed and had defects.
cont_sig_lvl	Computed by Update_Stats. Chi Square (χ^2) for defect map is defined as $\chi^2 = \frac{(a + b + c + d)(ad - bc)^2}{(a + b)(c + d)(a + c)(b + d)}$ where

	pass	fail
clean	a	b
defects	c	d

10.14 DF_IMGSRC

Abstract Holds all the possible image sources.

Table

DF_IMGSRC	
ims_key	
img_code	
img_src	

1 (ims_key) n
[DF_IMAGE](#)

Serial Key

ims_key Automatically generated based on img_src.

Foreign Keys / Link Description None.

**Other Fields in
DF_IMGSRC**

img_code Code by which user would identify an image source when using the proper format file function.

img_src Holds the name of the image source.

img_src	img_code
Unknown	1
SEM Internal	2
SEM External	3
EDX	4
Optical	5

10.15 DF_RECIP

Abstract Holds the main information relating to the defect recipe.

Table

DF_RECIP	
drcp_key	
drcp_name max_def rule_cnt min_clust	1 (drcp_key) n DF_LYRINFO
	1 (drcp_key) n DF_SAMPLE
	1 (drcp_key) n DF_TOLERANCE
	1 (drcp_key) n DRC2PDLYR

Serial Key

drcp_key Automatically generated based on drcp_name.

**Foreign Keys / Link
Description**

None.

**Other Fields in
DF_RECIP**

drcp_name Unique name identifier.

max_def Holds the maximum number of defects to sample using this recipe.

rule_cnt Holds the number of recipe rules.

min_clust Holds the minimum number of defects per cluster.

10.16 DF_SAMPLE

Abstract Holds the description of the sampling rules used by a defect recipe.

Table

DF_SAMPLE	
1 (drpc_key) n	DF_RECIP
DF_SAMPLE	
smpl_key	
drpc_key	
max_def	
sel_type	
cls_key	
df_clstr	
size_type	
low_size	
up_size	

Serial Key	smpl_key	Automatically generated based on drpc_key and the number of sampling rules for the recipe.
Foreign Keys / Link Description	drpc_key	Foreign key from DF_RECIP.
	cls_key	A foreign key from DF_CLASS that determines which class to use as a sampling criteria. When set to -1, class is not used as a sampling criteria.
Other Fields in DF_SAMPLE	max_def	Determines the maximum number of defects to sample for the given rule.
	sel_type	Selection type. This column determines how to select from the defect set. Valid values are:

Value	Description
F	First max_def defects
L	Last max_def defects
R	Random max_def defects

df_type Defect type valid values are:

Value	Description
F	First max_def defects
L	Last max_def defects
R	Random max_def defects

df_clstr Cluster sampling settings. Valid values are:

Adder State	Repeater State	Column Code
Yes	Yes	R
Yes	No	A
Yes	Don't care	B
No	Yes	S
No	No	C
No	Don't care	T
Don't care	Yes	Y
Don't care	No	N
Don't care	Don't care	Don't care

size_type Determines whether to use size as sampling criteria, as well as the size type criteria considered for sampling. Valid values are:

Value	Description
X	Use defect x-size as size type
Y	Use defect y-size as size type
A	Use defect area as size type
D	Use defect d-size as size type
I	Use intensity as size type (for TENCOR)
N	Do NOT use size as a sampling criteria

low_size Determines lower limit on size (when size_type is not "N").

up_size Determines upper limit on size.

10.17 DF_TOLERANCE

Abstract Maintains tolerances for clustering and defect type computations. The x and y tolerances (x_tol and y_tol) specify the major and minor axis of an ellipse respectively.

Table

DF_TOLERANCE	
1 (drpc_key) n DF_RECIPES	drpc_key tol_type x_tol y_tol

Serial Key None.

Foreign Keys / Link Description

drpc_key Foreign key from DF_RECIPES.

Other Fields in DF_TOLERANCE

tol_type Indicates whether there is a tolerance for clustering or defect. Valid values are:

Value	Description
C	Clustering
T	Defect type
R	Repeater (future)

x_tol Holds the dimension of the ellipse in the X-direction.

y_tol Holds the dimension of the ellipse in the Y-direction.

10.18 DRCP2PDLYR

Abstract Maintains an m2n relationship between a recipe and product and/or layer.

Table

DRCP2PDLYR	
1 (drcp_key) n	drcp_key pd_key step_key
DF_RECIPE	
1 (pd_key) n	
PRODUCT	
1 (step_key) n	
PROC_STEP	

Serial Key None.

Foreign Keys / Link Description

drcp_key Foreign key from DF_RECIPE.

pd_key Foreign key from PRODUCT. Set to -1 if not applicable.

step_key Foreign key from PROC_STEP.

Other Fields in DRCP2PDLYR None.

10.19 DF_CONTRIB

Abstract Holds adder defect step contribution summaries.

Table

DF_CONTRIB	
1 (loo_step) n	step_key lg_key wf_key lot_key loo_step def_cnt
PROC_STEP	
1 (lg_key) n	
OP_LOG	
1 (step_key) n	
PROC_STEP	
1 (lot_key) n	
LOT	
1 (wf_key) n	
WAFER	

Serial Key None.

Foreign Keys / Link Description

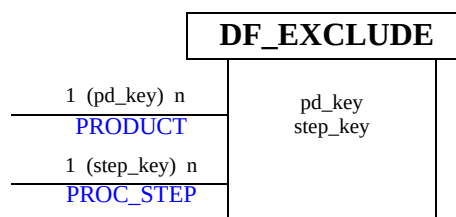
lg_key Foreign from OP_LOG. Indicates the wafer layer of interest.

Other Fields in DF_CONTRIB	wf_key	Foreign key from WAFER. The wafer for the above lg_key.
	lot_key	Foreign from LOT. The lot for the above lg_key.
	step_key	Foreign key from PROC_STEP. The step from the lg_key.
	loo_step	Foreign from PROC_STEP. The step of the layer of origin.
	def_cnt	Integer count of defects originating from loo_step for layer specified by lg_key.

10.20 DF_EXCLUDE

Abstract Maintains a list of layers in a product to be excluded from defect summary computations.

Table



Serial Key None.

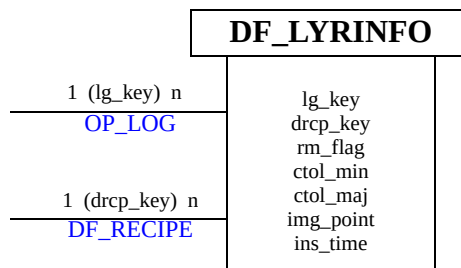
Foreign Keys / Link Description	pd_key	Foreign key from PRODUCT. Set to -1 if not applicable.
	step_key	Foreign key from PROC_STEP

**Other Fields in
DF_EXCLUDE** None.

10.21 DF_LYRINFO

Abstract Holds auxiliary data about the wafer layer.

Table



Serial Key None.

Foreign Keys / Link Description

lg_key Foreign key from OP_LOG.

drcp_key Foreign key from DF_RECIP.

Other Fields in DF_LYRINFO

rm_flag Allows removal of a wafer layer from search for adders and summary computation. Valid values are “R” (remove) and “K” (keep).

ctol_min Holds the Y tolerance for defect cluster calculation.

ctol_maj Holds the X tolerance for defect cluster calculation.

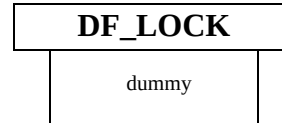
img_point Indicates if images for this layer are stored in “D” (DB) or “F” (file system pointers).

ins_time Date and time of when image pointers were reinserted into the database (when img_point is “F”).

10.22 DF_LOCK

Abstract Used to detect if another process is updating defect tables.

Table



Serial Key None.

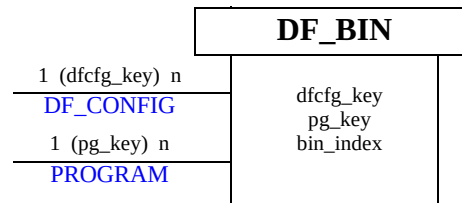
Foreign Keys / Link Description None.

Other Fields in DF_LOCK Dummy: unused

10.23 DF_BIN

Abstract Maps user-defined blocks to bins, per program. Not filled if [DF_CONFIG](#) type = S or Z (defect size bin).

Table



Serial Key None.

Foreign Keys / Link Description **dfcfg_key** Foreign key from DF_CONFIG.

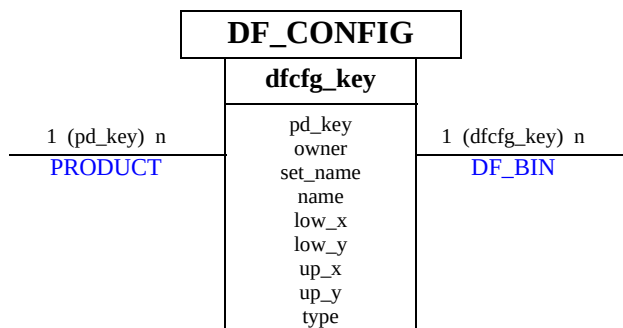
pg_key From PROGRAM table.

Other Fields in DF_BIN **bin_index** Ties each record to the bins defined in BIN_LOG.

10.24 DF_CONFIG

Abstract Contains block and multiple block set definitions, per product. Blocks are user-defined rectangular zones of interest within die, used for monitoring defect hot-spots. Alternatively, the table can contain size bin definitions.

Table



Serial Key

dfcfg_key Automatically generated serial key.

Foreign Keys / Link Description

pd_key The key to the related product information.

Other Fields in DF_CONFIG

owner The user who creates/defines the block or defect size bin set.

set_name Unique name for the block set if **type** = **B** or **D**; unique name for defect size bin set if **type** = **S** or **Z**.

name Unique name for the block if **type** = **B** or **D**; unique name for defect size bin if **type** = **S** or **Z**.

low_x Lower X-coordinate for defining block dimensions if **type** = **B** or **D**; defect size bin lower limit if **type** = **S** or **Z**.

low_y Lower Y-coordinate for defining block dimensions if **type** = **B** or **D**; defect size bin upper limit if **type** = **S** or **Z**.

up_x Upper X-coordinate for defining block dimensions.

up_y Upper Y-coordinate for defining block dimensions.

type Four possible states:

- **B** — blocks
- **D** — default blocks. Sets user-defined blocks as automatic default when *defectMAP* is opened.
- **S** — size bins.*
- **Z** — default size bins. Sets user-defined size bins as automatic default when *defectMAP* is opened.*

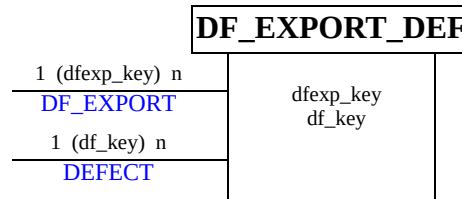
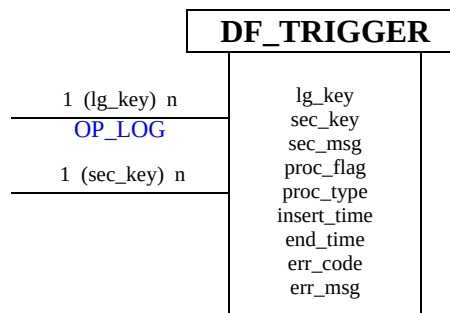
*When type = **S** or **Z**, DF_BIN is not used.

10.25 DF_EXPORT**Abstract****Table**

DF_EXPORT		
	dfexp_key	
1 (lg_key) n OP_LOG	insert_time end_time	1 (dfexp_key) n DF_EXPORT_DEF
1 (wf_key) n WAFER	export_type lg_key wf_key	
1 (step_key) n PROC_STEP	step_key wmap_key	
1 (wmap_key) n WMAP_CONFIG	pg_key status	
1 (pg_key) n PROGRAM	def_cnt file_name error_code err_msg owner	

Serial Key**dfexp_key** Automatically generated serial key.**Foreign Keys / Link Description****lg_key** The key of the OP_LOG which has been archived.**wf_key** The key to the related wafer information.**step_key** Foreign key from PROC_STEP. Uniquely identifies the process step where the genealogy event occurred.**wmap_key** Foreign key from WMAP_CONFIG. Identifies wafer map configuration.**pg_key** Foreign key from the PROGRAM table.**Other Fields in DF_EXPORT****insert_time** Time that the user inserted data into the database.**end_time** End time of run insertion into the database.**export_type****status****def_cnt** Integer count of defects.**file_name** The name of the file saved in the database.**error_code** Error code.**err_msg** Message related to a given error code.

owner

10.26 DF_EXPORT_DEF**Abstract****Table****Serial Key** None.**Foreign Keys / Link
Description****dfexp_key** From DF_EXPORT table.**df_key** From DEFECTION table. Uniquely identifies a defect.**Other Fields in
DF_EXPORT_DEF** None.**10.27 DF_TRIGGER****Abstract****Table****Serial Key** None.**Foreign Keys / Link
Description****lg_key** Foreign key from OP_LOG table.**sec_key****Other Fields in
DF_TRIGGER****sec_msg**

Front



Contents



Index

proc_flag A flag indicating whether a layer has been processed by the update_stats operation to find defect type. The following table indicates the possible values and their meanings:

proc_flag value	meaning
0	needs full df_type computations
1	already fully processed to find df_type
>1	only "repeater" computation of df_type complete

proc_type

insert_time The current date and time of insertion into the database. Date/time/year to the second. Defaults to current year to the second.

end_time End time for the data range to be considered.

err_code Error code value, given by the system.

err_msg Message related to a given error code.

11.0 dbTOOLS Tables

The *dbTOOLS* tables are all used for merging. Each row in these tables merges two entities of the same type — families, processes, products, or technologies.

11.1 DBTOOLS_MERGE_FAM

Abstract Used to merge two families per row. Families can be merged using this table and the *dbTOOLS* utility.

Table

DBTOOLS_MERGE_FAM		
1 (src_fam) n	src_fam dest_fam	
FAMILY		
1 (dest_fam) n		
FAMILY		

Serial Key None.

Foreign Keys / Link Description	src_fam	Referenced to the fm_key in the FAMILY table. This field refers to the source family.
	dest_fam	Referenced to the fm_key in the FAMILY table. This field refers to the destination family.

11.2 DBTOOLS_MERGE_PROC

Abstract Used to merge two processes per row. Processes can be merged using this table and the *dbTOOLS* utility.

Table

DBTOOLS_MERGE_PROC		
1 (src_proc) n	src_proc dest_proc	
PROCESS		
1 (dest_proc) n		
PROCESS		

Serial Key None.

Foreign Keys / Link Description	src_proc	Referenced to the pr_key in the PROCESS table. This field refers to the source process.
	dest_proc	Referenced to the pr_key in the PROCESS table. This field refers to the destination process.

11.3 DBTOOLS_MERGE_PROD

Abstract Used to merge two products per row. Products can be merged using this table and the *dbTOOLS* utility.

Table

DBTOOLS_MERGE_PROD	
1 (src_prod) n PRODUCT	src_prod dest_prod
1 (dest_prod) n PRODUCT	

Serial Key None.

Foreign Keys / Link Description

src_prod Referenced to the **pd_key** in the **PRODUCT** table. This field refers to the source product.

dest_prod Referenced to the **pd_key** in the **PRODUCT** table. This field refers to the destination product.

11.4 DBTOOLS_MERGE_TECH

Abstract Used to merge two technologies per row. Technologies can be merged using this table and the *dbTOOLS* utility.

Table

DBTOOLS_MERGE_TECH	
1 (src_tech) n TECHNOLOGY	src_tech dest_tech
1 (dest_tech) n TECHNOLOGY	

Serial Key None.

Foreign Keys / Link Description

src_tech Referenced to the **tech_key** in the **TECHNOLOGY** table. This field refers to the source technology.

dest_tech Referenced to the **tech_key** in the **TECHNOLOGY** table. This field refers to the destination technology.

A default value of **parent** = **0** indicates that the equipment has no parent.

Other Fields in EQUIPMENT

eq_name



Name of instantiation of particular piece of equipment.

Note that EQUIPMENT does not have a link to DEPARTMENT or FAB, thus eq_name must be unique throughout the fab. Some equipment (such as Vista Vision and Vista Logic testers) support unique serial numbers which can be used as eq_name. In all other cases, a site name must be prepended to any site internal equipment naming scheme.

Some standardization on the usage of this field - especially within one site - is desirable to allow easy data selection.

eq_level

Used to enable usage of **parent** to identify parent equipment. A value of 1 identifies parent equipment, 2 is a child, 3 a child of a child, etc. Default is 1.

For Metrology and LEH programs, this field will be filled with 101,102 or 103:

eq_level	Description
1	default (parent)
2 ...100	generation level (child, child of child, etc.)
101	MEQUIP for metrology
102	PEQUIP for metrology and LEH
103	MEQUIP and PEQUIP

12.2 EQUIP_TYPE

Abstract This table enables grouping of instantiations of **equipment**, managed in **EQUIPMENT**, into groups of identical type - such as one model of equipment.

Table

EQUIP_TYPE		
	eqt_key	
1 (ec_key) n	eqt_name ec_key ven_key pg_key	1 (eqt_key) n
EQUIP_CLASS		EQUIPMENT
1 (pg_key) n		1 (eqt_key) n
PROGRAM		PROC_STEP
1 (ven_key) n		1 (eqt_key) n
VENDOR		PROGRAM

Serial Key

eqt_key Automatically generated based on **eqt_name**.

Foreign Keys / Link Description

ec_key Each equipment type belongs to a class of equipment (such as testers, etc.) maintained in EQUIP_CLASS.

pg_key Link into PROGRAM for cases where the equipment firmware produces data independent of devices under process.

ven_key Links an equipment type to a vendor.

Other Fields in EQUIP_TYPE

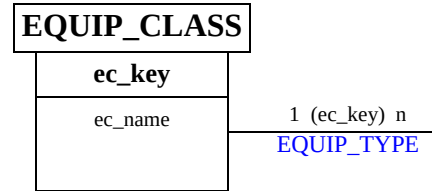
eqt_name Descriptive text per equipment type. The following are legal equipment types:

Value	User	Description
J953	Test	Teradyne J953
A575	Test	Teradyne A570, A570LC, A575
J273	Test	Teradyne J273A, J273B
Synchro_AC	Test	LTX Synchromaster AC
VV_1001	Test	Credence Vista Vision
VL_1001	Test	Credence Vista Logic
S-10	Test	Sentry S10
STS_6500	Test	Credence STS 6060 and 6500
SPEA	Test	SPEA MX300

12.3 EQUIP_CLASS

Abstract This table groups **equipment** into classes such as testers, probers, furnaces, etc.

Table



Serial Key

ec_key Automatically generated key based on **ec_name**.

Foreign Keys / Link Description None.

Other Fields in EQUIP_CLASS

ec_name Descriptive text for an equipment class. The following are legal values for **ec_name**:

Value	User	Description
Tester	Test	Test system (or part of it, such as test head)
Handler	Test	Handler
Prober	Test	Wafer prober
Probecard	Test	Wafer probe card
Loadboard	Test	Loadboard (both, device specific and standard)

12.4 PEOPLE

Abstract Employees, operators, engineers, etc., and related information.

Table

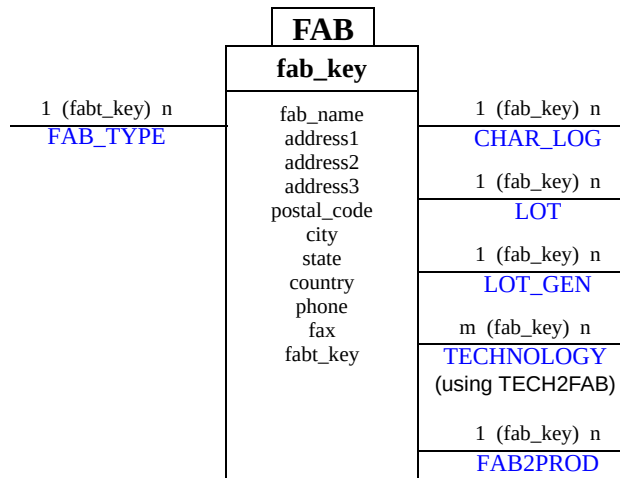
PEOPLE	
em_key	
name	1 (em_key) n
login	OP_LOG
access_level	1 (em_key) n
hire_date	EVENT_LOG
title	1 (em_key) n
role	PRODUCT
shift	1 (prod_engr) n
dep_key	PRODUCT
mailbox	1 (apps_engr) n
phone	PRODUCT
pager	1 (designer) n
email	PRODUCT
	1 (user1) n
	PRODUCT
	1 (user2) n
	PRODUCT
	1 (em_key) n
	CHAR_LOG
	1 (em_key) n
	DP_LOG
	1 (em_key) n
	LEH_LOG
	1 (em_key) n
	WEH_LOG
	1 (em_key) n
	MEM_LOG
	1 (em_key) n
	CV_LIMITS
	1 (em_key) n
	CV_MODEL
	1 (em_key) n
	PROGRAM_GROUP
	1 (em_key) n
	STEP_ORDER_SET
	1 (em_key) n
	DP_FOLDER
	1 (em_key) n
	DP_FILE
	1 (em_key) n
	DPMON_HOTLIST
	1 (em_key) n
	DPMON_TASK
	1 (em_key) n
	DPMON_USER_INFO
	1 (em_key) n
	DPMON_DASH_COMP
	1 (em_key) n
	DPMON_ANNOT
	1 (em_key) n
	DPMON_SCHEDULE_LOG
	1 (em_key) n
	AUTHORITIES

Serial Key	em_key	Automatically generated key based on name .
Foreign Keys / Link Description	dep_key	Relates PEOPLE to DEPARTMENT.
Other Fields in PEOPLE	name	Persons name.
	login	Login name.
	access_level	For future use, controls access to information beyond what can be achieved by database access privileges.
	hire_date	Date person is hired.
	title	Job title.
	role	Title description.
	shift	Example day or night shift.
	mailbox	Postal P.O. Box.
	phone	Telephone number.
	pager	Pager number.
	email	e-mail address.

12.5 FAB

Abstract Describes different Fab locations.

Table



Serial Key

fab_key Automatically generated key based on **fab_name**

Foreign Keys / Link Description

fabt_key From FAB_TYPE table. Identifies type of fab.

Other Fields in FAB

fab_name Name of fab.

address1, ...2, ...3 Fab addresses, up to three lines.

postal_code As relates to the address.

city As relates to the address.

state As relates to the address.

country As relates to the address.

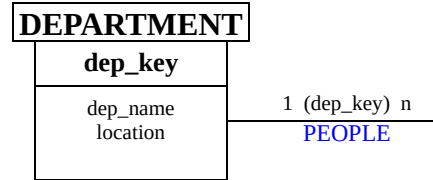
fax Fax number.

phone Telephone number.

12.6 DEPARTMENT

Abstract Departments, such as Test Engineering and Product Engineering.

Table



Serial Key **dep_key** Automatically generated key based on **dep_name**.

Foreign Keys / Link Description None.

Other Fields in DEPARTMENT

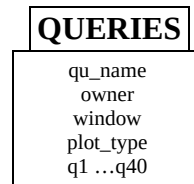
dep_name Department name.

location Department location.

12.7 QUERIES

Abstract Stores SQL queries edited by the user for later replay.

Table



Serial Key None.

Foreign Keys / Link Description None.

Other Fields in QUERIES

qu_name Query name

owner User that created the query.

window dataPOWER window. In the case of characterization data queries, this could be ROBUST or SHMOO.

plot_type The query chart type.

q1 ...q40 Fields used to store query.

12.8 DBINFO

Abstract This table is used to hold historical information on the database version(s) and date of creation (or update).

Table

DBINFO
version cdate waf_orient max_bins tool_date owner description por_level central_flag central_schema

Foreign Keys / Link Description None.

Serial Key None.

Other Fields in DBINFO

version	The database schema version number.
cdate	The date when the database was created or updated.
waf_orient	Determines the global wafer orientation in the database. Currently used only by Defect, <i>bitMAP</i> and aligned programs with wafermap configurations.
max_bins	Holds the number of bins the user enters to DpCreateDb, when prompted (32-950).
tool_date	Date/time stamp of the tool that created or updated the database.
owner	The user who ran the tool that created/updated the database.
description	Description of the update task.
por_level	
central_flag	Is the database a central schema? Y = yes; N = no.
central_schema	

12.9 PAR_GRP

Abstract This table stores the information about parameter groups to be retrieved and used by *dataPOWER* during the data retrieval from the database.

Table

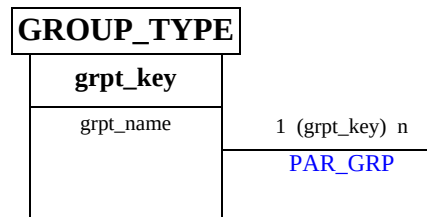
PAR_GRP		
grp_key		
1 (grpt_key) n GROUP_TYPE	grp_name	1 (grp_key) n PAR2PARGRP
	grpt_key	1 (grp_key) n T2T_REPORT
	owner	1 (grp_key) n CT_RULE_SET
		1 (grp_key) n CT_JOB
		1 (grp_key) n CV_DEF2TYPE_<lot_key>

Serial Key	grp_key	Automatically generated based on grp_name .
Foreign Keys / Link Description	grpt_key	A reference to the type of the parameter group.
Other Fields in PAR_GRP	grp_name	The name of the parameter group.
	owner	The name of the user (owner) of the parameter group.

12.10 GROUP_TYPE

Abstract This table holds the parameter group type info.

Table



Serial Key **grpt_key** Automatically generated based on grpt_name.

Foreign Keys / Link Description None.

Other Fields in GROUP_TYPE **grpt_name** The name of the parameter group type.

12.11 REPORTS

Abstract The users of *report*GENERATOR can save their report attributes in this table for convenience and fast generation of reports.

Table

REPORTS
rep_key
rep_name
rep_type
owner
trimpct
qxpct
qypct
filter_type
statcols
sortcols

Serial Key

rep_key Automatically generated based on **rep_name**.

Foreign Keys / Link Description None.

Other Fields in REPORTS

rep_name The name of the report as specified by the user.

rep_type The report type (e.g. Summary, DeltaCalc, ...etc.)

owner The user name (owner of the report).

trimpct The user specified value for Trim Mean.

qxpct The user specified quartile value.

qypct A second user specified quartile value.

filter_type The type of filter used:

Value	Description
1	Outlier Limits
2	What-If Limits
3	Specification Limits
4	Production Limits
5	Mean +/- nSigma
6	Mean +/- nSigma * IQR
7	Use All data

statcols A comma separated list of values representing the user-selected statistics. The values are the item numbers in the *report*GENERATOR dialog box in *dataPOWER*.

sortcols A comma separated list of values representing the order of the column as specified by the user.

12.12 ARCHIVE_LOG

Abstract This table is used by the *dpEXPORT* utility which is used to archive user specified lots of a user specified *program*.

Table

ARCHIVE_LOG		
1 (lg_key) n	lg_key	pg_key lot_key archive_time archive_level bring_back
OP_LOG		
1 (pg_key) n	pg_key	
PROGRAM		
1 (lot_key) n	lot_key	
LOT		

Serial Key None.

Foreign Keys / Link Description

lg_key The key of the OP_LOG which has been archived.

pg_key The key of the PROGRAM to which the archived lot belongs.

lot_key The key of the LOT which have been archived.

Other Fields in ARCHIVE_LOG

archive_time The date and time when the archive took place.

archive_level The archive level:

Value	Description
0	Raw data have been archived and both raw data and statistics (summaries) have been deleted from the database.
1	Raw data have been archived and deleted from the database, and statistics (summaries) remain in the database.
2	Raw data and statistics (summaries) have been deleted from the database
3	Raw data have been deleted from the database and statistics (summaries) remain in the database

bring_back Reserved for future use.

12.13 ARCHIVE_PLOG

Abstract This table is used by the *dpEXPORT* utility which is used to archive user specified lots of a user specified **program**.

Table

ARCHIVE_PLOG	
1 (lg_key) n OP_LOG	lg_key pg_key archive_time archive_level bring_back
1 (pg_key) n PROGRAM	
1 (lot_key) n LOT	

Serial Key None.

Foreign Keys / Link Description

lg_key The key of the OP_LOG which has been archived.

pg_key The key of the PROGRAM to which the archived lot belongs.

Other Fields in ARCHIVE_PLOG

archive_time The date and time when the archive took place.

archive_level The archive level:

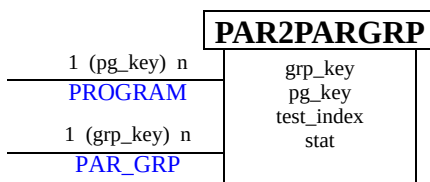
Value	Description
0	Raw data have been archived and both raw data and statistics (summaries) have been deleted from the database.
1	Raw data have been archived and deleted from the database, and statistics (summaries) remain in the database.
2	Raw data and statistics (summaries) have been deleted from the database
3	Raw data have been deleted from the database and statistics (summaries) remain in the database

bring_back Reserved for future use.

12.14 PAR2PARGRP

Abstract This table holds the relationship between individual programs and parameter groups.

Table



Serial Key None.

Foreign Keys / Link Description

grp_key A reference to the parameter group.

pg_key A reference to the program.

Other Fields in PAR2PARGRP

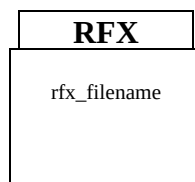
test_index The parameter index, i.e. in DEF..., the parameter name, a group of parameter names (using wild-cards), a formula using bin statistics (total failed/total passed).

stat The type of statistics to be calculated (e.g. “avg”, “q1”, “q2”,...etc.).

12.15 RFX

Abstract Used by *dataPOWER* for file tracking.

Table



Serial Key None.

Foreign Keys / Link Description

None.

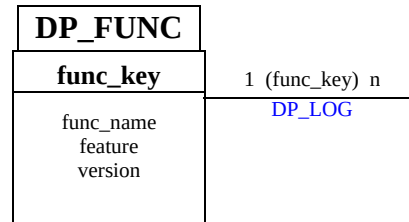
Other Fields in RFX

rfx_filename File name.

12.16 DP_FUNC

Abstract Maintains a list of *dataPOWER* functions that may be used in conjunction with the DP_LOG table for tracking function usage.

Table



Serial Key

func_key Automatically generated key from **func_name**.

Foreign Keys / Link Description

None.

Other Fields in DP_FUNC

func_name The name of the function in *dataPOWER*.

feature The name of the feature to which the called function belongs.

version Version of the function.

12.17 DP_LOG

Abstract Logs frequency of usage and users who call any of the dp_Extensions functions.

Table

DP_LOG		
	dplg_key	
1 (stdir_key) n STAGING_DIRS	stdir_key	1 (dplg_key) n META_TYPES
1 (rawfile_key) n RAW_FILE	rawfile_key	
1 (func_key) n DP_FUNC	func_key	
1 (option_key) n TOOL_OPTION	option_key	
1 (em_key) n PEOPLE	em_key	
1 (msg_key) n ERROR_MESSAGE	msg_key	
1 (supp_key) n	supp_key	
1 (fmt_key) n STAGE2STEP	fmt_key	
1 (pgc_key) n PROG_CLASS	pgc_key	
1 (db_key) n DATABASES	db_key	
	start_time	
	run_time	
	error_code	
	reload_flag	

Serial Key

dplg_key A unique key assigned to each entry in DP_LOG. It is stored automatically.

Foreign Keys / Link Description

stdir_key Foreign key from STAGING_DIRS.

rawfile_key Foreign key from RAW_FILE.

func_key Foreign key from DP_FUNC.

option_key Foreign key from TOOL_OPTION.

em_key Foreign key from PEOPLE.

msg_key Foreign key from ERROR_MESSAGE.

supp_key

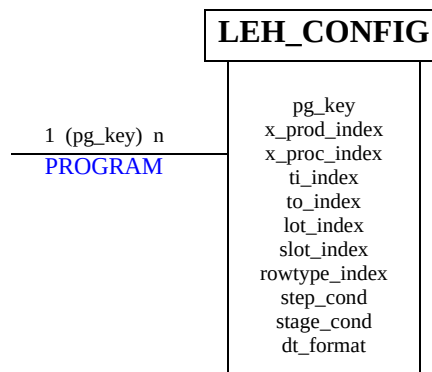
fmt_key Foreign key from FMT_FILE.

Other Fields in DP_LOG	pgc_key	Foreign key from PROG_CLASS.
	db_key	Foreign key from DATABASES.
	start_time	The time the function was executed.
	run_time	
	error_code	Error code value, given by the system.
	reload_flag	

12.18 LEH_CONFIG

Abstract Maintains information, extracted from the LEH format file, that describes the configuration of that format file with respect to the conditions and indexes.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key A reference to the PROGRAM table.

Other Fields in LEH_CONFIG

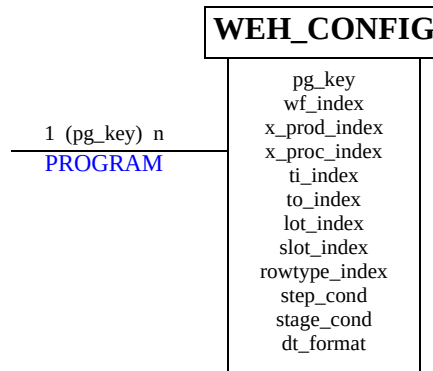
x_prod_index	Index number with product information. Will be deleted in future releases.
x_proc_index	Index number with process information. Will be deleted in future releases.
ti_index	Index number with track-in information.
to_index	Index number with track-out information.
lot_index	Index number with lot information.
slot_index	Index number with source lot information.
rowtype_index	Index number with row type information.

step_cond	Condition number with step information.
stage_cond	Condition number with stage information.
dt_format	Date-time format.

12.19 WEH_CONFIG

Abstract Maintains information, extracted from the WEH format file, that describes the configuration of that format file with respect to the conditions and indexes.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key A reference to the PROGRAM table.

Other Fields in WEH_CONFIG

wf_index	Index number with wafer information.
x_prod_index	Index number with product information. Will be deleted in future releases.
x_proc_index	Index number with process information. Will be deleted in future releases.
ti_index	Index number with track-in information.
to_index	Index number with track-out information.
lot_index	Index number with lot information.
slot_index	Index number with source lot information.
rowtype_index	Index number with row type information.
step_cond	Condition number with step information.
stage_cond	Condition number with stage information.
dt_format	Date-time format.

12.20 DATA_SET

Abstract Creation of reusable data set.

Table

DATA_SET	
dt_key	
dt_name	1 (dt_key) n
owner	T2T_REPORT
start_time	
end_time	1 (dt_key) n
nlots	CT_JOB
load_level	
period_value	1 (dt_key) n
period_type	CT_RULE_SET
period_offset	
every_n_lots	
which_time	

Serial Key

dt_key Serial key automatically generated.

Foreign Keys / Link Description

None.

Other Fields in LEH_CONFIG

dt_name Name of the data set.

owner Owner of the data set.

start_time Start time for the data range to be considered.

end_time End time for the data range to be considered.

nlots Number of last n lots.

load_level Level of data to be loaded. It can be:

Value	Description
L	Lot
W	Wafer
R	Raw

period_value Value of the relative period.

period_type Type of the relative period.
Possible values:

Value	Description
H	Hour
D	Day
W	Week
M	Month
Y	Year

period_offset Offset for the relative period. This field will be considered only if **period_value** and **period_type** fields are not empty. Default value is 0.



Any missing value will be substituted by retrieving all values, e.g. If the end_data was missing, then nlots will be retrieved until the time. The below table would explain how missing values are treated:

Absolute date/time table

Start_time	End_time	nlots	Description
SD	X	X	Consider all lots from start date SD till now
SD	ED	X	Consider all lots from start date SD till end date ED
SD	ED	N	Consider last N lots from start date SD till end date ED
SD	X	N	Consider last N lots from start date - now
X	ED	X	Consider all lots till end date ED
X	ED	N	Consider last N lots till end date ED
X	X	N	Consider last N lots
X	X	X	See next table (Relative date/time table)

Relative date/time table

Period_value	Period_type	period_offset	nlots	Description
Val	Type	X	X	Consider all lots for the period now - Val * Type (e.g. 5 * weeks)
Val	Type	Offset	X	Consider all lots for the period (now – Val * Type) - Offset (Offset has the same type as Type (e.g. 1 week)
Val	Type	X	N	Consider last N lots for the period now – Val * Type (e.g. 5 * weeks)
Val	Type	Offset	N	Consider last N lots for the period (now – Val * Type) - Offset (Offset has the same type as Type (e.g. 1 week)



*In the tables above:
X is the field is being left empty.
SD is the start_date
ED is the end_date*

every_n_lots Value for the “every *n*th lot” function — e.g. when value is 5, every 5th lot is selected.

which_time Specifies which time value to use from the OP_LOG table.
 Possible values:

Value	Description
start_time	start time
end_time	end time
insert_time	insertion time

12.21 BM_ARGS

Abstract Contains jobs names, types and owners, also contains different arguments related to scheduled jobs.

Table

BM_ARGS
bmargs_key
name job_type login n arg1...arg20

Serial Key **bmargs_key** Serial key automatically generated.

Foreign Keys / Link Description None.

Other Fields in BM_ARGS

name Name of the inserted job.

job_type Type of the inserted job (1: Tool2tool, 2: dataMINING).

login Login of the person who owns inserted job.

n Number of valid arguments.

arg1...arg20 Arguments, limited to 20.

12.22 DP_TEMP_RUN

Abstract DP_TEMP_RUN is used along with DP_TEMP_RUN_KEYS to temporarily hold intermediate sets of keys for front-end operations (also known as runs). Together, the two tables act as a temporary repository. Users will typically not need to access these tables. DP_TEMP_RUN holds information pertaining to the type of run being created and who created it.

Table

DP_TEMP_RUN
dpr_key
owner run_type
1 (dpr_key) n DP_TEMP_RUN_KEYS

Serial Key **dpr_key** The serial key generated by each use (run) of these scratch tables (DP_TEMP_RUN and DP_TEMP_RUN_KEYS).

**Foreign Keys / Link
Description**

None.

**Other Fields in
DP_TEMP_RUN****owner**

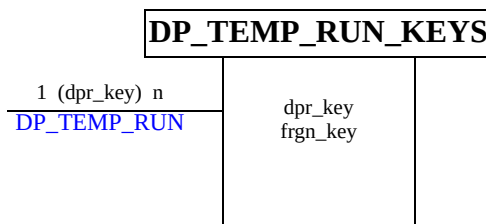
The name of the user that created the set of intermediate results (run).

run_type

An internal code to identify the type of run that was created.

12.23 DP_TEMP_RUN_KEYS**Abstract**

DP_TEMP_RUN_KEYS is used in conjunction with DP_TEMP_RUN to temporarily hold intermediate sets of keys for front-end operations.

Table**Serial Key**

None.

**Foreign Keys / Link
Description****dpr_key**

Foreign key from DP_TEMP_RUN to link the two tables together.

frgn_key

A generic holder to save any foreign key from any other table in the database.

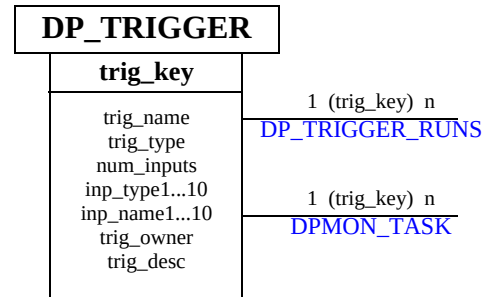
Other Fields in DP_TEMP_RUN_KEYS

None.

12.24 DP_TRIGGER

Abstract This table stores the definitions for triggers, from systems such as *dpMonitor*. Triggers are an automated method for launching *dpMonitor* jobs based on data loading events (values inserted into the DP_TRIGGER_RUNS table).

Table

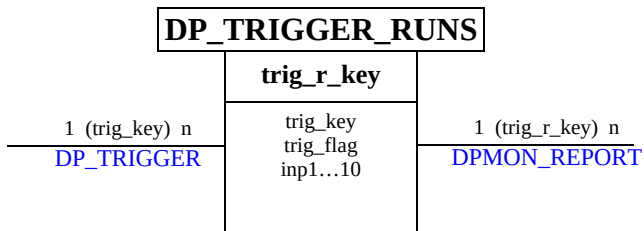


Serial Key	trig_key	Serial key.
Foreign Keys / Link Description	None.	
Other Fields in DP_TRIGGER	trig_name	The name of the trigger.
	trig_type	The user-defined trigger type.
	num_inputs	The number of trigger input parameters (integer between 1 and 10).
	inp_type1...10	The variable type, such as string, or integer, of each of the trigger inputs (the number of inputs defined in the num_inputs field).
	inp_name1...10	The name of each of the input parameters.
	trig_owner	The user name of the owner of the trigger.
	trig_desc	The description of the trigger.

12.25 DP_TRIGGER_RUNS

Abstract This table stores all instances of the triggers from DP_TRIGGER.

Table



Serial Key **trig_r_key** Serial key.

Foreign Keys / Link Description **trig_key** Foreign key from DP_TRIGGER.

Other Fields in DP_TRIGGER_RUNS

trig_flag The trigger status:

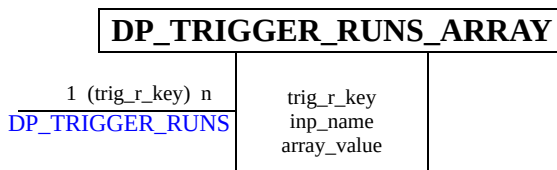
Value	Description
0	new
1	processed

inp...10 The values for each of the trigger input parameters as defined in dp_trigger.inp_name and dp_trigger.inp_type.

12.26 DP_TRIGGER_RUNS_ARRAY

Abstract Stores the distinct values for multi-value trigger run information.

Table



Serial Key None.

Foreign Keys / Link Description **trig_r_key** Foreign key from DP_TRIGGER_RUNS.

Other Fields in DP_TRIGGER_RUNS_ARRAY

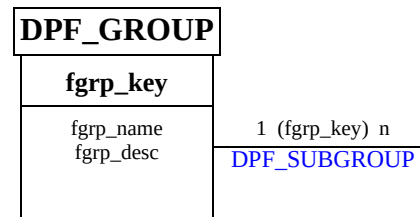
inp_name The trigger input name.

array_value The trigger array value.

12.27 DPF_GROUP

Abstract This table contains information about groups, which, in turn, may contain subgroups or files.

Table



Serial Key **fgrp_key** Serial key.

Foreign Keys / Link Description None.

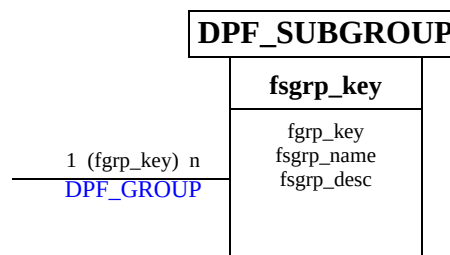
Other Fields in DPF_GROUP

fgrp_name	The name of the group as displayed in the interface
fgrp_desc	<i>For future use.</i>

12.28 DPF_SUBGROUP

Abstract This table contains information about subgroups, which contain files.

Table



Serial Key **fsgrp_key** Serial key.

Foreign Keys / Link Description **fgrp_key** From DPF_GROUP. The key to the group to which this subgroup belongs.

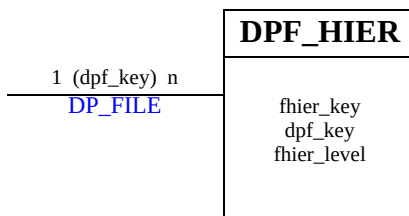
Other Fields in DPF_SUBGROUP

fsgrp_name	The name of the subgroup.
fsgrp_desc	<i>For future use.</i>

12.29 DPF_HIER

Abstract This table maintains the hierarchy of the file system, associating each file with a group, subgroup, or identifying the file as being on the root of the file system.

Table



Serial Key None.

Foreign Keys / Link Description

fhier_key This is the group key, if the file is saved under a group; or the subgroup key if the file is saved under a subgroup; and 0 if the file is saved at the root.

dpf_key From DP_FILE table.

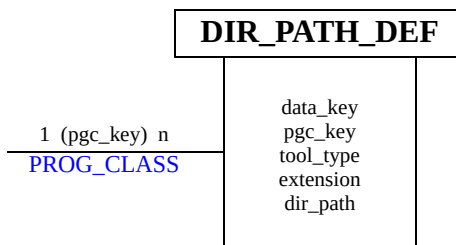
Other Fields in DPF_HIER

fhier_level The file level in the file system —
0: file at root
1: file under a group
2: file under a subgroup.

12.30 DIR_PATH_DEF

Abstract DIR_PATH_DEF is not currently used. It is intended for use in the future to allow front-end users to specify where and how to export certain types of data.

Table



Serial Key None.

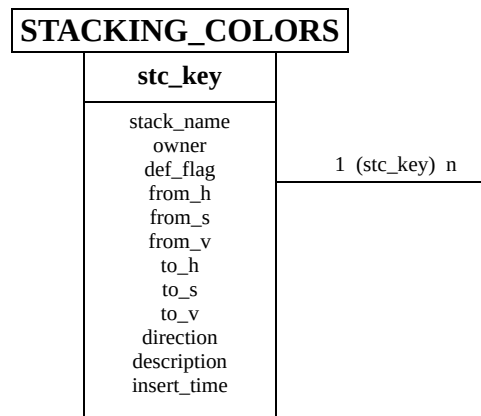
Foreign Keys / Link Description

data_key Generic foreign key from other database tables. The table from which this foreign key is referred from is determined by the tool type.

pgc_key Foreign key from PROG_CLASS table.

**Other Fields in
DIR_PATH_DEF**

tool_type	A string code that specifies which tool will use the row in this table.
extension	The three letter extension of the data file.
dir_path	The output directory into which the data files should be exported.

12.31 STACKING_COLORS**Abstract****Table****Serial Key****stc_key**

**Foreign Keys / Link
Description** None.

Other Fields in STACKING_COLORS**stack_name**

owner	The name/ID of the person who inserts this into the database.
def_flag	Name of the default.
from_h	
from_s	
from_v	
to_h	
to_s	

to_v	
direction	
description	A field to hold any user notes comments.
insert_time	The time of insertion of the file into the database. Date/time/year to the second, default is current year to the second.

12.32 RAW_PATH

Abstract This table stores the raw path, which is basically the first field of every line in a configuration file used by DpLoad.

Table

RAW_PATH	
path_key	1 (path_key) n
name_alias	RAW_FILE
name1	1 (path_key) n
name2	STAGING_DIRS
name3	
name4	1 (path_key) n
path_sequence	DPMON_APP

Serial Key **path_key** A unique key assigned to each entry in RAW_PATH.

Foreign Keys / Link Description None.

Other Fields in RAW_PATH

name_alias Alias for path name.

name1-4 Names concatenated to define path.

path_sequence Not currently used.

12.33 RAW_FILE

Abstract This table stores the data file name and the insertion time. The fully qualified filename is constructed by concatenating path names referenced by the rootpath_key, path_key, and the filename.

Table

RAW_FILE	
1 (path_key) n RAW_PATH	rawfile_key
1 (rootpath_key) n RAW_PATH	path_key rootpath_key file_name insert_time
	1 (rawfile_key) n DP_LOG
	1 (rawfile_key) n FILE2MEMCONFIG
	1 (rawfile_key) n MEM_RAW_FILE_DETAILS
	1 (rawfile_key) n MEM_RAW_FILE_DETAILS_TMP
	1 (rawfile_key) n MEM_FILE2SETUP

Serial Key	rawfile_key	A unique key assigned to each entry in RAW_FILE.
Foreign Keys / Link Description	path_key	A foreign key from the RAW_PATH table.
	rootpath_key	A foreign key from the RAW_PATH table.
Other Fields in RAW_FILE	file_name	The name of the file saved in the database.
	insert_time	The current date and time of insertion into the database. Date/time/year to the second. Defaults to current year to the second.

12.34 CLS_METHOD

Abstract Distinguishes different “methods” of lots and wafers. Classes are subsets of methods.

Table

CLS_METHOD	
method_name method_desc method_type	method_key
	1 (method_key) n WAFER_CLASS
	1 (method_key) n LOT_CLASS
	1 (method_key) n LOT2LOTGEN

Serial Key**method_key**

A unique key assigned to each entry in CLS_METHOD. It is stored automatically.

Foreign Keys / Link Description

None.

Other Fields in CLS_METHOD**method_name**

Method name.

method_desc

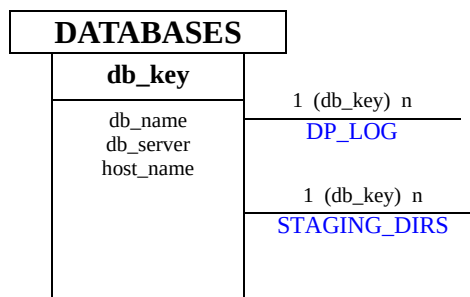
Method description.

method_type

Method type.

12.35 DATABASES**Abstract**

This table stores the database name, database server and host name being used.

Table**Serial Key****db_key**

A unique key assigned to each entry in DATABASES. It is stored automatically.

Foreign Keys / Link Description

None.

Other Fields in DATABASES**db_name**

Database name.

db_server

Database server name.

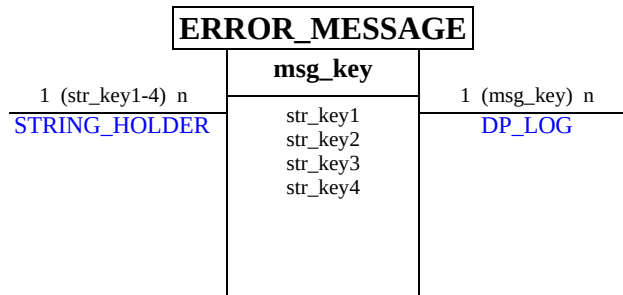
host_name

Host name.

12.36 ERROR_MESSAGE

Abstract This table stores the keys of the error message pieces that are stored in the STRING_HOLDER table. This table is similar to the TOOL_OPTIONS table.

Table



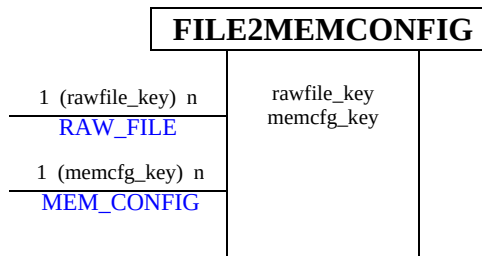
Serial Key **msg_key** A unique key assigned to each entry in ERROR_MESSAGE.

Foreign Keys / Link Description **STR_KEY1-4** A foreign key from the STRING_HOLDER table.

12.37 FILE2MEMCONFIG

Abstract This tables specifies which memory configurations to use for processing a given file.

Table



Serial Key None.

Foreign Keys / Link Description

rawfile_key Foreign key from RAW_FILE table.

memcfg_key Foreign key from the MEM_CONFIG table.

Other Fields in FILE2MEMCONFIG None.

12.38 FMT_FILE

Abstract This table stores the names of the format files.

Table

FMT_FILE	
fmt_key	1 (fmt_key) n
fmt_name	STAGING_DIRS

Serial Key **fmt_key** A unique key assigned to each entry in FMT_FILE.

Foreign Keys / Link Description None.

Other Fields in FMT_FILE **fmt_name** The name of the format file.

12.39 META_TYPES

Abstract

Table

META_TYPES	
meta_key	1 (meta_key) n
meta_name meta_desc	LIM_LOG
	1 (meta_key) n
	META_LOG

Serial Key **meta_key** A unique key assigned to each entry in META_TYPES. It is stored automatically.

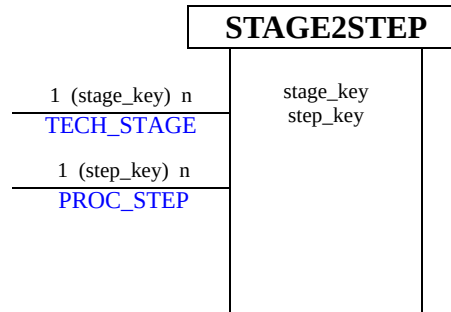
Foreign Keys / Link Description None.

Other Fields in META_TYPES **meta_name**
meta_desc

12.40 STAGE2STEP

Abstract This table contains information about the relationship between technology stages and process steps.

Table



Serial Key None.

Foreign Keys / Link Description

stage_key Foreign key from the TECH_STAGE table.

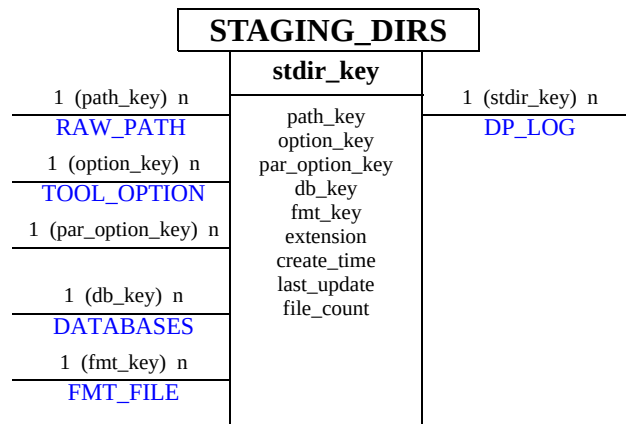
step_key Foreign key from the PROC_STEP table.

Other Fields in STAGE2STEP None.

12.41 STAGING_DIRS

Abstract This table stores the keys of the error message pieces that are stored in the STRING_HOLDER table. This table is similar to the TOOL_OPTIONS table.

Table



Serial Key **stdir_key** A unique key assigned to each entry in STAGING_DIRS. It is stored automatically.

**Foreign Keys / Link
Description**

path_key	A foreign key from the RAW_PATH table.
option_key	A foreign key from the TOOL_OPTION table.
par_option_key	A foreign key.
db_key	A foreign key from the DATABASES table.
fmt_key	A foreign key from the FMT_FILE table.
extension	The three letter extension for the data file type.
create_time	
last_update	
file_count	

**Other Fields in
STAGING_DIRS****12.42 STRING_HOLDER**

Abstract This table acts as a string placeholder for many tables. For example, it is used for the ERROR_MESSAGE and TOOL_OPTION tables. It is used to overcome the limitation of 255 characters per column.

Table

STRING_HOLDER	
str_key	1 (str_key) n
str_value	ERROR_MESSAGE
	1 (str_key) n
	TOOL_OPTION

Serial Key

str_key A unique key assigned to each entry in STRING_HOLDER. It is stored automatically.

**Foreign Keys / Link
Description**

None.

**Other Fields in
STRING_HOLDER****str_value**

13.0 Characterization Tables — Shmoo and Robustness

Abstract Pattern (group of vectors) generation tools create many more patterns than production testers can practically use; therefore, it is necessary to select the most robust patterns that will most exhaustively test device sensitivity. The *pattern*ROBUSTNESS tool provides the means for test engineers to select the most robust patterns.

Shmoo plots are routinely used by development and test engineers to evaluate digital devices under pattern test while one or more conditions are being varied. The *shmoo*PLOT tool is used for the analysis and display of shmoo data.

This section of the document contains description of the database tables that are used by the *pattern*ROBUSTNESS and *shmoo*PLOT products. Those tables are created by the DpCreateDb.pl utility when **-Char** switch is provided on the command line.

13.1 CHAR_LOG

Abstract This table holds most of the header information of shmoo and robustness data logs. Each data log read by the “robust” or “shmoo” readers has an entry here.

Table

CHAR_LOG		
	char_key	
1 (cpar_key) n	char_title	1 (char_key) n
CRITICAL_PARAM	char_cmt	PARAM_INFO
1 (datatype_key) n	data_desc	
DATA_TYPE	start_time	1 (char_key) n
1 (pg_key) n	end_time	ROBUST
PROGRAM	insert_time	
1 (pd_key) n	prog_load_date	
PRODUCT	invalid_flag	1 (char_key) n
1 (lot_key) n	cpar_key	ROB_FAIL
LOT	datatype_key	
1 (wf_key) n	pg_key	1 (char_key) n
WAFER	pd_key	LINE_DIST
1 (pk_key) n	lot_key	
PKG	wf_key	1 (char_key) n
1 (exp_key) n	pk_key	SHM_STATS
EXPERIMENT_STEP	exp_key	
1 (skew_key) n	skew_key	1 (char_key) n
FAB_SKEW	em_key	SIGN_PARETO
1 (em_key) n	fab_key	
PEOPLE	fabstep_key	1 (char_key) n
1 (fab_key) n	ftest_key	PIN_PARETO
FAB	patlist_key	
1 (fabstep_key) n	die_x	1 (char_key) n
FAB_STEPPING	die_y	SHMOO...
1 (ftest_key) n	unit_id	
FTEST	fuse_id	1 (char_key) n
1 (patlist_key) n	len	LINE...
PAT_LIST	lep	
1 (equip_ate) n	leff	1 (char_key) n
EQUIPMENT	equip_ate	FAIL...
1 (equip_interface) n	equip_interface	
EQUIPMENT	equip_temp	1 (char_key) n
1 (equip_temp)	equip_4	
EQUIPMENT	ate_ver	
1 (equip_4) n	ftemp_start	
EQUIPMENT	ftemp_end	
1 (pindef_key) n	mtemp_start	1 (char_key) n
TESTER_PIN_DEF	mtemp_end	
1 (pkgtype_key) n	vector_rev	
TESTER_PKG_TYPE	pindef_key	
1 (pre_pat_key) n	pkgtype_key	
PATTERN	timing_block	
	levels_block	
	pre_code	
	post_code	
	temp_meas_flag	
	test_pwr_down	
	pat_pwr_down	
	loops	
	burst_mode	
	compare_blocks	
	pre_pat_key	
	dimensions	
	transpose_flag	

Serial Key	char_key	Automatically generated.
Foreign Keys	cpar_key	The key to the critical parameter information. It is set by the shmoo reader only and it is left as NULL by the robust reader.
	datatype_key	The type of data log.
	pg_key	The key to the related program information.
	pd_key	The key to the related product information.
	lot_key	The key to the related lot information.
	wf_key	The key to the related wafer information.
	pk_key	The key to the related package info.
	exp_key	The key to the related experiment info.
	skew_key	The key to the related fab skew info.
	em_key	The key to the related employee info.
	fab_key	The key to the related fab info.
	fabstep_key	The key to the related fab stepping info.
	ftest_key	The key to the related ftest info.
	patlist_key	The key to the related pattern list info.
	equip_ate	The key to the related test equipment info.
	equip_interface	The key to the related interface equipment info.
	equip_temp	The key to the related temperature control equipment info.
	equip_4	Reserved for future use.
	pindef_key	The key to the related pin definition info.
	pkgtype_key	The key to the related package type info.
	pre_pat_key	The key to the related pre-test pattern info.
Other Fields in CHAR_LOG	char_title	The title as specified in the “Intention 1” item of the header section of the data log.

char_cmt	The comments as specified in the “Intention 2” item of the header section of the data log.
data_desc	The description as specified in the “Intention 3” item of the header section of the data log.
start_time	The start date and time of data collection as specified in “Test date” item of the header section of the data log.
end_time	The end date and time of data collection as specified in “Test date” item of the trailer section of the data log.
insert_time	The current date and time of insertion into the database.
prog_load_date	The date and time of loading the program as specified in “Program load date” item of the header section of the data log.
invalid_flag	The user can mark a characterization log as invalid by setting this flag to '1'.
die_x	The x-coordinate of a die under test as specified in “Die X” item of the header section of the data log.
die_y	The y-coordinate of a die under test as specified in “Die X” item of the header section of the data log.
unit_id	An identification of the unit being tested. Typically marked and/or scribed on the packaged part. This is specified in “Unit id” item of the header section of the data log.
fuse_id	Electrically programmed fuse elements (at wafer sort) can be read from the device under test. This is specified in “Fuse id” item of the header section of the data log.
len	N-channel transistor effective channel length as specified in “LEN” item of the header section of the data log.
lep	P-channel transistor effective channel length as specified in “LEP” item of the header section of the data log.
leff	An external value, if known, as specified in “LEFF” item of the header section of the data log.
ate_ver	Software version number of the tester/ATE manufacture's system being exercised as specified in “ASAP version” item of the header section of the data log.

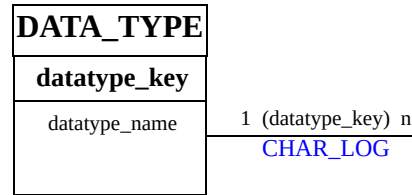
ftemp_start	Initial value of any temperature forcing device (e.g. wafer prober chuck, packaged part handler force temperature) as specified in “Force temperature” item of the header section of the data log.
ftemp_end	Initial value of any temperature forcing device (e.g. wafer prober chuck, packaged part handler force temperature) as specified in “Force temperature” item of the trailer section of the data log.
mtemp_start	Circuits on the device can report the die temperature sensed as specified in “Device temperature” item of the header section of the data log.
mtemp_end	Circuits on the device can report the die temperature sensed as specified in “Device temperature” item of the trailer section of the data log.
vector_rev	Tester pattern sets are distributed via a revision number. This number is critical for maintaining sanity of the test results gathered. This is specified in “Vector revision” item of the header section of the data log.
timing_block	The timing waveforms associated with the given Ftest as specified in “Timing block” item of the header section of the data log.
levels_block	The voltage levels associated with the given Ftest as specified in “Levels block” item of the header section of the data log.
pre_code	User function written code to execute before execution of the Ftest. This is specified in “Pre code” item of the header section of the data log.
post_code	User function written code to execute after execution of the Ftest. This is specified in “Post code” item of the header section of the data log.
temp_meas_flag	Determines whether temperature measurements are taken. This is specified in “Temperature measure” item of the header section of the data log. When the value is “on”, the flag will be '1'; otherwise, the flag will be '0'.
test_pwr_down	Determines whether the die or unit is powered down between consecutive Ftests. This is specified in “Per test power down” item of the header section of the data log. When the value is “on”, the flag will be '1'; otherwise, the flag will be '0'.

pat_pwr_down	Determines whether the die or unit is powered down between consecutive pattern tests. This is specified in “Per pattern power down” item of the header section of the data log. When the value is “on”, the flag will be '1'; otherwise, the flag will be '0'.
loops	For the robustness data, this sets the number of “test” sequences to perform (e.g. 1, 2, 3). This is specified in “Signature sequence” item of the header section of the data log. For the shmoo data, this is the number of unique values of the critical parameter. In the SHMOO... and LINE... tables, $t(n)$ is $t(\text{loops} * 3)$.
burst_mode	When all but the first pattern in a functional test require only a “short” reset pulse. This is specified in “Burst mode” item of the header section of the data log. When the value is “on”, the flag will be '1'; otherwise, the flag will be '0'.
compare_blocks	Pre-defined pins can be disabled during individual pattern testing. Record the fact “if any pattern” in the given list has a “compare block” associated with it. This is specified in “Compare blocks” item of the header section of the data log.
dimensions	Used for the shmoo data only. It holds the number of dimensions (number of parameters being swept) the shmoo data has. The “robust” reader will leave this field NULL.
transpose_flag	Used for the shmoo data only. When it is set to 'Y', it means the shmoo data has been transposed by the “shmoo” reader along the X-axis (critical parameter axis). Otherwise, it is set to 'N'. The “robust” reader will leave this field NULL.

13.2 DATA_TYPE

Abstract This table holds the different characterization data types.

Table



Serial Key **datatype_key** Automatically generated based on datatype_name.

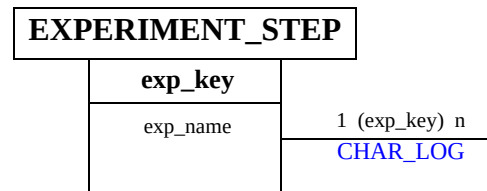
Foreign Keys / Link Description None.

Other Fields in DATA_TYPE **datatype_name** The name of the characterization type. This is specified in “Data type” item of the header section of the data log. The only valid values (up-to now) are:
 “robustness data-log”
 “shmoo data-log”
 “Line Shmoo data-log”

13.3 EXPERIMENT_STEP

Abstract Experiment step info. Some parts are “new”, “fib”-ed, temperature stressed, “latched up”, etc.

Table



Serial Key **exp_key** Automatically generated based on exp_name.

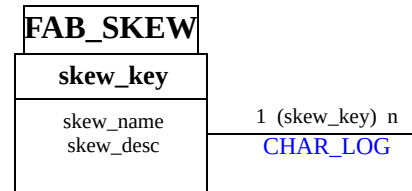
Foreign Keys / Link Description None.

Other Fields in EXPERIMENT_STEP **exp_name** This field is a short character string about the unit's “state”. This is specified in “Experiment step” item of the header section of the data log.

13.4 FAB_SKEW

Abstract The fab skew information. Typically a process corner of a unit or die.

Table



Serial Key **skew_key** Automatically generated based on skew_name.

Foreign Keys / Link Description None.

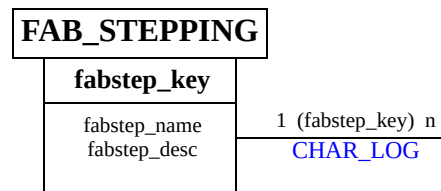
Other Fields in FAB_SKEW **skew_name** A short character string meaning “typ” (typical), “s” (slow), “f” (fast), “ff” (fast fast), etc. This is specified in “Fab skew” item of the header section of the data log.

skew_desc Any additional text description. Reserved for future use.

13.5 FAB_STEPPING

Abstract The fab stepping info. Which mask set is used for the product being tested.

Table



Serial Key **fabstep_key** Automatically generated based on fabstep_name.

Foreign Keys / Link Description None.

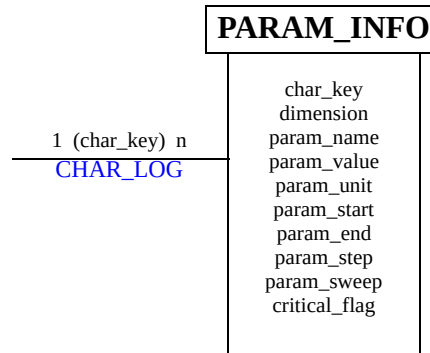
Other Fields in FAB_STEPPING **fabstep_name** A short char string (e.g. A0.. A9, B0.. B9, C0.. C9, ...). This is specified in “Fab stepping” item of the header section of the data log.

fabstep_desc Any additional text description. Reserved for future use.

13.6 PARAM_INFO

Abstract This table contain the information about the test parameters per characterization log.

Table



Serial Key None.

Foreign Keys / Link Description

char_key The characterization log which the parameter belongs to.

Other Fields in PARAM_INFO

dimension The order in which this parameter appears in the data log as “param name” item.

param_name The parameter name as specified in either “param name” or “Test” items in the header section of the data log. In the case of robustness data, this should also be the value of “param” in the data section of the data log.

param_value The parameter value as specified in either “param start” or “Test” items in the header section of the data log.

param_unit The parameter unit as specified in either “param start” or “Test” items in the header section of the data log following the parameter value.

param_start The swept-parameter's start value as specified in the “param start” item in the header section of the data log.

param_end The swept-parameter's end value as specified in the “param end” item in the header section of the data log.

param_step The swept-parameter's stepping value as specified in the “param step” item in the header section of the data log.

param_sweep Indicated whether the parameter is swept or not as specified in the “param sweep” item in the header section of the data log. When the value is “on”, the flag will be '1'; otherwise, the flag will be '0'.

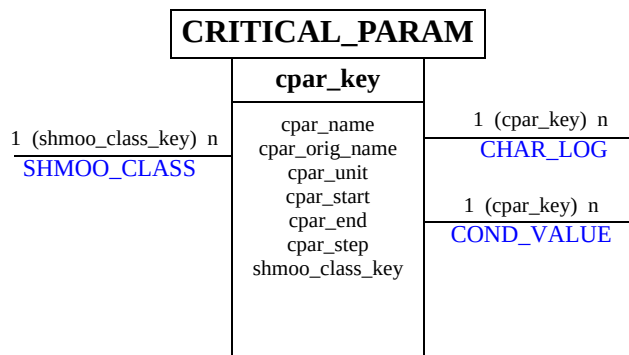
critical_flag **** With the shmoo data**, this flag indicates whether this parameter is the critical parameter which will be used as the X-axis parameter. It is the last parameter listed in the header section (the parameter in the most inner loop). The value would be either 'Y' for critical parameters or 'N' for others.

**** With the robust data**, this flag indicates whether the swept parameter has (min,max) results or (min,max,avg) results. The value would be '0' for (min,max) and '1' for (min,max,avg).

13.7 CRITICAL_PARAM

Abstract This table is used only by the shmoo reader and the shmoo analysis package. It contains information regarding the critical parameter, where the critical parameter is defined as the last parameter listed in the header section of the data log. This parameter is placed on the x-axis of the shmoo plot.

Table



Serial Key	cpar_key	Automatically generated based on cpar_name.
Foreign Keys / Link Description	shmoo_class_key	A reference to the shmoo class.
Other Fields in CRITICAL_PARAM	cpar_name	A unique name built using other fields in this table. It has the following format: “cpar_orig_name (cpar_start,cpar_end,cpar_step)” (e.g. “vcc (3.0, 6.0, 0.1)”).
	cpar_orig_name	The original name of the parameter as specified in the “param name” item of the header section of the data log.
	cpar_unit	The parameter unit that follows the parameter value as specified in the “param start” item of the header section of the data log.
	cpar_start	The parameter start value as specified in the “param start” item of the header section of the data log.

cpar_end	The parameter end value as specified in the “param end” item of the header section of the data log.
cpar_step	The parameter step value as specified in the “param step” item of the header section of the data log.

13.8 SHMOO_CLASS

Abstract There are, currently, two type of shmoo data. The first type is the standard shmoo data where one or more parameters get swept. The second type is the pattern based shmoo where a single parameter is swept across many patterns. This table holds the possible classes (types) of the shmoo data.

Table

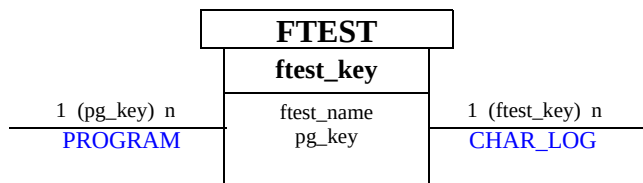
SHMOO_CLASS	
shmoo_class_key	
shmoo_class_name	1 (shmoo_class_key) n CRITICAL_PARAM

Serial Key	shmoo_class_key	Automatically generated based on shmoo_class_name.
Foreign Keys / Link Description	None.	
Other Fields in SHMOO_CLASS	shmoo_class_name	The name of the shmoo class. The only valid values are: “Pattern-Based Shmoo” and “Standard Shmoo”.

13.9 FTEST

Abstract This table holds information about the functional test block in the test program.

Table



Serial Key **fctest_key** Automatically generated based on **fctest_name**.

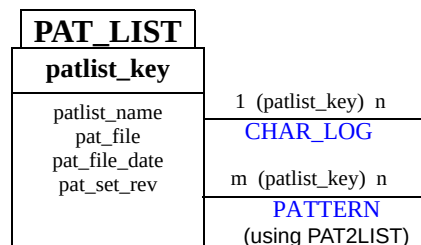
Foreign Keys / Link Description **pg_key** A link to the program which contains the test.

Other Fields in FTEST **fctest_name** The name of the functional test block.

13.10 PAT_LIST

Abstract This table contains information about the list of patterns used.

Table



Serial Key **patlist_key** Automatically generated based on patlist_name.

Foreign Keys / Link Description None.

Other Fields in PAT_LIST **patlist_name** The name of the pattern list used by Ftest as specified in the “Pattern list” item of the header section of the data log.

pat_file The path name to a pattern list file as specified in the “Pattern file” item of the header section of the data log.

pat_file_date Reserved for future use.

pat_set_rev Reserved for future use.

13.11 PATTERN

Abstract This table holds the relevant information about the patterns (test vectors). In the case of robustness data, the entries in this table are for each pattern tested. In the case of the shmoo data, the entries in this table correspond to the pattern info when the test result is “fail”.

Table

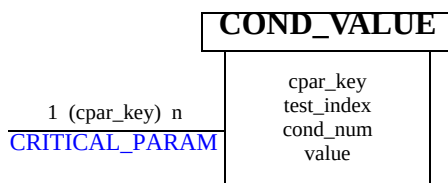
PATTERN		
pat_key		
1 (patttype_key) n PAT_TYPE	pat_name	1 (pat_key) n PAT_STATUS
	full_name	
1 (patmode_key) n PAT_MODE	vrev	1 (pat_key) n ROBUST
	patttype_key	
	patmode_key	1 (pat_key) n ROB_FAIL
	vector_cnt	1 (pre_pat_key) n CHAR_LOG
	cycle_cnt	1 (pat_key) n SHMOO...
		1 (pat_key) n LINE...
		1 (pat_key) n FAIL...
		m (pat_key) n PAT_LIST (using PAT2LIST)

Serial Key	pat_key	Automatically generated based on pat_name.
Foreign Keys / Link Description	patttype_key	A reference to the pattern type.
	patmode_key	A reference to the pattern mode.
Other Fields in PATTERN	pat_name	The first 8 characters of the full_name assuming this part is unique for every pattern.
	full_name	The full pattern name as specified in the “pattern name” item of the header section of the data log.
	vrev	Tester pattern sets are distributed via a revision number. This number is critical for maintaining sanity of the test results gathered. This is specified in the “Vector revision” item of the header section of the data log.
	vector_cnt	Reserved for future use.
	cycle_cnt	Reserved for future use.

13.12 COND_VALUE

Abstract This table is used by the shmoo reader and analysis tools only. It contains the values of the critical parameter as swept on the x-axis. The rest of the parameters values are stored as dim1 and dim2 in the SHMOO... tables.

Table



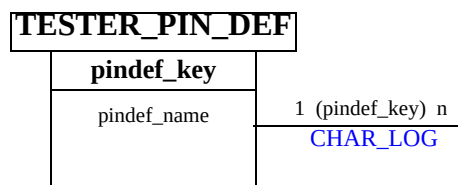
Serial Key None.

Foreign Keys / Link Description	cpar_key	A reference to the critical parameter info.
Other Fields in COND_VALUE	test_index	the parameter index (i.e. t1.. tn) in the SHMOO... tables.
	cond_num	Reserved for future use.
	value	the parameter value at each test_index .

13.13 TESTER_PIN_DEF

Abstract This table contains information regarding the tester pin definition.

Table



Serial Key **pindef_key** Automatically generated based on **pindef_name**.

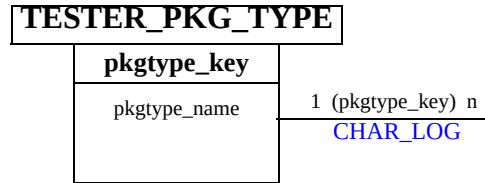
Foreign Keys / Link Description None.

Other Fields in TESTER_PIN_DEF **pindef_name** A short char string contains a list of “package types” and “pin groups”. This is specified in the “Tester pin definition” item of the header section of the data log.

13.14 TESTER_PKG_TYPE

Abstract This table contains information regarding the tester package type.

Table



Serial Key **pkgtype_key** Automatically generated based on pkgtype_name.

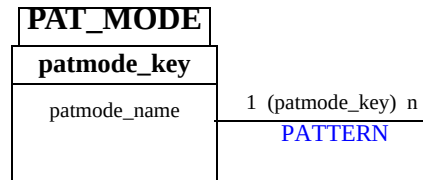
Foreign Keys / Link Description None.

Other Fields in TESTER_PGK_TYPE **pkgtype_name** A short char string contains a tester channel and pin name mapping. This is specified in the “Tester package type” item of the header section of the data log.

13.15 PAT_MODE

Abstract This table contains information regarding the tester specific layout of vectors of a pattern.

Table



Serial Key **patmode_key** Automatically generated based on patmode_name.

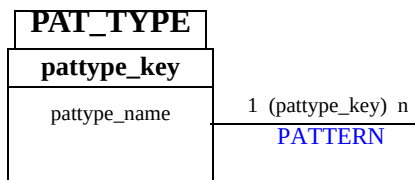
Foreign Keys / Link Description None.

Other Fields in PAT_MODE **patmode_name** The name of the pattern layout mode.

13.16 PAT_TYPE

Abstract This table contains information regarding the internal layout of vector opcodes in a pattern.

Table



Serial Key **patype_key** Automatically generated based on pattype_name.

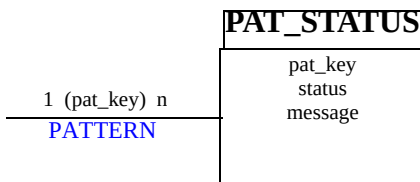
Foreign Keys / Link Description None.

Other Fields in PAT_TYPE **pattype_name** The name of the pattern layout type.

13.17 PAT_STATUS

Abstract This table is reserved for future use. It contains pattern status and user defined messages per pattern.

Table



Serial Key None.

Foreign Keys / Link Description **pat_key** A reference to the pattern.

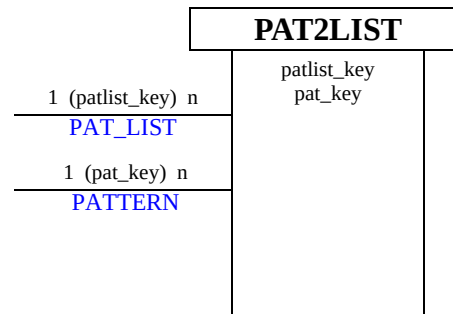
Other Fields in PAT_STATUS **status** A short string describing the status of the pattern (e.g. "INVALID", "Remapped", etc.).

message A user specified message associated with the pattern.

13.18 PAT2LIST

Abstract This table contains information about the relationship between patterns and the pattern list.

Table



Serial Key None.

Foreign Keys / Link Description

patlist_key Foreign key from the PAT_LIST table.

pat_key Foreign key from the PATTERN table.

Other Fields in PAT2LIST None.

13.19 ROB_FAIL

Abstract This table is used by the “robust” reader and the robustness analysis tools only. It contains a collection of information generated every time the pattern test result is “fail”. The list of failed pattern information is grouped by the characterization log (char_key).

Table

ROB_FAIL		
	rfail_key	
1 (char_key) n	char_key	1 (seq1) n
CHAR_LOG	pat_key	ROBUST
	pat_index	1 (seq2) n
1 (pat_key) n	sequence	ROBUST
PATTERN	cycle	1 (seq3) n
	rma	ROBUST
	sub	1 (seq4) n
	channels	ROBUST
	sim_curr	1 (seq5) n
	sim_next	ROBUST
	sim_prev	ROBUST

Serial Key

rfail_key Automatically generated serial number.

Foreign Keys / Link Description

char_key A reference to the characterization log which the fail-info belongs to.

pat_key A reference to the pattern that failed.

Other Fields in ROB_FAIL

pat_index The index number of the pattern in the list of all patterns. This is specified in the “pattern index” item of the data section of the data log.

sequence When the pattern is applied multiple times, the sequence of tests is stored in this field. This is specified in the “sequence” item of the data section of the data log.

cycle This value is the actual tester cycle for a failing pattern. This is specified in the “tester cycle” item of the data section of the data log.

rma Relative Memory Address (relative to 0). This is specified in the “rma” item of the data section of the data log.

sub If a pattern fails in a tester's subroutine memory, this is the absolute memory address in subroutine memory. This is specified in the “sub” item of the data section of the data log.

channels	Tester channel list of signals failing a vector. This is specified in the “fail channel” item of the data section of the data log.
sim_curr	The current simulation clock cycles as specified in the “sim_curr” item the data section of the data log.
sim_next	The current simulation clock cycles as specified in the “sim_next” item the data section of the data log.
sim_prev	The current simulation clock cycles as specified in the “sim_prev” item the data section of the data log.

13.20 ROBUST

Abstract This table is used by the “robust” reader and the robustness analysis tools only. It contains information and results of each test irrespective of whether the test passed or failed.

ROBUST	
1 (char_key) n	char_key
CHAR_LOG	pat_key
1 (pat_key) n	pat_index
PATTERN	burst
1 (seq1) n	compare
ROB_FAIL	chain
1 (seq2) n	loop
ROB_FAIL	keep_alive
1 (seq3) n	temp
ROB_FAIL	sign
1 (seq4) n	seq1
ROB_FAIL	seq2
1 (seq5) n	seq3
ROB_FAIL	seq4
	seq5
	sweep_flag
	min1
	min1_flag
	max1
	max1_flag
	avg1
	min2
	min2_flag
	max2
	max2_flag
	avg2
	min3
	min3_flag
	max3
	max3_flag
	avg3
	min4
	min4_flag
	max4
	max4_flag
	avg4
	min5
	min5_flag
	max5
	max5_flag
	avg5

Serial Key None.

Foreign Keys / Link Description

char_key	A reference to the characterization log which contain this data.
pat_key	A reference to the pattern under test.
seq1	The key to the fail info if test 1 failed.
seq2	The key to the fail info if test 2 failed.
seq3	The key to the fail info if test 3 failed.

**Other Fields in
ROBUST**

seq4	The key to the fail info if test 4 failed.
seq5	The key to the fail info if test 5 failed.
pat_index	The index number of the pattern in the list of all patterns. This is specified in the “pattern index” item of the data section of the data log.
burst	Burst mode as specified in the “burst” item of the data section of the data log. When burst is “on”, this flag is '1'; otherwise it is '0'.
compare	The compare flag as specified in the “compare” item of the data section of the data log. When compare is “on”, this flag is '1'; otherwise it is '0'.
chain	The chain flag as specified in the “chain” item of the data section of the datalog. When chain is “on”, this flag is '1'; otherwise it is '0'.
loop	The loop flag as specified in the “loop” item of the data section of the data log. When loop is “on”, this flag is '1'; otherwise it is '0'.
keep_alive	The keep_alive flag as specified in the “keep_alive” item of the data section of the data log. When keep_alive is “on”, this flag is '1'; otherwise it is '0'.
temp	The measured temperature as specified in the “temperature” item of the data section of the data log.
sign	The cumulative signature of all the tests. (e.g. “FpF” means the first test failed, the second test passed and the third test failed).
sweep_flag	The sweep_flag flag as specified in the “sweep” item of the data section of the data log. When sweep is “on”, this flag is '1'; otherwise it is '0'.
min1 ... min5	The minimum value of the n(th) parameter being swept (where n is 1 ... 5) as specified in the “min” item of the data section of the data log.
min1_flag ... min5_flag	An indicator of whether the parameter value reached the end or not. This could be either '+' or '-' if specified after the min value.

max1 ... max5 The maximum value of the n(th) parameter being swept (where n is 1 ... 5) as specified in the “max” item of the data section of the data log.

max1_flag ... max5_flag

An indicator of whether the parameter value reached the end or not. This could be either '+' or '-' if specified after the max value.

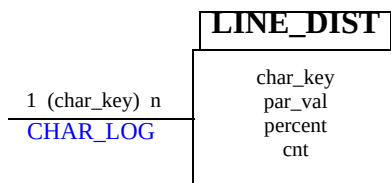
avg1 ... avg5

The average value of the n(th) parameter being swept (where n is 1 ... 5) as specified in the “avg” item of the data section of the data log.

13.21 LINE_DIST

Abstract This table is used by the “shmoo” reader and the shmoo analysis tools only. It contains the count and percentage of patterns which first failed at the specified critical parameter value.

Table



Serial Key None.

Foreign Keys / Link Description

char_key A reference to the corresponding characterization log.

Other Fields in LINE_DIST

par_val The critical parameter value (x-axis) at which the first-fail patterns are counted.

percent The percent of patterns failed at par_val.

cnt The count (number) of patterns failed at par_val.

13.22 SHM_STATS

Abstract This table is used by the “shmoo” reader and the shmoo analysis tools only. It contains the basic statistics of the critical parameter (x-axis) values where patterns first failed.

Table

SHM_STATS	
	char_key
	holes
	avg
	std
	q1
	q2
	q3
	min_val
	max_val
	cnt
	p1
	p5
	p10
	p90
	p95
	p99
	row_cnt

1 (char_key) n
[CHAR_LOG](#)

Serial Key None.

Foreign Keys / Link Description

char_key A reference to the corresponding characterization log.

Other Fields in SHM_STATS

holes The number of holes found. A hole is when a pattern passes then fails then passes again.

avg The average value of critical parameter (x-axis) where the patterns started to fail.

std The standard deviation value of critical parameter (x-axis) where the patterns started to fail.

q1 The first quartile value of critical parameter (x-axis) where the patterns started to fail.

q2 The second quartile value of critical parameter (x-axis) where the patterns started to fail.

q3 The third quartile value of critical parameter (x-axis) where the patterns started to fail.

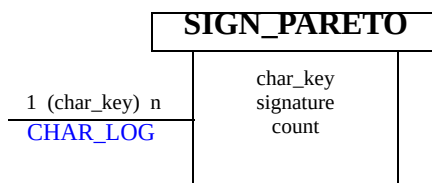
min_val The minimum value of critical parameter (x-axis) where the patterns started to fail.

max_val	The maximum value of critical parameter (x-axis) where the patterns started to fail.
cnt	The number of critical parameter (x-axis) values used in calculating the statistics.
p1	The 1% value of critical parameter (x-axis) where the patterns started to fail.
p5	The 5% value of critical parameter (x-axis) where the patterns started to fail.
p10	The 10% value of critical parameter (x-axis) where the patterns started to fail.
p90	The 90% value of critical parameter (x-axis) where the patterns started to fail.
p99	The 99% value of critical parameter (x-axis) where the patterns started to fail.
row_cnt	The number of rows in the SHMOO... table.

13.23 SIGN_PARETO

Abstract This table is used by the “robust” reader and robustness analysis tools only. It holds the count of repeated signature for various signatures per characterization log. The signatures are the test results when a pattern is tested multiple times in sequence. For example, “pFp” is a signature of a pattern passed the first test, failed the second test, and passed the third test.

Table



Serial Key None.

Foreign Keys / Link Description

char_key A reference to the characterization log.

Other Fields in SIGN_PARETO

signature The test signature (e.g. “pFp”).

count The number of times the above signature appeared in the above characterization log.

13.24 PIN_PARETO

Abstract This table holds the count (number of times) a certain pin (channel) has failed per characterization log. It can handle up to 500 pins.

Table

PIN_PARETO	
1 (char_key) n	char_key unique_flag p1 ... p500
CHAR_LOG	

Serial Key None.

Foreign Keys / Link Description

char_key A reference to the characterization log

Other Fields in PIN_PARETO

unique_flag **For robustness data**, this flag is either 'Y' or 'N'. When the flag is 'Y', that means the failed pin is counted only once per pattern. For example, if the pattern is tested five times and the same pin failed three times, then only one failure is counted for that pin. When the flag is 'N', that means anytime a pin fails gets counted. For example, if the pattern is tested five times and the same pin failed three times, then all three failures are counted for that pin.

For shmoo data, this flag is either 'S' or 'C'. When the flag is 'S', that means the failed pins are counted across the whole characterization log. When the flag is 'C', that means the failed pins are counted at the contour only. The contour is defined as the first-fail or last-pass point of the parameter under test.

p1 ... p500 The count of failures per pin.

13.25 HEADER_INFO

Abstract This tables is used to give the user the flexibility of setting which items of the header section of the characterization log wants to be included in the reports. Those item flags can be set using the characterization interface in *dataPOWER*.

Table

HEADER_INFO

name
 load_date
 test_date
 sys_id
 site
 interface
 login
 intention1
 intention2
 intention3
 pd_name
 exp_step
 fab_skew
 fab_step
 len
 lep
 leff
 unit_id
 fuse_id
 pkg_name
 lot_num
 waf_num
 die_x
 die_y
 force_temp
 dev_temp
 temp_ctrl
 asap
 test_pg
 vector_rev
 tester_pin
 tester_pkg
 ftest
 pre_code
 post_code
 timing_blk
 levels_blk
 pat_list
 pat_file
 pre_pat
 temp_meas
 pre_test_down
 pre_pat_down
 params

Serial Key None.

Foreign Keys / Link Description None.

Other Fields in HEADER_INFO

name The name associated with the flag settings.

(All Other Flags) The rest of the fields are flags with either 0 or 1 values. When the flag is 0, it is turned off. When it is 1, it is turned on. By default, all the flags are turned on.

13.26 SHMOO...

Abstract This table is used by the “shmoo” reader and the shmoo analysis tools only. It contains the standard shmoo data (1,2,or 3 dimensions) where all the swept parameters are numeric. The table name is based on the cpar_key value of CRITICAL_PARAM table. For example, if the cpar_key is 5, then the table name would be “shmoo5”.

Table

SHMOO...	
1 (char_key) n CHAR_LOG	char_key
1 (pat_key) n PATTERN	pat_key first_fail last_pass dim1 dim2
1 (t3, t6, t9, ...etc.) n FAIL...	hole_flag t1...tn

Serial Key None.

Foreign Keys / Link Description

char_key A reference to the characterization log.

pat_key A reference to the pattern info.

t3, t6, t9, ...etc. A reference to the FAIL... table.

Other Fields in SHMOO...

first_fail The critical parameter's value (x-axis) when the other parameters (y-axis, or z-axis) failed for the first time.

last_pass The critical parameter's value (x-axis) when the other parameters (y-axis, or z-axis) passed for the last time.

dim1 The value of the second parameter (y-axis); if available.

dim2 The value of the third parameter (z-axis); if available.

hole_flag If 'Y', that means there is at least one hole in test results. A hole is defined as a test failure between test passes (e.g. “pFp”, “ppppFFppp” have holes).

t1...tn The results are stored in a group of three (PASS_FLAG, TEMPERATURE, FAIL_KEY) for each value of the critical parameter (x-axis). Where, PASS_FLAG is 1 if the test passed, 0 if the test failed, and NULL if the test was not performed. The TEMPERATURE is the measured temperature whether the test passed or failed. The FAIL_KEY is a reference to the FAIL... table if the test is passed, otherwise it is NULL. For example:

t1	t2	t3	t4	t5	t6
-----	-----	-----	-----	-----	-----
1	13.6	(NULL)	0	20.4	123

For the first critical parameter value, the test passed where the temperature was 13.6. For the second critical parameter value, the test failed where the temperature was 20.4 and the fail_key value is 123.

13.27 LINE...

Abstract This table is used by the “shmoo” reader and the shmoo analysis tools only. It contains the line shmoo (pattern-based) data (2 dimensions) where the swept parameter is numeric on the x-axis the y-axis has patterns being tested. The table name is based on the cpar_key value of CRITICAL_PARAM table. For example, if the cpar_key is 12, then the table name would be “line12”.

Table

LINE...	
1 (char_key) n CHAR_LOG	char_key pat_key first_fail last_pass rank hole_flag t1...tn
1 (pat_key) n PATTERN	
1 (t3, t6, t9, ...etc.) n FAIL...	

Serial Key None.

Foreign Keys / Link Description

char_key A reference to the characterization log.

pat_key A reference to the pattern info.

t3, t6, t9, ...etc. A reference to the FAIL... table.

Other Fields in LINE

first_fail The critical parameter's value (x-axis) when the pattern (y-axis) failed for the first time.

last_pass The critical parameter's value (x-axis) when the pattern (y-axis) passed for the last time.

rank The rank is a number which represent the order in which the first fail has happened with respect to the other patterns. If multiple patterns have the same fail location, the ranking is assigned in sequence from top to bottom. For example:

pat1	pppFFFFF	rank = 3
pat2	pFFFFFFF	rank = 1
pat5	pppppppF	rank = 5

pat3 ppFFFFFF rank = 2

pat4 pppFFFFFF rank = 4

pat6 ppppppppp rank = NULL

hole_flag

If 'Y', that means there is at least one hole in test results. A hole is defined as a test failure between test passes (e.g. “pFp”, “ppppFFppp” have holes).

t1...tn

The results are stored in a group of three (PASS_FLAG, TEMPERATURE, FAIL_KEY) for each value of the critical parameter (x-axis). Where, PASS_FLAG is 1 if the test passed, 0 if the test failed, and NULL if the test was not performed. The TEMPERATURE is the measured temperature whether the test passed or failed. The FAIL_KEY is a reference to the FAIL... table if the test is passed, otherwise it is NULL.

For example:

t1	t2	t3	t4	t5	t6
-----	-----	-----	-----	-----	-----
1	13.6	(NULL) 0		20.4	123

for the first critical parameter value, the test passed where the temperature was 13.6. For the second critical parameter value, the test failed where the temperature was 20.4 and the fail_key value is 123.

13.28 FAIL...

Abstract This table is used by the “shmoo” reader and the shmoo analysis tools only. It contains the data produced when a test has failed. The table name is based on the cpar_key value of CRITICAL_PARAM table. For example, if the cpar_key is 20, then the table name would be “fail20”.

Table

FAIL...		
fail_key		
1 (pg_key) n PROGRAM	pg_key char_key fail_type dim1 dim2 dim3 pat_key tester_cycle rma sub fail_channel mtemp	1 (t3, t6, t9, ...etc.) n SHMOO...
1 (char_key) n CHAR_LOG		1 (t3, t6, t9, ...etc.) n LINE...
1 (pat_key) n PATTERN		

Serial Key

fail_key Automatically generated serial number.

Foreign Keys / Link Description

pg_key A reference to the program table.

char_key A reference to the characterization log.

pat_key A reference to the pattern info.

Other Fields in FAIL

fail_type Either 'C' or 'F'. When it is 'C' it means the failure happened at the contour line. When it is 'F', it means the failure happened anywhere else.

dim1 The value of the first parameter (x-axis).

dim2 The value of the second parameter (y-axis); if available (2D or 3D shmoo).

dim3 The value of the third parameter (z-axis); if available (3D shmoo).

tester_cycle This value is the actual tester cycle for a failing pattern. This is specified in the “tester cycle” item of the data section of the data log.

rma Relative Memory Address (relative to 0). This is specified in the “rma” item of the data section of the data log.

sub	If a pattern fails in a tester's subroutine memory, this is the absolute memory address in subroutine memory. This is specified in the “sub” item of the data section of the data log.
fail_channel	Tester channel list of signals failing a vector. This is specified in the “fail channel” item of the data section of the data log.
mtemp	The measured temperature when the test failed.

14.0 Lot Genealogy Tables

Lot Genealogy Tables describe the genealogy of processed lots. They document the different genealogy events (i.e. merge, split, and/or rename) that each lot has gone through.

14.1 LOT_GEN

Abstract This table describes the details of each genealogy event. Each genealogy event is described by either one row (i.e. rename) or multiple rows (i.e. split or merge) in this table.

Table

LOT_GEN		
	lgen_key	
1 (gevt_key) n LOT_GEVENT	gevt_key	1 (lgen_key) n WAF2LOTGEN
1 (lty_key) n LOT_TYPE	lty_key	
1 (from_lty_key) n LOT_TYPE	from_lty_key	
1 (pd_key) n PRODUCT	pd_key	
1 (lot_key) n LOT	lot_key	
1 (fab_key) n FAB	fab_key	
	from_lot	
	from_fab	
	src_lot	
	src_fab	
	event_type	
	waf_count	
	from_waf_count	
	part_cnt	
	from_part_cnt	
	lot_code	
	from_prod	

Serial Key

lgen_key Uniquely identifies an entry in LOT_GEN.
Automatically generated out of the unique combination of gevt_key, lot_key, and from_lot

Foreign Keys / Link Description

gevt_key From LOT_GEVENT table. Uniquely identifies a genealogy event.

lty_key From LOT_TYPE table. Identifies type of output lot.

from_lty_key From LOT_TYPE table. Identifies type of input lot.

pd_key Foreign key from PRODUCT. Identifies output product.

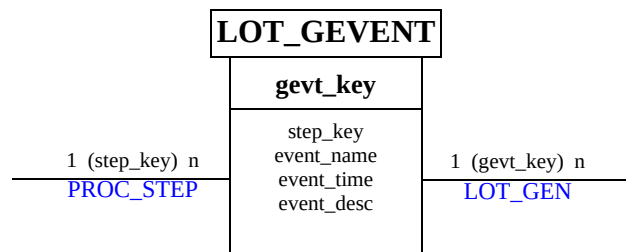
lot_key Foreign key from LOT. Identifies output lot.

**Other Fields in
LOT_GEN**

fab_key	Foreign key from FAB. Identifies output fab.
from_lot	Links into LOT table to identify input lot.
from_fab	Links into FAB table to identify input fab.
src_lot	Reference to source lot in LOT table, to identify input source lot.
src_fab	Links into FAB table to identify the fab of the input source lot.
event_type	The type (i.e. split, merge, or rename) of the genealogy event.
waf_count	The count of the wafers in the output lot.
from_waf_count	The count of the wafers in the input lot.
part_cnt	The count of the parts in the output lot.
from_part_cnt	The count of the parts in the input lot.
lot_code	Code of the output lot.
from_prod	Links into PRODUCT table, to identify input product.

14.2 LOT_GEVENT

Abstract LOT_GEVENT lists each genealogy event. Each row is a unique event.

Table**Serial Key**

gevt_key Uniquely identifies an entry in LOT_GEVENT. Automatically generated for every unique event_name.

**Foreign Keys / Link
Description**

step_key Foreign key from PROC_STEP. Uniquely identifies the process step where the genealogy event occurred.

**Other Fields in
LOT_GEVENT**

event_name The unique name of the genealogy event.

event_time The time stamp when the genealogy event occurred.

event_desc Description of the genealogy event.

14.3 WAF2LOTGEN

Abstract WAF2LOTGEN lists the wafers of each output lot associated with a genealogy event.

Table

WAF2LOTGEN	
1 (lgen_key) n	wf_key lot_key step_key lgen_key
LOT_GEN	
1 (wf_key) n	
WAFER	
1 (lot_key) n	
LOT	
1 (step_key) n	
PROC_STEP	

Serial Key None.

Foreign Keys / Link Description

lgen_key Foreign key from LOT_GEN table. Identifies the related genealogy event.

wf_key Foreign key from WAFER. Identifies the wafer.

lot_key Foreign key from LOT. Identifies the lot.

step_key Foreign key from PROC_STEP. Uniquely identifies the process step where the genealogy event occurred.

Other Fields in WAF2LOTGEN None.

14.4 FAB_TYPE

Abstract This table describes the type of a fab.

Table

FAB_TYPE	
fabt_key	
fabt_name fabt_desc	1 (fabt_key) n FAB

Serial Key **fabt_key** Uniquely identifies an entry in FAB_TYPE.
Automatically generated for every unique fabt_name.

Foreign Keys / Link Description None.

Other Fields in FAB_TYPE

fabt_name The name of fab type.

fabt_desc The description of the fab type.

14.5 LOT_TYPE

Abstract This table describes the type of a lot.

Table

LOT_TYPE	
lty_key	
lot_type	1 (lty_key) n LOT_GEN
	1 (from_lty_key) n LOT_GEN
	1 (lty_key) n LOT2TYPE

Serial Key **lty_key** Uniquely identifies an entry in LOT_TYPE.
Automatically generated for every unique lot_type.

Foreign Keys / Link Description None.

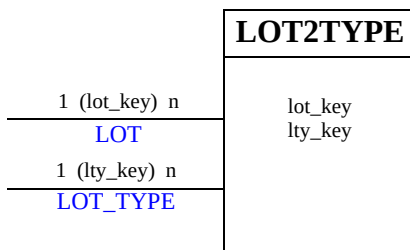
Other Fields in LOT_TYPE

lot_type The name of the lot type.

14.6 LOT2TYPE

Abstract LOT2TYPE is an m2n table holding all the available combinations of lty_keys and lot_keys.

Table



Serial Key None.

Foreign Keys / Link Description

lot_key Foreign key from LOT to identify the lot.

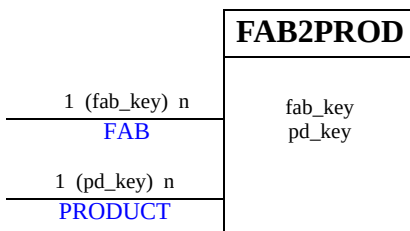
lty_key From LOT_TYPE table to identify type of lot.

Other Fields in LOT2TYPE None.

14.7 FAB2PROD

Abstract FAB2PROD is an m2n table holding all the available combinations of fab_keys and pd_keys.

Table



Serial Key None.

Foreign Keys / Link Description

fab_key Foreign key from FAB to identify the fab.

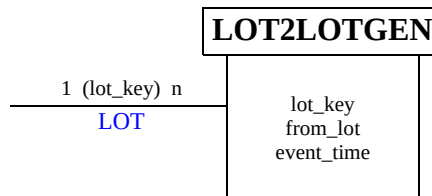
pd_key Foreign key from PRODUCT to identify the product.

Other Fields in FAB2PROD None.

14.8 LOT2LOTGEN

Abstract LOT2LOTGEN is an m2n table holding all the available combinations of lot_keys of lots linked by a genealogy event.

Table



Serial Key None.

Foreign Keys / Link Description

lot_key	From the LOT table to identify output lot.
----------------	--

Other Fields in LOT2LOTGEN

from_lot	Links into LOT table to identify input lot.
event_time	The time stamp when the genealogy events occurred.

15.0 *bit*MAP Tables

*bit*MAP tables are used to store raw bit information. These tables describe the portion of the schema that holds the *bit*MAP fail bit pattern data, and related fail pattern definitions and templates.

15.1 MEM_META_CLASS

Abstract MEM_META_CLASS contains meta class definitions for various bit failure patterns, for the purpose of broadly classifying memory cell failing patterns. MEM_META_CLASS is more general than MEM_CLASS, representing a family of memory pattern fail classes. MEM_META_CLASS is created and filled by DpCreateDb.pl, not by the reader. This table is part of the *bit*MAP application logic, and contents should not be modified.

Table

MEM_META_CLASS	
mmc_key	
mmc_name	
comp_flag	1 (mmc_key) n
tf_flag	MEM_CLASS
mind_flag	
maxd_flag	
rt_flag	1 (mmc_key) n
ct_flag	MEM_MCLASS_COLL
bw_flag	
rf_flag	
cf_flag	1 (mmc_key) n
bitg_flag	MEM_PI

Serial Key

mmc_key Formed from a unique mmc_name.

Foreign Keys / Link Description

None.

Other Fields in MEM_META_CLASS

mmc_name Memory meta class name, unique within the database.

comp_flag Indicates if the fail pattern is composite or not.
Y = composite meta class; N = not a composite.

tf_flag Reserved for future use (column twisting).

mind_flag Indicates if **min_density** column in the MEM_CLASS table is used by classes of that meta class.

maxd_flag Indicates if **max_density** column in the MEM_CLASS table is used by classes of that meta class.

rt_flag	Indicates if row_thickness and max_row_count columns in the MEM_CLASS table are used by classes of that meta class.
ct_flag	Indicates if col_thickness and max_col_count columns in the MEM_CLASS table are used by classes of that meta class.
bw_flag	Indicates if border_width column in the MEM_CLASS table is used by classes of that meta class.
rf_flag	Indicates if row_flag column in the MEM_CLASS table is used by classes of that meta class.
cf_flag	Indicates if col_flag column in the MEM_CLASS table is used by classes of that meta class.
bitg_flag	Indicates if bitg_flag column in the MEM_CLASS table is used by classes of that meta class.

15.2 MEM_CLASS

Abstract MEM_CLASS contains predefined memory cell fail pattern classes — class definitions for various bit failure patterns, for the purpose of classifying memory cell failing patterns. A set of predefined rows of MEM_CLASS is created and filled by DpCreateDb.pl, not by the reader. Additional classes are created by the configuration GUI.

For example, the meta class, **Row**, includes the memory classes **Row1** and **Row1Par**. So **Row1** and **Row1Par** are two different memory classes that belong to the same family — **Row**. Each memory class has its own properties that can be found in the MEM_CLASS table. However, two different memory classes can inherit common behavior from the meta class that both of them belong to. For example: both MEM_CLASS and MEM_META_CLASS contain the **bitg_flag** field. This means that for all memory classes in MEM_CLASS that belong to the same family or the same meta class, this **bitg_flag** field is only valid if **bitg_flag** is equal to **Y** in the meta class.

Table

MEM_CLASS	
1 (mmc_key) n MEM_META_CLASS	mc_key mc_name comp_flag mmc_key mc_level twisted_flag min_density max_density row_thickness col_thickness max_col_count max_row_count border_width row_flag col_flag bitg_flag fail_bit_count patt_key
	1 (mc_key) n MEM_CLS2CLS
1 (patt_key) n MEM_BIT_PATT	1 (mc_key) n MEM_MCL2MC
	1 (mc_key) n MEM_ROWS
	1 (mc_key) n MEM_BOX
	1 (mc_key) n MEM_CUSTOM
	1 (mc_key) n MEM_COLS
	1 (mc_key) n MEM_MISS
	1 (mc_key) n MEM_SPOT
	1 (mc_key) n MEM_CROSS
	1 (mc_key) n MEM_FOG
	1 (mc_key) n MEM_MULTI_ROW
	1 (mc_key) n MEM_MULTI_COL
	1 (mc_key) n MEM_CROSS_BITS
	1 (mc_key) n MEM_LAYER2PAT
	1 (mc_key) n MEM_MC2MCGRP
	1 (mc_key) n MEM_MULTI_ROW_BITS
	1 (mc_key) n MEM_MULTI_COL_BITS
	Links to MEM_BOP_... Related Tables ►

Links to
MEM_BOP_...
Related Tables

MEM_CLASS	
...	1 (mc_key) n MEM_BOP_MULTI_ROW
	1 (mc_key) n MEM_BOP_ROWS
	1 (mc_key) n MEM_BOP_COLS
	1 (mc_key) n MEM_BOP_BOX
	1 (mc_key) n MEM_BOP_CUSTOM
	1 (mc_key) n MEM_BOP_MISS
	1 (mc_key) n MEM_BOP_CROSS
	1 (mc_key) n MEM_BOP_CUSTOM
	1 (mc_key) n MEM_BOP_FOG
	1 (mc_key) n MEM_BOP2PAT
	1 (mc_key) n MEM_BOP_CUSTOM

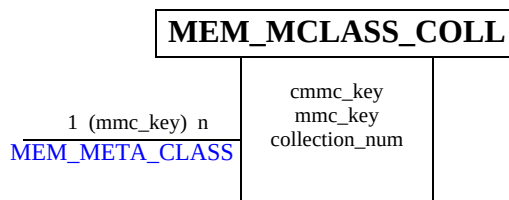
Serial Key	mc_key	A unique, non-zero entry, created from mc_name .
Foreign Keys / Link Description	mmc_key	Foreign key from MEM_META_CLASS. Refers to the meta-class this class belongs to.
	patt_key	Foreign key from MEM_BIT_PATT, used only for spot patterns; null otherwise.
Other Fields in MEM_CLASS	mc_name	Unique memory class name.
	comp_flag	Indicates whether the fail pattern class is composite or not. Y = composite meta class (cross, fog, multi_row, multi_column); N = not composite.
	mc_level	Level can be memory or block — M: memory or B: Block
	twisted_flag	Reserved for future use.
	min_density	Minimum fail bit density required for pattern matching.

max_density	Maximum fail bit density required for pattern matching.
row_thickness	For spot meta-class, number of rows which a given pattern contains. Otherwise, it means minimum number of rows.
col_thickness	For spot meta-class, number of columns which a given pattern contains. Otherwise, it means minimum number of columns.
max_col_count	For a given fail class, the maximum number of columns which a given pattern contains.
max_row_count	For a given fail class, the maximum number of rows which a given pattern contains.
border_width	Minimum required border width around the pattern, in bits, within which it is required that no failed bits be present.
row_flag	Indicates parity of the starting row origin of the pattern: 0 = all, 1 = odd, 2 = even.
col_flag	Indicates parity of the starting column origin of the pattern: 0 = all, 1 = odd, 2 = even.
bitg_flag	Bit groups flag. For box meta-class, indicates whether or not bit groups are used for pattern matching. Y = yes; N = no.
fail_bit_count	For spot patterns, it stores the number of failing bits. Valid if patt_key is not null.

15.3 MEM_MCLASS_COLL

Abstract For composite metaclasses, this table contains a list of possible metaclasses to be used.

Table



Serial Key

None.

Foreign Keys / Link Description

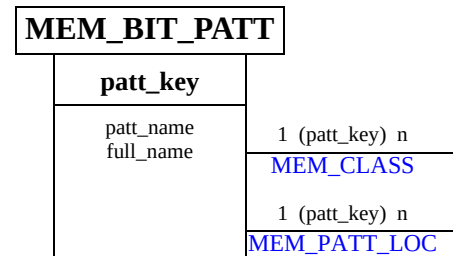
cmmc_key	Foreign key from MEM_META_CLASS. It references a component of composite meta class.
mmc_key	Foreign key from MEM_META_CLASS.

Other Fields in MEM_MCLASS_COLL

collection_num	0 = column; 1 = row.
-----------------------	----------------------

15.4 MEM_BIT_PATT

Abstract This table stores information for the pattern used to define spot pattern classes.

Table**Serial Key**

patt_key Serial key.

Foreign Keys / Link Description

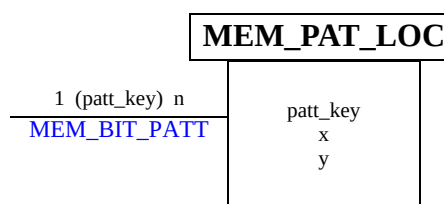
None.

Other Fields in MEM_BIT_PATT

patt_name	Unique name of pattern; typically the same as the spot pattern class.
full_name	Complete name of pattern.

15.5 MEM_PATT_LOC

Abstract For a given spot pattern, this table contains a list of failing bit positions within the pattern template.

Table

Serial Key None.

Foreign Keys / Link Description **patt_key** Foreign key from MEM_BIT_PATT. Identifies which rows in the MEM_PATT_LOC table are in that spot pattern.

Other Fields in MEM_PAT_LOC

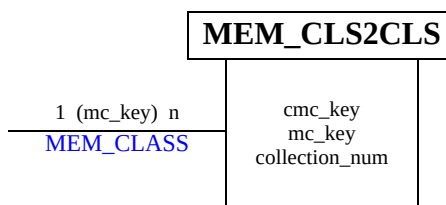
x X position of the failing bit in the pattern.

y Y position of the failing bit in the pattern.

15.6 MEM_CLS2CLS

Abstract For a given composite class from the MEM_CLASS table, this tables contains a list of classes used in its collections.

Table



Serial Key None.

Foreign Keys / Link Description **cmc_key** Foreign key from MEM_CLASS that references a composite class.

mc_key Foreign key from MEM_CLASS that references a component of a composite class.

Other Fields in MEM_CLS2CLS

collection_num 0 for a column, or 1 for a row.

15.7 MEM_MCLIB

Abstract This table contains a list of unique pattern set names.

Table

MEM_MCLIB	
mcl_key	
mcl_name	1 (mcl_key) n MEM_MCL2MC
	1 (mcl_key) n MEM_LIB2PROD
	1 (mcl_key) n MEM_LOG
	1 (mcl_key) n MEM_DEF2MEM
	1 (mcl_key) n MEM_UPSTAT
	1 (mcl_key) n MEM_LAYER2PAT
	1 (mcl_key) n MEM_BOP2PAT
	1 (mcl_key) n MEM_BOP_UPSTAT

Serial Key **mcl_key** Serial key.

Foreign Keys / Link Description None.

Other Fields in MEM_MCLIB **mcl_name** Unique name of pattern set.

15.8 MEM_MCL2MC

Abstract This table contains a list of pattern recognition stages for a given pattern set.

Table

MEM_MCL2MC	
1 (mcl_key) n MEM_MCLIB	mcl_key mcg_key mc_key destruct rec_order
1 (mcg_key) n MEM_CLASS_GROUP	
1 (mc_key) n MEM_CLASS	

Serial Key None.

Foreign Keys / Link Description

mcl_key Foreign key from MEM_MCLIB.

mcg_key Foreign key from MEM_MEM_CLASS_GROUP.

mc_key Foreign key from MEM_CLASS.

Other Fields in MEM_MCL2MC

destruct Mode of recognition: destruct or non-destruct.

rec_order Order of recognition.

15.9 MEM_MCGRP

Abstract This table contains a list of unique pattern group names.

Table

MEM_MCGRP	
1 (gl_key) n MEM_MCGRPLIB	gp_key gl_key gp_name
	1 (gp_key) n MEM_MC2MCGRP

Serial Key **gp_key** Serial key.

Foreign Keys / Link Description

gl_key Foreign key from MEM_MCGRPLIB.

Other Fields in MEM_MCGRP

gp_name Unique name of group.

15.10 MEM_MCGRPLIB

Abstract This table contains information about pattern groups.

Table

MEM_MCGRPLIB	
gl_key	
gl_name owner description insert_time def_flag	1 (gl_key) n MEM_MCGRP

Serial Key

gl_key Serial key.

Foreign Keys / Link Description None.

Other Fields in MEM_MCGRPLIB

gl_name Unique name of group library.

owner Name/ID of the person who created the record.

description Text field to enter comments for the record.

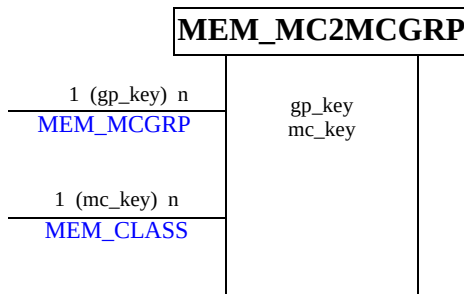
insert_time Time of data insertion into the database, defaults to the current time.

def_flag Flag indicating if this is “default.” Legal values are **Y** and **N**.

15.11 MEM_MC2MCGRP

Abstract This table defines which pattern classes belong to each group.

Table



Serial Key None.

Foreign Keys / Link Description

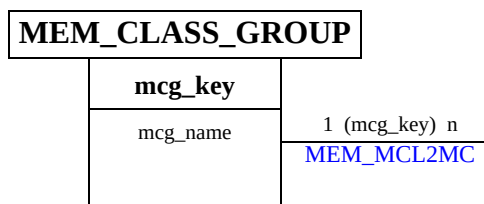
gp_key	Foreign key from MEM_MCGRP table.
mc_key	Foreign key from MEM_CLASS.

Other Fields in MEM_MC2MCGRP None.

15.12 MEM_CLASS_GROUP

Abstract This table is reserved for future use.

Table



Serial Key **mcg_key** Serial key.

Foreign Keys / Link Description None.

Other Fields in MEM_CLASS_GROUP **mcg_name** Name of meta class group.

15.13 MEM_LIB2PROD

Abstract This table associates a pattern set with a products and memory designs.

Table

MEM_LIB2PROD	
1 (pd_key) n PRODUCT	pd_key
1 (mdsn_key) n MEM_CELL_CLASS	mdsn_key
1 (mcl_key) n MEM_MCLIB	mcl_key
	is_default

Serial Key None.

Foreign Keys / Link Description

pd_key Foreign key from PRODUCT.

mdsn_key Foreign key from MEM_CELL_CLASS.

mcl_key Foreign key from MEM_MCLIB.

Other Fields in MEM_LIB2PROD

is_default Indicates that given pattern set is the default for a given product-memory design combination.

15.14 MEM_PI

Abstract This table associates a unique pattern ID with its memlog_key and metaclass.

When a new pattern instance is created, and by its meta-class its table established, then a new row is inserted into the MEM_PI table and this generated mi_key is used as a unique identifier of this pattern instance.

Table

MEM_PI	
mi_key	
1 (memlog_key) n MEM_LOG	1 (mi_key) n
	MEM_ROWS
	1 (mi_key) n
	MEM_BOP_ROWS
	1 (mi_key) n
	MEM_BOX
	1 (mi_key) n
	MEM_BOP_BOX
	1 (mi_key) n
	MEM_SPOT
	1 (mi_key) n
	MEM_BOP_SPOT
	1 (mi_key) n
	MEM_CROSS
	1 (mi_key) n
	MEM_CROSS_BITS
	1 (mi_key) n
	MEM_BOP_CROSS
	1 (mi_key) n
	MEM_FOG
	1 (mi_key) n
	MEM_BOP_FOG
	1 (mi_key) n
	MEM_MISS
	1 (mi_key) n
	MEM_BOP_MISS
	1 (mi_key) n
	MEM_COLS
	1 (mi_key) n
	MEM_MULTI_COL
	1 (mi_key) n
	MEM_BOP_COLS
	1 (mi_key) n
	MEM_MULTI_COL_BITS
	1 (mi_key) n
	MEM_BOP_MULTI_COL
	1 (mi_key) n
	MEM_MULTI_ROW
	1 (mi_key) n
	MEM_BOP_MULTI_ROW
	1 (mi_key) n
	MEM_MULTI_ROW_BITS

Serial Key	mi_key	A unique key for each pattern instance.
Foreign Keys / Link Description	memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.
Other Fields in MEM_PI	None.	

15.15 MEM_ROWS

Abstract This table contains a list of pattern instances of row meta-type.

Table

MEM_ROWS	
1 (mrd_key (mi_key)) n	mrd_key
MEM_PI	memlog_key
1 (memlog_key) n	memcfg_key
MEM_LOG	mc_key
1 (memcfg_key) n	membk_key
MEM_CONFIG	row_num
1 (mc_key) n	fail_count
MEM_CLASS	min_col
1 (membk_key) n	max_col
MEM_BLK_CFG	thickness

Serial Key	None.
Foreign Keys / Link Description	mrd_key Foreign key from MEM_PI table (using mi_key). memlog_key Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader. memcfg_key Foreign key from MEM_CONFIG. mc_key Foreign key from MEM_CLASS. membk_key Foreign key from MEM_BLK_CFG.
Other Fields in MEM_ROWS	row_num Row number. fail_count Fail bit count. min_col Column of first failing bit. max_col Column of last failing bit. thickness Number of rows.

15.16 MEM_BOX

Abstract This table contains a list of pattern instances of box meta-type.

Table

MEM_BOX	
1 (mbx_key (mi_key)) n	mbx_key
MEM_PI	memlog_key
1 (memlog_key) n	memcfg_key
MEM_LOG	mc_key
1 (memcfg_key) n	membld_key
MEM_CONFIG	min_col
1 (mc_key) n	min_row
MEM_CLASS	max_col
1 (membld_key) n	max_row
MEM_BLK_CFG	fail_count
	invert_fg

Serial Key None.

Foreign Keys / Link Description

mbx_key	Foreign key from MEM_PI table (using mi_key).
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS.
membld_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOX

min_col	Column of first failing bit.
min_row	Row of first failing bit.
max_col	Column of last failing bit.
max_row	Row of last failing bit.
fail_count	Total fail count in box.
invert_fg	Reserved for future use.

15.17 MEM_CUSTOM

Abstract This table contains a list of pattern instances of custom meta-type.

Table

MEM_CUSTOM	
1 (memlog_key) n MEM_LOG	memlog_key
1 (memcfg_key) n MEM_CONFIG	memcfg_key
1 (mc_key) n MEM_CLASS	mc_key
1 (memblk_key) n MEM_BLK_CFG	memblk_key
	min_col
	min_row
	max_col
	max_row
	fail_count

Serial Key None.

Foreign Keys / Link Description **memlog_key** Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.

memcfg_key Foreign key from MEM_CONFIG.

mc_key Foreign key from MEM_CLASS.

memblk_key Foreign key from MEM_BLK_CFG.

Other Fields in MEM_CUSTOM

min_col Column of first failing bit.

min_row Row of first failing bit.

max_col Column of last failing bit.

max_row Row of last failing bit.

fail_count Total fail count in box.

15.18 MEM_COLS

Abstract This table contains a list of pattern instances of column meta-type.

Table

MEM_COLS		
1 (mcd_key (mi_key)) n	mcd_key memlog_key memcfg_key memblk_key mc_key col_num fail_count min_row max_row thickness	1 (mcd_key) n
MEM_PI		MEM_MULTI_COL_BITS
1 (memlog_key) n		1 (mrd_key) n
MEM_LOG		MEM_MULTI_ROW_BITS
1 (memcfg_key) n		
MEM_CONFIG		
1 (mc_key) n		
MEM_CLASS		
1 (memblk_key) n		
MEM_BLK_CFG		

Serial Key None.

Foreign Keys / Link Description

mcd_key	Foreign key from MEM_PI table (using mi_key).
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS.
memblk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_COLS

col_num	Column number.
fail_count	Fail bit count.
min_row	Row of first failing bit.
max_row	Row of last failing bit.
thickness	Number of columns.

15.19 MEM_MISS

Abstract This table contains a list of pattern instances of “other” meta-type.

Table

MEM_MISS		
1 (mmd_key (mi_key)) n	MEM_PI	mmd_key
1 (memlog_key) n	MEM_LOG	memlog_key
1 (memcfg_key) n	MEM_CONFIG	memcfg_key
1 (mc_key) n	MEM_CLASS	membk_key
1 (membk_key) n	MEM_BLK_CFG	mc_key
		min_col
		min_row
		max_col
		max_row
		fail_count

Serial Key None.

Foreign Keys / Link Description

mmd_key	Foreign key from MEM_PI table (using mi_key).
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the <i>bitMAP</i> reader.
memcfg_key	Foreign key from MEM_CONFIG.
membk_key	Foreign key from MEM_BLK_CFG.
mc_key	Foreign key from MEM_CLASS.

Other Fields in MEM_MISS

min_col	Column of first failing bit.
min_row	Row of first failing bit.
max_col	Column of last failing bit.
max_row	Row of last failing bit.
fail_count	Total fail count in box.

15.20 MEM_SPOT

Abstract This table contains a list of pattern instances of spot meta-type.

Table

MEM_SPOT	
1 (msd_key (mi_key)) n	msd_key
MEM_PI	memlog_key
1 (memlog_key) n	memcfg_key
MEM_LOG	mc_key
1 (memcfg_key) n	membblk_key
MEM_CONFIG	min_col
1 (mc_key) n	min_row
MEM_CLASS	
1 (membblk_key) n	
MEM_BLK_CFG	

Serial Key None.

Foreign Keys / Link Description

msd_key	Foreign key from MEM_PI table (using mi_key).
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS.
membblk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_SPOT

min_col	Column of first failing bit.
min_row	Row of first failing bit.

15.21 MEM_CROSS

Abstract This table contains a list of pattern instances of cross meta-type.

Table

MEM_CROSS	
1 (mcs_key (mi_key)) n	mcs_key
MEM_PI	memlog_key
1 (memlog_key) n	memcfg_key
MEM_LOG	mc_key
1 (memcfg_key) n	mcd_key
MEM_CONFIG	mrd_key
1 (mc_key) n	membld_key
MEM_CLASS	fail_count
1 (membld_key) n	min_col
MEM_BLK_CFG	min_row
	max_col
	max_row
	min_inter_col
	min_inter_row
	max_inter_col
	max_inter_row

Serial Key None.

Foreign Keys / Link Description

mcs_key Foreign key from MEM_PI table (using mi_key).

memlog_key Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.

memcfg_key Foreign key from MEM_CONFIG.

mc_key Foreign key from MEM_CLASS.

membld_key Foreign key from MEM_BLK_CFG.

Other Fields in MEM_CROSS

fail_count Total fail count.

min_col Column of first failing bit.

min_row Row of first failing bit.

max_col Column of last failing bit.

max_row Row of last failing bit.

min_inter_col Minimum intersecting column.

min_inter_row Minimum intersecting row.

max_inter_col Maximum intersecting column.

max_inter_row Maximum intersecting row.

15.22 MEM_CROSS_BITS

Abstract This tables enumerates all failing bits in cross-pattern instances.

Table

MEM_CROSS_BITS	
1 (memlog_key) n	mcs_key
MEM_LOG	mcsc_key
1 (mcs_key (mi_key)) n	memlog_key
MEM_PI	memcfg_key
1 (mcsc_key) n	mc_key
...	membblk_key
1 (memcfg_key) n	min_col
MEM_CONFIG	min_row
1 (mc_key) n	max_col
MEM_CLASS	max_row
1 (membblk_key) n	fail_count
MEM_BLK_CFG	invert_fg

Serial Key None.

Foreign Keys / Link Description

mcs_key	Foreign key from MEM_PI table (using mi_key).
mcsc_key	Placeholder foreign key from relevant table: MEM_BOX, MEM_ROWS, MEM_COLS, or MEM_SPOT.
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS.
membblk_key	Foreign key from MEM_BLK_CFG.
min_col	Column of first failing bit.
min_row	Row of first failing bit.
max_col	Column of last failing bit.
max_row	Row of last failing bit.
fail_count	Total fail count.
invert_fg	Reserved for future use.

Other Fields in MEM_CROSS_BITS

15.23 MEM_FOG

Abstract This table contains a list of pattern instances of fog meta-type.

Table

MEM_FOG		
1 (mfg_key (mi_key))	n	
MEM_PI		mfg_key
1 (memlog_key)	n	memlog_key
MEM_LOG		memcfg_key
1 (memcfg_key)	n	membk_key
MEM_CONFIG		mc_key
1 (mc_key)	n	min_col
MEM_CLASS		min_row
1 (membk_key)	n	max_col
MEM_BLK_CFG		max_row
		fail_count

Serial Key None.

Foreign Keys / Link Description

mfg_key	Foreign key from MEM_PI table (using mi_key).
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS.
membk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_FOG

min_col	Column of first failing bit.
min_row	Row of first failing bit.
max_col	Column of last failing bit.
max_row	Row of last failing bit.
fail_count	Total fail count.

15.24 MEM_MULTI_ROW

Abstract This table contains component pattern instances of multi-row patterns.

Table

MEM_MULTI_ROW	
1 (m_mr_key (mi_key)) n	m_mr_key
MEM_PI	memlog_key
1 (memlog_key) n	memcfg_key
MEM_LOG	mc_key
1 (memcfg_key) n	membk_key
MEM_CONFIG	fail_count
1 (mc_key) n	min_col
MEM_CLASS	min_row
1 (membk_key) n	max_col
MEM_BLK_CFG	max_row

Serial Key

m_mr_key Foreign key from MEM_PI table (using mi_key).

Foreign Keys / Link Description

memlog_key Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the *bitMAP* reader.

memcfg_key Foreign key from MEM_CONFIG.

mc_key Foreign key from MEM_CLASS.

membk_key Foreign key from MEM_BLK_CFG.

Other Fields in MEM_MULTI_ROW

fail_count Total fail count.

min_col Column of first failing bit.

min_row Row of first failing bit.

max_col Column of last failing bit.

max_row Row of last failing bit.

15.25 MEM_MULTI_ROW_BITS

Abstract This table enumerates all failing bits in multi-row patterns.

Table

MEM_MULTI_ROW_BITS		
1 (m_mr_key (mi_key)) n	MEM_PI	m_mr_key
1 (mrd_key) n	MEM_COLS	mrd_key
1 (memlog_key) n	MEM_LOG	memlog_key
1 (memcfg_key) n	MEM_CONFIG	memcfg_key
1 (mc_key) n	MEM_CLASS	mc_key
1 (memblk_key) n	MEM_BLK_CFG	memblk_key
		row_num
		fail_count
		min_col
		max_col
		thickness

Serial Key None.

Foreign Keys / Link Description

m_mr_key	Foreign key from MEM_PI table (using mi_key).
mrd_key	Foreign key from MEM_COLS table (using mrd_key).
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the <i>bitMAP</i> reader.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS.
memblk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_MULTI_ROW_BITS

row_num	Row number.
fail_count	Fail bit count.
min_col	Column of first failing bit.
max_col	Column of last failing bit.
thickness	Number of rows.

15.26 MEM_MULTI_COL

Abstract This table contains pattern instances of multi-column patterns.

Table

MEM_MULTI_COL	
1 (m_mc_key (mi_key)) n	m_mc_key memlog_key memcfg_key mc_key memblk_key fail_count min_col min_row max_col max_row
MEM_PI	
1 (memlog_key) n	
MEM_LOG	
1 (memcfg_key) n	
MEM_CONFIG	
1 (mc_key) n	
MEM_CLASS	
1 (memblk_key) n	
MEM_BLK_CFG	

Serial Key None.

Foreign Keys / Link Description

m_mc_key	Foreign key from MEM_PI table (using mi_key).
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS.
memblk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_MULTI_COL

fail_count	Total fail count.
min_col	Column of first failing bit.
min_row	Row of first failing bit.
max_col	Column of last failing bit.
max_row	Row of last failing bit.

15.27 MEM_MULTI_COL_BITS

Abstract This table enumerates all failing bits in multi-column patterns.

Table

MEM_MULTI_COL_BITS		
1 (m_mc_key (mi_key)) n		m_mc_key
MEM_PI		mcd_key
1 (mcd_key) n		memlog_key
MEM_COLS		memcfg_key
1 (memlog_key) n		mc_key
MEM_LOG		mемblk_key
1 (memcfg_key) n		col_num
MEM_CONFIG		fail_count
1 (mc_key) n		min_row
MEM_CLASS		max_row
1 (memblk_key) n		thickness
MEM_BLK_CFG		

Serial Key None.

Foreign Keys / Link Description

m_mc_key	Foreign key from MEM_PI table (using mi_key).
mcd_key	Foreign key from MEM_COLS table (using mcd_key).
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the <i>bitMAP</i> reader.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS.
memblk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_MULTI_COL_BITS

col_num	Column number.
fail_count	Fail bit count.
min_row	Row of first failing bit.
max_row	Row of last failing bit.
thickness	Number of columns.

15.28 MEM_CELL_CLASS

Abstract This table contains memory designs used across products.

Table

MEM_CELL_CLASS	
mdsn_key	
mem_dsgn_name	1 (mdsn_key) n
mem_design	MEM_LIB2PROD
mem_desc	
mem_rows	1 (mdsn_key) n
mem_cols	MEM_CONFIG
mem_buswidth	
block_cnt	1 (mdsn_key) n
twist_scheme	
width	MEM_CELL_BLK
height	

Serial Key **mdsn_key** Serial key.

Foreign Keys / Link Description None.

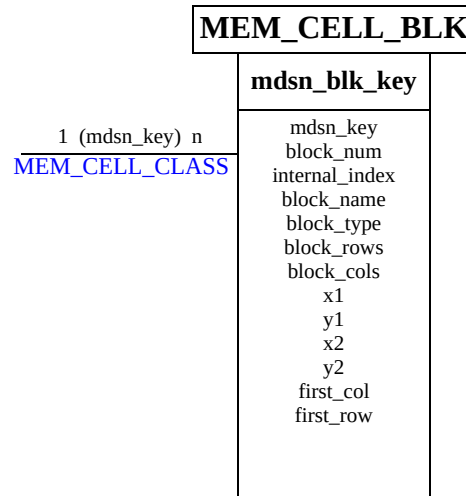
Other Fields in MEM_CELL_CLASS

mem_dsgn_name	Unique name of design.
mem_design	Type of design.
mem_desc	Text description of design.
mem_rows	Number of rows.
mem_cols	Number of columns.
mem_buswidth	Width of bus.
block_cnt	Number of blocks.
twist_scheme	Flag indicating if twist (Y/N).
width	Width in microns.
height	Height in microns.

15.29 MEM_CELL_BLK

Abstract This table contains blocks for all the memory designs used in a given database.

Table



Serial Key **mdsn_blk_key** Serial key.

Foreign Keys / Link Description **mdsn_key** Foreign key from MEM_CELL_CLASS table.

Other Fields in MEM_CELL_BLK

block_num Block number.

internal_index Internal index for block.

block_name Name of block.

block_type Type of block.

block_rows Number of rows in block.

block_cols Number of columns in block.

x1 First X coordinate of block relative to design, in microns.

y1 First Y coordinate of block relative to design, in microns.

x2 Last X coordinate of block relative to design, in microns.

y2 Last Y coordinate of block relative to design, in microns.

first_col Position of first column of block in design bitmap.

first_row Position of first row of block in design bitmap.

15.30 MEM_BLK_CFG

Abstract This table contains blocks of all memory instances for all products in a given database.

Table

MEM_BLK_CFG		
1 (memcfg_key) n MEM_CONFIG	memblk_key	1 (memblk_key) n
	memcfg_key	MEM_ROWS
	block_name	
	block_num	
	internal_index	1 (memblk_key) n
	block_type	MEM_BOX
	block_rows	
	block_cols	
	x1	1 (memblk_key) n
	y1	MEM_CUSTOM
	x2	
	y2	
	first_col	1 (memblk_key) n
	first_row	MEM_COLS
	1 (memblk_key) n	
	MEM_MISS	
	1 (memblk_key) n	
	MEM_SPOT	
	1 (memblk_key) n	
	MEM_CROSS	
	1 (memblk_key) n	
	MEM_FOG	
	1 (memblk_key) n	
	MEM_MULTI_ROW	
	1 (memblk_key) n	
	MEM_MULTI_COL	
	1 (memblk_key) n	
	MEM_MULTI_ROW_BITS	
	1 (memblk_key) n	
	MEM_MULTI_COL_BITS	
	1 (memblk_key) n	
	MEM_CROSS_BITS	
	Links to MEM_BOP_... Related Tables ▶	

**Links to
MEM_BOP_...
Related Tables**

MEM_BLK_CFG	
...	
1 (memblk_key) n	MEM_BOP_MULTI_COL
1 (memblk_key) n	MEM_BOP_ROWS
1 (memblk_key) n	MEM_BOP_COLS
1 (memblk_key) n	MEM_BOP_BOX
1 (memblk_key) n	MEM_BOP_CUSTOM
1 (memblk_key) n	MEM_BOP_MISS
1 (memblk_key) n	MEM_BOP_SPOT
1 (memblk_key) n	MEM_BOP_CROSS
1 (memblk_key) n	MEM_BOP_FOG
1 (memblk_key) n	MEM_BOP_MULTI_ROW

Serial Key	memblk_key	
Foreign Keys / Link Description	memcfg_key	Foreign key from MEM_CONFIG.
Other Fields in MEM_BLK_CFG	block_name	Name of block.
	block_num	Block number.
	internal_index	Internal index for block.
	block_type	Type of block.
	block_rows	Number of rows in block.
	block_cols	Number of columns in block.
	x1	First X coordinate of block relative to design, in microns.
	y1	First Y coordinate of block relative to design, in microns.
	x2	Last X coordinate of block relative to design, in microns.

y2	Last Y coordinate of block relative to design, in microns.
first_col	Position of first column of block in design bitmap.
first_row	Position of first row of block in design bitmap.

15.31 MEM_CONFIG

Abstract This table contains all memory instances for all products in a given database.

Table

MEM_CONFIG		
	memcfg_key	1 (memcfg_key) n
	pd_key	MEM_BOX
1 (pd_key) n	mdsn_key	1 (memcfg_key) n
PRODUCT	wmap_key	MEM_CUSTOM
	mem_name	
	mem_num	1 (memcfg_key) n
	mem_desc	MEM_COLS
	x1	
1 (mdsn_key) n	y1	1 (memcfg_key) n
MEM_CELL_CLASS	x2	MEM_MISS
	y2	
	mem_col_dir	1 (memcfg_key) n
1 (wmap_key) n	mem_row_dir	MEM_SPOT
WMAP_CONFIG	mem_type	
	mem_name	1 (memcfg_key) n
	mem_cols	MEM_CROSS
	mem_rows	
	mem_blk_cnt	1 (memcfg_key) n
	red_blk_cnt	MEM_FOG
		1 (memcfg_key) n
		MEM_MULTI_ROW
	1 (memcfg_key) n	
	MEM_MULTI_COL	
	1 (memcfg_key) n	
	MEM_BLK_CFG	
	1 (memcfg_key) n	
	MEM_MULTI_ROW_BITS	
	1 (memcfg_key) n	
	MEM_ROWS	
	1 (memcfg_key) n	
	MEM_DEF2MEM	
	1 (memcfg_key) n	
	MEM_MEMORY	
	1 (memcfg_key) n	
	MEM_LOG	
	1 (memcfg_key) n	
	MEM_CROSS_BITS	
	1 (memcfg_key) n	
	MEM_DEFAULT_TEST	
	1 (memcfg_key) n	
	MEM_FILE_SETUP	
	1 (memcfg_key) n	
	FILE2MEMCONFIG	
	1 (memcfg_key) n	
	MEM_MULTI_COL_BITS	

Links to MEM_BOP_...
Related Tables ►

**Links to
MEM_BOP_...
Related Tables**

MEM_CONFIG	
...	1 (memcfg_key) n MEM_BOP_ROWS
	1 (memcfg_key) n MEM_BOP_COLS
	1 (memcfg_key) n MEM_BOP_BOX
	1 (memcfg_key) n MEM_BOP_CUSTOM
	1 (memcfg_key) n MEM_BOP_MISS
	1 (memcfg_key) n MEM_BOP_SPOT
	1 (memcfg_key) n MEM_BOP_CROSS
	1 (memcfg_key) n MEM_BOP_FOG
	1 (memcfg_key) n MEM_BOP_MULTI_ROW
	1 (memcfg_key) n MEM_BOP_MULTI_COL
	1 (memcfg_key) n MEM_BOP2PAT
	1 (memcfg_key) n MEM_BOP_UPSTAT

Serial Key

memcfg_key Serial key.

**Foreign Keys / Link
Description**

pd_key Foreign key from PRODUCT table.

mdsn_key Foreign key from MEM_CELL_CLASS table.

wmap_key Foreign key from WMAP_CONFIG table. Identifies wafer map configuration.

**Other Fields in
MEM_CONFIG**

mem_name Name of memory instance.

mem_num Unique number of memory instance.

mem_desc Text description of memory instance.

x1 First X coordinate position of memory instance in product, in microns.



Front



Contents



Index

y1	First Y coordinate position of memory instance in product, in microns.
x2	Last X coordinate position of memory instance in product, in microns.
y2	Last X coordinate position of memory instance in product, in microns.
mem_col_dir	Direction of column.
mem_row_dir	Direction of row.
mem_type	Memory type.
mem_cols	Number of columns in memory instance.
mem_rows	Number of rows in memory instance.
mem_blk_cnt	Number of blocks in memory instance.
red_blk_cnt	Number of redundant blocks in memory instance.

15.32 MEM_DEF2MEM

Abstract This table lists memories that were tested and their respective die were inspected for defects.

Table

MEM_DEF2MEM	
1 (mcl_key) n	memcfg_key
MEM_MCLIB	memlog_key
1 (memlog_key) n	mcl_key
MEM_LOG	mem_key
1 (memcfg_key) n	step_key
MEM_CONFIG	die_key
1 (mem_key) n	lg_key
MEM_MEMORY	lot_key
1 (lg_key) n	wf_key
OP_LOG	diex
1 (lot_key) n	diey
LOT	test_index
1 (wf_key) n	
WAFER	
1 (die_key) n	
MEM_DIE	
1 (step_key) n	
PROC_STEP	

Serial Key None.

Foreign Keys / Link Description

memcfg_key	Foreign key from MEM_CONFIG.
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.
mem_key	Foreign key from MEM_MEMORY.
mcl_key	Foreign key from MEM_MCLIB.
lg_key	Foreign key from OP_LOG. The lot for the above lg_key. Defect log identifier from OP_LOG table.
lot_key	Foreign key from LOT. The key to the related lot information.
wf_key	Foreign key from WAFER. The key to the related wafer information.

die_key	Foreign key from MEM_DIE. Die identifier from MEM_DIE table.
step_key	Foreign key from PROC_STEP. Identifier of a defect inspection process step from PROC_STEP table. It identifies the layer of the defect.
Other Fields in MEM_DEF2MEN	diex Die X index from MEM_DIE table.
	diey Die y index from MEM_DIE table.

15.33 MEM_UPSTAT

Abstract This table contains the processing schedule for bm_upstat.

Table

MEM_UPSTAT	
1 (memlog_key) n MEM_LOG	memlog_key
1 (mcl_key) n MEM_MCLIB	mcl_key
1 (mem_key) n MEM_MEMORY	mem_key
1 (lg_key) n OP_LOG	lg_key
1 (lot_key) n LOT	lot_key
1 (wf_key) n WAFER	pd_key
1 (die_key) n MEM_DIE	wf_key
1 (pd_key) n PRODUCT	pg_key
1 (pg_key) n PROGRAM	die_key
1 (step_key) n PROC_STEP	step_key
	error_code
	err_msg
	flag
	diex
	diey
	test_index

Serial Key None.

Foreign Keys / Link Description

step_key	Foreign key from PROC_STEP. Uniquely identifies a process step.
memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.

mcl_key	Foreign key from MEM_MCLIB.
mem_key	Foreign key from MEM_MEMORY.
lg_key	Foreign key from OP_LOG. The lot for the above lg_key. Unique identifier of a log entry from OP_LOG table, set by the defect reader.
lot_key	Foreign key from LOT. The key to the related lot information, set by both <i>bit</i> MAP reader and defect reader.
wf_key	Foreign key from WAFER. The key to the related wafer information, set by both <i>bit</i> MAP reader and defect reader.
die_key	Foreign key from MEM_DIE. Die identifier from MEM_DIE table.
pd_key	Foreign key from PRODUCT. Set by both <i>bit</i> MAP reader and defect reader.
pg_key	Foreign key from PROGRAM. Set by both <i>bit</i> MAP reader and defect reader.
Other Fields in MEM_UPSTAT	error_code Error code for a given mem_log key or lg_key if any error occurred during the overlay process.
	err_msg Message related to a given error code. Neither reader uses these fields.
	flag 1 - if the entry was inserted by the defect reader 2 - if the entry was inserted by the <i>bit</i> MAP reader, or an overlay was forced from the command line 3 - if the entry was inserted by UpStat.
	diex Die X index from MEM_DIE table.
	diey Die y index from MEM_DIE table.
test_index	Identifies the test. Referenced from DEF... table.

15.34 MEM_LAYER2PAT

Abstract This table contains overlay associations.

Table

MEM_LAYER2PAT	
1 (mc_key) n MEM_CLASS	mc_key step_key pd_key mcl_key
1 (step_key) n PROC_STEP	
1 (pd_key) n PRODUCT	
1 (mcl_key) n MEM_MCLIB	

Serial Key None.

Foreign Keys / Link Description

mc_key	Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.
step_key	Identifier of a defect inspection process step from PROC_STEP table. It identifies the layer of the defect.
pd_key	Foreign key from PRODUCT. Unique identifier of a product form OP_LOG table.
mcl_key	Foreign key from MEM_MCLIB.

Other Fields in MEM_LAYER2PAT None.

15.35 MEM_MEMLOG2LAYER

Abstract This table is not currently used.

Table

MEM_MEMLOG2LAYER	
1 (memlog_key) n MEM_LOG	memlog_key lg_key
1 (lg_key) n OP_LOG	

Serial Key None.

Foreign Keys / Link Description

memlog_key	Foreign key from MEM_LOG. Unique identifier of a log entry from MEM_LOG table, set by the bitMAP reader.
lg_key	Foreign key from OP_LOG. The lot for the above lg_key.

Other Fields in MEM_MEMLOG2LAYER

None.

15.36 MEM_MEMORY**Abstract** This table contains one row per memory tested on a die.**Table**

MEM_MEMORY		
	mem_key	
1 (memcfg_key) n MEM_CONFIG	memcfg_key	1 (mem_key) n MEM_DEF2MEM
1 (lot_key) n LOT	lot_key	1 (mem_key) n MEM_UPSTAT
1 (wf_key) n WAFER	wf_key	1 (mem_key) n MEM_LOG
1 (die_key) n MEM_DIE	die_key	1 (mem_key) n MEM_BOP_UPSTAT
	src_lot	
	diex	
	diey	
	dev_num	

Serial Key**mem_key** Serial key.**Foreign Keys / Link Description**

memcfg_key	Foreign key from MEM_CONFIG.
lot_key	Foreign key from LOT. The key to the related lot information.
wf_key	Foreign key from WAFER. The key to the related wafer information.
die_key	Foreign key from MEM_DIE.
src_lot	Source lot.
diex	X coordinate of die.
diey	Y coordinate of die.
dev_num	Device number on wafer.

Other Fields in MEM_MEMORY

15.37 MEM_LOG

Abstract This table contains one row per each combination of memory instance, memory test, and pattern set.

Table

MEM_LOG	
memlog_key	
1 (lg_key) n OP_LOG	1 (memlog_key) n MEM_PI
1 (pd_key) n PRODUCT	1 (memlog_key) n MEM_ROWS
1 (pg_key) n PROGRAM	1 (memlog_key) n MEM_BOX
1 (mcl_key) n MEM_MCLIB	1 (memlog_key) n MEM_CUSTOM
1 (mem_key) n MEM_MEMORY	1 (memlog_key) n MEM_COLS
1 (memcfg_key) n MEM_CONFIG	1 (memlog_key) n MEM_MISS
1 (step_key) n PROC_STEP	1 (memlog_key) n MEM_SPOT
1 (stage_key) n TECH_STAGE	1 (memlog_key) n MEM_CROSS
1 (eqkey1..3) n EQUIPMENT	1 (memlog_key) n MEM_FOG
1 (em_key) n PEOPLE	1 (memlog_key) n MEM_MULTI_ROW
	1 (memlog_key) n MEM_MULTI_COL
	1 (memlog_key) n MEM_UPSTAT
	1 (memlog_key) n MEM_MEMLOG2LAYER
	1 (memlog_key) n MEM_DEF2MEM
	1 (memlog_key) n MEM_CROSS_BITS
	1 (memlog_key) n MEM_MULTI_ROW_BITS
	1 (memlog_key) n MEM_GALIMG
	1 (memlog_key) n MEM_MULTI_COL_BITS
	Links to MEM_BOP_... Related Tables ▶

Links to MEM_BOP_... Related Tables

MEM_LOG	
...	1 (memlog_key) n MEM_BOP_ROWS
	1 (memlog_key) n MEM_BOP_COLS
	1 (memlog_key) n MEM_BOP_CUSTOM
	1 (memlog_key) n MEM_BOP_BOX
	1 (memlog_key) n MEM_BOP_MISS
	1 (memlog_key) n MEM_BOP_CROSS
	1 (memlog_key) n MEM_BOP_FOG
	1 (memlog_key) n MEM_BOP_MULTI_ROW
	1 (memlog_key) n MEM_BOP_SPOT
	1 (memlog_key) n MEM_BOP_MULTI_COL
	1 (memlog_key) n MEM_BOP_UPSTAT

Serial Keys

memlog_key Automatically generated based on mem_key, test_key and, optionally, start_time or end_time.

Foreign Keys / Link Description

lg_key Foreign key from OP_LOG. The lot for the above lg_key.

pd_key Foreign key from PRODUCT table. Identifies the product.

pg_key Foreign key from PROGRAM table. Identifies the test.

mcl_key Foreign key from MEM_MCLIB.

mem_key Foreign key from MEM_MEMORY.

memcfg_key Foreign key from MEM_CONFIG.

step_key Foreign key from PROC_STEP table. Identifies the process step.

stage_key Foreign key from TECH_STAGE table. Identifies the stage.

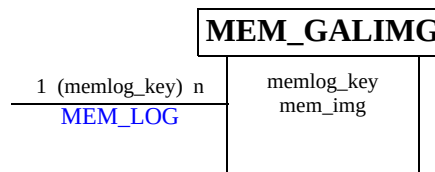
eqkey1-3 Foreign key from EQUIPMENT table. Identifies the equipment. Eqkey 1 is tester.

Other Fields in MEM_LOG	em_key	Foreign key from PEOPLE table. Identifies the operator. The key to the related employee info.
	test_index	From DEF... table. Identifies the test.
	start_time	Start time of tracking defaults.
	end_time	End time of tracking defaults.
	insert_time	Time of data insertion into the database, defaults to the current time.
	rework_flag	Functional as OP_LOG. Rework criteria flag; will append and mark.
	default_test	Flag indicating if given test is the default test.

15.38 MEM_GALIMG

Abstract This table optionally contains thumbnail images of memory bitmaps.

Table



Serial Key None.

Foreign Keys / Link Description

memlog_key	Foreign key from MEM_LOG.
-------------------	---------------------------

Other Fields in MEM_GALIMG	mem_img	Thumbnail image.
---------------------------------------	----------------	------------------

15.39 MEM_DIE

Abstract This table contains one row, per die location on a given wafer.

Table

MEM_DIE	
die_key	1 (die_key) n MEM_DEF2MEM
1 (wf_key) n WAFER	wf_key diex diey 1 (die_key) n MEM_MEMORY 1 (die_key) n MEM_UPSTAT

Serial Key

die_key Serial key.

Foreign Keys / Link Description

wf_key Foreign key from WAFER. The key to the related wafer information.

Other Fields in MEM_DIE

diex X coordinate of die.

diey Y coordinate of die.

15.40 MEM_DEFAULT_TEST

Abstract This table contains information about the default test set most recently used for a product-memory design pair.

Table

MEM_DEFAULT_TEST	
1 (pd_key) n PRODUCT	pd_key memcfg_key test_index
1 (memcfg_key) n MEM_CONFIG	

Serial Key None.

Foreign Keys / Link Description

pd_key Foreign key from PRODUCT table. Identifies the product.

memcfg_key Foreign key from MEM_CONFIG.

Other Fields in MEM_DEFAULT_TEST

test_index Default test index.

15.41 MEM_BOP_ROWS

Abstract This table contains a list of pattern instances of row meta-type for synthetic tests.

Table

MEM_BOP_ROWS	
1 (mrd_key (mi_key)) n	mrd_key
MEM_PI	value
1 (memlog_key) n	memlog_key
MEM_LOG	memcfg_key
1 (memcfg_key) n	mc_key
MEM_CONFIG	row_num
1 (mc_key) n	fail_count
MEM_CLASS	min_col
1 (memblk_key) n	max_col
MEM_BLK_CFG	thickness
	memblk_key

Serial Key None.

Foreign Keys / Link Description

mrd_key	Foreign key from MEM_PI table (using mi_key).
memlog_key	Foreign key from MEM_LOG.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.
memblk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOP_ROWS

value	Instance value.
row_num	Row number.
fail_count	Fail bit count.
min_col	Column of first failing bit.
max_col	Column of last failing bit.
thickness	Number of rows.

15.42 MEM_BOP_MULTI_ROW

Abstract This table contains component pattern instances of multi-row patterns for synthetic tests.

Table

MEM_BOP_MULTI_ROW	
1 (m_mr_key (mi_key)) n	m_mr_key
MEM_PI	m_mr_org_key
1 (m_mr_org_key) n	memlog_key
MEM_MULTI_ROW	memcfg_key
1 (memlog_key) n	mc_key
MEM_LOG	membk_key
1 (memcfg_key) n	fail_count
MEM_CONFIG	min_col
1 (mc_key) n	min_row
MEM_CLASS	max_col
1 (membk_key) n	max_row
MEM_BLK_CFG	value

Serial Key None.

Foreign Keys / Link Description

m_mr_key	Foreign key from MEM_PI table (using mi_key).
m_mr_org_key	From the MEM_MULTI_ROW table.
memlog_key	Foreign key from MEM_LOG.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.
membk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOP_MULTI_ROW

value	Instance value.
fail_count	Total fail count
min_col	Column of first failing bit.
min_row	Row of first failing bit.
max_col	Column of last failing bit.
max_row	Row of last failing bit.

15.43 MEM_BOP_COLS

Abstract This table contains a list of pattern instances of column meta-type for synthetic tests.

Table

MEM_BOP_COLS	
1 (mcd_key (mi_key)) n	mcd_key memlog_key memcfg_key mc_key memblk_key col_num fail_count min_row max_row thickness value
MEM_PI	
1 (memlog_key) n	
MEM_LOG	
1 (memcfg_key) n	
MEM_CONFIG	
1 (mc_key) n	
MEM_CLASS	
1 (memblk_key) n	
MEM_BLK_CFG	

Serial Key None.

Foreign Keys / Link Description

mcd_key	Foreign key from MEM_PI table (using mi_key).
memlog_key	Foreign key from MEM_LOG.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.
memblk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOP_COLS

value	Instance value.
col_num	Column number.
fail_count	Total fail count.
min_row	Row of first failing bit.
max_row	Row of last failing bit.
thickness	Number of columns.

15.44 MEM_BOP_MULTI_COL

Abstract This table contains pattern instances of multi-column patterns for synthetic tests.

Table

MEM_BOP_MULTI_COL		
1 (m_mc_key (mi_key)) n		
MEM_PI	m_mc_key	
1 (memlog_key) n	m_mc_org_key	
MEM_LOG	memlog_key	
1 (memcfg_key) n	memcfg_key	
MEM_CONFIG	mc_key	
1 (mc_key) n	mемblk_key	
MEM_CLASS	fail_count	
1 (memblk_key) n	min_col	
MEM_BLK_CFG	min_row	
	max_col	
	max_row	
	value	

Serial Key None.

Foreign Keys / Link Description

m_mc_key	Foreign key from MEM_PI table (using mi_key).
m_mc_org_key	The original mi_key.
memlog_key	Foreign key from MEM_LOG.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.
mемblk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOP_MULTI_COL

value	Instance value.
col_num	Column number.
fail_count	Total fail count.
min_col	Column of first failing bit.
min_row	Row of first failing bit.
max_col	Column of last failing bit.
max_row	Row of last failing bit.

15.45 MEM_BOP_BOX

Abstract This table contains a list of pattern instances of box meta-type for synthetic tests.

Table

MEM_BOP_BOX	
1 (mbx_key (mi_key)) n	mbx_key
MEM_PI	memlog_key
1 (memlog_key) n	memcfg_key
MEM_LOG	mc_key
1 (memcfg_key) n	membld_key
MEM_CONFIG	min_col
1 (mc_key) n	min_row
MEM_CLASS	max_col
1 (membld_key) n	max_row
MEM_BLK_CFG	fail_count
	invert_fg
	value

Serial Key None.

Foreign Keys / Link Description

mbx_key Foreign key from MEM_PI table (using mi_key).

memlog_key Foreign key from MEM_LOG.

memcfg_key Foreign key from MEM_CONFIG.

mc_key Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.

membld_key Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOP_BOX

value Instance value.

min_col Column of first failing bit.

min_row Row of first failing bit.

max_col Column of last failing bit.

max_row Row of last failing bit.

fail_count Total fail count.

15.46 MEM_BOP_MISS

Abstract This table contains a list of pattern instances of “other” meta-type for synthetic tests.

Table

MEM_BOP_MISS	
1 (mmd_key (mi_key)) n	mmd_key memlog_key memcfg_key mc_key memblk_key min_col min_row max_col max_row fail_count value
MEM_PI	
1 (memlog_key) n	
MEM_LOG	
1 (memcfg_key) n	
MEM_CONFIG	
1 (mc_key) n	
MEM_CLASS	
1 (memblk_key) n	
MEM_BLK_CFG	

Serial Key None.

Foreign Keys / Link Description

mmd_key	Foreign key from MEM_PI table (using mi_key).
memlog_key	Foreign key from MEM_LOG.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.
memblk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOP_MISS

value	Instance value.
min_col	Column of first failing bit.
min_row	Row of first failing bit.
max_col	Column of last failing bit.
max_row	Row of last failing bit.
fail_count	Total fail count.

15.47 MEM_BOP_SPOT

Abstract This table contains a list of pattern instances of spot meta-type for synthetic tests.

Table

MEM_BOP_SPOT	
1 (mmd_key (mi_key)) n	msd_key memlog_key memcfg_key memblk_key mc_key min_col min_row value
MEM_PI	
1 (memlog_key) n	
MEM_LOG	
1 (memcfg_key) n	
MEM_CONFIG	
1 (mc_key) n	
MEM_CLASS	
1 (memblk_key) n	
MEM_BLK_CFG	

Serial Key None.

Foreign Keys / Link Description

msd_key Foreign key from MEM_PI table (using mi_key).

memlog_key Foreign key from MEM_LOG.

memcfg_key Foreign key from MEM_CONFIG.

mc_key Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.

memblk_key Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOP_SPOT

value Instance value.

min_col Column of first failing bit.

min_row Row of first failing bit.

15.48 MEM_BOP_FOG

Abstract This table contains a list of pattern instances of fog meta-type for synthetic tests.

Table

MEM_BOP_FOG	
1 (mfg_key (mi_key)) n	mfg_key
MEM_PI	mfg_org_key
1 (mfg_org_key) n	memlog_key
MEM_FOG	memcfg_key
1 (memlog_key) n	mc_key
MEM_LOG	membld_key
1 (memcfg_key) n	min_col
MEM_CONFIG	min_row
1 (mc_key) n	max_col
MEM_CLASS	max_row
1 (membld_key) n	fail_count
MEM_BLK_CFG	value

Serial Key None.

Foreign Keys / Link Description

mfg_key	Foreign key from MEM_PI table (using mi_key).
mfg_org_key	Foreign from the MEM_FOG table, using mfg_org_key.
memlog_key	Foreign key from MEM_LOG.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.
membld_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOP_FOG

value	Instance value.
min_col	Column of first failing bit.
min_row	Row of first failing bit.
max_col	Column of last failing bit.
max_row	Row of last failing bit.
fail_count	Total fail count in box.

15.49 MEM_BOP_CROSS

Abstract This table contains a list of pattern instances of cross meta-type for synthetic tests.

Table

MEM_BOP_CROSS	
1 (mcs_key (mi_key)) n MEM_PI	mcs_key mcs_org_key mcd_key mcd_mmc_key mrd_mmc_key memlog_key memcfg_key mc_key mrd_key fail_count memblk_key min_col min_row max_col max_row min_inter_col min_inter_row max_inter_col max_inter_row value
1 (memlog_key) n MEM_LOG	
1 (memcfg_key) n MEM_CONFIG	
1 (mc_key) n MEM_CLASS	
1 (memblk_key) n MEM_BLK_CFG	

Serial Key None.

Foreign Keys / Link Description

mcs_key Foreign key from MEM_PI table (using mi_key).

mcs_org_key

mcd_mmc_key

mrd_mmc_key

mcd_key

mrd_key

memlog_key Foreign key from MEM_LOG.

memcfg_key Foreign key from MEM_CONFIG.

mc_key Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.

memblk_key Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOP_CROSS

value Instance value.

fail_count Total fail count in box.

min_col	Column of first failing bit
min_row	Row of first failing bit.
max_col	Column of last failing bit.
max_row	Row of last failing bit.

15.50 MEM_BOP_CUSTOM

Abstract This table contains a list of pattern instances of custom meta-type for synthetic tests.

Table

MEM_BOP_CUSTOM	
1 (mcst_key) n MEM_CUSTOM	mcst_key
1 (memlog_key) n MEM_LOG	memlog_key
1 (memcfg_key) n MEM_CONFIG	memcfg_key
1 (mc_key) n MEM_CLASS	mc_key
1 (membblk_key) n MEM_BLK_CFG	membblk_key
	value
	min_col
	min_row
	max_col
	max_row
	fail_count
	invert_fg

Serial Key None.

Foreign Keys / Link Description

mcst_key	Foreign key from MEM_CUSTOM table.
memlog_key	Foreign key from MEM_LOG.
memcfg_key	Foreign key from MEM_CONFIG.
mc_key	Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.
membblk_key	Foreign key from MEM_BLK_CFG.

Other Fields in MEM_BOP_CUSTOM

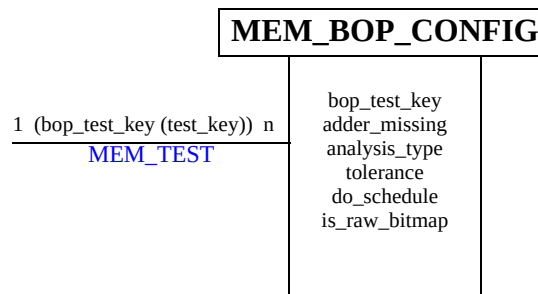
value	Instance value.
min_col	Column of first failing bit
min_row	Row of first failing bit.

max_col	Column of last failing bit.
max_row	Row of last failing bit.
fail_count	Total fail count.
invert_fg	Reserved for future use.

15.51 MEM_BOP_CONFIG

Abstract This table contains all memory instances for all products in a given database for synthetic tests.

Table



Serial Key None.

Foreign Keys / Link Description

bop_test_key Foreign key from MEM_TEST table (using test_key).

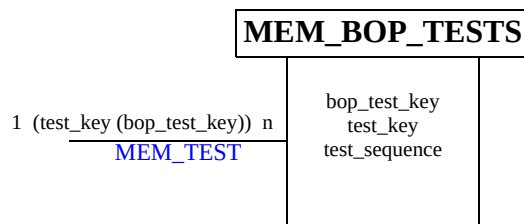
Other Fields in MEM_BOP_CONFIG

adder_missing	Reserved for future use.
analysis_type	Type of analysis. Possible values: C=pattern class, M=pattern meta-class, S=pattern set
tolerance	Overlay tolerance.
do_schedule	Indicates if bitmaps for test are to be auto-scheduled.
is_raw_bitmap	Flag indicating if boolean operation is to be applied to raw bitmap files (pre-classification) or to classified bitmap files.

15.52 MEM_BOP_TESTS

Abstract This table contains definitions of synthetic tests in reverse order notation.

Table



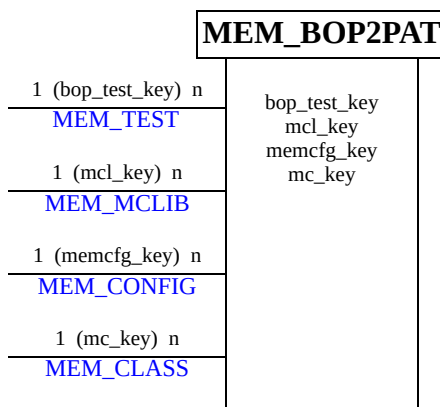
Serial Key None.

Foreign Keys / Link Description	test_key	Foreign key from MEM_TEST.
	bop_test_key	Foreign key from MEM_TEST (using test_key).
Other Fields in MEM_BOP_TESTS	test_sequence	Sequence of test.

15.53 MEM_BOP2PAT

Abstract This table contains the set of pattern classes and memory instances to be analyzed for synthetic tests.

Table



Serial Key None.

Foreign Keys / Link Description	bop_test_key	Foreign key from MEM_TEST (using test_key).
	mcl_key	Foreign key from MEM_MCLIB.
	memcfg_key	Foreign key from MEM_CONFIG.

mc_key Foreign key from MEM_CLASS. Unique identifier of the memory-fail-pattern class from MEM_CLASS table.

Other Fields in MEM_BOP2PAT None.

15.54 MEM_BOP_UPSTAT

Abstract This table contains the processing schedule for bm_upstat for synthetic tests.

Table

MEM_BOP_UPSTAT		
1 (pd_key) n	pd_key	
PRODUCT	pg_key	
1 (pg_key) n	lot_key	
PROGRAM	wf_key	
1 (lot_key) n	mem_key	
LOT	memcfg_key	
1 (wf_key) n	mcl_key	
WAFER	bop_test_key	
1 (mem_key) n	memlog_key	
MEM_MEMORY	error_code	
1 (memcfg_key) n	err_msg	
MEM_CONFIG		
1 (mcl_key) n		
MEM_MCLIB		
1 (memlog_key) n		
MEM_LOG		
1 (bop_test_key (test_key)) n		
MEM_TEST		

Serial Key None.

Foreign Keys / Link Description

pd_key Foreign key from PRODUCT table. Identifies the product.

pg_key Foreign key from PROGRAM table. Identifies the test.

lot_key Foreign key from LOT. The key to the related lot information.

wf_key Foreign key from WAFER. The key to the related wafer information.

mem_key Foreign key from MEM_MEMORY.

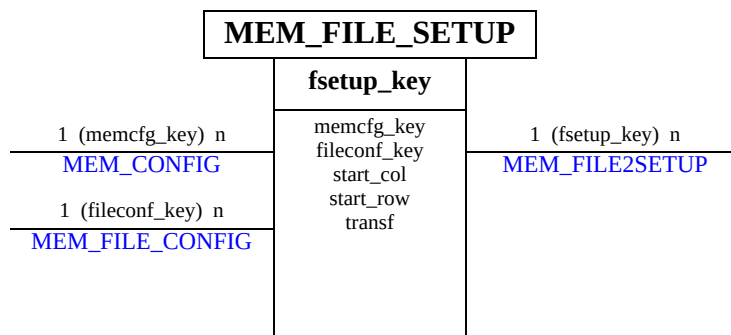
memcfg_key	Foreign key from MEM_CONFIG.
mcl_key	Foreign key from MEM_MCLIB.
bop_key	Foreign key from MEM_BOP_CONFIG.
memlog_key	Foreign key from MEM_LOG.
bop_test_key	Foreign key from the MEM_TEST table, using test_key.
error_code	Error code value, given by the system.
err_msg	Error message generated by the system.

Other Fields in MEM_BOP_UPSTAT

15.55 MEM_FILE_SETUP

Abstract This table contains definitions of file configuration items in the MEM_FILE_CONFIG table.

Table



Serial Key

fsetup_key Serial key.

Foreign Keys / Link Description

memcfg_key Foreign key from MEM_CONFIG.

fileconf_key Foreign key from MEM_FILE_CONFIG.

Other Fields in MEM_FILE_SETUP

start_col Starting column position.

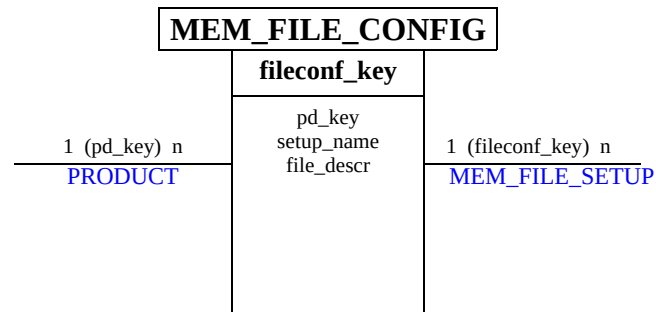
start_row Starting row position.

transf Type of transformation.

15.56 MEM_FILE_CONFIG

Abstract This table contains a list of bitmap file configurations which define how bitmap is configured from smaller bitmaps belonging to different memory instances.

Table

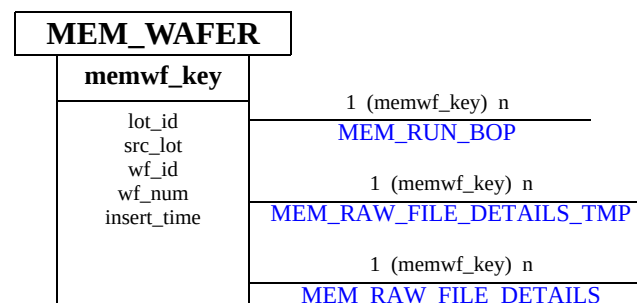


Serial Key	fileconf_key	A unique key assigned to each entry in MEM_FILECONFIG. It is stored automatically.
Foreign Keys / Link Description	pd_key	Foreign key from PRODUCT table.
Other Fields in MEM_FILE_CONFIG	setup_name	Unique name of setup configuration.
	file_descr	Text description of file configuration.

15.57 MEM_WAFER

Abstract This table contains a list of wafers which have bitmap information.

Table



Serial Key	memwf_key	A unique key assigned to each entry in MEM_WAFER. It is stored automatically.
Foreign Keys / Link Description	None.	

**Other Fields in
MEM_WAFER**

lot_id	Lot ID, unique.
src_lot	Reference to source lot in LOT table, to identify input source lot.
wf_id	Wafer ID. (The combination of wafer ID and lot key is unique.)
wf_num	Wafer number.
insert_time	The current date and time of insertion into the database. Date/time/year to the second. Defaults to current year to the second.

15.58 MEM_RAW_FILE_DETAILS

Abstract This table contains information about raw files stored in the RAW_FILE table.

Table

MEM_RAW_FILE_DETAILS	
1 (rawfile_key) n RAW_FILE	rawfile_key
1 (test_key) n MEM_TEST	test_key
1 (ndie_key) n NWMAP2DIE	ndie_key
1 (descramb_key) n MEM_SCRAMBLE	descramb_key
1 (memwf_key) n MEM_WAFER	memwf_key
1 (wfrf_key) n MEM_WF_RF	wfrf_key
	diex
	diey
	insert_time

Serial Key None.

**Foreign Keys / Link
Description**

rawfile_key	Foreign key from RAW_FILE table.
test_key	Foreign key from the MEM_TEST table.
ndie_key	Foreign key from NWMAP2DIE table.
descramb_key	Foreign key from the MEM_SCRAMBLE table.
memwf_key	Foreign key from MEM_WAFER table.

wfrf_key Foreign key from the MEM_WF_RF table.

Other Fields in MEM_RAW_FILE_DETAILS

diex X coordinate of die.

diey Y coordinate of die.

insert_time The current date and time of insertion into the database. Date/time/year to the second. Defaults to current year to the second.

15.59 MEM_RAW_FILE_DETAILS_TMP

Abstract This table contains information about raw files stored in the RAW_FILE table that have not yet been scheduled for processing by MEMBRAIN.

Table

MEM_RAW_FILE_DETAILS_TMP		
1 (rawfile_key) n RAW_FILE	rawfile_key	
1 (test_key) n MEM_TEST	test_key	
1 (descramb_key) n MEM_SCRAMBLE	descramb_key	
1 (memwf_key) n MEM_WAFER	memwf_key	
1 (wfrf_key) n MEM_WF_RF	wfrf_key	
	diex	
	diey	

Serial Key None.

Foreign Keys / Link Description

rawfile_key Foreign key from RAW_FILE table.

test_key Foreign key from the MEM_TEST table.

descramb_key Foreign key from the MEM_SCRAMBLE table.

memwf_key Foreign key from MEM_WAFER table.

wfrf_key Foreign key from the MEM_WF_RF table.

Other Fields in MEM_RAW_FILE_DETAILS_TMP

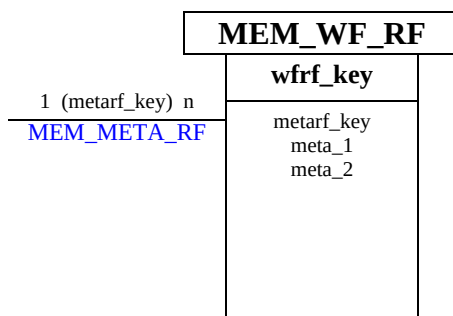
diex X coordinate of die.

diey Y coordinate of die.

15.60 MEM_WF_RF

Abstract This table contains meta information about memory products.

Table



Serial Key

wfrf_key A unique key assigned to each entry in MEM_WF_RF. It is stored automatically.

Foreign Keys / Link Description

metarf_key Foreign key from the MEM_META_RF table.

Other Fields in MEM_WF_RF

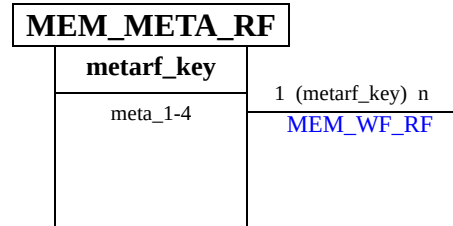
meta_1 Concatenated with meta_2 to form a string of semicolon-separated key value pairs.

meta_2 Concatenated with meta_1 to form a string of semicolon-separated key value pairs.

15.61 MEM_META_RF

Abstract This table stores meta information about memory products.

Table



Serial Key **metarf_key** A unique key assigned to each entry in META_RF. It is stored automatically.

Foreign Keys / Link Description None.

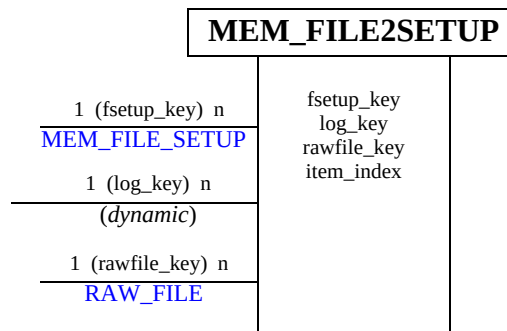
Other Fields in MEM_META_RF **metarf1-4** Can be one of the following values:

Value	Description
tech	technology name
step	step name
stag	stage

15.62 MEM_FILE2SETUP

Abstract This table links raw files with file setups.

Table



Serial Key None.

Foreign Keys / Link Description

fsetup_key Foreign key from MEM_FILE_SETUP.

rawfile_key Foreign key from RAW_FILE.

log_key Dynamic foreign key that can link to any log table, primarily OP_LOG.

Other Fields in MEM_FILE2SETUP

item_index Reserved for use with future multi-die setup file formats.

15.63 MEM_RUN_TEMPLATE

Abstract This table defines the templates that instruct the MEMBRAIN runs.

Table

MEM_RUN_TEMPLATE	
	run_key
1 (test_key) n PROGRAM	test_key
1 (mcl_key) n WAFER_CLASS	mcl_key
1 (fsetup_key) n WAFER_CLASS	fsetup_key
1 (pd_key) n WAFER_CLASS	pd_key
	is_synthetic
	run_name
	sched_flag
	run_descr
	out_fname_template
	is_new_flag

Serial Key

run_key A unique key assigned to each entry in MEM_RUN_TEMPLATE. It is stored automatically.

Foreign Keys / Link Description

test_key Foreign key from the MEM_TEST table.

mcl_key Foreign key from the MEM_MCLIB table.

fsetup_key Foreign key from the MEM_FILE_SETUP table.

pd_key Foreign key from the PRODUCT table.

Other Fields in MEM_RUN_TEMPLATE

run_name Name of run template.

sched_flag Flag indicating schedule status: Y = template can be used for auto-scheduling; N = template cannot be used for auto-scheduling.

run_descr Text description of run.

out_fname_template Output filename template for MEMBRAIN.

is_new_flag Indicates if the run is new. Y = yes.



Front



Contents



Index

15.64 MEM_RUN_SCHEDULE

Abstract This table serves as the MEMBRAIN input queue. Each record in this table is equivalent to a single MEMBRAIN run.

Table

MEM_RUN_SCHEDULE		
1 (rawfile_key) n RAW_FILE	msched_key	1 (msched_key) n MEM_RUN_BOP
1 (run_key) n MEM_SCRAMBLE	rawfile_key run_key status insert_time start_time end_time error_code err_msg priority_flag	

Serial Key **msched_key** A unique key assigned to each entry in MEM_RUN_SCHEDULE. It is stored automatically.

Foreign Keys / Link Description

rawfile_key Foreign key from the RAWFILE table.

run_key Foreign key from the MEM_RUN_TEMPLATE table.

Other Fields in MEM_RUN_SCHEDULE

status Flag indicating status of run.

Value	Description
0	Not in queue
1	Running
2	Finished successfully
-1	Finished with error
-99	Postponed. Component of synthetic test waiting for other component to become available

insert_time The current date and time of insertion into the database. Date/time/year to the second. Defaults to current year to the second.

start_time Time MEMBRAIN started run.

end_time Time MEMBRAIN ended run.

error_code Error code value, given by the system when status field = -1.

err_msg Text description of error.

priority_flag Flag indicating run priority.

15.65 MEM_RUN_BOP

Abstract This table holds detailed information associated with records in MEM_RUN_SCHEDULE for synthetic tests.

Table

MEM_RUN_BOP	
1 (msched_key) n MEM_SCRAMBLE	msched_key
1 (memwf_key) n MEM_WAFER	memwf_key
1 (ndie_key) n NWMAP2DIE	ndie_key
1 (memcfg_key) n MEM_CONFIG	memcfg_key
1 (test_key) n MEM_TEST	test_key
1 (wmap_key) n WMAP_CONFIG	wmap_key
1 (bop_test_key) n MEM_TEST	bop_test_key

Serial Key None.

Foreign Keys / Link Description

msched_key Foreign key from MEM_RUN_SCHEDULE table.

memwf_key Foreign key from the MEM_WAFER table.

ndie_key Foreign key from the NWMAP2DIE table.

memcfg_key Foreign key from the MEM_CONFIG table.

test_key Foreign key from the MEM_TEST table.

wmap_key Foreign key from the WMAP_CONFIG table.

bop_test_key Foreign key from the MEM_TEST table.
Other Fields in MEM_RUN_BOP

None.

15.66 MEM_SCRAMBLE

Abstract This table contains scrambling rules.

Table

MEM_SCRAMBLE	
descramb_key	
rule_name rule_text	1 (descramb_key) n MEM_RUN_BOP
	1 (descramb_key) n MEM_RAW_FILE_DETAILS
	1 (descramb_key) n MEM_RAW_FILE_DETAILS_TMP

Serial Key **descramb_key** A unique key assigned to each entry in MEM_SCRAMBLE. It is stored automatically.

Foreign Keys / Link Description None.

Other Fields in MEM_SCRAMBLE

rule_name Name of the rule.

rule_text Text description of rule.

15.67 MEM_TEST

Abstract This table contains a list of bitmap tests.

Table

MEM_TEST	
	test_key
1 (pd_key) n PRODUCT	1 (test_key) n MEM_RAW_FILE_DETAILS
1 (pg_key) n PROGRAM	1 (test_key) n MEM_RAW_FILE_DETAILS_TMP
1 (wmap_key) n WMAP_CONFIG	1 (test_key) n MEM_RUN_BOP
1 (bop_test_key) n MEM_BOP_TESTS	1 (test_key) n MEM_BOP_TESTS
	1 (test_key) n MEM_BOP_CONFIG
	1 (test_key) n MEM_BOP_UPSTAT

Serial Key **test_key** A unique key assigned to each entry in MEM_TEST. It is stored automatically.

**Foreign Keys / Link
Description****pd_key** Foreign key from the PRODUCT table.**pg_key** Foreign key from the PROGRAM table.**wmap_key** Foreign key from the WMAP_CONFIG table.**Other Fields in
MEM_TEST****test_name** Name of test.**test_index** From DEF... table. Identifies the test.

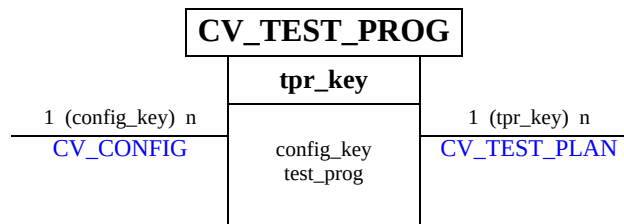
16.0 CV Tables

These tables describe the portion of the schema that holds the pdCV Characterization Vehicle data and related summaries.

16.1 CV_TEST_PROG

Abstract Provides details about the test program.

Table



Serial Key **tpr_key** Identifies the test program entry.

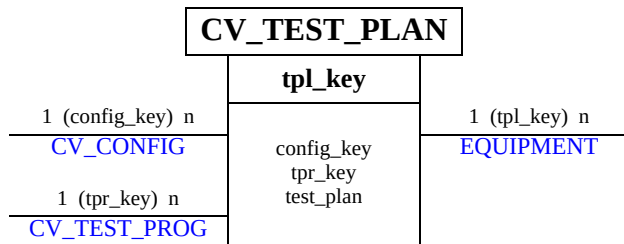
Foreign Keys / Link Description **config_key** From CV_CONFIG table. Identifies the configuration entry.

Other Fields in CV_TEST_PROG **test_prog** Test program description.

16.2 CV_TEST_PLAN

Abstract Provides details about the test plan.

Table



Serial Key **tpl_key** Identifies the Test Plan entry.

Foreign Keys / Link Description **config_key** From CV_CONFIG table. Identifies the CV configuration entry.

tpr_key From CV_TEST_PROG table. Identifies the Test Program entry.

Other Fields in CV_TEST_PLAN **test_plan** Test plan description.

16.3 CV_CONFIG

Abstract Provides overall information about CV configuration.

Table

CV_CONFIG	
	1 (config_key) n
config_key	CV_TEST_PLAN
1 (pg_key) n	1 (config_key) n
PROGRAM	CV_UNQCI
1 (wmap_key) n	1 (config_key) n
WMAP_CONFIG	CV_MEAS2PIN
pg_key	
wmap_key	
config_name	
revision	
ret_align	1 (config_key) n
in_x_offset	CV_MEAS
in_y_offset	
gds_x_offset	1 (config_key) n
gds_y_offset	CV_TEST_PROG
	1 (config_key) n
	CV_MODULE2CELL
	1 (config_key) n
	CV_LIMITS
	1 (config_key) n
	CV_FLTR
	1 (config_key) n
	CV_FLTR_INFO
	1 (config_key) n
	CV_COEF_LABEL
	1 (config_key) n
	CV_FLTR
	1 (config_key) n
	CV_PR_MOD
	1 (config_key) n
	CV_ADL
	1 (config_key) n
	CV_PIN_RESULTS
	1 (config_key) n
	CV_DIE_RESULTS
	1 (config_key) n
	CV_FINE_FTYPE
	1 (config_key) n
	CV_CA
	1 (config_key) n
	CV_CA_VAL
	Links to CV Cell Related Tables ▶
	Links to Fail Rate Modeling/Mode Tables ▶
	Links to Attributes Related Tables ▶
	Links to Formula and Function Tables ▶
	Links to Analysis Items Tables ▶

Table continued...

CV_CONFIG	
config_key	
1 (pg_key) n PROGRAM	1 (config_key) n CV_CA_NV
1 (wmap_key) n WMAP_CONFIG	1 (config_key) n CV_P_VAL
pg_key wmap_key config_name revision ret_align in_x_offset in_y_offset gds_x_offset gds_y_offset	1 (config_key) n CV_STEP_ORDER
	1 (config_key) n CV_PAR2MEAS
	1 (config_key) n CV_UPSTAT
	1 (config_key) n CV_LOT2CFG
	1 (config_key) n CV_DOE
	1 (config_key) n CV_EXPORT
	1 (config_key) n CV_INSP_SET
	1 (config_key) n CV_MINOR_CONFIG
	1 (config_key) n CV_PIN2PADGRP
	1 (config_key) n CV_PAD_GROUP
	1 (config_key) n CV_MEAS2PADGRP
	1 (config_key) n CV_UCONSTANT
	1 (config_key) n CV_PIN_SUM
	1 (config_key) n CV_PADGRP_SUM
	1 (config_key) n CV_UOPERATION
	1 (config_key) n CV_FRS_SETUP
	1 (config_key) n CV_FRS_RUN
	Links to CV Cell Related Tables ▶
	Links to Fail Rate Modeling/Mode Tables ▶
	Links to Attributes Related Tables ▶
	Links to Formula and Function Tables ▶
	Links to Analysis Items Tables ▶

Links to CV Cell Related Tables

CV_CONFIG	
...	1 (config_key) n CV_CELLI2IDEF
	1 (config_key) n CV_CELL
	1 (config_key) n CV_CELL_I
	1 (config_key) n CV_CELLI_BLOCK_NUM
	1 (config_key) n CV_CELLI_ZONES
	1 (config_key) n CV_CELLI_ZONE_NUM
	1 (config_key) n CV_CELLI_BLOCK_DEF

Links to Fail Rate Modeling/Mode Tables

CV_CONFIG	
...	1 (config_key) n CV_MODEL
	1 (config_key) n CV_MODEL_DEF
	1 (config_key) n CV_FAIL_MODE
	1 (config_key) n CV_FAIL_LAYER
	1 (config_key) n CV_FMD_TYPE
	1 (config_key) n CV_FMD2MDL
	1 (config_key) n CV_FMD2PAR
	1 (config_key) n CV_FMD2LSPLT
	1 (config_key) n CV_FRM_DIAG_ATTR
	1 (config_key) n CV_FRM_PARAM
	1 (config_key) n CV_FRM_DIAG_V
	1 (config_key) n CV_FRM_MOD
	1 (config_key) n CV_FRM_RUN
	1 (config_key) n CV_FRM_RUN_DET
	1 (config_key) n CV_FRM_CLUST_DET

Links to Attributes Related Tables

CV_CONFIG		
...	1 (config_key) n	
	CV_CATTR	
		1 (config_key) n
	CV_AATTR	
		1 (config_key) n
	CV_CATTR	
	1 (config_key) n	
	CV_GATTR	
		1 (config_key) n
	CV_GATTR_V	
		1 (config_key) n
	1 (config_key) n	
	CV_MATTR	
	CV_MODULE2CATTR	

Links to Formula and Function Tables

CV_CONFIG		
...	1 (config_key) n	
	CV_FUNCTION	
		1 (config_key) n
	CV_FUNC2FORM	
		1 (config_key) n
	CV_FUNC2INP	
	1 (config_key) n	
	CV_FORM2PARAM	
		1 (config_key) n
	CV_FORM	
		1 (config_key) n
	CV_FORM2LSPLT	
	1 (config_key) n	
	CV_UFORM2FILTER	
		1 (config_key) n
	CV_FILTER_UFORM	
		1 (config_key) n
	CV_UFORM	

Links to Analysis Items Tables

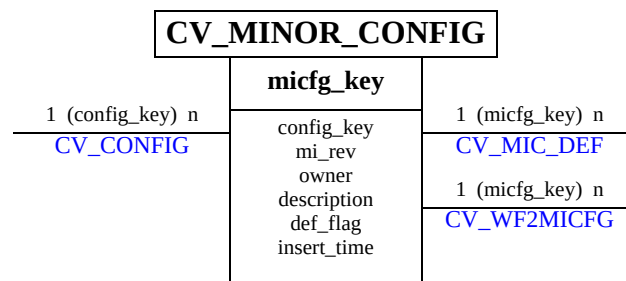
CV_CONFIG		
...	1 (config_key) n	
	CV_AI_MAP	
		1 (config_key) n
	CV_AI	
		1 (config_key) n
	CV_AI_MAPC	
	1 (config_key) n	
	CV_AI_MOD	

Serial Key	config_key	Identifies the CV configuration entry.									
Foreign Keys / Link Description	pg_key	From PROGRAM table. Identifies the Program entry.									
	wmap_key	From WMAP_CONFIG. Identifies the Wafer configuration entry.									
Other Fields in CV_CONFIG	config_name	CV configuration description.									
	revision	Describes the revision of the CV configuration.									
	ret_align	Can be one of the following to indicate location of flat (or notch) side of wafer. Values:									
		<table><tr><th>Value</th><th>Description</th></tr><tr><td>R</td><td>right</td></tr><tr><td>L</td><td>left</td></tr><tr><td>T</td><td>top</td></tr><tr><td>B</td><td>bottom</td></tr></table>	Value	Description	R	right	L	left	T	top	B
Value	Description										
R	right										
L	left										
T	top										
B	bottom										
	in_x_offset	Indicates the offset value within die along X-axis.									
	in_y_offset	Indicates the offset value within die along Y-axis.									
	gds_x_offset	Indicates the GDS offset value within die along X-axis.									
	gds_y_offset	Indicates the GDS offset value within die along Y-axis.									

16.4 CV_MINOR_CONFIG

Abstract Allows the end user to define a subset of CV configuration options, through the pdCV GUI.

Table



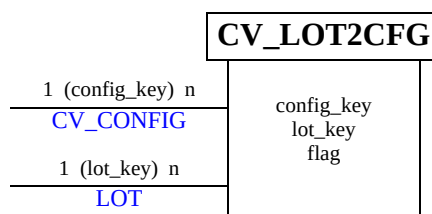
Serial Key	micfg_key	Serial key.
Foreign Keys / Link Description	config_key	From CV_CONFIG table. Identifies CV configuration entry.

**Other Fields in
CV_MINOR_CONFIG**

mi_rev	Minor revision.
owner	The name/ID of the person who creates the minor configuration in the database.
description	A field to hold any comments or notes a user has on the configuration (entered in the GUI).
def_flag	Y or N . Y = default minor configuration; N = not default minor configuration. As defined in the GUI.
insert_time	Time of insertion of the minor configuration into the database, defaults to the current time.

16.5 CV_LOT2CFG

Abstract Holds information on relationships between CV configurations and lots.

Table

Serial Key None.

**Foreign Keys / Link
Description**

config_key	From CV_CONFIG table. Identifies CV configuration entry.
lot_key	From LOT table. Identifies lot entry.
flag	Reserved for future use. Currently hard-coded to 1.

**Other Fields in
CV_LOT2CFG**

16.6 CV_CELL

Abstract Describes each cell within the CV configuration.

Table

CV_CELL		
	cell_key	
1 (config_key) n	config_key pmod_key ctype_key step_key cell_name meta_flag	1 (cell_key) n
CV_CONFIG		CV_CATTR_V
1 (pmod_key) n		1 (cell_key) n
CV_PR_MOD		CV_CELL2AMOD
1 (ctype_key) n		1 (cell_key) n
CV_CELL_TYPE		CV_CELL_I
1 (step_key) n		1 (cell_key) n
PROC_STEP		CV_MEAS
		1 (cell_key) n
		CV_AI
		1 (cell_key) n
		CV_DOE2CELL
		1 (cell_key) n
		CV_MODULE2CELL
		1 (cell_key) n
		CV_CA
		1 (cell_key) n
		CV_CA_NV
		1 (cell_key) n
		CV_DEFECT
		1 (cell_key) n
		CV_EDEF2IDEF
		1 (cell_key) n
		CV_CELLI2IDEF
		1 (cell_key) n
		CV_SUM_ZONE
		1 (cell_key) n
		CV_FRN
		1 (cell_key) n
		CV_FRM_OUT_DATA_<model_key>
		1 (cell_key) n
		CV_EDEF2IDEF_ALL
		1 (cell_key) n
		CV_E2I_INHERIT

Serial Key **cell_key** Identifies the cell entry.

Foreign Keys / Link Description

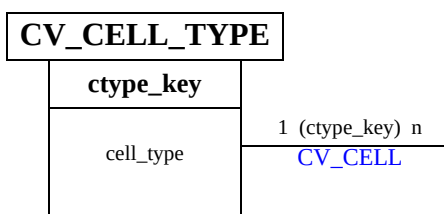
config_key	From CV_CONFIG table. Identifies the CV configuration entry.
pmod_key	From CV_PR_MOD table. Identifies the process module entry.
ctype_key	From CV_CELL_TYPE. Identifies the cell type.
step_key	From PROC_STEP table. Identifies the process step entry.

Other Fields in CV_CELL

cell_name	Provides the cell description.
meta_flag	Specifies the “meta” flag for the cell.

16.7 CV_CELL_TYPE

Abstract Provides information about cell type.

Table**Serial Key**

ctype_key	Identifies the cell type entry
------------------	--------------------------------

Foreign Keys / Link Description

None.

Other Fields in CV_CELL_TYPE

cell_type	Describes the cell type
------------------	-------------------------

16.8 CV_CELL2AMOD

Abstract Describes relationships between cells and analysis modules.

Table

CV_CELL2AMOD		
1 (cell_key) n	cell_key amod_key	
CV_CELL		
1 (amod_key) n		
CV_AI_MOD		

Serial Key None.

Foreign Keys / Link Description

cell_key From CV_CELL table. Identifies cell entry.

amod_key From CV_AI_MOD table. Identifies analysis module entry.

Other Fields in CV_CELL2AMOD None.

16.9 CV_CELL_I

Abstract Holds information on cell instances within CV configuration.

Table

CV_CELL_I		
	celli_key	
1 (config_key) n	config_key cell_key celli_name llx lly urx ury llx_org lly_org urx_org ury_org insp_flag insp_prio	1 (celli_key) n
CV_CONFIG		CV_CIATTR_V
1 (cell_key) n		1 (celli_key) n
CV_CELL		CV_MEAS
		1 (celli_key) n
		CV_AI
		1 (celli_key) n
		CV_UNQCI
		1 (celli_key) n
		CV_DEFECT
		1 (celli_key) n
		CV_WF_CELLI
		1 (celli_key) n
		CV_INSP_SET
		1 (celli_key) n
		CV_CELLI_BLOCK_DEF

Serial Key **celli_key** Identifies the cell instance entry.

**Foreign Keys / Link
Description**

config_key From CV_CONFIG table. Identifies CV configuration entry.

cell_key From CV_CELL table. Identifies cell entry.

**Other Fields in
CV_CELL_I**

celli_name Describes cell instance.

llx X position of the lower left corner of cell instance within die.

lly Y position of the lower left corner cell instance within die

urx X position of the upper right corner of cell instance within die.

ury Y position of the upper right corner of cell instance within die.

llx_org Original X position of the lower left corner of cell instance.

lly_org Original Y position of the lower left corner of cell instance.

urx_org Original X position of the upper right corner of cell instance.

ury_org Original Y position of the upper right corner of cell instance.

insp_flag Flag to indicate if the cell was inspected.

insp_prio Flag to indicate inspection priority.

16.10 CV_CELLI2IDEF

Abstract Holds information on overlay between cell instances and inline defects.

Table

CV_CELLI2IDEF	
1 (config_key) n	config_key cell_key wf_key s_lg_key estep_key istep_key unqci_key pmod_key df_source diex diey et_flag
CV_CONFIG	
1 (cell_key) n	
CV_CELL	
1 (wf_key) n	
WAFER	
1 (s_lg_key) n	
OP_LOG	
1 (estep_key) n	
PROC_STEP	
1 (istep_key) n	
PROC_STEP	
1 (unqci_key) n	
CV_UNQCI	
1 (pmod_key) n	
CV_PR_MOD	

Serial Key None.

Foreign Keys / Link Description

config_key	From CV_CONFIG table. Identifies CV configuration entry.
cell_key	From CV_CELL table. Identifies cell entry.
wf_key	From WAFER table. Identifies wafer entry.
s_lg_key	From OP_LOG table. Contains sourced lg_key entry.
estep_key	From PROC_STEP table. Identifies electrical step.
istep_key	From PROC_STEP table. Identifies inline step.
unqci_key	From CV_UNQCI table. Identifies entry for unique cell instance per die entry.
pmod_key	From CV_PR_MOD table. Identifies process module entry.

Other Fields in CV_CELLI2IDEF

df_source	Contains entry for the inline source defect.
diex	X index of the die.
diey	Y index of the die.

et_flag Flag indicating result of electrical test: S =short, O = Open, B = both short and open.

16.11 CV_CELLI_BLOCK_NUM

Abstract Describes cell instance blocks.

Table

CV_CELLI_BLOCK_NUM		
1 (ciznm_key) n CV_CELLI_ZONE_NUM	ciblnm_key	1 (ciblnm_key) n CV_CELLI_BLOCK_DEF
1 (cizns_key) n CV_CELLI_ZONES	ciznm_key cizns_key celli_block_num celli_block_name low_x low_y high_x high_y config_key	
1 (config_key) n CV_CONFIG		

Serial Key

ciblnm_key Serial, a unique key is assigned to each entry in CV_CELLI_BLOCK_NUM. It is stored automatically.

Foreign Keys / Link Description

ciznm_key From CV_CELLI_ZONE_NUM table.

cizns_key From CV_CELLI_ZONES table.

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_CELLI_BLOCK_NUM

celli_block_num A cell instance block number.

celli_block_name A cell instance block name.

low_x Lower-left X-coordinate of the cell instance block.

low_y Lower-left Y-coordinate of the cell instance block.

high_x Upper-right X-coordinate of the cell instance block.

high_y Upper-right Y-coordinate of the cell instance block.

16.12 CV_CELLI_BLOCK_DEF

Abstract An m2n table holding all the available combinations of cell instances, cell blocks and cell zones.

Table

CV_CELLI_BLOCK_DEF	
1 (ciblnm_key) n CV_CELLI_BLOCK_NUM	ciblnm_key cizns_key config_key celli_key
1 (cizns_key) n CV_CELLI_ZONES	
1 (config_key) n CV_CONFIG	
1 (celli_key) n CV_CELL_I	

Serial Key None

Foreign Keys / Link Description

ciblnm_key	Foreign key from CV_CELLI_BLOCK_NUM.
celli_key	Foreign key from CV_CELL_I table. Identifies the cell instance entry.
cizns_key	Foreign key from CV_CELLI_ZONES table.
config_key	Foreign key from CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_CELLI_BLOCK_DEF

None.

16.13 CV_CELLI_ZONES

Abstract Describes cell instance zone sets.

Table

CV_CELLI_ZONES		
1 (config_key) n CV_CONFIG	cizns_key	1 (cizns_key) n CV_CELLI_ZONE_NUM
	cizns_name	
	config_key	
	owner	
	def_flag	1 (cizns_key) n CV_CELLI_BLOCK_NUM
	num_cizones	
	description	
	insert_time	1 (cizns_key) n CV_CELLI_BLOCK_DEF
	create_method	

Serial Key

cizns_key A unique key is assigned to each entry in CV_CELLI_ZONES. It is stored automatically.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_CELLI_ZONES

cizns_name A cell instance zone name.

owner The name/ID of the person who created the record.

def_flag Flag, which indicates if this is “default.” Legal values are **Y** and **N**.

num_cizones Number of cell instance zones.

description Description of cell instance zones.

insert_time Time of data insertion into the database, defaults to the current time.

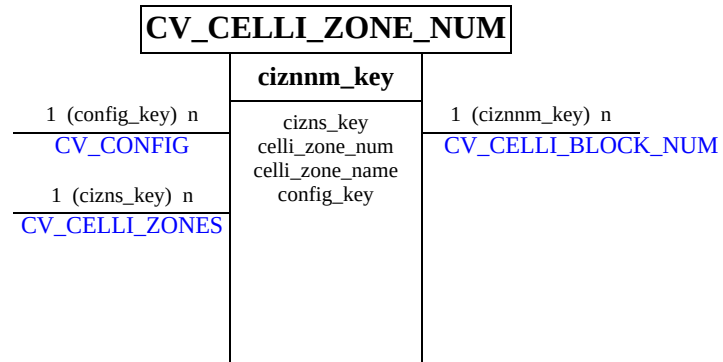
create_method Flag indicating how the record was created.

Value	Description
R	Reader: zones are created by Reader; this is default.
G	Grid: zones are created using Grid method
F	Filter: zones are created using Filter method
M	Manual: zones are created using Manual method

16.14 CV_CELLI_ZONE_NUM

Abstract Describes cell instance zone number and names.

Table



Serial Key

ciznnm_key

A unique key is assigned to each entry in CV_CELLI_ZONES. It is stored automatically.

Foreign Keys / Link Description

cizns_key

Foreign key from CV_CELLI_ZONES.

config_key

From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_CELLI_ZONE_NUM

celli_zone_num

Cell instance zone number.

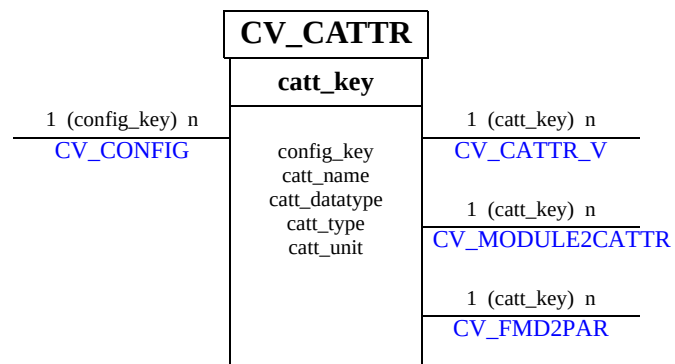
celli_zone_name

Cell instance zone name.

16.15 CV_CATTR

Abstract Describes cell attributes within CV configuration.

Table



Serial Key

catt_key

Identifies the cell attribute entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_CELL

catt_name Provides cell attribute description.

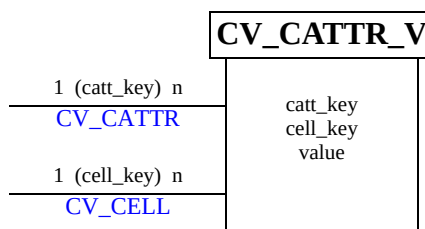
catt_datatype Provides cell attribute data type.

catt_type Provides cell attribute type.

catt_unit Provides cell attribute units.

16.16 CV_CATTR_V

Abstract Contains values for cell attributes within the CV configuration.

Table

Serial Key None.

Foreign Keys / Link Description

catt_key From CV_CATTR table. Identifies the cell attribute entry.

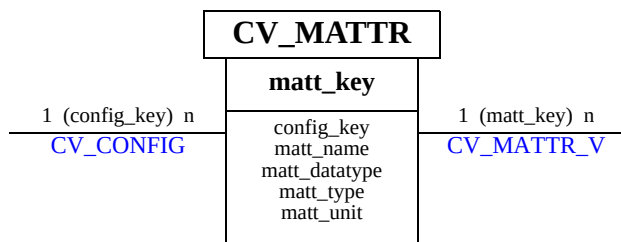
cell_key From CV_CELL table. Identifies cell entry.

Other Fields in CV_CATTR_V

value Holds the value for a particular cell attribute.

16.17 CV_MATTR

Abstract Holds measurement attributes.

Table**Serial Key**

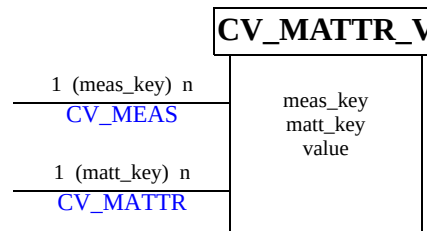
matt_key Identifies measurement attribute entry

Foreign Keys / Link Description	config_key	From CV_CONFIG table. Identifies CV configuration entry.
Other Fields in CV_MATTR	matt_name	Contains measurement attribute name
	matt_datatype	Contains measurement attribute data type
	matt_type	Contains measurement attribute type
	matt_unit	Contains measurement attribute unit

16.18 CV_MATTR_V

Abstract Holds measurement attribute values.

Table

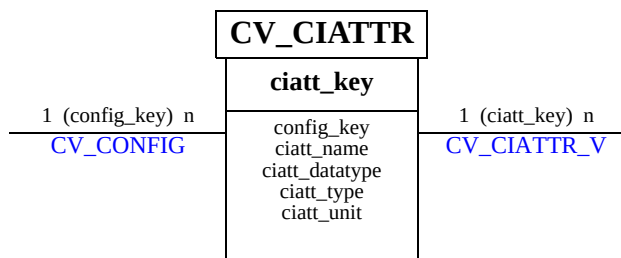


Serial Key None.

Foreign Keys / Link Description	meas_key	From CV_MEAS table. Identifies measurement entry.
	matt_key	From CV_MATTR table. Identifies measurement attribute entry.
Other Fields in CV_MATTR_V	value	Contains a value for specific measurement attribute.

16.19 CV_CIATTR

Abstract Holds information on cell instance attributes within CV configuration.

Table**Serial Key**

ciatt_key Identifies the cell instance attribute entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_CIATTR

ciatt_name Describes the cell instance attribute.

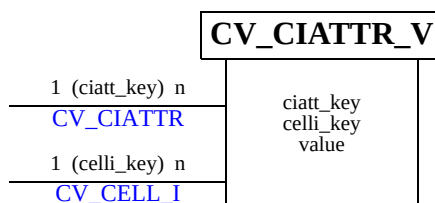
ciatt_datatype Describes the cell instance attribute data type.

ciatt_type Describes the cell instance attribute type.

ciatt_unit Describes the cell instance attribute unit.

16.20 CV_CIATTR_V

Abstract Holds information on values of cell instance attributes within CV configuration.

Table

Serial Key None.

Foreign Keys / Link Description

ciatt_key From CV_CIATTR table. Identifies the cell instance attribute entry.

celli_key From CV_CELL_I table. Identifies the cell instance entry.

Other Fields in CV_CIATTR_V

value Holds the value for a particular cell instance attribute.

16.21 CV_MEAS

Abstract Provides information on measurements within the CV configuration.

Table

CV_MEAS		
	meas_key	
1 (config_key) n CV_CONFIG	config_key cell_key celli_key meas_name meas_type	1 (meas_key) n CV_MEAS2PIN
1 (cell_key) n CV_CELL		1 (meas_key) n CV_LIMITS_V
1 (celli_key) n CV_CELL_I		1 (meas_key) n CV_MAP2IN
		1 (meas_key) n CV_MATTR_V
		1 (par_key) n CV_PAR2MEAS

Serial Key

meas_key Identifies the measurement entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

cell_key From CV_CELL table. Identifies cell entry.

celli_key From CV_CELL_I table. Identifies the cell instance entry.

Other Fields in CV_MEAS

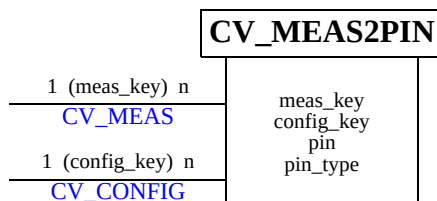
meas_name Describes the measurement name.

meas_type Describes the measurement type.

16.22 CV_MEAS2PIN

Abstract Holds relationships between measurements and pins within the CV configuration.

Table



Serial Key None.

Foreign Keys / Link Description

meas_key From CV_MEAS table. Identifies the measurement entry.

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_MEAS2PIN

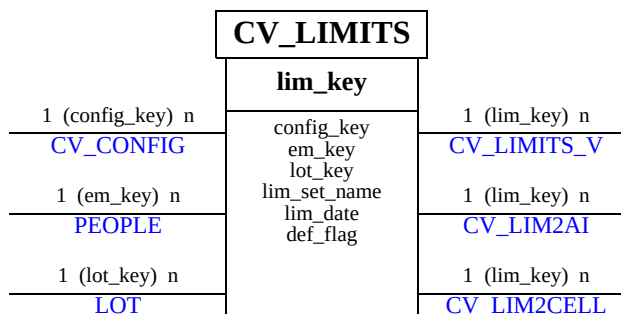
pin Contains the pin number.

pin_type Describes pin type.

16.23 CV_LIMITS

Abstract Holds specification limits for measurements.

Table



Serial Key

lim_key Identifies limit entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

em_key From PEOPLE table. Identifies employee entry.

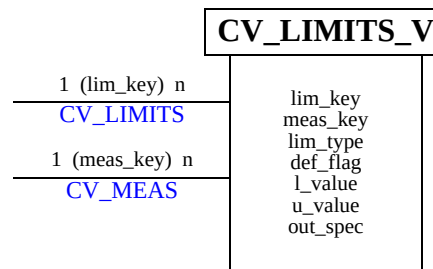
lot_key From LOT table. Identifies lot entry.

**Other Fields in
CV_LIMITS**

lim_set_name	Describes limit set.
lim_date	Time stamp of the limit set.
def_flag	Flag to specify default limit set.

16.24 CV_LIMITS_V

Abstract Holds the specification limit values for measurements.

Table

Serial Key None.

**Foreign Keys / Link
Description**

lim_key	From CV_LIMITS table. Identifies the limit entry.
meas_key	From CV_MEAS table. Identifies measurement entry.

**Other Fields in
CV_LIMITS_V**

lim_type	Specifies limit type.
def_flag	Specifies default flag.
l_value	Contains value for a particular lower specification limit.
u_value	Contains value for a particular upper specification limit.
out_spec	Flag to specify outer specification limit.

16.25 CV_LIM2CELL

Abstract An m2n table holding all the available combination of lim_keys and cell_keys.

Table

CV_LIM2CELL	
1 (lim_key) n CV_LIMITS	lim_key meas_key lim_type def_flag l_value u_value out_spec
1 (meas_key) n CV_MEAS	

Serial Key None.

Foreign Keys / Link Description

lim_key From CV_LIMITS table. Identifies the limit entry.

meas_key From CV_MEAS table. Identifies measurement entry.

Other Fields in CV_LIM2CELL

lim_type Specifies limit type.

16.26 CV_AI

Abstract Holds CV analysis items.

Table

CV_AI	
ai_key	
1 (config_key) n CV_CONFIG	1 (ai_key) n CV_AATTR_V
1 (cell_key) n CV_CELL	1 (ai_key) n CV_MAP2OUT
1 (celli_key) n CV_CELL_I	1 (ai_key) n CV_DEFECT
1 (fft_key) n CV_FINE_FTYPE	1 (ai_key) n CV_LIM2AI
1 (pmod_key) n CV_PR_MOD	

Serial Key **ai_key** Identifies an.item entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies the CV configuration entry.

cell_key From CV_CELL table. Identifies the cell entry.

	celli_key	From CV_CELL_I table. Identifies the cell instance entry.
	fft_key	From CV_FINE_FTYPE table. Identifies the fine fail type entry.
	pmod_key	From CV_PR_MOD table. Identifies the process module entry.
Other Fields in CV_AI	ai_name	Describes analysis map name.
	fail_type	Describes fail type.

16.27 CV_AI2AI

Abstract Holds relations between analysis items.

Table

CV_AI2AI
ai_key_i ai_key_o
yield_type

Serial Key	ai_key_i	Holds reference to input analysis item.
	ai_key_o	Holds reference to the output analysis item.

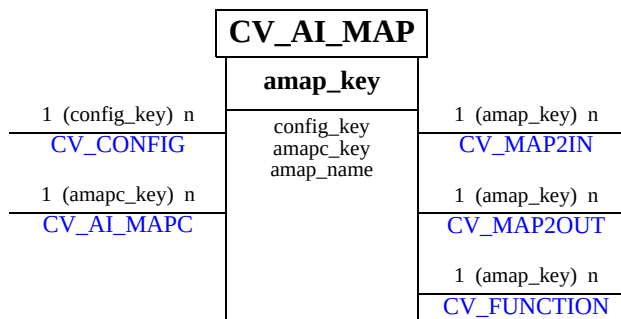
Foreign Keys / Link Description None.

Other Fields in CV_AI2AI	yield_type	Contains yield type.
---------------------------------	-------------------	----------------------

16.28 CV_AI_MAP

Abstract Holds analysis map names.

Table



Serial Key

amap_key Identifies analysis map entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

amapc_key From CV_AI_MAPC table. Identifies analysis map class entry.

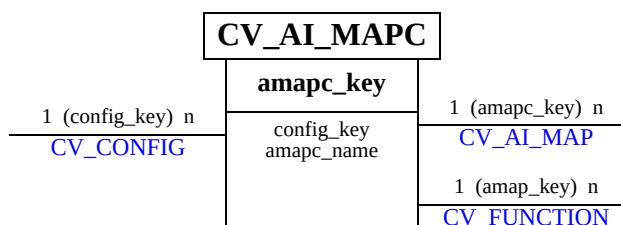
Other Fields in CV_AI_MAP

amap_name Describes analysis map name.

16.29 CV_AI_MAPC

Abstract Holds analysis map classes.

Table



Serial Key

amapc_key Identifies analysis map class entry

config_key From CV_CONFIG table. Identifies CV configuration entry.

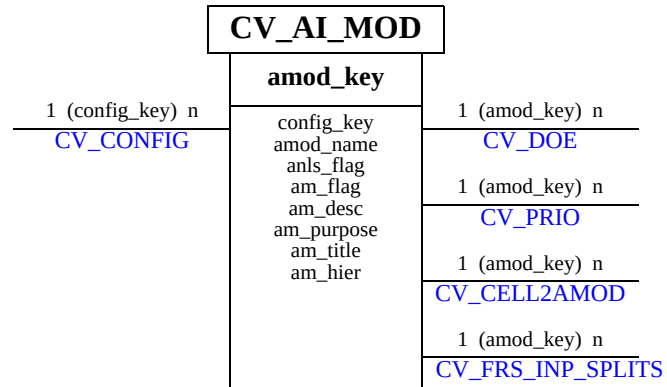
Other Fields in CV_AI_MAPC

amapc_name Describes analysis map class name.

16.30 CV_AI_MOD

Abstract Holds information on analysis modules.

Table



Serial Key

amod_key Identifies an. module.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_AI_MOD

amod_name Describes an. module.

anls_flag Contains analysis flag.

am_flag Contains an.module flag.

am_desc Contains an.module description.

am_purpose Contains an.module purpose.

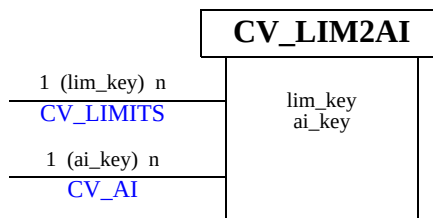
am_title Contains an.module title.

am_hier Contains an.module hierarchy.

16.31 CV_LIM2AI

Abstract ai_key relationship to lim_key.

Table



Serial Key None.

Foreign Keys / Link Description

lim_key From the LIM_LOG table. This table has information about the limit sets.

ai_key From CV_AI table. Identifies an item entry.

Other Fields in CV_LIM2AI None.

16.32 CV_PR_MOD**Abstract** Holds information on the process module.**Table**

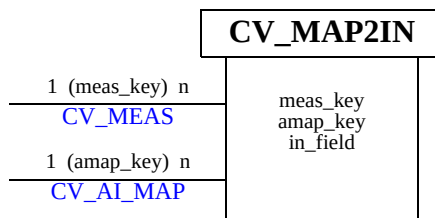
CV_PR_MOD		
	pmod_key	
1 (config_key) n CV_CONFIG	config_key pmod_name pm_order pm_type	1 (pmod_key) n CV_CELL
		1 (pmod_key) n CV_DOE2CELL
		1 (pmod_key) n CV_DEFECT
		1 (pmod_key) n CV_EDEF2IDEF
		1 (pmod_key) n CV_EDEF2IDEF_ALL
		1 (pmod_key) n CV_CELLI2IDEF
		1 (pmod_key) n CV_SUM_ZONE
		1 (pmod_key) n CV_AI
		1 (pmod_key) n CV_E2I_INHERIT
		1 (pmod_key) n CV_FRM_CLUST_DET
		1 (pmod_key) n CV_FRS_INP_SPLITS

Serial Key	pmod_key	Identifies process module entry.
Foreign Keys / Link Description	config_key	From CV_CONFIG table. Identifies CV configuration entry.
Other Fields in CV_PR_MOD	pmod_name	Describes process module.
	pm_order	Contains process module order.
	pm_type	Contains process module type.

16.33 CV_MAP2IN

Abstract Holds relationships between measurements and analysis maps.

Table



Serial Key None.

Foreign Keys / Link Description

meas_key From CV_MEAS table. Identifies measurement entry.

amap_key From CV_AI_MAP table. Identifies analysis map entry.

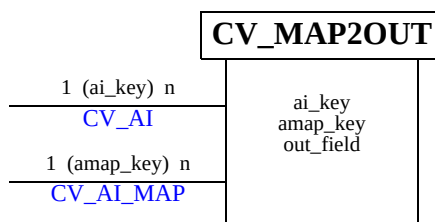
Other Fields in CV_MAP2IN

in_field Contains input field.

16.34 CV_MAP2OUT

Abstract Holds information on analysis items and output fields.

Table



Serial Key None.

Foreign Keys / Link Description

ai_key From CV_AI table. Identifies an item entry.

amap_key From CV_AI_MAP table. Identifies analysis map entry.

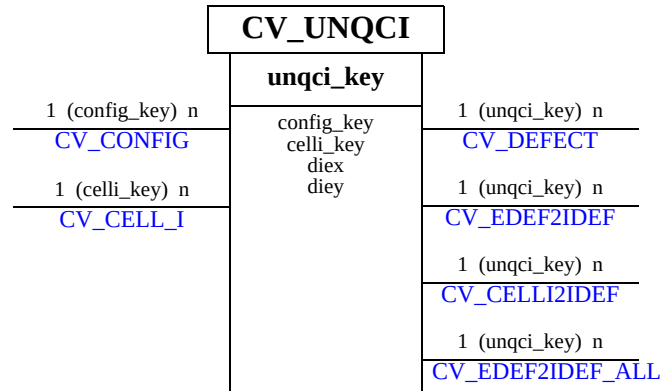
Other Fields in CV_MAP2OUT

out_field Describes output field.

16.35 CV_UNQCI

Abstract Holds information on unique cell instances.

Table



Serial Key

unqci_key Identifies entry for unique cell instance per die entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry

celli_key From CV_CELL_I table. Identifies the cell instance entry.

Other Fields in CV_UNQCI

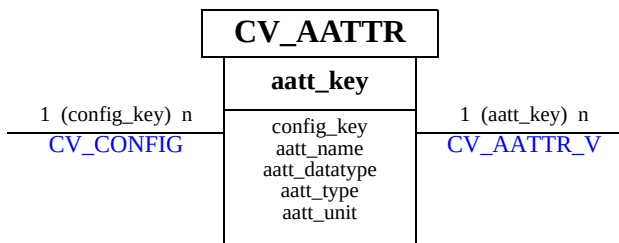
diex X index of the die

diey Y index of the die

16.36 CV_AATTR

Abstract Holds information on analysis item attributes.

Table



Serial Key **aatt_key** Identifies an.item entry.

Foreign Keys / Link Description **config_key** From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_AATTR **aatt_name** Describes an.item attribute name.

aatt_datatype Describes an.item attribute data type.

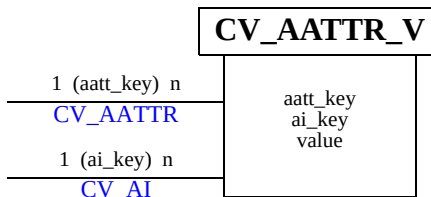
aatt_type Describes an.item attribute type.

aatt_unit Describes an.item attribute unit.

16.37 CV_AATTR_V

Abstract Holds values of analysis item attributes.

Table



Serial Key None.

Foreign Keys / Link Description **aatt_key** From CV_AATTR table. Identifies an.item attribute entry.

ai_key From CV_AI table. Identifies an.item entry.

Other Fields in CV_AATTR_V **value** Contains a value for a particular an.item attribute.

16.38 CV_DOE**Abstract** Holds information on the DOE.**Table**

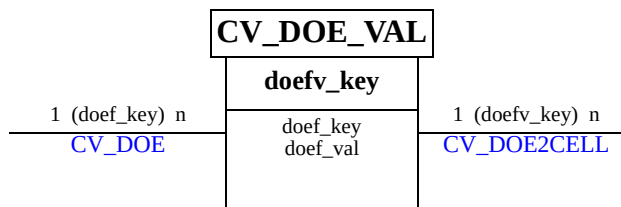
CV_DOE		
doef_key		
1 (config_key) n	config_key amod_key doef_name doef_sname catt_key1 catt_key2 catt_key3 catt_key4 pmod_flag doe_type plot_map data_type anls_flag doe_hier	1 (doef_key) n
CV_CONFIG		CV_DOE_VAL
1 (amod_key) n		1 (doef_key) n
CV_AI_MOD		CV_DOE2CELL
		1 (doef_key) n
		CV_FRS_INP_SPLITS

Serial Key**doef_key** Identifies DOE entry**Foreign Keys / Link Description****config_key** From CV_CONFIG table. Identifies the CV configuration entry.**amod_key** From CV_AI_MOD table. Identifies analysis module entry**Other Fields in CV_DOE****doef_name** Describes DOE factor name**doef_sname** Describes DOE factor short name.**catt_key1** Contains attribute key1.**catt_key2** Contains attribute key2.**catt_key3** Contains attribute key3.**catt_key4** Contains attribute key4.**pmod_flag** Contains process module flag.**doe_type** Contains DOE type.**plot_map** Integer that specifies the default plot variables.**data_type** Contains data type.**anls_flag** Contains analysis flag.**doe_hier** Reserved for future use.

16.39 CV_DOE_VAL

Abstract Holds information on DOE values.

Table



Serial Key

doefv_key Identifies DOE factor value entry.

Foreign Keys / Link Description

doef_key From CV_DOE table. Identifies DOE entry.

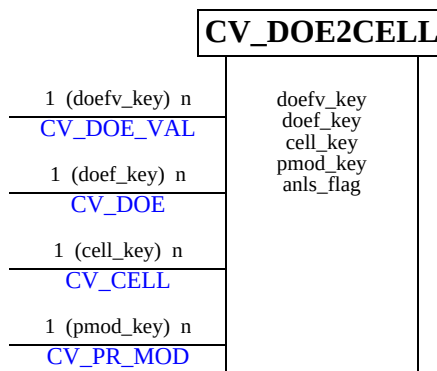
Other Fields in CV_DOE_VAL

doef_val Contains DOE factor value.

16.40 CV_DOE2CELL

Abstract Holds relationships between DOE factor values and cells.

Table



Serial Key None.

Foreign Keys / Link Description

doefv_key From CV_DOE_VAL table. Identifies DOE factor value entry.

doef_key From CV_DOE table. Identifies DOE entry.

cell_key From CV_CELL table. Identifies cell entry.

pmod_key From CV_PR_MOD table. Identifies Process module entry.

Other Fields in CV_DOE2CELL

anls_flag Contains analysis flag.

16.41 CV_MODEL

Abstract Holds information on the model.

Table

CV_MODEL		
	model_key	
1 (config_key) n CV_CONFIG	config_key em_key frm_mod_key model_id	1 (model_key) n CV_FMD2MDL
1 (em_key) n PEOPLE		1 (model_key) n CV_FRM_DEF
1 (frm_mod_key) n CV_FRM_MOD		1 (model_key) n CV_FRM_COV
		1 (model_key) n CV_FRM_RUN
		1 (model_key) n CV_USC_DEF
		1 (model_key) n CV_FRM_DIAG_V
		1 (model_key) n CV_FRM_OUT_SUM

Serial Key

model_key Identifies model attribute entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

em_key From PEOPLE table. Identifies employee entry.

frm_mod_key From CV_FRM_MOD table. Identifies FRM module entry.

Other Fields in CV_MODEL

model_id Model name.

16.42 CV_MODEL_DEF

Abstract Holds information on which cells, fine-fail type, fail layer are related to the fail mode.

Table

CV_MODEL_DEF	
1 (config_key) n CV_CONFIG	config_key fmd_key model_key cell_key fft_key flyr_key fail_type
1 (fmd_key) n CV_FAIL_MODE	
1 (model_key) n CV_MODEL	
1 (cell_key) n CV_CELL	
1 (fft_key) n CV_FINE_FTYPE	
1 (flyr_key) n CV_FAIL_LAYER	

Serial Key None.

Foreign Keys / Link Description

config_key	From CV_CONFIG table. Identifies CV configuration entry
cell_key	From CV_CELL table. Identifies cell entry.
fft_key	From table CV_FINE_FTYPE table. Identifies fine fail type entry.
flyr_key	From CV_FAIL_LAYER table. Identifies fail layer entry.
fmd_key	From CV_FAIL_MODE table. Identifies fail mode key.
model_key	From CV_MODEL table. Identifies the model entry.
fail_type	Contains fail type.

Other Fields in CV_MODEL_DEF

16.43 CV_FAIL_MODE

Abstract Holds information on fail mode.

Table

CV_FAIL_MODE		
	fmd_key	
1 (config_key) n	config_key	1 (fmd_key) n
CV_CONFIG	scale_key	CV_FMD2MDL
1 (config_key) n	ftp_key	1 (fmd_key) n
CV_FMD_TYPE	fmode_name	CV_FRM_COV
	x0	1 (fmd_key) n
	weight	CV_FRM_DEF
	fail_type	1 (fmd_key) n
	xmax	CV_FRM_OUT_SUM
	scale_type	CV_USC_DEF
		1 (fmd_key) n
		CV_MODULE2CELL
		1 (fmd_key) n
		CV_MODULE2CATTR
		1 (fmd_key) n
		CV_FRM_USC_<model_key>

Serial Key

fmd_key Fail mode key (serial key).

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

scale_key Same as [CV_CATTR](#) table serial key, **catt_key**, if **scale_type = C**. Otherwise, NULL. Used if fail mode is being scaled by cell attribute.

ftp_key From CV_FMD_TYPE. Fail mode type (Do, Lambda, Df).

Other Fields in CV_FAIL_MODE

fmode_name Fail mode name (character string).

x0 Minimum defect size used in Ac(p) integration (float).

weight Scaler used to multiply scale factor (float).

fail_type Fail type: **O** = open, **S** = short.

xmax Maximum defect size used in Ac(p) integration (float).

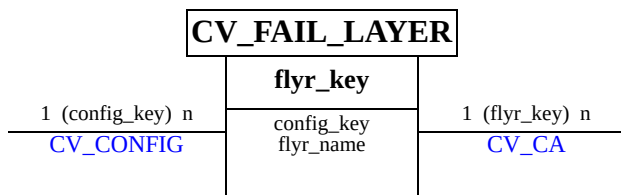
scale_type Scale type:

Value	Description
V	number of vias — scale fail mode by cv_ca_nv.acp_nv
C	cell attribute — scale fail mode by cv_cattr_v.value
A	critical area — scale fail mode by integrated critical area

16.44 CV_FAIL_LAYER

Abstract Holds information on fail layer.

Table



Serial Key

flyr_key Identifies fail layer entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_FAIL_LAYER

flyr_name Contains fail layer name.

16.45 CV_FINE_FTYPE

Abstract Holds information on fine fail type.

Table

CV_FINE_FTYPE		
	fft_key	
1 (config_key) n CV_CONFIG	config_key fine_fail_type fail_type adl_flag	1 (fft_key) n CV_MODEL_DEF
		1 (fft_key) n CV_CA
		1 (fft_key) n CV_CA_NV
		1 (fft_key) n CV_DEFECT
		1 (fft_key) n CV_FRN
		1 (fft_key) n CV_AI
		1 (fft_key) n CV_FRM_OUT_DATA_<model_key>

Serial Key	fft_key	Identifies fine fail type entry.
Foreign Keys / Link Description	config_key	From CV_CONFIG table. Identifies the CV configuration entry.
Other Fields in CV_FINE_FTYPE	fine_fail_type	Describes the fine fail type.
	fail_type	Contains the fail type.
	adl_flag	Contains the ADL flag.

16.46 CV_FRN

Abstract Holds information on fail rates.

Table

CV_FRN	
1 (cell_key) n	cell_key wf_key fft_key p_key fail_type pm_type num_die n
CV_CELL	
1 (wf_key) n	
WAFER	
1 (fft_key) n	
CV_FINE_FTYPE	
1 (p_key) n	
CV_P_VAL	

Serial Key None.

Foreign Keys / Link Description

cell_key From CV_CELL table. Identifies cell entry.

wf_key From WAFER table. Identifies wafer entry.

fft_key From CV_FINE_FTYPE table. Identifies fine fail type entry.

p_key From CV_P_VAL table. Identifies p entry.

Other Fields in CV_FRN

fail_type Contains fail type.

pm_type Contains process module type.

num_die Contains number of dies.

n Contains number of cell instances.

16.47 CV_FMD_TYPE

Abstract Holds information on fail mode type.

Table

CV_FMD_TYPE	
1 (config_key) n	1 (ftp_key) n
CV_CONFIG	CV_FAIL_MODE
ftp_key config_key fmd_type	

Serial Key

ftp_key Fail mode type key.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies the CV configuration entry.



Front



Contents



Index

Other Fields in
CV_FMD_TYPE**fmd_type** Fail mode type.

16.48 CV_FMD2MDL

Abstract Holds information on the relationship between model and fail mode.**Table**

CV_FMD2MDL		
	fail_key	
1 (fmd_key) n CV_FAIL_MODE	fmd_key	1 (fail_key) n CV_FMD2PAR
1 (model_key) n CV_MODEL	model_key	1 (fail_key) n CV_FMD2LSPLT
1 (frm_pm_key) n CV_FRM_PARM	frm_pm_key	1 (fail_key) n CV_FRM_PARAM
1 (config_key) n CV_CONFIG	config_key	1 (fail_key) n CV_FORM2LSPLT
	formula_id	
	formula_id1	
	formula_id2	
	p_formula_id	
	p_formula_id1	
	p_formula_id2	
	inter_flag	
	p_val	
	x0	
	xmax	

Serial Key**fail_key** Serial key.**Foreign Keys / Link
Description****fmd_key** From the CV_FAIL_MODE table.**model_key** From the CV_MODEL table.**frm_pm_key** From the CV_FRM_PARM table.**config_key** From CV_CONFIG table. Identifies CV configuration entry.**Other Fields in
CV_FMD2MDL****formula_id** User defined parameter formula.**formula_id1** User defined parameter formula.**formula_id2** User defined parameter formula.**p_formula_id** User defined parameter formula.**p_formula_id1** User defined parameter formula.**p_formula_id2** User defined parameter formula.**inter_flag** Reserved for future use.

p_val	When modeling, p-value, as defined by the user in the GUI.
x0	When modeling, X0, as defined by the user in the GUI.
xmax	Maximum defect size used in Ac(p) integration (float).

16.49 CV_FMD2PAR

Abstract Holds information on the relationship of fail mode to parameters.

Table

CV_FMD2PAR	
1 (config_key) n CV_CONFIG	config_key
1 (par_key) n CV_FRM_PARAM	par_key fail_key att_key param_type grp_index att_type comp_type
1 (fail_key) n CV_FMD2MDL	
1 (catt_key) n CV_CATTR	

Serial Key None.

Foreign Keys / Link Description

config_key	From CV_CONFIG table. Identifies CV configuration entry.
par_key	From CV_FRMPARAM table. Identifies parameter entry.
fail_key	From CV_FMD2MDL table. Identifies fail rate mode entry.
catt_key	From CV_CATTR table. Identifies cell attribute entry.

Other Fields in CV_FMD2PAR

param_type Parameter type:

Value	Description
P	process module
W	wafer
Z	zones
C	cell attribute

grp_index Index for grouping cell attribute.

att_type Cell attribute type:

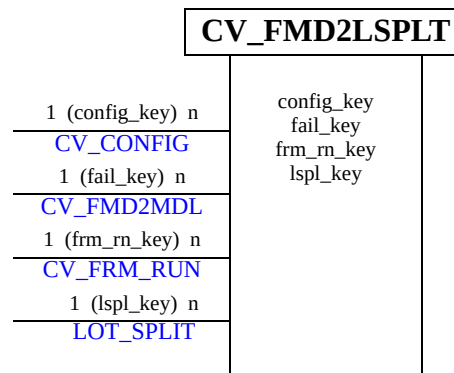
Value	Description
S	string
N	numeric

comp_type Reserved for future use.

16.50 CV_FMD2LSPLT

Abstract Holds information on relationship of fail mode to lot splits.

Table



Serial Key None.

Foreign Keys / Link Description

config_key	From CV_CONFIG table. Identifies CV configuration entry.
fail_key	From CV_FMD2MDL table. Identifies fail rate mode entry.
frm_rn_key	From the CV_FRM_RUN table. Identifies run entry.
lspl_key	From LOT_SPLIT table. A reference to the lot split.

Other Fields in CV_FMD2LSPLT None.

16.51 CV_FRM_PARAM

Abstract Holds information on fail rate parameters.

Table

CV_FRM_PARAM		
	par_key	
1 (config_key) n CV_CONFIG	config_key	1 (par_key) n CV_USC_DEF
1 (fail_key) n CV_FMD2MDL	fail_key	1 (par_key) n CV_FMD2PAR
	grp_index	

Serial Key

par_key

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

fail_key From CV_FMD2MDL table. Identifies fail rate mode entry.

Other Fields in CV_FRM_PARAM

grp_index Grouping parameters.

16.52 CV_FRM_PARM

Abstract Holds the user defined parameters list for the model.

Table

CV_FRM_PARM		
	frm_pm_key	
1 (config_key) n CV_CONFIG	config_key	1 (frm_pm_key) n CV_FMD2MDL
	model_key	
	frm_pm_name	
	owner	1 (frm_pm_key) n CV_FRM_RUN
1 (model_key) n CV_MODEL	frm_pm_desc	1 (frm_pm_key) n CV_USC_DEF
	insert_time	

Serial Key

frm_pm_key

Foreign Keys / Link Description

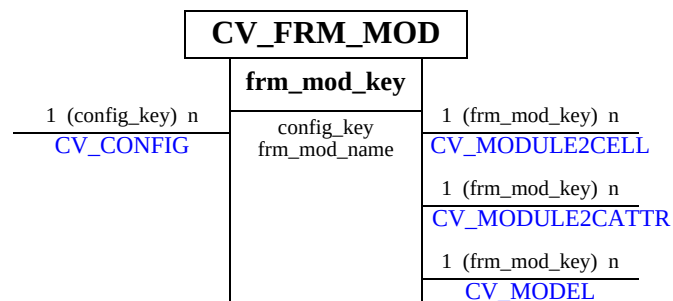
config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_FRM_PARM	model_key	From the CV_MODEL table.
	frm_pm_name	User-defined parameter name.
	owner	The name/ID of the person who saved the parameter into the database.
	frm_pm_desc	Parameter description.
	insert_time	Time of parameter insertion into the database, defaults to the current time.

16.53 CV_FRM_MOD

Abstract Holds information on Fail Rate Mode modules.

Table



Serial Key	frm_mod_key	
Foreign Keys / Link Description	config_key	From CV_CONFIG table. Identifies CV configuration entry.
	frm_mod_name	Failure rate mode name.
Other Fields in CV_FRM_MOD		

16.54 CV_FRM_CLUST_DET

Abstract Clustering results. Percentage of clustered die per wafer, per process module.

Table

CV_FRM_CLUST_DET	
1 (frm_rn_key) n CV_FRM_RUN	frm_rn_key
1 (lot_key) n LOT	lot_key
1 (wf_key) n WAFER	wf_key
1 (pmod_key) n CV_PR_MOD	pmod_key
1 (config_key) n CV_CONFIG	config_key
	bad_die_pct
	cluster_flag

Serial Key None.

Foreign Keys / Link Description	
frm_rn_key	From the CV_FRM_RUN table. Identifies the run entry.
lot_key	From LOT table. Identifies lot entry.
wf_key	From WAFER table. Identifies wafer entry.
pmod_key	From CV_PR_MOD table. Identifies the process module entry.
config_key	From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_FRM_CLUST_DET

bad_die_pct	Percentage of clustered die.
cluster_flag	Y = greater than the value set in crit_bad_die_pct in the CV_FRM_RUN table; otherwise = N.

16.55 CV_FRM_COV

Abstract Holds information on covariance results.

Table

CV_FRM_COV	
1 (model_key) n CV_MODEL	model_key
1 (lot_key) n LOT	lot_key
1 (fmd_key1-2) n CV_FAIL_MODE	fmd_key1
1 (cvlbl_key1-2) n CV_COEF_LABEL	fmd_key2
1 (frm_rn_key) n CV_FRM_RUN	cvlbl_key1
	cvlbl_key2
	frm_rn_key
	coef_order
	coef_value

Serial Key None.

Foreign Keys / Link Description

model_key From the CV_MODEL table. Identifies model entry.

lot_key From LOT table. Identifies lot entry.

fmd_key1 From the CV_FAIL_MODE table. Identifies fail mode entry.

fmd_key2 From the CV_FAIL_MODE table. Identifies fail mode entry.

cvlbl_key1 From the CV_COEF_LABEL table. Identifies coefficient label.

cvlbl_key2 From the CV_COEF_LABEL table. Identifies coefficient label.

frm_rn_key From the CV_FRM_RUN table. Identifies the run entry.

Other Fields in CV_FRM_COV

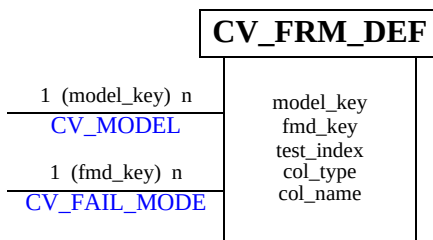
coef_value Covariance value of coefficients.

coef_order Coefficients order.

16.56 CV_FRM_DEF

Abstract Defines the corresponding columns in the CV_FRM_OUT_DATA_<model_key> table (“CV_FRM_OUT_DATA_<model_key>,” page 384).

Table



Serial Key None.

Foreign Keys / Link Description

model_key From the CV_MODEL table. Identifies model entry.

fmd_key From the CV_FAIL_MODE table. Identifies fail mode entry.

Other Fields in CV_FRM_DEF

test_index The corresponding t... column index from the CV_FRM_OUT_DATA_<model_key> table.

col_type The column type:

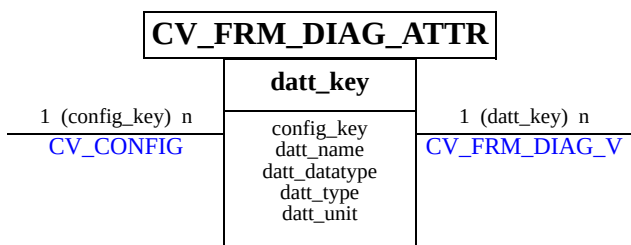
Value	Description
M	mu
S	scale
T	standard error

col_name Column name.

16.57 CV_FRM_DIAG_ATTR

Abstract Holds information on Fail Rate Modeling diagnostic attributes.

Table



Serial Key **datt_key** Diagnostic attribute key.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_FRM_DIAG_ATTR

datt_name Diagnostic attribute name.

datt_datatype Date type of diagnostic attribute.

Value	Description
S	string
N	numeric

datt_type Diagnostic attribute type.

datt_unit Diagnostic attribute unit.

16.58 CV_FRM_DIAG_V

Abstract Holds Fail Rate Modeling diagnostic value, per run.

Table

CV_FRM_DIAG_V	
1 (config_key) n	config_key model_key datt_key frm_rn_key value value1 value2
CV_CONFIG	
1 (model_key) n	
CV_MODEL	
1 (frm_rn_key) n	
CV_FRM_RUN	
1 (datt_key) n	
CV_FRM_DIAG_ATTR	

Serial Key None.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

model_key From the CV_MODEL table. Identifies model entry.

frm_rn_key From the CV_FRM_RUN table.

datt_key From the CV_FRM_DIAG_ATTR table.

Other Fields in CV_FRM_DIAG_V

value Diagnostic value.

value1 continuation of **value**.

value2 continuation of **value1**.

16.59 CV_FRM_OUT_SUM

Abstract Holds information on Fail Rate Modeling output summary results.

Table

CV_FRM_OUT_SUM	
1 (model_key) n CV_MODEL	model_key
1 (fmd_key) n CV_FAIL_MODE	frm_rn_key fmd_key cvlbl_key
1 (frm_rn_key) n CV_FRM_RUN	out_type fresult1 fresult2 quantile prob_val
1 (cvlbl_key) n CV_COEF_LABEL	

Serial Key None.

Foreign Keys / Link Description

model_key	From the CV_MODEL table. Identifies model entry.
frm_rn_key	From the CV_FRM_RUN table. Identifies run entry.
fmd_key	From the CV_FAIL_MODE table. Identifies fail mode entry.
cvlbl_key	From the CV_COEF_LABEL table. Identifies coefficient label.

Other Fields in CV_FRM_OUT_SUM

out_type	Output type of FRM results: A = ANOVA; C = coefficient.
fresult1	If out_type is A : fresult1 = Df — degrees of freedom for source. If out_type is C : fresult1 = coef — parameter estimates.
fresult2	If out_type is A : fresult2 = Dev — model deviance with source dropped. If out_type is C : fresult2 = std_err — standard errors for parameters.

quantile If **out_type** is **A**: **quantile** = **FVal** — F statistics of refitted model.
 If **out_type** is **C**: **quantile** = **Zval** — z-score for parameter,

prob_val If **out_type** is **A**: **prob_val** = **Pr(>F)** — p-value of significance for dropping source.
 If **out_type** is **C**: **prob_val** = **Pr(>|Z|)** — p-value significance for parameter = 0

16.60 CV_FRM_RUN

Abstract This table holds run information when the user creates a run.

Table

CV_FRM_RUN		
	frm_rn_key	
1 (model_key) n	model_key	1 (frm_rn_key) n
CV_MODEL	frm_pm_key	CV_FRM_OUT_SUM
1 (frm_pm_key) n	config_key	1 (frm_rn_key) n
CV_FRM_PARM	zone_key	CV_FRM_DIAG_V
	fltr_key	
	frm_rn_name	1 (frm_rn_key) n
	frm_rn_desc	CV_FRM_COV
	insert_time	
1 (config_key) n	end_time	1 (frm_rn_key) n
CV_CONFIG	fail_style	CV_FMD2LSPLT
	aggregate	
	agg_level	1 (frm_rn_key) n
1 (fltr_key) n	status	CV_FORM2LSPLT
CV_FLTR	apply_cluster_flag	
	min_prob	1 (frm_rn_key) n
	min_count	CV_FRM_RUN_DET
1 (zone_key) n	crit_bad_die_pct	
ZONES	cluster_alg_type	1 (frm_rn_key) n
	all_cluster_flag	CV_FRM_CLUST_DET
	fltr_getmisval_flag	
	owner	1 (frm_rn_key) n
	error_code	CV_FRM_OUT_DATA_<model_key>
	err_msg	
	err_msg1	1 (frm_rn_key) n
	err_msg2	CV_FRM_USC_<model_key>

Serial Key

frm_rn_key Serial key.

Foreign Keys / Link Description

model_key From the CV_MODEL table. Identifies model entry.

frm_pm_key From the CV_FRM_PARM table. Identifies user-defined parameter.

Other Fields in CV_FRM_RUN

config_key From CV_CONFIG table. Identifies CV configuration entry.

zone_key From ZONES table.

fltr_key From CV_FLTR table.

frm_rn_name The name the run, specified by the user in the GUI.

frm_rn_desc The description for the run, specified by the user.

insert_time Time of run insertion into the database, defaults to the current time.

end_time End time of run insertion into the database.

fail_style Fail style: **H** = hard; **S** = soft.

aggregate Fail count format: **S** = structure; **D** = die.

agg_level Aggregation level of the data, as defined in the GUI. Four-character string: LDWZ — L = lot, D = die, W = wafer, Z = zone. For each character 1 = on, 0 = off.

status Status of the run.

Value	Description
-1	error
0	new
1	in progress
2	done

apply_cluster_flag Whether or not user wants to apply cluster, as defined in the GUI. Y = yes, N = no.

min_prob Minimum probability for clustering, as defined by the user in the GUI.

min_count Minimum fail count for clustering, as defined by the user in the GUI.

crit_bad_die_pct Critical bad die percentage, as defined by the user in the GUI.

cluster_alg_type Cluster algorithm type for clustering. Currently hard-coded to 1, only.

all_cluster_flag Y = all wafers are clustered; N = otherwise.

fltr_getmisval_flag	Y = include undefined cell attributes for filtering; N = do not include.
owner	The user who created the run.
error_code	Error code value, given by the system.
err_msg	Error message generated by the system.
err_msg1	Concatenated error message, used when message is greater than 255 characters (capacity of err_msg).
err_msg2	Concatenated error message, used when message is greater than 510 characters (capacity of err_msg and err_msg1).

16.61 CV_FRM_RUN_DET

Abstract Holds detailed information related to a run.

Table

CV_FRM_RUN_DET	
1 (frm_rn_key) n CV_FRM_RUN	frm_rn_key lot_key wf_key config_key
1 (lot_key) n LOT	
1 (wf_key) n WAFER	
1 (config_key) n CV_CONFIG	

Serial Key None.

Foreign Keys / Link Description

frm_rn_key	From the CV_FRM_RUN table.
lot_key	From LOT table. Identifies lot entry.
wf_key	From WAFER table. Identifies wafer entry.
config_key	From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_FRM_RUN_DET None.

16.62 CV_EXPORT

Abstract CV_EXPORT contains information on automatic and custom requests for summary exports.

Table

CV_EXPORT	
	cv_exp_key
1 (lot_key) n LOT	lot_key
1 (wf_key) n WAFER	wf_key
1 (config_key) n CV_CONFIG	config_key
1 (zone_key) n ZONES	zone_key
1 (config_key) n CV_FLTR	fltr_key
	insert_time
	end_time
	flag
	status
	owner
	error_code
	err_msg
	exp_dir

Serial Key

cv_exp_key Serial key for export operation.

Foreign Keys / Link Description

lot_key From LOT table. Identifies lot entry.

wf_key From WAFER table. Identifies wafer entry.

config_key From CV_CONFIG table. Identifies CV configuration entry.

zone_key Value from ZONES table.

fltr_key Value from CV_FLTR table.

Other Fields in CV_EXPORT

insert_time Time of data insertion into the database, defaults to the current time.

end_time Time when data insertion was finished.

flag Defines output summary type.

status

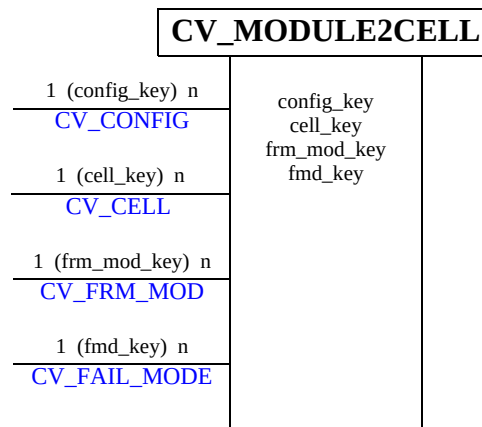
Value	Description
-1	error
0	new
1	in progress
2	done

owner	User description.
error_code	Error code number.
err_msg	Error description.
exp_dir	Destination directory for export.

16.63 CV_MODULE2CELL

Abstract Holds information on the relationship between fail mode and cells.

Table



Serial Key None.

Foreign Keys / Link Description

config_key	From CV_CONFIG table. Identifies CV configuration entry.
cell_key	From CV_CELL table. Identifies cell entry.
frm_mod_key	From CV_FRM_MOD table.
fmd_key	Fail mode key from CV_FAIL_MODE table.

16.64 CV_FRM_OUT_DATA_<model_key>

Abstract Holds Fail Rate Modeling output data, per run. Tables are dynamically generated when database is created, one table per model.

Table

CV_FRM_OUT_DATA_<model_key>		
1 (lot_key) n	frm_rn_key cell_key fft_key zoned_key lot_key wf_key fail_type diex diey fitted std_err residual fail_cnt struct_cnt t1 ... tY	
LOT		
1 (wf_key) n		
WAFER		
1 (frm_rn_key) n		
CV_FRM_RUN		
1 (cell_key) n		
CV_CELL		
1 (fft_key) n		
CV_FINE_FTYPE		
1 (zoned_key) n		
ZONE_NUM		

Serial Key None.

Foreign Keys / Link Description

frm_rn_key	From the CV_FRM_RUN table.
cell_key	From CV_CELL table. Identifies cell entry.
fft_key	From CV_FINE_FTYPE table. Identifies fine fail type entry.
zoned_key	From ZONE_NUM table.
lot_key	From LOT table. Identifies lot entry.
wf_key	From WAFER table. Identifies wafer entry.
fail_type	Contains fail type: O = open, S = short.
diex	X index of the die.
diey	Y index of the die.
fitted	Fitted value.
std_err	Standard error value.
residual	Residual value.

Other Fields in CV_FRM_OUT_DATA_ <model_key>

fail_cnt Fail count.

struct_cnt Structure count.

t1 ...tY Mu, standard error or scale value.

16.65 CV_FRM_USC_<model_key>

Abstract Holds information for unique splitter combinations.

Table

CV_FRM_USC_<model_key>	
1 (frm_run_key) n CV_FRM_RUN	frm_run_key
1 (wf_key) n WAFER	wf_key
1 (fmd_key) n CV_FAIL_MODE	fmd_key
	d1
	d2
	dx

Serial Key None.

Foreign Keys / Link Description

frm_run_key From the CV_FRM_RUN table.

wf_key From the WAFER table.

fmd_key From the CV_FAIL_MODE table.

Other Fields in CV_FRM_USC_<model_key>

d1 Parameter name.

d2 Parameter name.

dx Parameter name.

16.66 CV_MODULE2CATTR

Abstract Holds information on fail mode to cell attribute relationship.

Table

CV_MODULE2CATTR	
1 (config_key) n	config_key catt_key frm_mod_key fmd_key
CV_CONFIG	
1 (catt_key) n	
CV_CATTR	
1 (frm_mod_key) n	
CV_FRM_MOD	
1 (fmd_key) n	
CV_FAIL_MODE	

Serial Key None.

Foreign Keys / Link Description

config_key	From CV_CONFIG table. Identifies CV configuration entry
catt_key	From CV_CATTR table. Identifies cell attribute entry.
frm_mod_key	From the CV_FRM_MOD table.
fmd_key	From CV_FAIL_MODE table.

Other Fields in CV_MODULE2CATTR

None.

16.67 CV_COEF_LABEL

Abstract Holds information on coefficient label, per configuration.

Table

CV_COEF_LABEL	
	cvlbl_key
1 (config_key) n	config_key coef_label
CV_CONFIG	
	1 (cvlbl_key) n
	CV_FRM_COV
	1 (cvlbl_key) n
	CV_FRM_OUT_SUM

Serial Key

cvlbl_key

Foreign Keys / Link Description

config_key	From CV_CONFIG table. Identifies CV configuration entry.
-------------------	--



**Other Fields in
CV_COEF_LABEL**

coef_label Coefficient label.

16.68 CV_USC_DEF

Abstract Holds information on the relationship between the fail mode and DOE index.

Table

CV_USC_DEF	
1 (par_key) n	par_key
CV_FRM_PARAM	model_key
1 (model_key) n	fmd_key
CV_MODEL	frm_pm_key
1 (fmd_key) n	doe_index
CV_FAIL_MODE	
1 (frm_pm_key) n	
CV_FRM_PARM	

Serial Key None.

**Foreign Keys / Link
Description**

par_key From CV_FRM_PARAM table.

model_key From the CV_MODEL table.

fmd_key From the CV_FAIL_MODE table.

frm_pm_key From CV_FRM_PARM table.

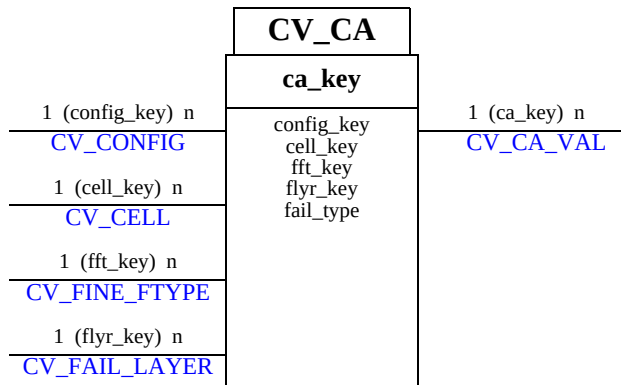
**Other Fields in
CV_USC_DEF**

doe_index Corresponding d... column index from CV_FRM_USC_<model_key> table.

16.69 CV_CA

Abstract Holds information on critical areas.

Table



Serial Key

ca_key Identifies critical area entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

cell_key From CV_CELL table. Identifies cell entry.

fft_key From CV_FINE_FTYPE table. Identifies fine fail type entry.

flyr_key From CV_FAIL_LAYER table. Identifies fail layer entry.

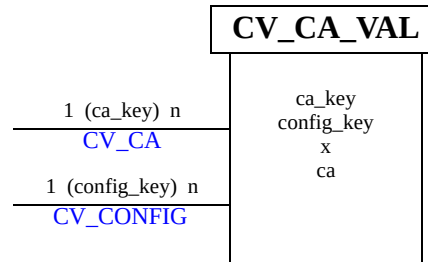
Other Fields in CV_CA

fail_type Contains fail type.

16.70 CV_CA_VAL

Abstract Holds information on critical area values.

Table



Serial Key None.

Foreign Keys / Link Description

ca_key From CV_CA table. Identifies critical area entry.

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_CA_VAL

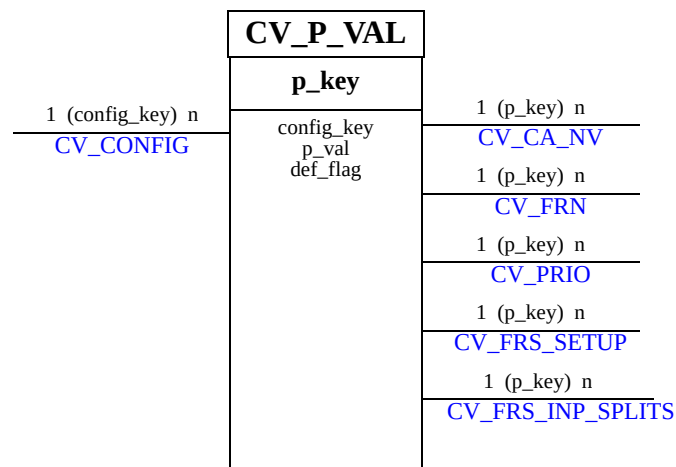
x Contains x value.

ca Contains critical area value.

16.71 CV_P_VAL

Abstract Holds information on p values.

Table



Serial Key

p_key

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

p_val Contains p value.

Other Fields in CV_P_VAL

def_flag Contains default flag.

16.72 CV_CA_NV

Abstract Holds information on number of vias.

Table

CV_CA_NV	
1 (config_key) n	config_key fft_key p_key cell_key pm_type fail_type acp_nv
CV_CONFIG	
1 (fft_key) n	
CV_FINE_FTYPE	
1 (p_key) n	
CV_P_VAL	
1 (cell_key) n	
CV_CELL	

Serial Key None.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

fft_key From CV_FINE_FTYPE table. Identifies fine fail type entry.

p_key From CV_P_VAL table. Identifies p value entry.

cell_key From CV_CELL table. Identifies cell entry.

Other Fields in CV_CA_NV

pm_type Contains process module type.

fail_type Contains fail type.

acp_nv Contains Acp or number of vias.

16.73 CV_BIN**Abstract** Holds information on yield types.**Table**

CV_BIN	
cvb_key	
config_key fs_bin h_flag s_flag r_flag l_flag u_flag w_flag ui_flag li_flag sc_flag sr_flag	1 (cvb_key) n CV_DEFECT

Serial Key**cvb_key** Identifies combination of yields entry.**Foreign Keys / Link Description**

None.

Other Fields in CV_BIN

fs_bin	Describes fail style.
h_flag	Contains hard fail flag.
s_flag	Contains soft fail flag.
r_flag	Contains r-fail flag.
l_flag	Contains l-fail flag.
u_flag	Contains u-fail flag.
w_flag	Contains w-fail flag.
ui_flag	Contains ui-fail flag.
li_flag	Contains li-fail flag.
sc_flag	Contains sc-fail flag.
sr_flag	Contains sr-fail flag.

16.74 CV_STEP_ORDER**Abstract** Holds information on step order

Table

CV_STEP_ORDER	
1 (config_key) n CV_CONFIG	config_key
1 (step_key) n PROC_STEP	step_key wf_key step_order tolerance step_type exclude flag
1 (wf_key) n WAFER	

Serial Key None.

**Foreign Keys / Link
Description**

config_key From CV_CONFIG table. Identifies CV configuration entry.

step_key From PROC_STEP table. Identifies step entry.

wf_key From WAFER table. Identifies wafer entry.

**Other Fields in
CV_STEP_ORDER**

step_order Contains step order.

tolerance Contains overlay tolerance.

step_type Step type: I = inline, E = electrical.

exclude Contains exclude flag.

flag Contain automatic or manual flag.

16.75 CV_DEFECT

Abstract Holds information on CV defects.

Table

CV_DEFECT		
	cv_df_key	
1 (cvb_key) n CV_BIN	cvb_key	1 (df_key) n CV_DEF2TYPE_<lot_key>
1 (ai_key) n CV_AI	ai_key	
1 (cell_key) n CV_CELL	cell_key	
1 (celli_key) n CV_CELL_I	celli_key	
1 (unqci_key) n CV_UNQCI	unqci_key	
1 (pmod_key) n CV_PR_MOD	pmod_key	
1 (fft_key) n CV_FINE_FTYPE	fft_key	
1 (lg_key) n OP_LOG	lg_key	
	df_index	
	diex	
	diey	
	xAbs	
	yAbs	
	x_size	
	y_size	
	value	
	num_vias	
	num_lines	
	df_type	
	nest	
	loo	
	df_source	
	df_info	
	fail_type	
	fs_bin	
	adl_flag	
	cift_flag	

Serial Key	cv_df_key	Identifies CV defect entry.
Foreign Keys / Link Description	cvb_key	From CV_BIN table. Identifies combination of yields entry.
	ai_key	From CV_AI table. Identifies an.item entry.
	cell_key	From CV_CELL table. Identifies cell entry.
	celli_key	From CV_CELL_I table. Identifies the cell instance entry.
	unqci_key	From CV_UNQCI table. Identifies entry for unique cell instance per die entry.
	pmod_key	From CV_PR_MOD table. Identifies process module entry.
	fft_key	From CV_FINE_FTYPE table. Identifies fine fail type entry.
	lg_key	Identifies entry from OP_LOG table.

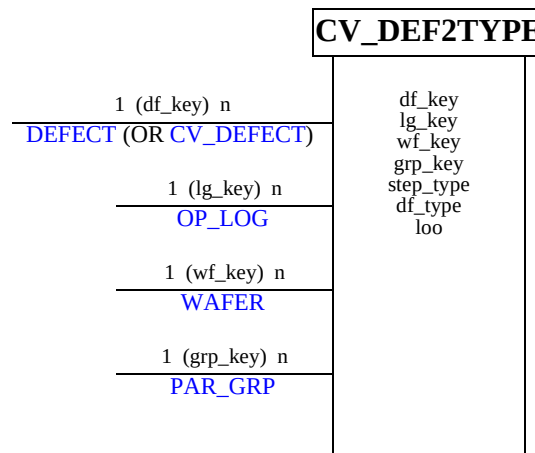
**Other Fields in
CV_DEFECT**

df_index	Contains index of CV defect.
diex	X index of the die.
diey	Y index of the die.
xAbs	X position of the defect from the wafer center.
yAbs	Y position of the defect from the wafer center.
x_size	X size of the defect in micron.
y_size	Y size of the defect in micron.
value	Contains electrical measurement value.
num_vias	Contains number of vias.
num_lines	Contains number of lines.
df_type	Contains defect type.
nest	Contains nest flag.
loo	Contains layer of origin — lg_key entry from OP_LOG table.
df_source	Contains df_key entry as a reference to the source of overlay.
df_info	Contains defect info.
fail_type	Describes fail type.
fs_bin	Describes fail style.
adl_flag	Contains ADL flag.
cift_flag	Contains cift flag.

16.76 CV_DEF2TYPE_<lot_key>

Abstract Contains relationships between defects and their overlay types. Tables are dynamically generated when database is created, one table per lot.

Table



Foreign Keys / Link Description

df_key Uses the entry from CV_DEFECT or DEFECT table.

lg_key Identifies entry from OP_LOG table.

wf_key From WAFER table. Identifies wafer entry.

grp_key From PAR_GRP table. Identifies parameter group.

Other Fields in CV_DEFECT

step_type Step type: I = inline, E = electrical.

df_type Contains defect type.

loo Contains layer of origin — lg_key entry from OP_LOG table.

16.77 CV_EDEF2IDEF

Abstract Holds information on relationship between electrical and inline defects. Only the largest, earliest inline defect information for defects that overlay with electrical defect bounding box is retained.

Table

CV_EDEF2IDEF	
1 (wf_key) n WAFER	wf_key
1 (estep_key) n PROC_STEP	estep_key
1 (istep_key) n PROC_STEP	istep_key
1 (unqci_key) n CV_UNQCI	unqci_key
1 (cell_key) n CV_CELL	cell_key
1 (s_lg_key) n OP_LOG	s_lg_key
1 (a_lg_key) n OP_LOG	a_lg_key
1 (pmod_key) n CV_PR_MOD	pmod_key
	df_source
	df_align
	diex
	diey
	fs_bin
	fail_type

Serial Key None.

Foreign Keys / Link Description

wf_key	From WAFER table. Identifies wafer entry.
estep_key	From PROC_STEP table. Identifies step key for electrical step.
istep_key	From PROC_STEP table. Identifies step key for inline step.
unqci_key	From CV_UNQCI table. Identifies entry for unique cell instance per die entry.
cell_key	From CV_CELL table. Identifies cell entry.
s_lg_key	Contains inline lg_key entry from OP_LOG table.
a_lg_key	Contains electrical lg_key entry from OP_LOG table.

Other Fields in CV_EDEF2IDEF	pmod_key	From CV_PR_MOD table. Identifies process module entry.
	df_source	Contains inline df_key defect entry from DEFECT table, the entry for the inline source defect.
	df_align	Contains electrical defect entry from CV_DEFECT table.
	diex	X index of the die.
	diey	Y index of the die.
	fail_type	Describes fail type.
	fs_bin	Describes fail style.

16.78 CV_EDEF2IDEF_ALL

Abstract Holds information on relationship between electrical and inline defects. All inline defect information for defects that overlay with electrical defect bounding box is retained.

Table

CV_EDEF2IDEF_ALL		
1 (wf_key) n		estep_key istep_key unqci_key cell_key s_lg_key a_lg_key pmod_key wf_key df_source df_align diex diey fs_bin fail_type
WAFER		
1 (estep_key) n		
PROC_STEP		
1 (istep_key) n		
PROC_STEP		
1 (unqci_key) n		
CV_UNQCI		
1 (cell_key) n		
CV_CELL		
1 (s_lg_key) n		
OP_LOG		
1 (a_lg_key) n		
OP_LOG		
1 (pmod_key) n		
CV_PR_MOD		

Serial Key None.

Foreign Keys / Link Description

estep_key	From PROC_STEP table. Identifies step key for electrical step.
istep_key	From PROC_STEP table. Identifies step key for inline step.

unqci_key	From CV_UNQCI table. Identifies entry for unique cell instance per die entry.
cell_key	From CV_CELL table. Identifies cell entry.
s_lg_key	Contains inline lg_key entry from OP_LOG table.
a_lg_key	Contains electrical lg_key entry from OP_LOG table.
pmod_key	From CV_PR_MOD table. Identifies process module entry.
wf_key	From WAFER table. Identifies wafer entry.

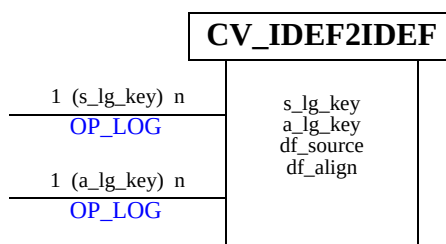
Other Fields in CV_EDEF2IDEF_ALL

df_source	Contains inline df_key defect entry from DEFECT table, the entry for the inline source defect.
df_align	Contains electrical defect entry from CV_DEFECT table.
diex	X index of the die.
diey	Y index of the die.
fail_type	Describes fail type.
fs_bin	Describes fail style.

16.79 CV_IDEF2IDEF

Abstract Holds information on overlays between inline defects.

Table



Serial Key None.

Foreign Keys / Link Description

s_lg_key	Contains inline lg_key entry from OP_LOG table.
a_lg_key	Contains electrical lg_key entry from OP_LOG table.

**Other Fields in
CV_IDEF2IDEF**

df_source	Contains inline df_key defect entry from DEFECT table, the entry for the inline source defect.
df_align	Contains entry for the overlaid inline source defect.

16.80 CV_UPSTAT

Abstract Holds information on the various stages of the flow.

Table

CV_UPSTAT	
1 (config_key) n	config_key
CV_CONFIG	lg_key
1 (lg_key) n	pg_key
OP_LOG	pd_key
1 (pg_key) n	lot_key
PROGRAM	wf_key
1 (pd_key) n	src_lot
PRODUCT	rework_flag
1 (lot_key) n	error_code
LOT	err_msg
1 (wf_key) n	flag
WAFER	

Serial Key None.

**Foreign Keys / Link
Description**

config_key	From CV_CONFIG table. Identifies CV configuration entry.
lg_key	Identifies entry from OP_LOG table.
pg_key	From PROGRAM table. Identifies program entry.
pd_key	From PRODUCT table. Identifies product entry.
lot_key	From LOT table. Identifies lot entry.
wf_key	From WAFER table. Identifies wafer entry.

**Other Fields in
CV_UPSTAT**

src_lot	From PROG2LOT table. Identifies lot entry.
rework_flag	Contains rework flag.
error_code	Contains error code.
err_msg	Describes error.
flag	Flag to indicate the last processing stage for current entry.

16.81 CV_SUM_ZONE

Abstract Holds information on CV zone summaries.

Table

CV_SUM_ZONE	
1 (lot_key) n	lot_key wf_key step_key pmod_key cell_key zoned_key fail_type tci_count fci_count hfc_i_count sfci_count tci_in_count fci_in_count hfc_i_in_count sfci_in_count
LOT	
1 (wf_key) n	
WAFER	
1 (step_key) n	
PROC_STEP	
1 (pmod_key) n	
CV_PR_MOD	
1 (cell_key) n	
CV_CELL	
1 (zoned_key) n	
ZONE_NUM	

Serial Key None.

Foreign Keys / Link Description

lot_key	From LOT table. Identifies lot entry.
wf_key	From WAFER table. Identifies wafer entry.
step_key	From PROC_STEP table. Identifies process step entry.
pmod_key	From CV_PR_MOD table. Identifies process module entry.
cell_key	From CV_CELL table. Identifies cell entry.
zoned_key	From ZONE_NUM table. Identifies zone entry.

Other Fields in CV_SUM_ZONE

fail_type	Contains fail type.
tci_count	Contains count of tested cell instances.
fci_count	Contains count of failed cell instances.
hfc_i_count	Contains count of failed "hard" cell instances.
sfci_count	Contains count of failed "soft" cell instances.
tci_in_count	Contains count of tested cell instances within inspected areas.
fci_in_count	Contains count of failed cell instances within inspected areas.

hfci_in_count Contains count of failed "hard" cell instances within inspected areas.

sfci_in_count Contains count of failed "soft" cell instances within inspected areas.

16.82 CV_PRIO

Abstract Holds information on prioritization results.

Table

CV_PRIO	
1 (lot_key) n	lot_key
LOT	p_key
1 (p_key) n	amod_key
CV_P_VAL	fail_type
1 (amod_key) n	fail_style
CV_AI_MOD	var_src
	sum_sq
	prio_per
	df
	mean_sq
	f_val
	prob_val
	baseline

Serial Key None.

Foreign Keys / Link Description

lot_key From LOT table. Identifies lot entry.

p_key From CV_P_VAL table. Identifies p entry.

amod_key From CV_AI_MOD. Identifies analysis module entry.

Other Fields in CV_PRIO

fail_type Contains fail type.

fail_style Fail style: **H** = hard; **S** = soft.

var_src Contains variation source.

sum_sq Contains sum of squares.

prio_per Contains prioritization level (in %).

df Contains Df term.

mean_sq Contains mean squares.

f_val Contains f value.

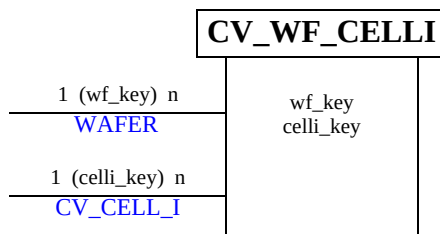
prob_val Contains probability value.

baseline Contains baseline value.

16.83 CV_WF_CELLI

Abstract Holds information on relationships between cell instances and wafers

Table



Serial Key None.

Foreign Keys / Link Description

wf_key From WAFER table. Identifies wafer entry.

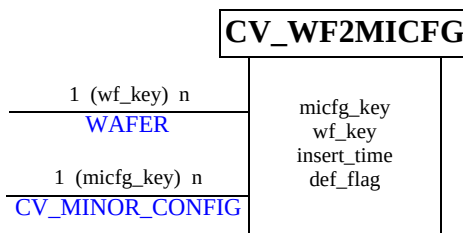
celli_key From CV_CELL_I table. Identifies the cell instance entry.

Other Fields in CV_WF_CELLI None.

16.84 CV_WF2MICFG

Abstract Holds information on relationships between wafers and minor configurations.

Table



Serial Key None.

Foreign Keys / Link Description

wf_key From WAFER table. Identifies wafer entry.

micfg_key Foreign key from CV_MINOR_CONFIG.

Other Fields in CV_WF2MICFG

insert_time Time of data insertion into the database, defaults to the current time.

def_flag Flag indicating if minor configurations the default: Y/N.

16.85 CV_FLTR

Abstract Lists the user-defined filters.

Table

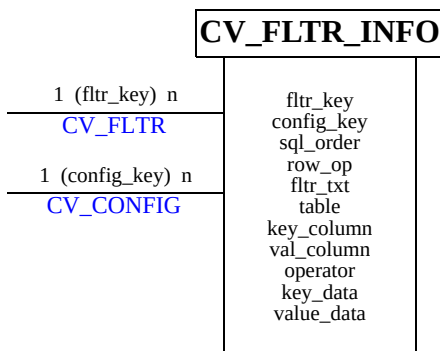
CV_FLTR		
fltr_key		
1 (config_key) n	config_key	1 (fltr_key) n
CV_CONFIG	fltr_name	CV_FLTR_INFO
	sql_count	1 (fltr_key) n
	owner	CV_FRM_RUN
	fltr_type	1 (fltr_key) n
		CV_EXPORT
		1 (fltr_key) n
		CV_FILTER_UFORM
		1 (fltr_key) n
		CV_FRS_INP_SPLITS

Serial Key	fltr_key	
Foreign Keys / Link Description	config_key	From CV_CONFIG table. Identifies CV configuration entry.
Other Fields in CV_FLTR	fltr_name	Name of filter.
	sql_count	Count of SQL sentences in filter.
	owner	The name of the owner.
	fltr_type	Type of filter.

16.86 CV_FLTR_INFO

Abstract Holds information on a run's user-defined filtering configuration.

Table



Serial Key None.

Foreign Keys / Link Description

fltr_key From CV_FLTR table.

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_FLTR_INFO

sql_order Order of SQL sentences.

row_op Row operators.

fltr_txt Text description of filter.

table Name of table in database that is providing values.

key_column Name of Key column in table.

val_column Name of Value column in table.

value_data Data in Value column in table.

operator Operator defining relationship between column and data.

key_data Data in Key column in table

16.87 CV_ADL

Abstract Holds information on what defect wafer-layer (inline or electrical) the user wants to export into a file.

Table

CV_ADL		
adl_key		
1 (config_key) n	config_key	1 (adl_key) n
CV_CONFIG	lg_key	CV_ADL_DEF
1 (lg_key) n	wf_key	
OP_LOG	step_key	
1 (wf_key) n	wmap_key	
WAFER	insert_time	
1 (step_key) n	end_time	
PROC_STEP	export_type	
1 (wmap_key) n	rule_cnt	
WMAP_CONFIG	filt_cnt	
	status	
	def_cnt	
	file_name	
	error_code	
	err_msg	
	owner	

Serial Key	adl_key	
Foreign Keys / Link Description	config_key	From CV_CONFIG table. Identifies CV configuration entry.
	lg_key	From OP_LOG table. Identifies lg_key entry.
	wf_key	From WAFER table. Identifies wafer entry.
	step_key	From PROC_STEP table. Identifies process step.
	wmap_key	From WMAP_CONFIG. Identifies wafer map configuration entry.
Other Fields in CV_ADL	insert_time	Time of data insertion into the database, defaults to the current time.
	end_time	Time when data insertion was finished.
	export_type	Placeholder column.
	rule_cnt	Placeholder column.
	filt_cnt	Placeholder column.

status

Value	Description
-1	error
0	new
1	in progress
2	done

def_cnt Number of defects.

file_name Output filename.

error_code Error code value, given by the system.

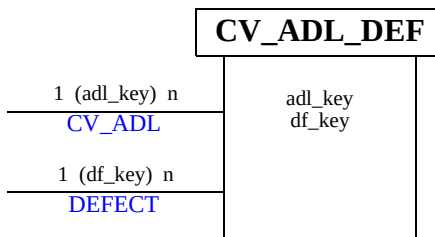
err_msg Error message generated by the system.

owner Name of user who submitted request.

16.88 CV_ADL_DEF

Abstract Holds information on which defects, for the layer specified in [CV_ADL](#), should be exported.

Table



Serial Key None.

Foreign Keys / Link Description

adl_key From CV_ADL table.

df_key From DEFECT table.

Other Fields in CV_ADL_DEF

None.

16.89 CV_PIN_RESULTS

Abstract Only used for standard CVs. Holds information for every pin on the wafer pertaining to test results.

Table

CV_PIN_RESULTS	
1 (config_key) n CV_CONFIG	config_key
1 (lg_key) n OP_LOG	lg_key
1 (wf_key) n WAFER	wf_key
	pin
	open_pass_cnt
	short_pass_cnt
	open_test_cnt
	short_test_cnt
	pass_flag

Serial Key None.

Foreign Keys / Link Description **config_key** From CV_CONFIG table. Identifies CV configuration entry.

lg_key From OP_LOG table. Identifies lg_key entry.

wf_key From WAFER table. Identifies wafer entry.

Other Fields in CV_PIN_RESULTS

pin Contains pin number.

open_pass_cnt Number of open passing bins.

short_pass_cnt Number of short passing bins.

open_test_cnt Number of open pins tested.

short_test_cnt Number of short pins tested.

pass_flag Flag indicating pass status.

16.90 CV_PIN_SUM

Abstract This table is used to store calculated pin-related summaries for the input dataset, such as pad yield, open/short tested and pass pin count.

Table

CV_PIN_SUM	
1 (config_key) n CV_CONFIG	lg_key wf_key config_key pdgrp_key
1 (lg_key) n OP_LOG	pin test_index diex diey
1 (wf_key) n WAFER	pad_value pad_yield open_pass_cnt short_pass_cnt
1 (pdgrp_key) n CV_PAD_GROUP	open_test_cnt short_test_cnt dut_open_yield dut_short_yield
1 (cvcat_key) n CV_CATEGORY	cvcat_key

Serial Key None.

Foreign Keys / Link Description **config_key** From CV_CONFIG table. Identifies CV configuration entry.

lg_key From OP_LOG table. Identifies lg_key entry.

wf_key From WAFER table. Identifies wafer entry.

pdgrp_key From CV_PAD_GROUP table.

cvcat_key From CV_CATEGORY table.

Other Fields in CV_PIN_SUM

pin Contains pin number.

test_index Contains test index.

diex X index of the die.

diey Y index of the die.

pad_value Measured value for pad.

pad_yield Yield for pad.

open_pass_cnt Number of open passing pins.

short_pass_cnt Number of short passing pins.

open_test_cnt	Number of open pins tested.
short_test_cnt	Number of short pins tested.
dut_open_yield	Open yield for device under test.
dut_short_yield	Short yield for device under test.

16.91 CV_DIE_RESULTS

Abstract Only used for standard CVs. Holds information for every die on the wafer pertaining to test results.

Table

CV_DIE_RESULTS	
1 (config_key) n CV_CONFIG	config_key lg_key wf_key test_order test_index diex diey
1 (lg_key) n OP_LOG	open_pass_cnt short_pass_cnt open_test_cnt short_test_cnt pass_flag
1 (wf_key) n WAFER	

Serial Key None.

Foreign Keys / Link Description

config_key	From CV_CONFIG table. Identifies CV configuration entry.
-------------------	--

lg_key	From OP_LOG table. Identifies lg_key entry.
---------------	---

wf_key	From WAFER table. Identifies wafer entry.
---------------	---

Other Fields in CV_DIE_RESULTS

test_order	Contains Die order during testing.
-------------------	------------------------------------

test_index	Contains Test index.
-------------------	----------------------

diex	X index of the die.
-------------	---------------------

diey	Y index of the die.
-------------	---------------------

open_pass_cnt	Number of open passing pins.
----------------------	------------------------------

short_pass_cnt	Number of short passing pins.
-----------------------	-------------------------------

open_test_cnt	Number of open pins tested.
----------------------	-----------------------------

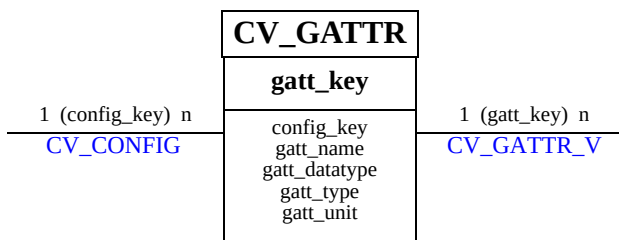
short_test_cnt	Number of short pins tested.
-----------------------	------------------------------

pass_flag Contains passing flag.

16.92 CV_GATTR

Abstract Holds information on global attributes.

Table



Serial Key

gatt_key Identifies global attribute entry.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_GATTR

gatt_name Describes global attribute.

gatt_datatype Describes global attribute data type.

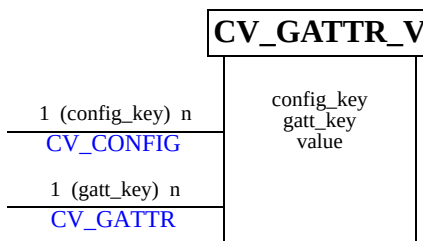
gatt_type Describes global attribute type.

gatt_unit Describes global attribute unit.

16.93 CV_GATTR_V

Abstract Holds information on global attribute values.

Table

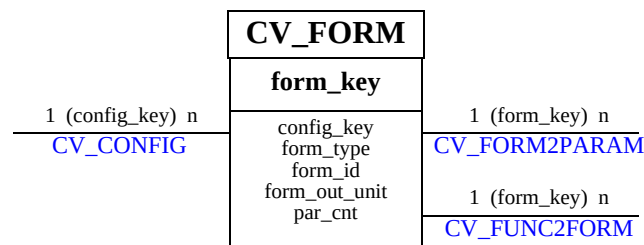
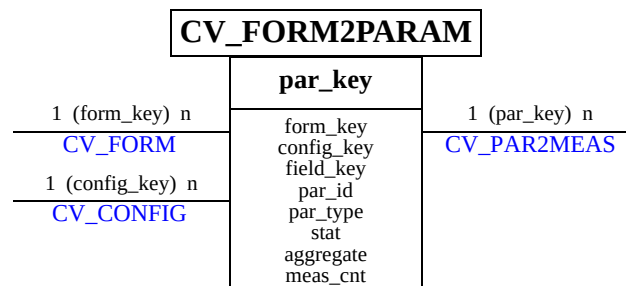


Serial Key None.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

gatt_key From CV_GATTR table. Identifies global attribute entry.

**Other Fields in
CV_GATTR****value** Contains global attribute value.**16.94 CV_FORM****Abstract** Holds information on CV formulas.**Table****Serial Key****form_key** Identifies formula entry.**Foreign Keys / Link
Description****config_key** From CV_CONFIG table. Identifies CV configuration entry.**Other Fields in
CV_FORM****form_type** Identifies formula type. Supported choices are 1,2,3,4,5.**form_id** Contains formula id.**form_out_unit** Contains formula output unit.**par_cnt** Contains number of parameters for units.**16.95 CV_FORM2PARAM****Abstract** Holds information on relationships between formulas and parameters.**Table****Serial Key****par_key** Identifies parameter entry.**Foreign Keys / Link
Description****form_key** From CV_FORM table. Identifies formula entry.

Other Fields in CV_FORM2PARAM

config_key	From CV_CONFIG table. Identifies CV configuration entry.
field_key	Contains the field entry from a related table.
par_id	Contains parameter id.
par_type	Contains parameter type.
param_order	Order of parameters.
stat	Describes type of used statistics. Possible values:

Value	Description
MIN	minimum
MAX	maximum
AVG	average
MED	median
NULL	null

aggregate	Contains aggregation level. Possible values:
------------------	--

Value	Description
D	die
W	wafer
NULL	null

meas_cnt	Contains number of measurements.
-----------------	----------------------------------

16.96 CV_FORM2LSPLT

Abstract Hold information on the relationship between formulas and lot splits.

Table

CV_FORM2LSPLT	
1 (fail_key) n CV_FMD2MDL	fail_key
1 (frm_rn_key) n CV_FRM_RUN	frm_rn_key
1 (config_key) n CV_CONFIG	config_key
	lspl_formula_id
	p_lspl_formula_id

Serial Key None.

Foreign Keys / Link Description

fail_key From CV_FMD2MDL table. Identifies relationship between fail mode and model.

frm_rn_key From the CV_FRM_RUN table.

config_key From CV_CONFIG table. Identifies CV configuration entry.

lspl_key Lot split.

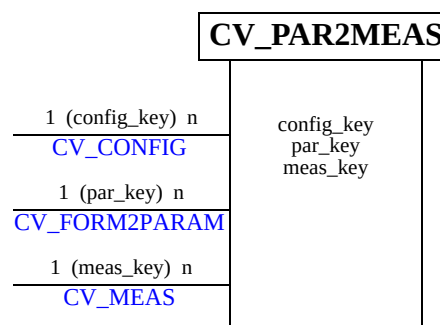
Other Fields in CV_FORM2LSPLT

l_lspl_formula_id Not currently implemented; reserved for future use.

p_lspl_formula_id Not currently implemented; reserved for future use.

16.97 CV_PAR2MEAS

Abstract Holds information on relationships between measurements and parameters.

Table

Serial Key None.

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

par_key From CV_FORM2PARAM table. Identifies parameter entry.

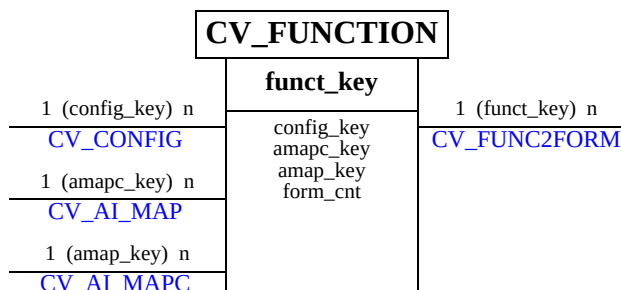
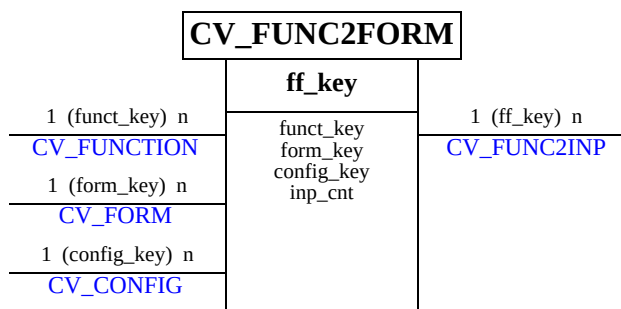
meas_key From CV_MEAS table. Identifies measurement entry.

Other Fields in CV_PAR2MEAS

None.

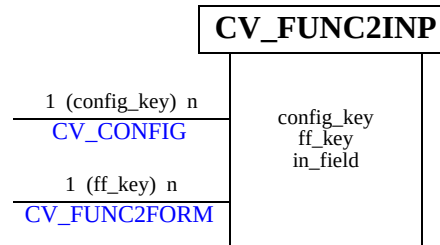
16.98 CV_FUNCTION

Abstract Holds information on relationships between functions and analysis map classes in CV configuration.

Table**Serial Key****funct_key** Identifies function entry.**Foreign Keys / Link Description****config_key** From CV_CONFIG table. Identifies CV configuration entry.**amapc_key** From CV_AI_MAP table. Identifies analysis map class entry.**amap_key** From CV_AI_MAPC table. Identifies analysis map entry.**Other Fields in CV_FUNCTION****form_cnt** Contains number of formulas.**16.99 CV_FUNC2FORM****Abstract** Holds information on relationships between functions and formulas.**Table****Serial Key****ff_key** Identifies function-formula relationship.**Foreign Keys / Link Description****funct_key** From CV_FUNCTION table. Identifies function.**form_key** From CV_FORM table. Identifies formula.**config_key** From CV_CONFIG table. Identifies CV configuration entry.**Other Fields in CV_FUNC2FORM****inp_cnt** Contains number of inputs for this entity.

16.100 CV_FUNC2INP

Abstract Holds information on input fields to function-formula entity.

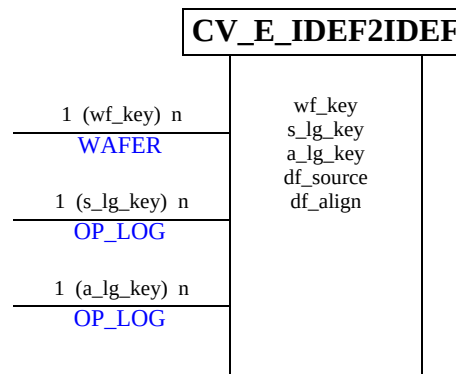
Table

Serial Key None.

Foreign Keys / Link Description	config_key	From CV_CONFIG table. Identifies CV configuration entry.
	ff_key	From CV_FUNC2FORM table. Identifies function-formula relationship.
	in_field	Contains input field.
Other Fields in CV_FUNC2INP		

16.101 CV_E_IDEF2IDEF

Abstract Holds information on overlaid inline defects which fall inside of an electrical defect's bounding box.

Table

Serial Key None.

Foreign Keys / Link Description	wf_key	From WAFER table. Identifies wafer entry.
	s_lg_key	Contains inline lg_key entry from OP_LOG table.
	a_lg_key	Contains electrical lg_key entry from OP_LOG table.

**Other Fields in
CV_E_IDEF2IDEF**

df_source	Contains inline df_key defect entry from DEFECT table, the entry for the inline source defect.
df_align	Contains entry for the overlaid inline source defect.

16.102 CV_E2I_INHERIT

Abstract Holds information on inline defects that overlaid with electrical defect bounding box, and also either have classification or image.

Table

CV_E2I_INHERIT	
1 (wf_key) n WAFER	wf_key estep_key istep_key s_lg_key a_lg_key cell_key pmod_key df_source df_align fail_type is_src fs_bin df_info
1 (estep_key) n PROC_STEP	
1 (istep_key) n PROC_STEP	
1 (s_lg_key) n OP_LOG	
1 (a_lg_key) n OP_LOG	
1 (cell_key) n CV_CELL	
1 (pmod_key) n CV_PR_MOD	

Serial Key None.

**Foreign Keys / Link
Description**

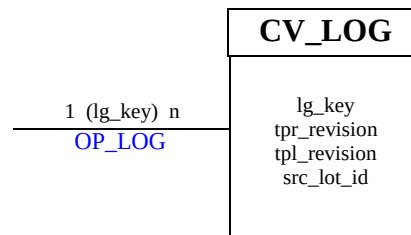
wf_key	From WAFER table. Identifies wafer entry.
estep_key	From PROC_STEP table. Identifies electrical step.
istep_key	From PROC_STEP table. Identifies inline step.
s_lg_key	Contains inline lg_key entry from OP_LOG table.
a_lg_key	Contains electrical lg_key entry from OP_LOG table.
cell_key	From CV_CELL table. Identifies cell entry.
pmod_key	From CV_PR_MOD table. Identifies process module entry.

**Other Fields in
CV_E2I_INHERIT**

df_source	Contains inline df_key defect entry from DEFECT table, the entry for the inline source defect.
df_align	Contains entry for the overlaid inline source defect.
fail_type	Contains fail type.
fs_bin	Describes fail style.
is_src	Flag indicating if defect is source: Y/N.
df_info	Contains defect info.

16.103 CV_LOG

Abstract Holds information specifying which test plan and test program are associated with the given data set.

Table

Serial Key None.

**Foreign Keys / Link
Description**

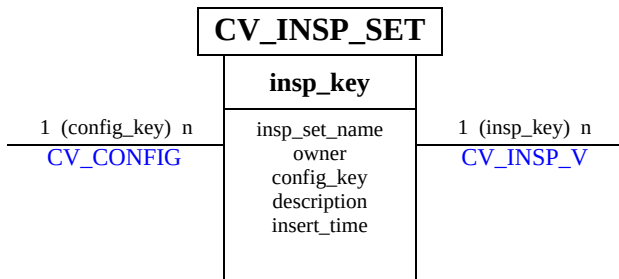
lg_key From OP_LOG table. Identifies lg_key entry.

**Other Fields in
CV_LOG**

tpr_revision	Test program version.
tpl_revision	Test plan version.
src_lot_id	Source lot ID.

16.104 CV_INSP_SET

Abstract Holds value of inspection set.

Table**Serial Key****insp_key****Foreign Keys / Link Description****config_key**

From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_INSP_SET**insp_set_name**

Name of inspection set.

owner

Name of user who submitted the request.

description

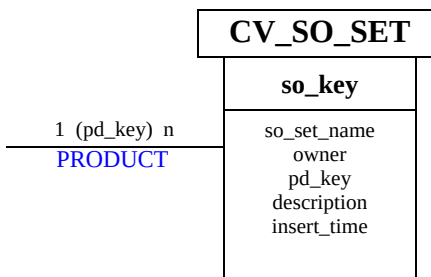
Text description of inspection set.

insert_time

Time of data insertion into the database, defaults to the current time.

16.105 CV_SO_SET

Abstract Holds information describing the step order set.

Table**Serial Key****so_key****Foreign Keys / Link Description****pd_key**

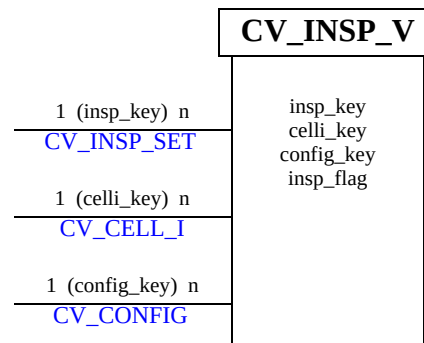
From PRODUCT table. Identifies product entry.

**Other Fields in
CV_SO_SET**

so_set_name	Name of step order set.
owner	Name of user who submitted the request.
description	Text description of step order set.
insert_time	Time of data insertion into the database, defaults to the current time.

16.106 CV_INSP_V

Abstract Holds the value of the inspection.

Table

Serial Key None.

**Foreign Keys / Link
Description**

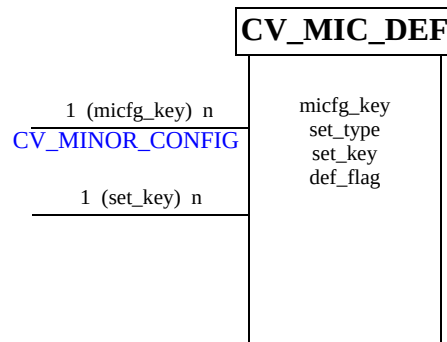
insp_key	Foreign key from CV_INSP_SET table.
celli_key	From CV_CELL_I table. Identifies the cell instance entry.
config_key	From CV_CONFIG table. Identifies CV configuration entry.

**Other Fields in
CV_INSP_V**

insp_flag	Flag to indicate if the cell was inspected.
------------------	---

16.107 CV_MIC_DEF

Abstract Holds information defining minor configuration.

Table

Serial Key None.

**Foreign Keys / Link
Description**

micfg_key Foreign key from CV_MINOR_CONFIG.

set_key Contains value for given set type.

**Other Fields in
CV_MIC_DEF**

set_type Indicates type of minor configuration.

def_flag Default flag for given set type.

16.108 CV_PAD_GROUP**Abstract** Holds pad group name.**Table**

CV_PAD_GROUP		
1 (config_key) n CV_CONFIG	pdgrp_key pdgrp_name config_key	1 (pdgrp_key) n CV_MEAS2PADGRP
		1 (pdgrp_key) n CV_PIN2PADGRP
		1 (pdgrp_key) n CV_PIN_SUM
		1 (pdgrp_key) n CV_PADGRP_SUM

Serial Key**pdgrp_key****Foreign Keys / Link Description****config_key** From CV_CONFIG table. Identifies CV configuration entry.**Other Fields in CV_PAD_GROUP****pdgrp_name** Name of pad group.**16.109 CV_PADGRP_SUM****Abstract** Holds pad group summaries calculated for the given pad group.**Table**

CV_PADGRP_SUM		
1 (config_key) n CV_CONFIG	lg_key wf_key config_key pdgrp_key test_index diex diey cvcat_key	
1 (lg_key) n OP_LOG		
1 (wf_key) n WAFER		
1 (pdgrp_key) n CV_PAD_GROUP		
1 (cvcat_key) n CV_CATEGORY		

Serial Key None.**Foreign Keys / Link Description****lg_key** From OP_LOG table. Identifies lg_key entry.

Other Fields in CV_PADGRP_SUM	config_key	From CV_CONFIG table. Identifies CV configuration entry.
	wf_key	From WAFER table. Identifies wafer entry.
	pdgrp_key	From CV_PAD_GROUP.
	cvcat_key	From CV_CATEGORY.
	test_index	Test index.
	diex	X index of the die.
	diey	Y index of the die.

16.110 CV_MEAS2PADGRP

Abstract Holds information of relationship between measurements and pad groups.

Table

CV_MEAS2PADGRP	
1 (meas_key) n CV_MEAS	meas_key pdgrp_key config_key
1 (pdgrp_key) n CV_PAD_GROUP	
1 (config_key) n CV_CONFIG	

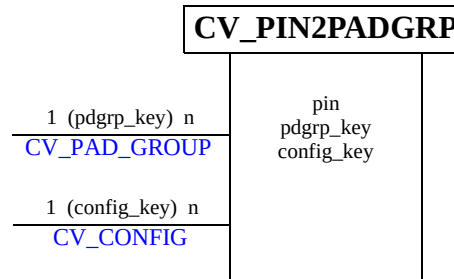
Serial Key None.

Foreign Keys / Link Description	meas_key	From CV_MEAS table. Identifies measurement entry.
	pdgrp_key	From CV_PAD_GROUP table.
	config_key	From CV_CONFIG table. Identifies CV configuration entry.

**Other Fields in
CV_MEAS2PADGRP** None.

16.111 CV_PIN2PADGRP

Abstract Holds information on relationship between pins and pad groups.

Table

Serial Key None.

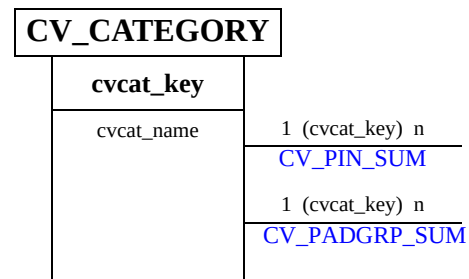
Foreign Keys / Link Description

pdgrp_key From CV_PAD_GROUP table.

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_PIN2PADGRP

pin Contains pin number.

16.112 CV_CATEGORY**Abstract****Table**

Serial Key **cvcat_key**

Foreign Keys / Link Description

None.

Other Fields in CV_CATEGORY

cvcat_name Name of CV category.

16.113 CV_UFORM

Abstract Holds information used to string user-defined formulas together.

Table

CV_UFORM		
	uform_key	
1 (uoper_key) n CV_UOPERATION	uoper_key uformd_key uform_order config_key	1 (uform_key) n CV_UPARAM
1 (uformd_key) n CV_UFORM_DEF		
1 (config_key) n CV_CONFIG		

Serial Key

uform_key

Foreign Keys / Link Description

uoper_key Foreign key from CV_UOPERATION table.

uformd_key Foreign key from CV_UFORM_DEF table. Defines formula

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_UFORM

uform_order Order of operations in formula.

16.114 CV_UFORM_DEF

Abstract Holds information that defines user-defined formulas.

Table

CV_UFORM_DEF		
	uformd_key	
	uform_type uform_id par_cnt uform_equation1 uform_equation2	1 (uformd_key) n CV_UFORM
		1 (uformd_key) n CV_UPARAM_DEF

Serial Key

uformd_key

Foreign Keys / Link Description

None.

Other Fields in CV_UFORM_DEF

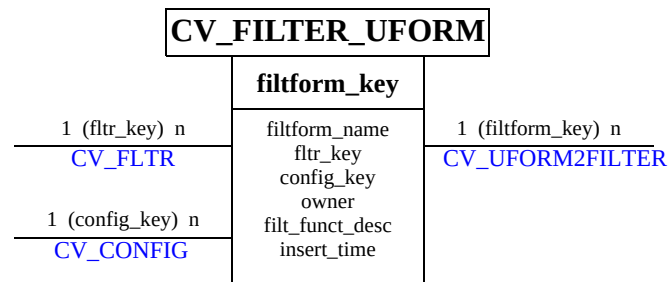
uform_type Numeric code for type of function.

uform_id	String description of formula.
par_cnt	Contains number of parameters for units.
uform_equation1	String equation to be parsed. Support 255*2 chars
uform_equation2	String equation to be parsed. Support 255*2 chars

16.115 CV_FILTER_UFORM

Abstract A grouping of one filter and many user formulas.

Table



Serial Key	filtform_key	
Foreign Keys / Link Description	fltr_key	From CV_FLTR table.
	config_key	From CV_CONFIG table. Identifies CV configuration entry.
Other Fields in CV_FILTER_UFORM	filtform_name	Name of filter formula.
	owner	The name of owner who submitted request.
	filt_funct_desc	Text description of filter function.
	insert_time	Automatically filled in by engine; do not populate this field.

16.116 CV_UFORM2FILTER

Abstract Relates multiple user-formulas to a single filter.

Table

CV_UFORM2FILTER	
1 (filtform_key) n CV_FILTER_UFORM	filtform_key uoper_key config_key
1 (uoper_key) n CV_UOPERATION	
1 (config_key) n CV_CONFIG	

Serial Key None.

Foreign Keys / Link Description

filtform_key	Foreign key from CV_FILTER_UFORM table.
uoper_key	Foreign key from CV_UOPERATION table.
config_key	From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_UFORM2FILTER

None.

16.117 CV_UPARAM

Abstract Holds user-created parameters in formula.

Table

CV_UPARAM	
	upar_key
1 (upard_key) n CV_UPARAM_DEF	upard_key uform_key field_key config_key stat aggregate
1 (uform_key) n CV_UFORM	
1 (config_key) n CV_CONFIG	

Serial Key

upar_key

Foreign Keys / Link Description

upard_key	Foreign key from CV_UPARAM_DEF table.
uform_key	Foreign key from CV_UFORM table.
field_key	Contains the field entry from a related table.

Corresponds to **upar_type** in [CV_UPARAM_DEF](#):

- Cell attribute key
- global attribute key
- cell instance attribute key
- measurement attribute key
- constant key (ucon_key)
- filter key fltr_key

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_UPARAM

stat Valid values only when **upar_type** (in [CV_UPARAM_DEF](#)) = **S**: MIN, MAX, AVG, MED, STD (standard deviation), RAN (range).

Null if **upar_type** <> **S** stat.

aggregate Contains aggregation level. Valid values only when **upar_type** (in [CV_UPARAM_DEF](#)) = **S**:

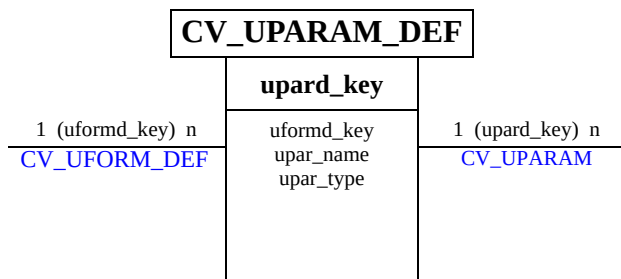
Value	Description
A	all data
D	die
W	wafer

Null if **upar_type** <> **S** stat.

16.118 CV_UPARAM_DEF

Abstract Hardcoded table that defines supported parameters for hardcoded set of formulas in CV_UFORM_DEF table.

Table



Serial Key

upard_key

Foreign Keys / Link Description

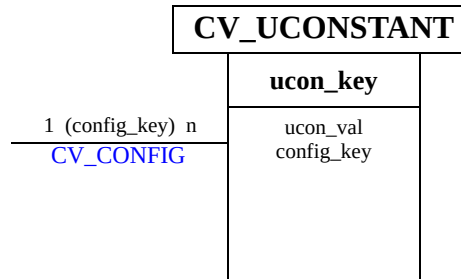
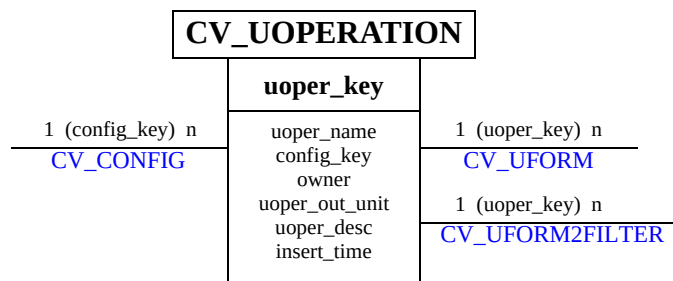
uformd_key From CV_UFOR_DEF table.

Other Fields in CV_UPARAM_DEF

upar_name Name of parameter.

upar_type Type of parameters:

Value	Description	Table
C	cell attribute	CV_CATTR
G	global attribute	CV_GATTR
I	cell instance attribute	CV_CIATTR
M	measurement attribute	CV_MATTR
K	constant	CV_UCONSTANT
S	stat of other data	

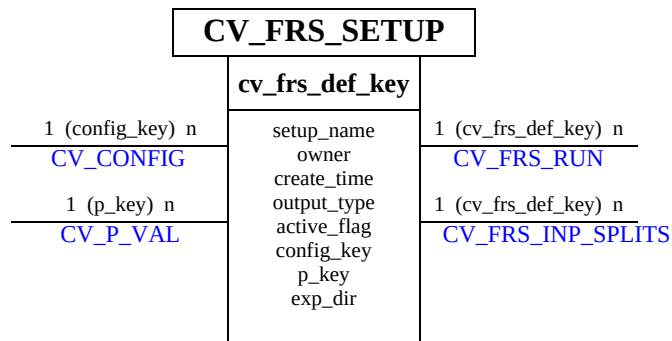
16.119 CV_UCONSTANT**Abstract** Holds user-defined constants.**Table****Serial Key****ucon_key****Foreign Keys / Link Description****config_key** From CV_CONFIG table. Identifies CV configuration entry.**Other Fields in CV_UCONSTANT****ucon_val** Value of user-defined constant.**16.120 CV_UOPERATION****Abstract** Holds user-created operation.**Table****Serial Key****uoper_key****Foreign Keys / Link Description****config_key** From CV_CONFIG table. Identifies CV configuration entry.**Other Fields in CV_UOPERATION****uoper_name** Name of user operation.**owner** The name of user who submitted request.**uoper_out_unit** Unit, if user wants to specify one.

uoper_desc	Text description.
insert_time	Automatically filled in by engine; do not populate this field.

16.121 CV_FRS_SETUP

Abstract Holds information describing fail rate summary setups.

Table



Serial Key

cv_frs_def_key

Foreign Keys / Link Description

config_key From CV_CONFIG table. Identifies CV configuration entry.

p_key From CV_P_VAL table. Identifies p entry.

Other Fields in CV_FRS_SETUP

setup_name Name of setup.

owner The name of the user who submitted request.

create_time Time fail rate summary is set up.

output_type Type of output.

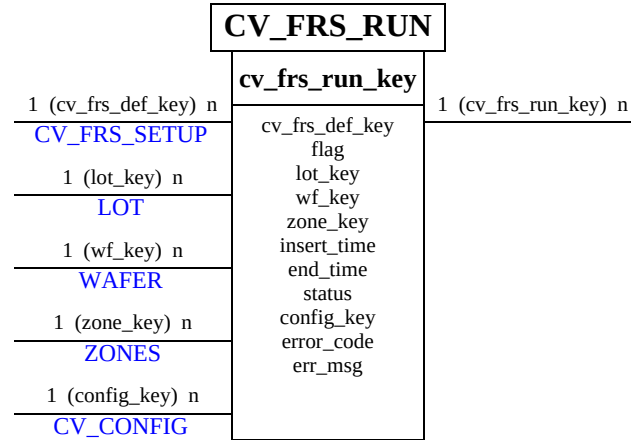
active_flag Flag indicating if setup is active (Y/N).

exp_dir Destination directory for export.

16.122 CV_FRS_RUN

Abstract This table is the run table for fail rate summaries.

Table



Serial Key

cv_frs_run_key

Foreign Keys / Link Description

cv_frs_def_key From CV_FRS_SETUP table.

lot_key From LOT table. Identifies lot entry.

wf_key From WAFER table. Identifies wafer entry.

zone_key From ZONES table.

config_key From CV_CONFIG table. Identifies CV configuration entry.

Other Fields in CV_FRS_RUN

flag Flag indicating type of fail rate summary.

insert_time Time of data insertion into the database, defaults to the current time.

end_time Time when data insertion was finished.

status Status of the run.

Value	Description
-1	error
0	new
1	in progress
2	done

error_code Error code value, given by the system.

err_msg Error message generated by the system.

16.123 CV_FRS_INP_SPLITS

Abstract This table is used to specify “input splitters” (like DOE factors/cell attributes) for Fail Rate Summary runs.

Table

CV_FRS_INP_SPLITS	
1 (cv_frs_def_key) n CV_FRS_SETUP	cv_frs_def_key inp_ndx
1 (amod_key) n CV_AI_MOD	amod_key pmod_key
1 (pmod_key) n CV_PR_MOD	split_key1 split_type1
1 (split_key1-4 (using doef_key)) n CV_DOE	split_key2 split_type2 split_key3 split_type3 split_key4 split_type4
1 (fltr_key) n CV_FLTR	fft_flag fltr_key
1 (p_key) n CV_P_VAL	p_key

Serial Key None.

Foreign Keys / Link Description

cv_frs_def_key	Foreign key from CV_FRS_SETUP
amod_key	From CV_AI_MOD. Identifies analysis module entry.
pmod_key	From CV_PR_MOD table. Identifies process module entry.
split_key1-4	Foreign key from CV_DOE table, using doef_key . Identifies DOE entry. split_key1 , split_key2 , split_key3 , and split_key4 can be -1 or the value of doef_key (from CV_DOE table).
fltr_key	From CV_FLTR table.
p_key	From CV_P_VAL table. Identifies p entry.

Other Fields in CV_FRS_INP_SPLITS

inp_ndx	Sequential order of input split combinations.
split_type1-4	Type of split, up to four types.

fft_flag Flag indicating if fine fail type for cell attribute or DOE factor (Y/N).

16.124 CV_CELL_POSITION

Abstract This table holds information about the position of a cell instance.

Table

CV_CELL_POSITION	
	clp_key
1 (config_key) n CT_CONFIG	config_key celli_key diex diey test_index absx absy radius angle
1 (celli_key) n CV_CELL_I	

Serial Key	clp_key	A unique key assigned to each entry in CV_CELL_POSITION. It is stored automatically.
Foreign Keys / Link Description	config_key	Foreign key from CT_CONFIG table.
	celli_key	Foreign key from CV_CELLI table.
Other Fields in CV_CELL_POSITION	diex	X index of the die.
	diey	Y index of the die.
	test_index	From DEF... table. Identifies the test.
	absx	Absolute X
	absy	Absolute Y
	radius	Length of radius
	angle	Angle of radius

16.125 CV_FRM_STEPWREG_COEF

Abstract This table holds the coefficient for the FRM stepwise regression.

Table

CV_FRM_STEPWREG_COEF		
1 (stepwreg_key) n	stepwreg_key cvlbl_key coef_val	
CV_FRM_STEPWREG_OUT		
1 (cvlbl_key) n		
CV_COEF_LABEL		

Serial Key	stepwreg_key	A unique key assigned to each entry in CV_FRM_STEPWREG_COEF. It is stored automatically.
Foreign Keys / Link Description	stepwreg_key	Foreign key from the CV_FRM_STEPWREG_OUT table.
	cvlbl_key	Foreign key from the CV_COEF_LABEL table.
Other Fields in CV_FRM_STEPWREG_COEF	coef_val	Covariance value of coefficients.

16.126 CV_FRM_STEPWREG_OUT

Abstract This table holds the output of the FRM stepwise regression.

Table

CV_FRM_STEPWREG_OUT		
1 (frm_run_key) n	stepwreg_key frm_rn_key cvlbl_key step type aic_val num_coef fresult1-4	1 (stepwreg_key) n
CV_FRM_RUN		CV_FRM_STEPWREG_COEF
1 (cvlbl_key) n		
CV_COEF_LABEL		

Serial Key	stepwreg_key	A unique key assigned to each entry in CV_FRM_STEPWREG_OUT. It is stored automatically.
Foreign Keys / Link Description	frm_rn_key	Foreign key from CV_FRM_RUN table.
	cvlbl_key	Foreign key from the CV_COEF_LABEL table.

**Other Fields in
CV_FRM_STEPREG
_OUTCV_FRN**

step Identifies the step in the regression.

type Identifies the step type. Possible values are:

Value	Description
A	Add
D	Delete
I	Initial
F	Final

aic_val Akaike Information Criteria (AIC) value from each step in the regression.

num_coef Number of coefficients.

fresult1 Degrees of freedom.

fresult2 Deviance.

fresult3 Residual degrees of freedom.

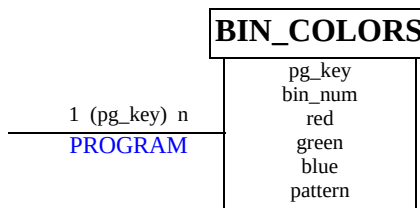
fresult4 Residual deviance.

17.0 binMAP tables

17.1 BIN_COLORS

Abstract Contains bin colors.

Table



Serial Keys None.

Foreign Keys **pg_key** A foreign key from the PROGRAM table. Identifies to which program the bin and its colors belong.

Other Fields in BIN_COLORS

bin_num Bin number.

red Red.

green Green.

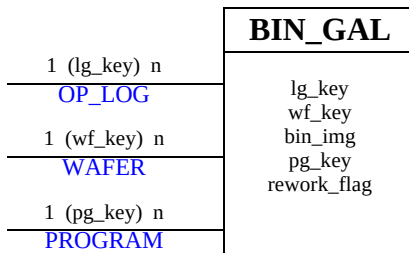
blue Blue.

pattern The hash pattern code used. Not used in current dataPOWER. It defaults to 1.

17.2 BIN_GAL

Abstract Contains bin images.

Table



Serial Keys None.

Foreign Keys / Link Description

pg_key Foreign key from PROGRAM table.

Other Fields in BIN_GAL	lg_key	Foreign key from OP_LOG table.
	wf_key	Foreign key from WAFER table.
	bin_img	The thumbnail image of the binmap of the given wafer.
	rework_flag	Indicates whether data is reworked or not. Currently not used.

18.0 SPLIT Tables

18.1 SPLIT

Abstract Contains information about a lot split type.

Table

SPLIT	
split_key	
split_type split_desc	1 (split_key) n LOT_SPLIT

Serial Key **split_key** Uniquely identifies an entry in SPLIT. Automatically generated out of the split type, **split_type**.

Other Fields in SPLIT **split_type** Unique name of split type.

split_desc description of the split type, string.

18.2 LOT_SPLIT

Abstract Contains entries describing the lot splits in a certain step.

Table

LOT_SPLIT	
lspl_key	
1 (lot_key) n LOT	lot_key step_key split_key split_date split_grp_desc
1 (step_key) n PROC_STEP	1 (lspl_key) n WAFER_SPLIT
1 (split_key) n SPLIT	1 (lspl_key) n CV_FMD2LSPLT

Serial Key **lspl_key** Uniquely identifies an entry in LOT_SPLIT. Automatically generated out of the lot key and the step key, **lot_key**, **step_key** respectively.

Foreign Keys / Link Description **lot_key** A reference to the lot.

step_key A reference to the step.

split_key A reference to the split.

Other Fields in LOT_SPLIT **split_date** Date and time of the split.

split_grp_desc Split group description.

18.3 WAFER_SPLIT

Abstract Contains entries describing the wafer splits in a certain step.

Table

WAFER_SPLIT	
1 (lspl_key) n	lspl_key
LOT_SPLIT	lot_key
1 (lot_key) n	wf_key
LOT	split_grp
1 (wf_key) n	base_flag
WAFER	

Serial Key None.

Foreign Keys / Link Description

lspl_key A reference to the lot split.

lot_key A reference to the lot.

wf_key A reference to the wafer.

Other Fields in WAFER_SPLIT

split_grp Wafer group after splitting the lot.

base_flag Indicates whether the wafer is a baseline wafer or not.
Y = yes, N = no.

19.0 Event Tables

19.1 EVENT_LOG

Abstract Some **equipment** or **lot** may be subjected to events such as preventive maintenance, etc. Such events are time stamped and logged in **EVENT_LOG**. Events include out of control situations, operator remarks to lot or equipment, etc.

Table

EVENT_LOG		
	ev_key	
1 (pg_key) n PROGRAM	pg_key	1 (ev_key) 1 RES...
1 (pgc_key) n PROG_CLASS	pgc_key	
1 (lot_key) n LOT	lot_key	
1 (scr_lot) n LOT	src_lot	
1 (eq_key) n EQUIPMENT	eq_key	
1 (em_key) n PEOPLE	em_key	
1 (step_key) n PROC_STEP	step_key	
1 (pd_key) n PRODUCT	pd_key	
1 (pr_key) n PROCESS	pr_key	
1 (stage_key) n TECH_STAGE	start_time	
	end_time	
	insert_time	
	stage_key	

Serial Key

ev_key A unique serial key, automatically generated to identify each event.

Foreign Keys / Link Description

pg_key Reference to PROGRAM table.

pgc_key Reference to PROG_CLASS table.

lot_key Reference to lot in LOT table.

src_lot Reference to source lot in LOT table.

eq_key Reference to EQUIPMENT table.

em_key Reference to PEOPLE table.

step_key Reference to PROC_STEP table.

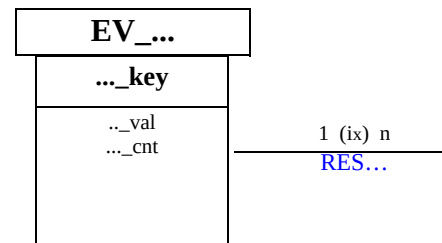
	pd_key	Reference to PRODUCT table.
	pr_key	Reference to PROCESS table.
	stage_key	Reference to TECH_STAGE table.
Other Fields in EVENT_LOG	start_time/end_time	Event start/end time.
	insert_time	Event insert time.

19.2 EV_...

Abstract A table will be created dynamically for every index that the user asks to be normalized using the built-in function **NormalizeIndex**.

The table name is equivalent to the index name prefixed with “EV_...”. For example, if FAB is declared as index and then the user asks to normalize it, a table is created with the name “EV_FAB”. The “...” in the table name and columns is replaced by the name of the index.

Table



Serial Key **..._key** A unique serial key, automatically generated to identify each unique value of the index.

Other Fields in EV_... **..._val** Value of the index.

..._cnt Holds the count of the index history.

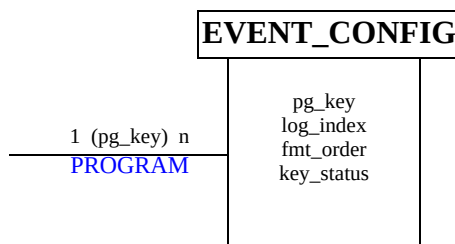


EV_... table has a link to one of the index columns (i's) in RES... table depending on the choice of the user. Whenever the user normalizes an index in RES..., an EV_... table is created for that index and a link to RES... is established. The 'x' in (ix) at the link represents the number of the index at RES...

19.3 EVENT_CONFIG

Abstract Describes the hidden-index columns of EVENT_LOG table. It has a row per populated hidden_index column in EVENT_LOG per program.

Table



Serial Key None.

Foreign Keys / Link Description

pg_key Reference to the PROGRAM table.

Other Fields in EVENT_CONFIG

log_index Holds a code that represents which hidden index this column refers to. Possible values are:

Value	Description
LT	Lot
SL	Source Lot
PE	Equipment
OP	Operator
ST	Process Step
PD	Product
PR	Process
SG	Stage

fmt_order Holds the sequence of that specific index in the original format file.

key_status Holds the status of the index, whether key or non-key.

Value	Description
0	key
1	non-key

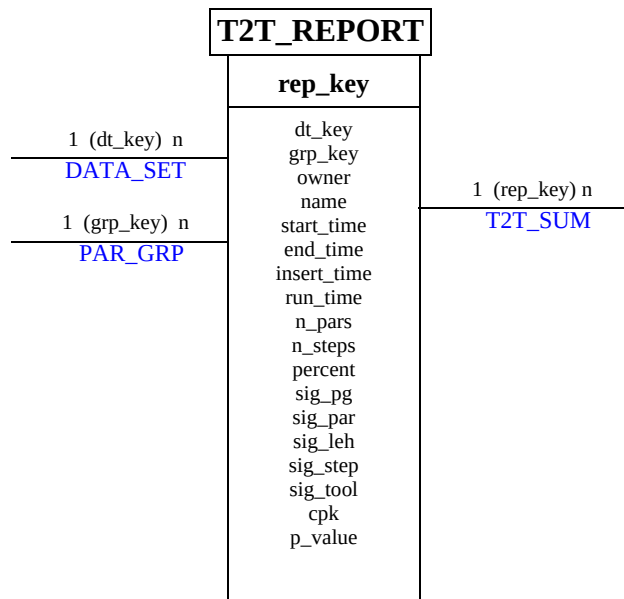
20.0 Tool-to-Tool Tables

Tool-to-Tool tables (T2T_REPORT, T2T_SUM, T2T_TEST, T2T_TOOL) are only used by the *dataPOWER* Tool-to-Tool module. They are used to store exception results generated by Tool-to-Tool analyses done in the background.

20.1 T2T_REPORT

Abstract This table contains an entry for each exception report generated by the Tool-to-Tool analysis.

Table



Serial Key	rep_key	Automatically generated based on the name field.
Foreign Keys / Link Description	dt_key	A reference to DATA_SET.
	grp_key	A reference to PAR_GRP.
Other Fields in T2T_REPORT	owner	The name of the owner.
	name	The name of the report.
	start_time	The start time for report data retrieval.
	end_time	The end time for report data retrieval.
	insert_time	The insert time for report data retrieval.
	run_time	The run time for report data retrieval.

n_pars	The number of parameters.
n_steps	The number of production steps.
percent	Percentage of exceptions.
sig_pg	The program that contains the parameter that generated the most significant exception.
sig_par	The parameter that generated the most significant exception.
sig_leh	The LEH program that contains the step that generated the most significant exception.
sig_step	The step that generated the most significant exception.
sig_tool	The tool that has the lowest Cpk.
cpk	The Cpk value for the tool specified in sig_tool.
p_value	The probability value for the most significant statistical test.

20.2 T2T_SUM

Abstract This table contains the most significant statistical test for each parameter/step combination.

Table

T2T_SUM		
sum_key		
1 (rep_key) n	rep_key	1 (sum_key) n
T2T_REPORT	pg_key	T2T_TEST
	par	
1 (pg_key) n	leh_key	1 (sum_key) n
PROGRAM	step	T2T_TOOL
	row_type	
1 (leh_key) n	n_tools	
PROGRAM	n_grps	
	ll	
	ul	
	imp_tool	
	cpk	
	stat_test	
	p_value	

Serial Key

sum_key Automatically generated based on the parameter/step combination.

**Foreign Keys / Link
Description**

rep_key A reference to the exception report that generated this entry.

pg_key The program containing the parameter associated with sum_key.

leh_key The LEH program containing the step associated with sum_key.

**Other Fields in
T2T_SUM**

par The parameter associated with sum_key.

step The step associated with sum_key.

row_type Defines the step information — equipment, operator, etc.

n_tools Number of tools used in the step field.

n_grps The number of groups that the tools are distributed in.

ll Lower specification limit for the parameter in the par field.

ul Upper specification limit for the parameter in the par field.

imp_tool The tool that has the lowest Cpk.

cpk The Cpk value of imp_tool.

stat_test The most significant statistical test.

p_value The probability value for the most significant statistical test.

20.3 T2T_TEST

Abstract This table contains the results for all the statistical tests performed for each parameter/step combination.

Table

T2T_TEST	
1 (sum_key) n	sum_key stat_test df1 df2 val crit sp p_value
T2T_SUM	

Serial Key None.

Foreign Keys / Link Description

sum_key A reference to the parameter/step combination.

Other Fields in T2T_TEST

stat_test The name of the statistical test.

df1 The first set of degrees of freedom.

df2 The second set of degrees of freedom, when applicable.

val The statistical test value.

crit The statistical test critical value.

sp The pooled standard deviation, if applicable.

p_value The probability value for the statistical test.

20.4 T2T_TOOL

Abstract This table contains the tools and tool statistics for each parameter/step combination.

Table

T2T_TOOL	
1 (sum_key) n	sum_key tool cnt row_cnt mean stddev min q1 q2 q3 max
T2T_SUM	

Serial Key None.

Foreign Keys / Link Description

sum_key The reference to the parameter/step combination.

Other Fields in T2T_TOOL

tool The tool name.

cnt The number of observations/data points associated with the tool for that parameter and step.

row_cnt The average number of raw data points used to compute each of the observations.

mean The mean of the observations for the parameter.

stddev The standard deviation of the observations for the parameter.

min The minimum value of the observations for the parameter.

q1-q3 Quartiles 1, 2, and 3 of the observations for the parameter.

max The maximum value of the observations for the parameter.

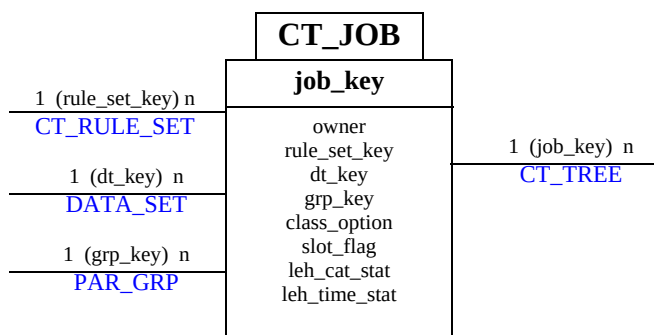
21.0 Data Mining Tables

The Data Mining tables are only used by the *dataPOWER* Data Mining classification trees. They are used to store the tree model generated by the CT classification tree module.

21.1 CT_JOB

Abstract This table contains references to a set of classification trees containing rules distinguishing a common set of data into a common set of classes.

Table



Serial Key

job_key A unique key that identifies a specific classification job.

Foreign Keys / Link Description

rule_set_key A key referencing a record in the CT_RULE_SET table.

dt_key A key referencing a record in the DATA_SET table.

grp_key A key referencing a record in the PAR_GRP table.

Other Fields in CT_JOB

owner The name of the owner.

class_option The action the system takes when there are duplicate class values. 1 = First class, 2 = Last class, 3 = Ignore.

slot_flag Source-lot flag.

leh_cat_stat LEH statistics name the system uses in conjunction with time. For example, "OP", "PE" and so on.

leh_time_stat LEH time the system uses in conjunction with the selected LEH statistic. For example, "TI", "TO" and so on.

21.2 CT_TREE

Abstract Contains an entry for each classification tree.

Table

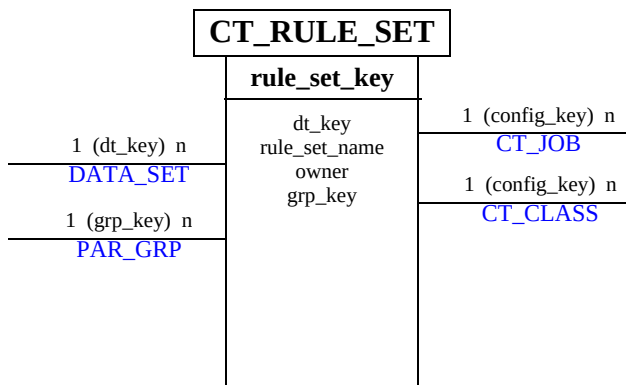
CT_TREE	
tree_key	
1 (job_key) n CT_JOB	tree_name parent_key config_key job_key start_time end_time insert_time train_accuracy test_accuracy level_cnt isstub xml_file
1 (config_key) n CT_CONFIG	
1 (parent_key) n CT_TREE	

Serial Key	tree_key	A unique key that identifies a specific tree classification.
Foreign Keys / Link Description	parent_key	A reference to the parent CT_TREE.
	config_key	A reference to a classification tree configuration in CT_CONFIG.
	job_key	A reference to the CT_JOB job instance to which this tree belongs.
Other Fields in CT_TREE	tree_name	The name of the tree.
	start_time	The time that the classification tree job was begun in the database.
	end_time	The time that the classification tree job was finished in the database.
	insert_time	The time that the tree was inserted in the database.
	train_accuracy	The classification tree accuracy for training data.
	test_accuracy	The classification tree accuracy for test data.
	level_cnt	The number of levels in the tree.
	isstub	Indicates that the tree has been modified or has not yet been filled. Values: N = no; Y = yes.
	xml_file	The XML file containing the classification tree.

21.3 CT_RULE_SET

Abstract Contains classification rule sets.

Table



Serial Key **rule_set_key** Automatically generated based on **rule_set_name**.

Foreign Keys / Link Description

dt_key A reference to the data set — DATA_SET.

grp_key A key referencing a record in the PAR_GRP table.

Other Fields in CT_RULE_SET

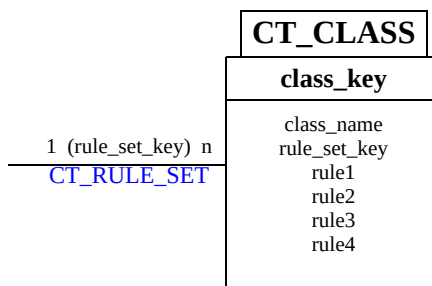
rule_set_name The name of the rule set.

owner The name of the owner.

21.4 CT_CLASS

Abstract Contains the list of classes in each rule set and the rules that define them.

Table



Serial Key **class_key** Automatically generated based on class_name.

Foreign Keys / Link Description

rule_set_key A reference to the rule set — CT_RULE_SET.

Other Fields in CT_CLASS	class_name	The name of the classification.
	rule1-4	Contains the formula that defines the class.

21.5 CT_CONFIG

Abstract Contains all the default and user settings needed to run Classification Tree analysis.

Table

CT_CONFIG
config_key
owner
config_name
test_proportion
max_cat_bins
subsample_size
subsample_method
max_depth
max_terminals
min_child_size
min_child_method
splitter_method
num_competitors
num_cat_srgts
num_cont_srgts
surrogate_imp
missing_factor_cut
rec_weighting
pruning_conf
date_range
random_seed
branch_replace

Serial Key	config_key	Automatically generated based on config_name .
Foreign Keys / Link Description	None.	
Other Fields in CT_CONFIG	owner	The owner name.
	config_name	The configuration name.
	test_proportion	The proportion of the data reserved for testing when a dedicated testing set is used.
	max_cat_bins	The maximum number of categorical states that an explanation variable is limited to.
	subsample_size	Defines the maximum number of records extracted from very large data sets, on which the calculation will be performed.

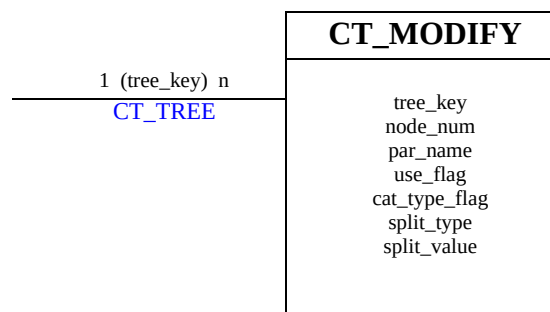
subsample_method	Defines the method in which the subsample is extracted: random = randomly; first = extracted from the beginning of the data set; last = extracted from the end of the data set; spaced = evenly spaced throughout the data set.
max_depth	The (user-defined) maximum allowed number of levels in the tree.
max_terminals	The (user-defined) maximum allowed number of terminal nodes.
min_child_size	The (user-defined) minimum number of records allowed in each node before it can be created.
min_child_method	The (user-defined) method for defining the minimum number of records allowed in each node before it can be created: fraction = % of parent; records = number of readings.
splitter_method	The (user-defined) method for defining <i>Splitter Method</i> — splitter rule methodology. In this release, there are no options other than the default, Gini .
num_competitors	An integer that defines the maximum number of parameters that can be used to choose the splitter parameter.
num_cat_srgts	An integer that defines the maximum number of surrogate rules that can be used to replace a categorical splitter parameter if it is missing.
num_cont_srgts	An integer that defines the maximum number of surrogate rules that can be used to replace a continuous splitter parameter if it is missing.
surrogate_imp	The weight on surrogate improvements.
missing_factor_cut	The proportion of missing records that are allowed in a explanation variable, below which this variable is not considered.
rec_weighting	The method used to modify the weightings of the input records: unit — none; equal class — such that each class has an equal number of records; advanced — uses an advanced algorithm to determine the weighting.
pruning_conf	Controls the extent of the tree pruning. The smaller the number the larger the tree.

date_range	Instruction for the system to consider date range in the splitter rule.
random_seed	Used in the random selection of test samples.
branch_replace	Apply a special branch replacement algorithm.

21.6 CT_MODIFY

Abstract Contains information about branches the system removes, or nodes the system replaces with different parameters. It is used when rebuilding trees.

Table



Serial Key None.

Foreign Keys / Link Description

tree_key Foreign key from CT_TREE. A reference to a classification tree in CT_TREE.

Other Fields in CT_MODIFY

node_num The node serial number within the tree.

par_name The name of the parameter to be included or excluded from the tree at the specified node.

use_flag Indicates whether to keep or exclude the specified parameter.

cat_type_flag Indicates whether the specified parameter is continuous or categorical.

split_type The logic operation for a split at a specified node. For example, ">" or "<=" and so on.

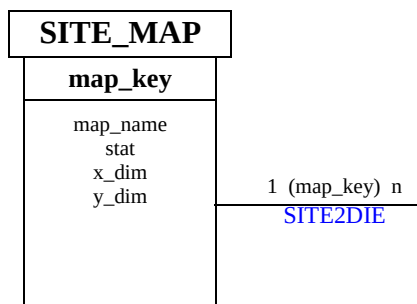
split_value The value of a split at a specified node.

22.0 PCM Wafer Sort Alignment

22.1 SITE_MAP

Abstract This table includes the available site maps.

Table



Serial Key **map_key** Serial key which uniquely identifies a map name.

Foreign Keys / Link Description None.

Other Fields in SITE_MAP

stat The statistic used to collapse data, e.g. average, sum, etc.

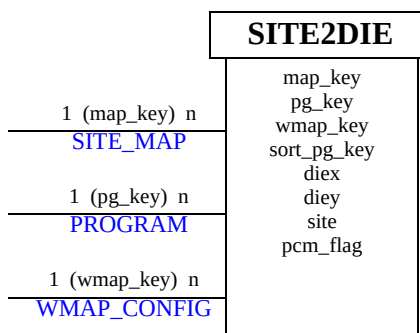
x_dim The number of die on the X-axis. It has to be odd.

y_dim The number of die on the Y-axis. It has to be odd.

22.2 SITE2DIE

Abstract This table maps PCM sites to wafer sort die based on the selected map of influence.

Table



Serial Key None.

**Foreign Keys / Link
Description****map_key** Reference to SITE_MAP. It contains the map key.**pg_key** PCM program key.**wmap_key** Link into WMAP_CONFIG. Identifies wafer map configuration.**sort_pg_key****Other Fields in
SITE2DIE****diex** die x index.**diey** die y index.**site** Site number.**pcm_flag** A flag stating whether the site is included in the dice or not.

23.0 Multi-Bin Tables

23.1 MB_UPSTAT

Abstract *dataPOWER* and the dbascii reader inserts a row into MB_UPSTAT identifying a set of programs that needs to be consolidated by BinTools. BinTools processes MB_UPSTAT entries and logically combines the tests of more than one program for the same product. The resulting file is then loaded by the dbascii reader into a new program.

Table

MB_UPSTAT	
1 (lot_key) n LOT	lot_key
1 (wf_key) n WAFER	wf_key
1 (mbset_key) n MB_SET	mbset_key
	error_code
	err_msg
	flag

Serial Key None.

Foreign Keys / Link Description

lot_key Reference to lot in LOT table.

wf_key Foreign key from WAFER. The key to the related wafer information.

mbset_key Foreign key from MB_SET.

Other Fields in MB_UPSTAT

error_code Contains error code.

err_msg Describes error.

flag Flag to indicate the last processing stage for current entry.

23.2 MB_SET

Abstract Used by BinTools. Holds the name of the new consolidated program, and the program keys of the programs to be consolidated.

Table

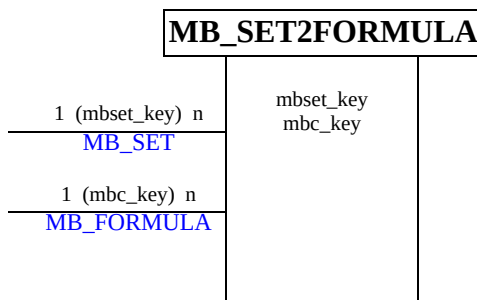
MB_SET	
mbset_key	
pg_key1-10	1 (mbset_key) n MB_UPSTAT
set_type	
ppid	
test_cnt	
pg_cnt	1 (mbset_key) n MB_SET2FORMULA
	1 (mbset_key) n MB_RULE

Serial Key	mbset_key	Serial key.
Foreign Keys / Link Description	pg_key1-10	Program keys of the programs to be consolidated (maximum of 10).
Other Fields in MB_SET	set_type	Identifies the program type — MultiBin (M) or SuperBin (S).
	ppid	The name of the new consolidated program.
	test_cnt	Number of tests for the new program in MultiBin and number of rules (in MB_RULE) if program is SuperBin.
	pg_cnt	Number of programs to consolidate.

23.3 MB_SET2FORMULA

Abstract An m2n table holding all the available combinations of **mbset_key** and **mbc_key**.

Table



Serial Key None.

Foreign Keys / Link Description

mbset_key Foreign key from MB_SET.

mbc_key Foreign key from MB_FORMULA.

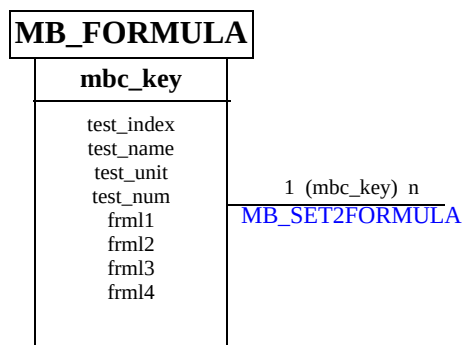
Other Fields in MB_SET2FORMULA

None.

23.4 MB_FORMULA

Abstract This table is used for MultiBin. MB_FORMULA holds information loaded by the client from the MultiBin window, it holds the logical formulas for combining the tests, and conditions information.

Table



Serial Key **mbc_key** Serial key.

Foreign Keys / Link Description

None.

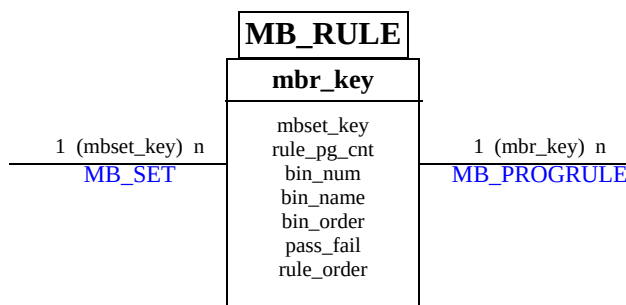
Other Fields in MB_FORMULA

test_index	Contains Test index.
test_name	Test name.
test_unit	Test unit.
test_num	Test number.
frml1-4	Contains the logical formulas for combining the tests.

23.5 MB_RULE

Abstract Used for SuperBin. It holds information about the rules and the resulting bin from the consolidation.

Table

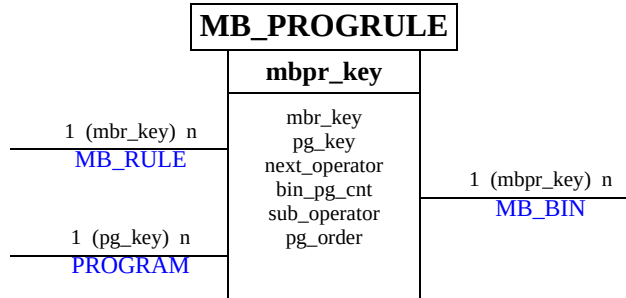


Serial Key	mbr_key	Serial key.						
Foreign Keys / Link Description	mbset_key	Foreign key from MB_SET.						
Other Fields in MB_RULE	rule_pg_cnt	Number of programs in the rule.						
	bin_num	Bin number.						
	bin_name	Descriptive text on bin.						
	bin_order	The order in which bins were assigned to the die.						
	pass_fail	Flag indicating if bin is a pass or fail bin.						
	Allowed values:							
	<table><tr><th>Value</th><th>Description</th></tr><tr><td>P</td><td>pass bin</td></tr><tr><td>F</td><td>fail bin</td></tr></table>		Value	Description	P	pass bin	F	fail bin
Value	Description							
P	pass bin							
F	fail bin							
	rule_order	The order in which rules were assigned to the bin.						

23.6 MB_PROGRULE

Abstract Used for SuperBin. It holds details on each rule set in MB_RULE.

Table



Serial Key

mbpr_key Serial key.

Foreign Keys / Link Description

mbr_key From MB_RULE table.

pg_key A reference to the PROGRAM table.

Other Fields in MB_PROGRULE

next_operator And/or between programs of each rule.

bin_pg_cnt Count of how many bins are used for each program.

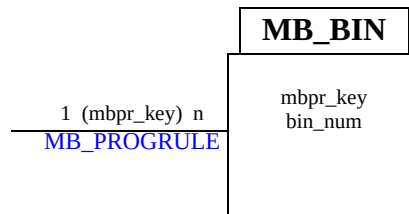
sub_operator N- not equal (similar to AND), E- equal (similar to OR).

pg_order Order of program rule to be executed first.

23.7 MB_BIN

Abstract Holds bin number for each program in MB_PROGRULE.

Table



Serial Key None.

Foreign Keys / Link Description

mbpr_key From MB_PROGRULE table.

Other Fields in MB_BIN

bin_num Bin number. -99999 for null.

24.0 Spatial Signature Analysis Tables

The purpose of Spatial Signature Analysis (SSA) is to enable the user to submit batch Fail Signature Detection and Analysis (FSDA) and Fail Signature Classification (FSC) jobs via a GUI, accessible from *dataPOWER*. The user will be able to monitor the progress of the run(s), including when the run has completed.

The user submits a job via the GUI by inserting it into certain database tables, including *SSA_RUN*. The program queries the *SSA_RUN* table for the "status", "error_code" and "err_msg" fields; if status = 0, error_code = null, and err_msg = null, then the entry is considered a new job and the program proceeds.

SSA stores the output data into a set of database tables; it also creates the images and stores them in the *SSA_IMAGE* table:

- *SSA_DIE_<pg_key>*
 - *Tn* columns - die-level statistic (mean, median, etc.) aggregated at the cluster level for each cluster identified by FSDA. One column per selected parameter (# of selected parameters = *n*).
 - *Cn* columns - # of die containing data for the selected parameter.
- *SSA_ZONE_<pg_key>*
 - *Tn* columns - zone-level statistic (mean, median, etc.) aggregated at the wafer level for each cluster identified by FSDA. One column per selected parameter (# of selected parameters = *n*).
 - *Cn* columns - # of die in each wafer/zone containing data for the selected parameter.
- *SSA_CLUSTER_<pg_key>*
 - *Tn* columns - zone-level statistic (mean, median, etc.) aggregated at the cluster level for each cluster identified by FSDA. One column per selected parameter (# of selected parameters = *n*).
 - *Cn* columns - # of die in each cluster/zone containing data for the selected parameter.

SSA stores data into other tables and also create the images and store them into *ssa_image*.

24.1 SSA_DIE_...

Abstract This table is by program class, and is filled by the SSA tool. It stores the die-level statistics and count aggregated by cluster level.

Table

SSA_DIE_\${pgc_key}	
1 (ssa_rn_key) n SSA_RUN	ssa_rn_key die_x die_y ssa_cluster_key
1 (ssa_cluster_key) n SSA_CLUSTER	Tn Cn

Serial Key None.

Foreign Keys / Link Description

ssa_rn_key Foreign key from SSA_RUN table.

ssa_cluster_key Foreign key from SSA_CLUSTER table.

Other Fields in SSA_DIE_...

die_x The x-coordinate of a die under test.

die_y The y-coordinate of a die under test.

Tn The die-level statistic (yield) aggregated at the cluster level for each cluster identified by FSDA. One column per selected parameter (# of selected parameters = n).

Cn The number of die containing data for the selected parameter. The y-coordinate of a die under test.

24.2 SSA_ZONE_...

Abstract This table is by program class and is filled by the SSA tool. It stores the zone-level statistics and count aggregated by wafer for each cluster identified by FSDA.

Table

SSA_ZONE_\${pgc_key}	
1 (ssa_rn_key) n SSA_RUN	ssa_rn_key zone_key wf_key
1 (zone_key) n ZONES	Tn Cn
1 (wf_key) n WAFER	

Serial Key	None.	
Foreign Keys / Link Description	ssa_rn_key	Foreign key from SSA_RUN table.
	zone_key	Value from ZONES table.
	wf_key	Foreign key from WAFER. The key to the related wafer information.
Other Fields in SSA_ZONE_...	Tn	The zone-level statistic (yield) aggregated at the wafer level for each cluster identified by FSDA. One column per selected parameter (# of selected parameters = n).
	Cn	The number of die in each wafer/zone containing data for the selected parameter.

24.3 SSA_CLUSTER_\$....

Abstract This table is by program class and is filled by the SSA tool. It stores the zone-level statistics and count aggregated at cluster level for each cluster identified by FSDA.

Table

SSA_CLUSTER_\${pgc_key}		
1 (ssa_rn_key) n	ssa_rn_key	
SSA_RUN	zone_key	
1 (ssa_cluster_key) n	ssa_cluster_key	
SSA_CLUSTER	Tn	
1 (zone_key) n	Cn	
ZONES		

Serial Key	None.	
Foreign Keys / Link Description	ssa_rn_key	Foreign key from SSA_RUN table.
	zone_key	Value from ZONES table.
	ssa_cluster_key	Foreign key from SSA_CLUSTER table.
Other Fields in SSA_CLUSTER_...	Tn	The zone-level statistic (yield) aggregated at the cluster level for each cluster identified by FSDA. One column per selected parameter (# of selected parameters = n).
	Cn	The number of die in each cluster/zone containing data for the selected parameter.

24.4 SSA_RUN

Abstract This table is populated by the GUI and each entry represents a job for FSDA or the signature library; it holds all the information about this job. The SSA tool queries this table and processes each entry.

Table

SSA_RUN		
	ssa_rn_key	
1 (pgc_key) n PROG_CLASS	ssa_rn_name	1 (ssa_rn_key) n SSA_DIE_...
	pgc_key	
	ssa_rn_type	1 (ssa_rn_key) n
	ssa_rn_desc	SSA_ZONE_\$....
	insert_time	
	end_time	1 (ssa_rn_key) n
	status	
	owner	SSA_CLUSTER_\$....
	test_cnt	
	col_type	1 (ssa_rn_key) n
	cfg1_r	SSA_RUN_DET
	cfg2_r	
	cfg3_r	1 (ssa_rn_key) n
	cfg4_r	SSA_RUN2ZONE
	cfg5_r	
	cfg6_r	1 (ssa_rn_key) n
	cfg1_s	SSA_DEF
	cfg2_s	
	error_code	1 (ssa_rn_key) n
	err_msg	SSA_DEF_DET
		1 (ssa_rn_key) n
		SSA_IMAGE
		1 (ssa_rn_key) n
		SSA_CLUSTER
		1 (ssa_rn_key) n
		SSA_WF2CLUSTER
		1 (ssa_rn_key) n
		SSA_OPER_HIST

Serial Key

ssa_rn_key A unique key that identifies a specific FSDA or Signature library job.

Foreign Keys / Link Description

pgc_key Reference to PROG_CLASS table.

Other Fields in SSA_RUN

ssa_rn_name The name of the job.

ssa_rn_type The report type (FSDA, Signature Library).

ssa_rn_desc The job description.



Front



Contents



Index

insert_time The time data was inserted into the database (populated by the GUI).

end_time The time the SSA tool finished processing this entry. (populated by SSA tool).

status The status of the entry.

Value	Description
0	new
1	in progress
2	successful
3	signature library update
-1	error

owner The name of the user who created the run.

test_cnt The number of tests/parameters selected when the job was created. If it was calculated, then -1 is used.

col_type Column type drop-down list from GUI.

Field	binMAP (pgc_key=4)	defects (pgc_key=14)	parametric (other pgc_key except 10)
col_type	Null	Column Type drop-down list: L = layer C = class S = size B = block, LC = layer/ class LS = layer/ size	Null

cfg1-6_r Values for cfg1-6_r are listed below:

Field	binMAP (pgc_key=4)	defects (pgc_key=14)	parametric (other pgc_key except 10)
cfg1_r	max cluster	max cluster	max cluster
cfg2_r	zone eng sig	zone eng sig	zone eng sig
cfg3_r	wfr eng sig	wfr eng sig	wfr eng sig
cfg4_r	target fail %	target fail count	part lot %

Field	binMAP (pgc_key=4)	defects (pgc_key=14)	parametric (other pgc_key except 10)
cfg5_r	part lot %	part lot %	If Statistics = Yield, 1=P 2=S 3=O 4=W
cfg6_r	If Bin Criteria is "Calculated", cfg6_4 = Top <i>n</i> Bins (plus 2 for others and yield).	If Level Criteria = 'Calculated', cfg6_r = Top <i>n</i> Levels (plus 1 for others)	If Filter-Method=Value, 1=P 2=S 3=O 4=W If FilterMethod=IQR or Mean, cfg6_r = Lim Wheel-Button; If FilterMethod = none, boxplot or tri-median, cfg6_r = -1

cfg1-2_s Values for **cfg1-2_s** are listed below:

Field	binMAP (pgc_key=4)	defects (pgc_key=14)	parametric (other pgc_key except 10)
cfg1_2	Null	WhichSize or ClassMethod drop-down list	Statistics drop-down list
cfg2_s	"Selected" or "Calculated"	"Selected" or "Calculated"	FilterMethod drop-down list

error_code The error codes and descriptions are listed below:
err_msg A text message describing the error.

24.5 SSA_RUN_DET

Abstract This table is populated by the GUI; it contains lot and wafer information for each run.

Table

SSA_RUN_DET	
1 (ssa_rn_key) n SSA_RUN	ssa_rn_key pg_key lot_key lg_key wf_key ssa_cluster_key
1 (pg_key) n PROGRAM	
1 (lot_key) n LOT	
1 (lg_key) n OP_LOG	
1 (wf_key) n WAFER	
1 (ssa_cluster_key) n SSA_CLUSTER	

Serial Key None.

Foreign Keys / Link Description

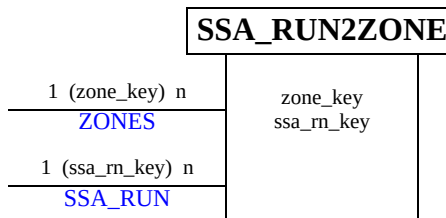
ssa_rn_key	Foreign key from SSA_RUN table.
pg_key	A reference to the PROGRAM table.
lot_key	Foreign key from the LOT table. The key to the related lot information.
lg_key	Foreign key from OP_LOG table.
wf_key	Foreign key from WAFER. The key to the related wafer information.
ssa_cluster_key	Foreign key from SSA_CLUSTER table.

Other Fields in SSA_RUN_DET None.

24.6 SSA_RUN2ZONE

Abstract This table is an m-to-n table that assigns zones to an SSA run.

Table



Serial Key None.

Foreign Keys / Link Description

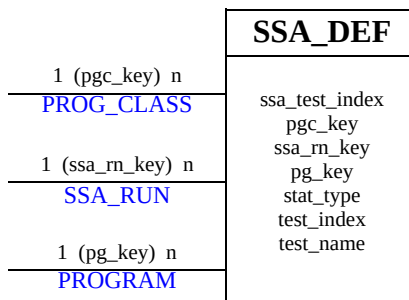
zone_key	Value from ZONES table.
ssa_rn_key	Foreign key from SSA_RUN table.

Other Fields in SSA_RUN2ZONE None.

24.7 SSA_DEF

Abstract This table is populated by the user via the GUI and is updated by the SSA tool. It contains information about the bins, parameters, and layers and is part of the FSDA job.

Table



Serial Key None.

Foreign Keys / Link Description

pgc_key	Reference to PROG_CLASS table.
ssa_rn_key	Foreign key from SSA_RUN table.
pg_key	A reference to the PROGRAM table.

Other Fields in SSA_DEF **ssa_test_index** A unique value.

stat_type The type of statistics stored in the current record.

For Defect data, the legal values for **stat_type** are:

Value	Description
TC	total count
AC	adder count
LTC	log of total count
LAC	log of adder count
Y	yield

For other data types, the legal values for **stat_type** are:

Value	Description
avg	Average

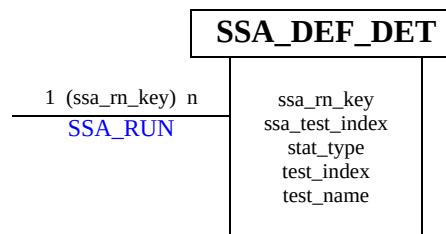
test_index Contains Test index value.

test_name Test name.

24.8 SSA_DEF_DET

Abstract This table is populated by the user via the GUI and is updated by the SSA tool. It holds information about the classes, blocks, size, and layers and it is part of the FSDA Job (for Defect data only).

Table



Serial Key None.

Foreign Keys / Link Description

ssa_rn_key Foreign key from SSA_RUN table.

Other Fields in SSA_DEF_DET

ssa_test_index Foreign key from SSA_DEF.

stat_type The type of statistics stored in the current record. The legal values for **stat_type** are:

Value	Description
L	layer
C	class
S	size
B	block

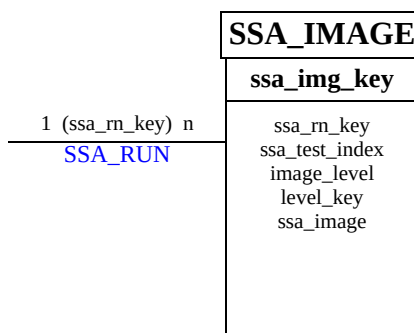
test_index The key for type of defect.

test_name Test name.

24.9 SSA_IMAGE

Abstract This table is filled by SSA tool. It contains one bin failure map for each bin/test. It also creates stacked bin failure maps for each bin.

Table



Serial Key

ssa_img_key Serial key for SSA_IMAGE.

Foreign Keys / Link Description

ssa_rn_key Foreign key from SSA_RUN table.

level_key Level key (ssa_cluster_key).

Other Fields in SSA_IMAGE

ssa_test_index A unique value for each (bin/parameter).

image_level Family or cluster.

ssa_image (Bin/parameter/..), image blob.

24.10 SSA_CLUSTER

Abstract The SSA Tool divides data into statistically significant clusters; this table holds the information about each cluster.

Table

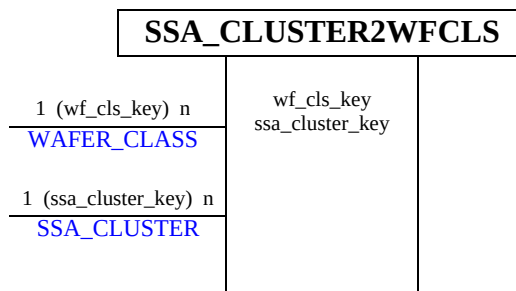
SSA_CLUSTER		
1 (ssa_m_key) n SSA_RUN	ssa_cluster_key	1 (ssa_cluster_key) n
	ssa_m_key	SSA_DIE_...
	ssa_cluster_name	1 (ssa_cluster_key) n
	ssa_cluster_desc	SSA_CLUSTER_\$....
	wafer_cnt	1 (ssa_cluster_key) n
	stat1	SSA_RUN_DET
	stat2	1 (ssa_cluster_key) n
		SSA_CLUSTER2WFCLS
		1 (ssa_cluster_key) n
		SSA_WF2CLUSTER

Serial Key	ssa_cluster_key	Serial key for SSA_CLUSTER.
Foreign Keys / Link Description	ssa_rn_key	Foreign key from SSA_RUN table.
Other Fields in SSA_CLUSTER	ssa_cluster_name	Cluster name.
	ssa_cluster_desc	Cluster description.
	wafer_cnt	The number of wafers in each cluster.
	stat1	Reserved for future use.
	stat2	Reserved for future use.

24.11 SSA_CLUSTER2WFCLS

Abstract An m-to-n table that assigns clusters to a wafer class.

Table



Serial Key None.

Foreign Keys / Link Description

wf_cls_key	Foreign key from WAFER_CLASS table.
ssa_cluster_key	Foreign key from SSA_CLUSTER table.

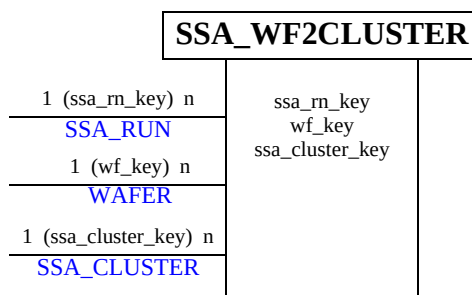
Other Fields in SSA_CLUSTER2WFCLS

None.

24.12 SSA_WF2CLUSTER

Abstract An m-to-n table that assigns wafers to a cluster.

Table



Serial Key None.

Foreign Keys / Link Description

ssa_rn_key	Foreign key from SSA_RUN table.
wf_key	Foreign key from WAFER. The key to the related wafer information.

ssa_cluster_key Foreign key from SSA_CLUSTER table.

**Other Fields in
SSA_WF2CLUSTER** None.

24.13 SSA_WAFER_DISTANCE

Abstract This table relates to the FSC module when executed as part of the SSA-Monitor; it maintains all distances between a wafer being classified and all classes in the library being used for classification.

Table

SSA_WAFER_DISTANCE	
1 (wf_key) n	wf_key lib_wf_key wf_cls_key lwc_lg_key distance
WAFER	
1 (lib_wf_key) n	
WAFER	
1 (wf_cls_key) n	
WAFER_CLASS	
(1 lwc_lg_key) n	
LW_CLASS_LOG	

Serial Key None.

**Foreign Keys / Link
Description**

wf_key Foreign key from WAFER. Automatically generated, unique wafer key. Based on **wf_id**.

lib_wf_key Foreign key from WAFER. Defines which wafer is being classified.

wf_cls_key Foreign key from WAFER_CLASS. Defines the wafer class.

lwc_lg_key Foreign key from LW_CLASS_LOG. Log entry that classified the wafer

**Other Fields in
SSA_WAFER_DIST
ANCE**

Distance The distance between the class and the wafer.

24.14 SSA_OPER_HIST

Abstract This table records modifications to the signature library.

Table

SSA_OPER_HIST		
1 (oper_type_key) n	oper_hist_key	1 (oper_hist_key) n
SSA_OPER_TYPES	oper_type_key	SSA_OPER
1 (trgt_rn_key) n	trgt_rn_key	
SSA_RUN	wf_cls_key	
1 (wf_cls_key) n	oper_level	
WAFER_CLASS	insert_time	
	owner	
	comments	

Serial Key

oper_hist_key A unique key assigned to each entry in SSA_OPER_HIST. It is stored automatically.

Foreign Keys / Link Description

oper_type_key Foreign key from the SSA_OPER_TYPES table, where it specifies what type of operation was performed on the signature library — such as *delete/add wafers* or *create signature library*.

trgt_rn_key Foreign key from the SSA_RUN table. This key specifies which signature library these modifications belong too.

wf_cls_key Foreign key from the WAFER_CLASS table. This key specifies which class these modifications belong too.

Other Fields in SSA_OPER_HIST

oper_level Indicates which level that these modifications belong too. Legal values are:

Value	Description
S	Signature Library
C	Class
W	Wafer

insert_time The time that data was inserted into the SSA_OPER_HIST table (populated by the GUI).

owner The name of the user who performed the modifications

comments User-defined comments string.

24.15 SSA_OPER_TYPES

Abstract Indicates what types of operations were performed on the signature library.

Table

SSA_OPER_TYPES	
oper_type_key	
oper_type	1 (oper_type_key) n SSA_OPER_HIST

Serial Key **oper_type_key** A unique key assigned to each entry in SSA_OPER_TYPES. It is stored automatically.

Foreign Keys / Link Description None.

Other Fields in SSA_OPER_TYPES **oper_type** Possible values of oper_type:

Value	Description
1	create: create signature library
2	delete: delete wafers
3	add: add wafers or classes
4	remove: remove wafers
5	rename
6	copy: copy signature library

24.16 SSA_OPER

Abstract Indicates which wafers belong to specific operations in the SSA_OPER_HIST table.

Table

SSA_OPER	
1 (oper_hist_key) n SSA_OPER_HIST	oper_hist_key wf_key
1 (wf_key) n WAFER	

Serial Key None.

Foreign Keys / Link Description

oper_hist_key Foreign key from the SSA_OPER_HIST table, where one or many wafers can belong to one operation (such as *add/delete wafers*) in the SSA_OPER_HIST table.

wf_key Foreign key from the WAFER table, where these keys indicate which wafers belong to specific operations in SSA_OPER_HIST.

Other Fields in SSA_OPER None.

24.17 SSA_YLD_RULE

Abstract This table is populated by the GUI. Each entry represents a rule that will be used in generating Lot Reports; it holds all the information about these rules.

Table

SSA_YLD_RULE		
1 (pg_key) n	ssa_rule_key	1 (ssa_rule_key) n
PROGRAM	pg_key cls_key rule_name def_flag owner operator target_yld description	SSA_YLD_RULE_DEF
1 (cls_key) n		
WAFER_CLASS		

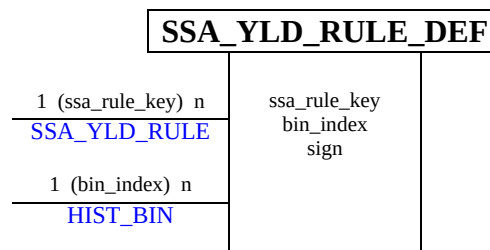
Serial Key **ssa_rule_key** A unique key assigned to each entry in SSA_YLD_RULE. It is stored automatically.

Foreign Keys / Link Description**pg_key** Foreign key from the PROGRAM table.**cls_key** Foreign key from the WAFER_CLASS table.**Other Fields in SSA_YLD_RULE****rule_name** The name of the rule.**def_flag** For each owner and program, there is a default rule. This field flags whether the rule is default or not. Default values of def_flag:

Value	Description
0	not a default rule
1	default rule

owner The name of the user who inserts the rule.**operator** For each rule there is an operator. Possible values of operator:

Value	Description
>	greater than
<	less than

target_yld The number that represents the target yield.**description** User-defined comment string.**24.18 SSA_YLD_RULE_DEF****Abstract** This table is populated by the GUI. It describes the formula definitions for specific rules in the SSA_YLD_RULE table.**Table****Serial Key** None.**Foreign Keys / Link Description****ssa_rule_key** Foreign key from the SSA_YLD_RULE table.**bin_index** Foreign key from the HIST_BIN table.

**Other Fields in
SSA_YLD_RULE_D
EF****sign**

Possible values of operator:

Value	Description
+	plus
-	minus

Central Schema — Overview

A *dataPOWER* central schema is an optional schema that can be generated using the **-central** function in *DpCreateDb.pl*. It has the capability to store information at database-engine level as opposed to a data-schema level. Things such as workflows, load balancing configuration, and *dpMonitor* information are among a few of these data storage categories.

When creating a standard data *dataPOWER* schema, there is an prompt to attach that schema to a central schema. All *dataPOWER* database schemas on one engine can be attached to one central schema on the same engine.

The following sections describe tables that can exist in *both* the standard data schema and a central schema, and also tables that exist *only* in the central schema.

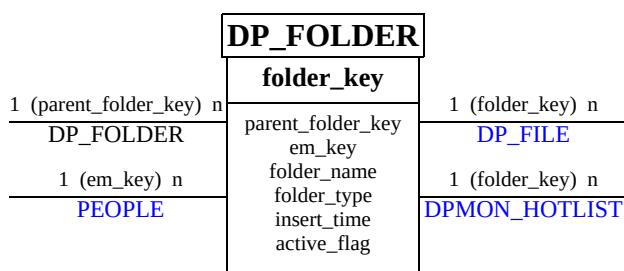
25.0 Central Schema or General Schema — File Storage Tables

The File Storage tables in this section are always present if a Central Schema is created. But they can also be created when a normal *dataPOWER* schema is generated, if the **-file_store** command-line argument is used with DpCreateDb.pl. This section describes how files, folders and their attributes will be stored in the database.

25.1 DP_FOLDER

Abstract This table contains information for folders stored in the database.

Table



Serial Key

folder_key A unique key that will be assigned to each entry in DP_FOLDER table. It is stored automatically.

Foreign Keys / Link Description

parent_folder_key Reference to **folder_key** in the same table, where each folder should have a Parent Folder, except the root will be -1.

em_key Foreign key from PEOPLE table.

Other Fields in DP_FOLDER

folder_name The name of the folder saved in the database.

folder_type An integer, it identifies which tool created the folder:

Value	Description
0	dpMonitor
1	workflow report
2	workflow
3	other

insert_time The time of insertion of the file into the database. Date/time/year to the second; default is current year to the second.

active_flag Flag indicating if setup is active (Y/N).

25.2 DP_FILE

Abstract This table contains information on worksheet files stored in the database.

Table

DP_FILE		
dpf_key		
1 (em_key) n	em_key folder_key file_name file_type file_blob dp_version dpsh_version start_time end_time insert_time file_desc	1 (dpf_key) n
PEOPLE		DPF2FIELD
1 (folder_key) n		1 (dp_wr_key) 1
DP_FOLDER		DPFILE2FILE
		1 (dpf_w_key) n
		DPFILE2FILE

Serial Key

dpf_key Assigned to each entry in the DP_FILE table. It is stored automatically.

Foreign Keys / Link Description

em_key Foreign key from PEOPLE table.

folder_key Foreign key from DP_FOLDER table.

Other Fields in DP_FILE

file_name The name of the worksheet file saved in the database

file_type L — limits worksheet
D — data worksheet
O — other type of worksheet
U — unknown

Value	Description
L	limits worksheet
D	data worksheet
O	other type of worksheet
U	unknown
W	workflow
R	workflow report
C	control sheet
S	script

file_blob Pointer to file BLOB.

dp_version date-time year to second. *For future use.*

dpsh_version dpSHEET (Wingz) version.

start_time	Date-time-year to the second.
end_time	Date-time-year to the second.
insert_time	The time of insertion of the file into the database. Date/time/year to the second; default is current year to the second.
file_desc	<i>For future use.</i>

25.3 DPF2FIELD

Abstract This table associates the file with field keys and the values for that field (file property).

Table

DPF2FIELD		
1 (dpf_key) n DP_FILE	dpfldval_key	1 (dpfldval_key) n DPFILE2FILE
1 (dpfld_key) n DPF_FIELD	dpf_key dpfld_key field_value	

Serial Key

dpfldval_key A unique key assigned to each entry in DPF2FIELD. It is stored automatically.

Foreign Keys / Link Description

dpf_key Foreign key from DP_FILE table.

dpfld_key Foreign key from DPF_FIELD table.

Other Fields in DPF2FIELD

field_value The value of the current field for the associated file.

25.4 DPF2FILE

Abstract This table is for *workFLOW*. It is used to store the relationship between the *workFLOW* report and the workflows that are related to it.

Table

DPF2FILE	
1 (dpf_wr_key) n DP_FILE	dpf_wr_key dpf_w_key dpfldval_key dpf_order
1 (dpf_w_key) n DP_FILE	
1 (dpfldval_key) n DPF2FIELD	

Serial Key None.

Foreign Keys / Link Description	dpf_wr_key	Foreign key from the DP_FILE table, that represents workflow report key.
	dpf_w_key	Foreign key from the DP_FILE table, that represents workflow key.
	dpfldval_key	Foreign key from DPF2FIELD table.
Other Fields in DPF2FILE	dpf_order	Integer that represents the order of the workflow in workflow report.

25.5 DPF_FIELD

Abstract This table provides information regarding the properties name associated with files.

Table

DPF_FIELD	
dpfld_key	1 (dpfld_key) n DPF2FIELD
field_name field_type	

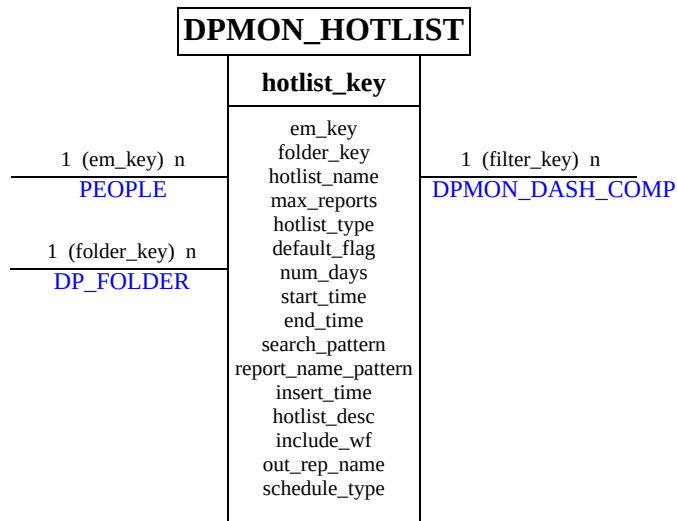
Serial Key	dpfld_key	A unique key assigned to each entry in DPF_FIELD. It is stored automatically.
Foreign Keys / Link Description	None.	
Other Fields in DPF_FIELD	field_name	The value of the current field for the associated file.
	field_type	<i>For future use.</i>

26.0 Central Schema — dPMonitor Tables

26.1 DPMON_HOTLIST

Abstract This table contains the information for configuration of *dpMonitor* hot lists (filters).

Table



Serial Key

hotlist_key A unique key that assigned to each entry in DPMON_HOTLIST. It is stored automatically.

Foreign Keys / Link Description

em_key Foreign key from PEOPLE table.

folder_key Foreign key from DP_FOLDER table.

Other Fields in DPMON_HOTLIST

hotlist_name String Value for the hotlist label.

max_reports The search criteria setting for this hotlist for the maximum number of reports to retrieve.

hotlist_type Character to indicate the type of hot list:

Value	Description
R	report hotlist
S	task hotlist
W	workflow report hotlist

default_flag Flag to mark the default hotlist.

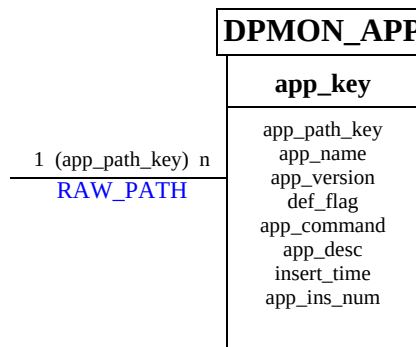
num_days Number of days after the start time to include in the retrieval.

start_time	Limit the search by start time, to take all the records of the insert time after the start date.
end_time	Limit the search to end time.
search_pattern	This enables the search for the contents of the record.
report_name_pattern	Search pattern to search on the name of report.
insert_time	The time of insertion of the file into the database. Date/time/year to the second; default is current year to the second.
hotlist_desc	Brief description of the hotlist (filter).
include_wf	This field is used when the search is to include both the workflow report <i>and</i> the associated workflows.
out_rep_name	Search on the task that will include the output report name.
schedule_type	The schedule options which are described in DPMON_TASK .

26.2 DPMON_APP

Abstract This table contains the installed application's version and configuration.

Table



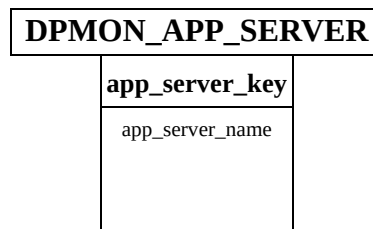
Serial Key	app_key	A unique key that assigned to each entry in DPMON_APP. It is stored automatically.
Foreign Keys / Link Description	app_path_key	Foreign key from RAW_PATH (path_key).
Other Fields in DPMON_APP	app_name	Unique string value with the name of the application.
	app_version	String value for the version number of the application.

def_flag	Marks the default application.
app_command	String value for the specific application command initial arguments.
app_desc	Brief description of the application.
insert_time	The time of insertion of the file into the database. Date/time/year to the second; default is current year to the second.
app_ins_num	This is integer value that indicates the number of instances the application should run.

26.3 DPMON_APP_SERVER

Abstract This table contains the name of the *dpMonitor* application server that uses this schema. The purpose is to block other instances from running on the same schema.

Table



Serial Key **app_server_key** A unique key that assigned to each entry in DPMON_APP_SERVER. It is stored automatically.

Foreign Keys / Link Description None.

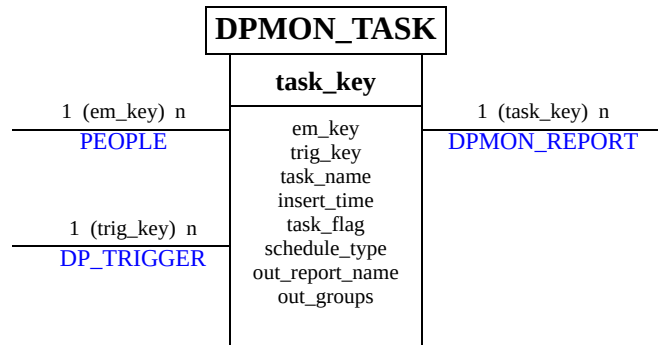
Other Fields in DPMON_APP_SERVER

app_server_name String value for the hostname that is running on the schema.

26.4 DPMON_TASK

Abstract This table contains all the *dpMonitor* tasks that have been scheduled. It contains the information for the tasks and is used to help the search for the tasks and to retrieve information from the scheduler system tables (indexing). It is also used to join the report with this task_key.

Table

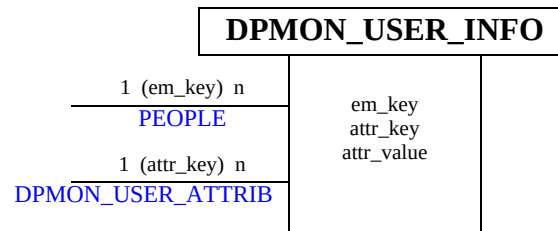


Serial Key	task_key	A unique key that identifies the task.												
Foreign Keys / Link Description	em_key	Foreign key from PEOPLE table.												
	trig_key	Foreign key from DP_TRIGGER table. Links to the type of trigger used.												
Other Fields in DPMON_TASK	task_name	The string value for the unique <i>dpMonitor</i> task name.												
	insert_time	The time of insertion of the file into the database. Date/time/year to the second; default is current year to the second.												
	task_flag	The integer value to indicate the task status.												
	schedule_type	The option that defines the task scheduling: <table><tr><th>Value</th><th>Description</th></tr><tr><td>0</td><td>Run now</td></tr><tr><td>1</td><td>Run once</td></tr><tr><td>2</td><td>Run periodically</td></tr><tr><td>3</td><td>Event triggered</td></tr><tr><td>4</td><td>Advanced</td></tr></table>	Value	Description	0	Run now	1	Run once	2	Run periodically	3	Event triggered	4	Advanced
	Value	Description												
0	Run now													
1	Run once													
2	Run periodically													
3	Event triggered													
4	Advanced													
out_report_name	The string value that defines the output report name generated when this task is triggered.													
out_groups	The comma separated string value that contains the OLS group's name to update the report record with the OLS settings, if enabled.													

26.5 DPMON_USER_INFO

Abstract This table contains the settings information for the user — such as email settings, language, default values, etc.

Table



Serial Key None.

Foreign Keys / Link Description

em_key Foreign key from PEOPLE table.

attr_key Foreign key from DPMON_USER_ATTRIB table.

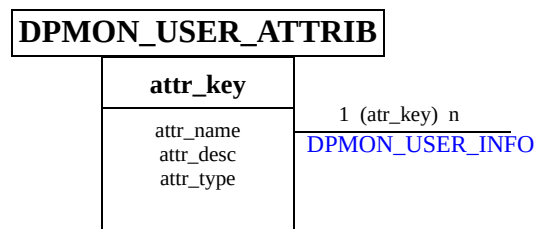
Other Fields in DPMON_USER_INFO

attr_value Contains different string structures to enable multiple attribute values (compound settings like in the table header the column shown, column sorted, etc.).

26.6 DPMON_USER_ATTRIB

Abstract This table contains the attribute definitions, such as the table header configuration.

Table



Serial Key

attr_key A unique key assigned to each entry in DPMON_USER_ATTRIB. It is stored automatically.

Foreign Keys / Link Description

None.

Other Fields in DPMON_USER_ATTRIB

attr_name The string value for the attribute name.



Front



Contents



Index

attr_desc A short description for the attribute.

attr_type Reserved for future use.

26.7 DPMON_REPORT

Abstract This table contains the report information that is generated from the application.

Table

DPMON_REPORT		
1 (em_key) n	dpmon_rep_key	1 (dpmon_rep_key) n
PEOPLE	folder_key	DPMON_ANNOT
1 (folder_key) n	em_key	
DP_FOLDER	task_key	
1 (task_key) n	trig_r_key	
DPMON_TASK	dpmon_rep_name	
1 (trig_r_key) n	insert_time	
DP_TRIGGER_RUNS	rep_flag	
	dpmon_rep_desc	
	aging_days	
	report_type	

Serial Key **dpmon_rep_key** A unique key that is assigned to each entry in DPMON_REPORT. It is stored automatically.

Foreign Keys / Link Description

folder_key Foreign key from DP_FOLDER table.

em_key Foreign key from PEOPLE table.

task_key Foreign key from DPMON_TASK table.

trig_r_key Foreign key from DP_TRIGGER_RUNS table.

Other Fields in DPMON_REPORT

dpmon_rep_name String value for the report name.

insert_time The time of insertion of the file into the database. Date/ time/year to the second; default is current year to the second.

rep_flag Integer value that maps the status for the report:

Value	Description
0	Ready
1	Done with warnings
2	under construction
3	Deleted
10	uploaded
11	restored
12	child

dpmon_rep_desc	Brief description of the report.
aging_days	Age of the report (in days) when the report needs to be sent to Recycle bin.
report_type	Reserved for future use.

26.8 DPMON_DASH_COMP

Abstract This table contains the configuration information for the users' dashboard settings.

Table

DPMON_DASH_COMP	
1 (em_key) n PEOPLE	comp_key
1 (filter_key) n DPMON_HOTLIST	em_key filter_key comp_col_order comp_row_order comp_type minimize_flag comp_options insert_time

Serial Key **comp_key** A unique key assigned to each entry in DPMON_DASH_COMP. It is stored automatically.

Foreign Keys / Link Description

em_key Foreign key from PEOPLE table.

filter_key Foreign key from DPMON_HOTLIST table.

Other Fields in DPMON_DASH_COMP

comp_col_order Component column order.

comp_row_order Component row order.

comp_type Define the component type:

Value	Description
0	hotlist
1	other

minimize_flag Flag to mark the component as minimized.

comp_options Component options, as string formulas.

insert_time The time of insertion of the file into the database. Date/ time/year to the second; default is current year to the second.

26.9 DPMON_ANNOT

Abstract This table contains the information that defines annotations.

Table

DPMON_ANNOT	
	annot_key
1 (em_key) n PEOPLE	em_key
1 (dpmon_rep_key) n DPMON_REPORT	dpmon_rep_key
1 (text1...4_key) n STRING HOLDER	text1_key text2_key text3_key text4_key xpos ypos insert_time annot_type

Serial Key **annot_key** A unique key assigned to each entry in DPMON_ANNOT. It is stored automatically.

Foreign Keys / Link Description

em_key Foreign key from PEOPLE table.

dpmon_rep_key Foreign key from DPMON_REPORT table.

text1_key ...text4_key Key from STRING HOLDER table which stores the contents of the added notes.

Other Fields in DPMON_ANNOT

xpos X position relative to the report page.

ypos Y position relative to the report page.

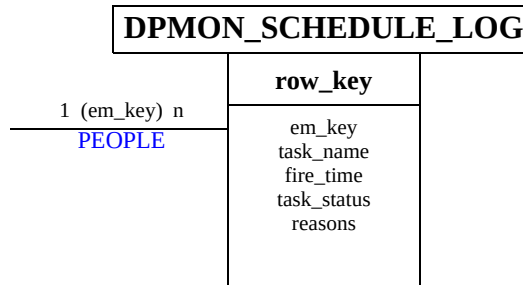
insert_time The time of insertion of the file into the database. Date/ time/year to the second; default is current year to the second.

annot_type Reserved for future use.

26.10 DPMON_SCHEDULE_LOG

Abstract This table contains the logs of warning and errors generated from running tasks.

Table



Serial Key **row_key** A unique key assigned to each entry in DPMON_SCHEDULE_LOG. It is stored automatically.

Foreign Keys / Link Description **em_key** Foreign key from PEOPLE table.

Other Fields in DPMON_SCHEDULE_LOG **task_name** String value for unique task name.

fire_time Task fire date-time.

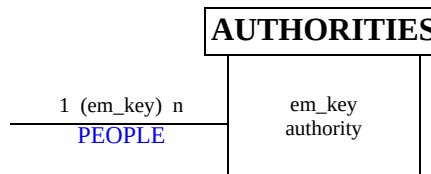
task_status String value to define the status of the task.

reasons The cause (brief description) for failure or warning.

26.11 AUTHORITIES

Abstract This table contains the role levels for the users. It defines the administrator and user roles.

Table



Serial Key None.

Foreign Keys / Link Description **em_key** Foreign key from PEOPLE table.

Other Fields in AUTHORITIES **authority** String value for the role assigned to users, as in ROLE_ADMIN for administrator and ROLE_USER for users.

26.12 ACL_OBJECT_IDENTITY

Abstract This table is reserved for future use.

Table

ACL_OBJECT_IDENTITY
id
object_identity parent_object acl_class

Serial Key **id** Serial key.

Foreign Keys / Link Description None.

Other Fields in ACL_OBJECT_IDENTITY

object_identity

parent_object

acl_class

26.13 ACL_PERMISSION

Abstract This table is reserved for future use.

Table

ACL_PERMISSION
id
acl_object_identity recipient mask

Serial Key **id** Serial key.

Foreign Keys / Link Description None.

Other Fields in ACL_PERMISSION

acl_object_identity

recipient

mask

27.0 Central Schema — Quartz Tables

27.1 QRTZ_JOB_DETAILS

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_JOB_DETAILS

job_name
job_group
description
job_class_name
is_durable
is_volatile
is_stateful
requests_recovery
job_data

Serial Key None.

**Foreign Keys / Link
Description** None.

Other Fields in QRTZ_JOB_DETAILS

job_name

job_group

description

job_class_name

is_durable

is_volatile

is_stateful

requests_recovery

job_data

27.2 QRTZ_JOB_LISTENERS

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_JOB_LISTENERS
job_name job_group job_listener

Serial Key None.

**Foreign Keys / Link
Description** None.

Other Fields in QRTZ_JOB_LISTENERS

job_name

job_group

job_listener

27.3 QRTZ_TRIGGERS

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_TRIGGERS
trigger_name trigger_group job_name job_group is_volatile description next_fire_time prev_fire_time priority trigger_state trigger_type start_time end_time calendar_name misfire_instr job_data byte

Serial Key None.

**Foreign Keys / Link
Description** None.

**Other Fields in
QRTZ_TRIGGERS**

trigger_name
trigger_group
job_name
job_group
is_volatile
description
next_fire_time
prev_fire_time
priority
trigger_state
trigger_type
start_time
end_time
calendar_name
misfire_instr
job_data

27.4 QRTZ_SIMPLE_TRIGGERS

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_SIMPLE_TRIGGERS
trigger_name trigger_group repeat_count repeat_interval times_triggered

Serial Key None.

Foreign Keys / Link Description None.

Other Fields in QRTZ_SIMPLE_TRIGGERS

trigger_name

trigger_group

repeat_count

repeat_interval

times_triggered

27.5 QRTZ_CRON_TRIGGERS

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_CRON_TRIGGERS
trigger_name trigger_group cron_expression time_zone_id

Serial Key None.

Foreign Keys / Link Description None.

Other Fields in QRTZ_CRON_TRIGGERS

trigger_name

trigger_group

cron_expression

time_zone_id

27.6 QRTZ_BLOB_TRIGGERS

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_BLOB_TRIGGERS

trigger_name trigger_group blob_data
--

Serial Key None.

**Foreign Keys / Link
Description** None.

Other Fields in QRTZ_BLOB_TRIGGERS

trigger_name

trigger_group

blob_data

27.7 QRTZ_TRIGGER_LISTENERS

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_TRIGGER_LISTENERS
trigger_name trigger_group trigger_listener

Serial Key None.

**Foreign Keys / Link
Description** None.

Other Fields in QRTZ_TRIGGER_LISTENERS

trigger_name

trigger_group

trigger_listener

27.8 QRTZ_CALENDARS

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_CALENDARS
calendar_name calendar

Serial Key None.

**Foreign Keys / Link
Description** None.

Other Fields in QRTZ_CALEDNARS

calendar_name

calendar

27.9 QRTZ_PAUSED_TRIGGER_GRP

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_PAUSED_TRIGGER_GRP

trigger_group

Serial Key None.

**Foreign Keys / Link
Description** None.

Other Fields in QRTZ_PAUSED_TRIGGER_GRP

trigger_groups

27.10 QRTZ_FIRED_TRIGGERS

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_FIRED_TRIGGERS

entry_id
trigger_name
trigger_group
is_volatile
instance_name
fired_time
priority
state
job_name
job_group
is_stateful
requests_recovery

Serial Key None.

**Foreign Keys / Link
Description** None.

Other Fields in QRTZ_FIRED_TRIGGERS

entry_id
trigger_name
trigger_group
is_volatile
instance_name
fired_time
priority
state
job_name
job_group
is_stateful
requests_recovery

27.11 QRTZ_SCHEDULER_STATE

Abstract This table is Quartz framework special table, for internal use only.

Table

QRTZ_SCHEDULER_STATE

instance_name
last_checkin_time
checkin_interval

Serial Key None.

**Foreign Keys / Link
Description** None.

**Other Fields in
QRTZ_SCHEDULER_STATE****instance_name****last_checkin_time****checkin_interval****27.12 QRTZ_LOCKS****Abstract** This table is Quartz framework special table, for internal use only.**Table****Serial Key** None.**Foreign Keys / Link
Description** None.**Other Fields in
QRTZ_LOCKS** **lock_name**

28.0 Central Schema — Load Balancing Tables

The Load Balancer system collects performance information from *dataPOWER* database servers and recommends the least loaded server for connection.

28.1 GENERAL_CONFIG

Abstract This table stores general configuration information and status of the system.

Table

GENERAL_CONFIG
status sleep_time node_perf_coll_interval machine_perf_coll_interval win_perf_wait_time win_perf_measure_interval

Serial Key None.

Foreign Keys / Link Description None.

Other Fields in GENERAL_CONFIG

status System status flag:

Value	Description
1	up
2	debugging
4	disable stat tables cleaning

Please note that all bits can be up at the same time.

sleep_time Pause between cyclic action.

node_perf_coll_interval
Reserved for future use.

machine_perf_coll_interval
Reserved for future use.

win_perf_wait_time Reserved for future use.

win_perf_measure_interval
The length of time during which the window's performance parameters are measured.

28.2 MACHINES

Abstract This table represents machines.

Table

MACHINES	
machine_key	
machine_name	1 (machine_key) n
machine_dsc	MACHINE_STAT
os_info	1 (machine_key) n
architecture	MACHINE_STAT_ARCHIVE
cpu_info	
mem_info	1 (machine_key) n
fs_info	NODES
machine_status	
lock_time_ends_seconds	
agent_time_updated	
time_updated	

Serial Key **machine_key** A unique key for the machine.

Foreign Keys / Link Description None.

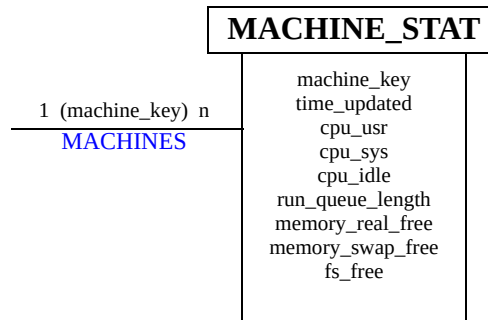
Other Fields in MACHINE

machine_name	Name of the machine.
machine_dsc	Description of the machine.
os_info	Operating System information.
architecture	Architecture information.
cpu_info	CPU Information.
mem_info	Memory information about.
fs_info	Information System information.
machine_status	Machine status. Reserved for future use.
lock_time_ends_seconds	Reserved for future use.
agent_time_updated	Reserved for future use.
time_updated	Time of last machine record update.

28.3 MACHINE_STAT

Abstract This table is intended for storing machine performance information. The information is written to the table by the machine agent every *n* seconds (according to configuration).

Table



Serial Key None.

Foreign Keys / Link Description

machine_key A foreign key from MACHINE.

Other Fields in MACHINE_STAT

time_updated Time of last machine record update.

cpu_usr Percentage of time the machine spent in the **usr** mode.

cpu_sys Percentage of time the machine spent in the **sys** mode.

cpu_idle Percentage of time the machine was idle.

run_queue_length Length of the run queue in

memory_real_free Amount of free memory in kb.

memory_swap_free Amount of free swap memory in kb.

fs_free Amount of free memory (in kb) in the root file system.

28.4 MACHINE_STAT_ARCHIVE

Abstract This table serves as an archive for the [MACHINE_STAT](#) table.

Table

MACHINE_STAT_ARCHIVE	
1 (machine_key) n MACHINES	machine_key time_updated stat_type time_range cpu_usr cpu_sys cpu_idle run_queue_length memory_real_free memory_swap_free fs_free

Serial Key None.

Foreign Keys / Link Description

machine_key A foreign key from MACHINE.

Other Fields in MACHINE_STAT

time_updated The date/time of the archive record.

stat_type Type of statistic:

Value	Description
1	min
2	max
3	avg
4	count
5	sum
6	stddev

time_range

Value	Description
1	per hour
2	per day

cpu_usr Aggregation of **cpu_usr** from [MACHINE_STAT](#) table.

cpu_sys Aggregation of **cpu_sys** from [MACHINE_STAT](#) table.

cpu_idle Aggregation of **cpu_idle** from [MACHINE_STAT](#) table.

run_queue_length Aggregation of **run_queue_length** from [MACHINE_STAT](#) table.

memory_real_free	Aggregation of memory_real_free from MACHINE_STAT table.
memory_swap_free	Aggregation of memory_swap_free from MACHINE_STAT table.
fs_free	Aggregation of fs_free from MACHINE_STAT table.

28.5 NODES

Abstract This table stores node information. Each node typically represents a *dataPOWER* server.

Table

NODES		
node_key		
1 (machine_key) n MACHINES	machine_key server_key node_name machine_dsc node_status node_type node_subtype node_port node_version dp_db max_mem current_mem shutdown_command startup_command test_command process_id jmx_port param_name_1 ...param_name_10 param_val_1 ...param_val_10 time_updated last_picked	1 (node_key) n NODE_STAT
1 (server_key) n SERVERS		1 (node_key) n NODE_STAT_ARCHIVE
		1 (node_key) n SCHEDULING_DETAILS

Serial Key	node_key	A unique key for each node.
Foreign Keys / Link Description	machine_key	Foreign key from MACHINES table.
	server_key	Foreign key from SERVERS table.
Other Fields in NODES	node_name	Node name.
	machine_dsc	Nose description.
	node_status	Node status (0 - normal).

node_type Note type:

Value	Description
0	normal
1	test
2	DPLM
3	STAT_CLEANING

node_subtype 0 — reserved for future use.

node_port dataPOWER server port.

node_version Internal version of the dataPOWER server.

dp_db String value of the database the dataPOWER server is connecting to.

max_mem 0 — reserved for future use.

current_mem 0 — reserved for future use.

shutdown_command Reserved for future use.

startup_command Reserved for future use.

test_command Reserved for future use.

process_id Reserved for future use.

jmx_port jmx port number, using the load balancer that collects dataPOWER server's performance information.

param_name_1 ...param_name_10
Custom property names.

param_val_1 ...param_val_10
Custom property values.

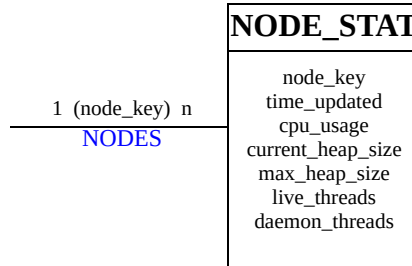
time_updated Date/time when the node record was last updated.

last_picked Date/time when the node was last picked by the load balancer.

28.6 NODE_STAT

Abstract This table stores performance information on the nodes.

Table



Serial Key None.

Foreign Keys / Link Description **node_key** A foreign key from the NODES table.

Other Fields in NODE_STAT

time_updated Date/time of record insertion.

cpu_usage Percentage of the cpu usage of the node.

current_heap_size Current heap size.

max_heap_size Maximum heap size.

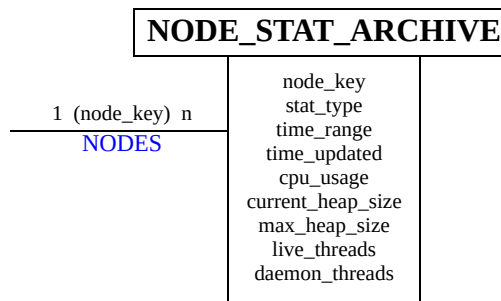
live_threads Number of live threads.

daemon_threads Number of daemon threads.

28.7 NODE_STAT_ARCHIVE

Abstract This table is an archive of the [NODE_STAT](#) table.

Table



Serial Key None.

Foreign Keys / Link Description

node_key A foreign key from the NODES table.

Other Fields in NODE_STAT_ARCHIVE

stat_type Type of statistic:

Value	Description
1	min
2	max
3	avg
4	count
5	sum
6	stddev

time_range

Value	Description
1	per hour
2	per day

time_updated The date/time of the archive record.

cpu_usage The aggregation of **cpu_usage** from the NODE_STAT table.

current_heap_size The aggregation of **current_heap_size** from the NODE_STAT table.

max_heap_size The aggregation of **max_heap_size** from the NODE_STAT table.

live_threads The aggregation of **live_threads** from the NODE_STAT table.

daemon_threads The aggregation of **daemon_threads** from the NODE_STAT table.

28.8 SERVERS

Abstract This table stores information about logical *dataPOWER* servers.

Table

SERVERS	
server_key	1 (server_key) n
server_name server_dsc server_version time_updated	MAP_SRV2DB 1 (server_key) n NODES

Serial Key	server_key	A unique key for each server.
Foreign Keys / Link Description	None.	
Other Fields in SERVERS	server_name	Server name.
	server_dsc	Server description.
	server_version	Server version.
	time_updated	Date/time of record insertion.

28.9 DB_ENGINES

Abstract This table stores information about database connections.

Table

DB_ENGINES	
db_key	1 (db_key) n
db_name db_host db_port db_server db_user_name db_password db_vendor db_schema db_time_updated	MAP_SRV2DB

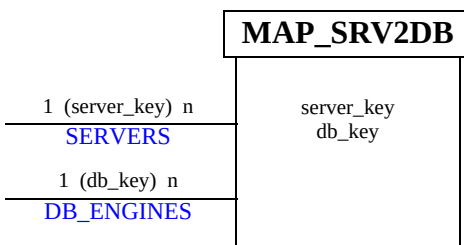
Serial Key	db_key	A unique key for each database connection.
Foreign Keys / Link Description	None.	

**Other Fields in
DB_ENGINES**

db_name	Name of the database which the record points to.
db_host	Host name.
db_port	Database connection port name.
db_server	Database server name.
db_user_name	Database account user name.
db_password	Database account password.
db_vendor	Database platform vendor (Informix or Oracle).
db_schema	Database schema name.
db_time_updated	Date/time of record update.

28.10 MAP_SRV2DB

Abstract This table contains mapping information between servers and databases.

Table

Serial Key None.

**Foreign Keys / Link
Description**

server_key	A foreign key from the SERVERS table.
db_key	A foreign key from the DB_ENGINES table.

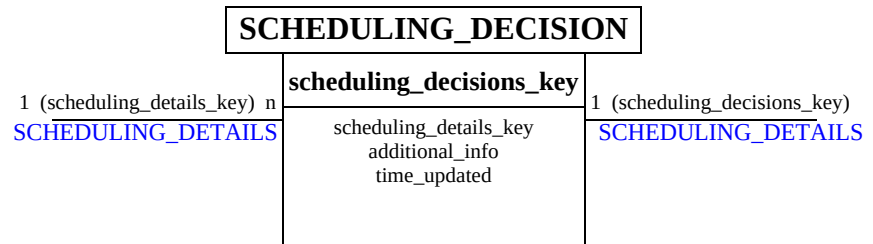
**Other Fields in
MAP_SRV2DB**

None

28.11 SCHEDULING_DECISION

Abstract This table stores information about load balancer scheduling decisions. It references one record in the SCHEDULING_DETAILS table. Each record in this table represents one decision act by the load balancer.

Table



Serial Key **scheduling_decisions_key**

A key, which uniquely identifies scheduling decision.

Foreign Keys / Link Description **scheduling_details_key**

A foreign key from the SCHEDULING_DETAILS table.

Other Fields in SCHEDULING_DECISION

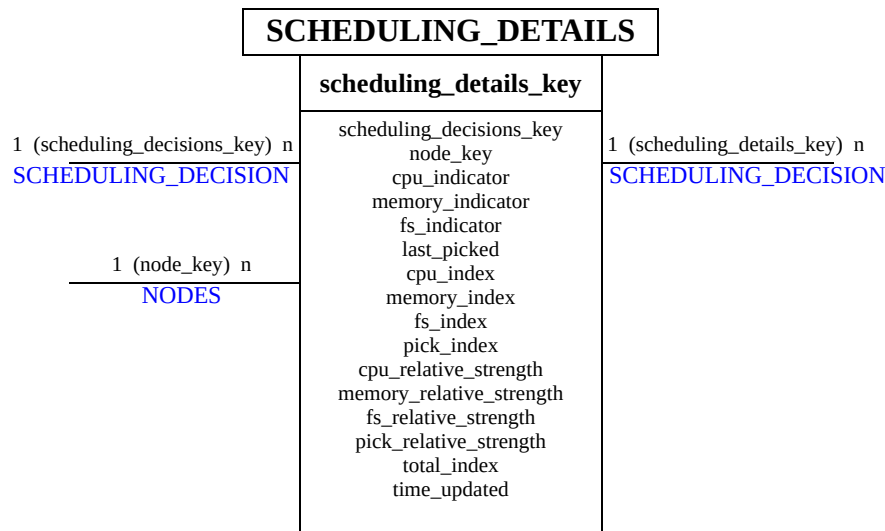
additional_info Currently null.

time_updated Date/time of record insertion.

28.12 SCHEDULING_DETAILS

Abstract This table contains information about scheduling details. For a decision act (one record in the SCHEDULING_DECISION table), there are one or more records in this table, representing alternatives, which the load balancer analyses.

Table



Serial Key **scheduling_details_key**

A unique key uniquely identifying a scheduling decision alternative.

Foreign Keys / Link Description **scheduling_decisions_key**

A foreign key from the SCHEDULING_DECISION table.

node_key A foreign key from the NODES table.

Other Fields in SCHEDULING_DETAILS

cpu_indicator CPU strength.

memory_indicator Memory strength.

fs_indicator Value is **1**, if there is enough space on root file system, otherwise **0.001**.

last_picked The last time the node was picked by the load balancer.

cpu_index Index order of the CPU.

memory_index Index order of the memory.

fs_index	Index order of the file system.
pick_index	Index order of the last node picked by the load balancer.
cpu_relative_strength	Relative strength of CPU.
memory_relative_strength	Relative strength of memory.
fs_relative_strength	Relative strength of file system.
pick_relative_strength	Relative strength of picked (selected node)
total_index	The combination of CPU, memory, file system and pick indexes.
time_updated	Date/time of record insertion.

29.0 Glossary

batch A batch (or run) is in most cases identical to a **lot**. However, a batch is subjected to common treatment in a single process step. This becomes important in a fab environment, where some equipment can process multiple lots (or less than one lot) at once.

For example, some furnaces exist which can process up to 150 wafers. With a normal lot size of 25 wafers, this corresponds to six lots. Because wafers are kept in their respective pods, traceability remains guaranteed despite lot merging. In the database, the data from this furnace would result in six entries in **OP_LOG** with a common **run_id**.

As another example, some implanters can only handle 17 wafers at once (less than a full lot of 25 wafers). However, they work such that they expect a flow of lots at 25 wafers each at the input and deliver them in the same order at the output. dataPOWER does not handle batch composition at a sub-lot level as would be needed in this example.

bin A tester or wafer sorter sorts devices into multiple bins.

Pass Bin devices pass all tests. For grading purposes, multiple pass bins may exist, associated to different test limits.

Fail Bin devices do not fall into any pass bin, thus fails some test(s).

Hard Bin information offered from the tester to the prober or handler.

Operator Bin as indicated to the operator on a bin box or through some other visual indication.

Soft Bin set by the test program (and typically reported on the **tester summary**). This is often also referred to as test counter information.

Each test typically increments a separate counter when failing. If the same test is executed on multiple device pins (such as output drive current measurements) typically the same counter is used.

Test Numbers on some test system have an even finer gradation than soft bins and refer to individual measurements.

DBA Database administrator, whose task is split into three major parts:

- guaranteeing the consistency of the data in dataPOWER by adequately configuring the database and the data readers.
- maintaining the size of the database by implementing appropriate backup strategies.
- tuning the performance of the database using standard Informix tools.

datalog A raw data file containing device data log information as produced by a tester.

equipment A piece of hardware on which a particular manufacturing step is executed. Each step requires one or more kinds of equipment (such as a tester plus a prober plus a probecard for a particular product, etc.).

Each kind of equipment may exist in multiple instantiations (such as different identical probe cards for a given product, etc.)

Equipment is dealt with in [EQUIPMENT](#) (instantiations) and [EQUIP_TYPE](#).

Esort Wafer testing, also called pretest.

event An action which affects equipment used for a manufacturing process or manufactured goods (wafers, lots, batches).

Events are logged in [EVENT_LOG](#).

Fab Wafer fabrication (as opposed to test centers).

lot A lot is a collection of manufacturing goods which are subjected to a sequence of identical processing. Lots are traced in the **LOT** table. (See [batch](#) for a definition of the difference between a lot and a batch.)

lot splitting Occurs when a lot is split into two or more parts, each of which is subsequently subjected to different processing steps or to the same steps at a different time. Lot splitting typically occurs for final test: e.g. a lot consisting of 24 or 50 wafers is split and tested into smaller, more easily tested lots.

lot merging Occurs when two or more previously unrelated lots (or lots which have been generated from a single lot by lot splitting) are merged and subjected to further common treatment.

lot renaming Occurs if the same lot is renamed as it proceeds through the manufacturing process.

PCM Process Control Module - an area on the wafer with structures allowing the measurement of performance of fab processes.

Formerly, PCMs were in dedicated die locations on the wafer. However in modern processes, PCMs are also integrated in the scribe lines — thus there are many more PCMs than are actually tested in normal production. Also, the relation to a physical location becomes slightly less obvious.

process A fabrication process for an IC. A given process technology consists of multiple technical stages which may be identical for different technologies. A technical stage consists of different flow stages (such as cleaning, implanting, etc.) which are subdivided into process steps, maintained in [PROC_STEP](#).

[Esort](#) and final test are special process steps, independent of process technology.

product A product (in the context of *dataPOWER*) is a die type. It does not distinguish between different packaging variants or speed grades (as derived from final testing).

Products are maintained in **PRODUCT**. Package variants are controlled through **PKG**.

program A unique output file format on a per-product basis, which always contains an identical set of information.

The file format per program is unique in that:

- it adheres to semantic rules unique to the equipment on which the **raw data** file is produced.
- it contains a given defined set of parameters which is measured on the product.
- it uses a predefined set of limits for the measured parameters.

A program is closely related to a test program (executed on a tester) or a **recipe** (executed on fab equipment). Programs are maintained in **PROGRAM**.

raw data Data as produced by the equipment (or after a pre-processing step not visible to *dataPOWER*).

reader Regularly executed software which inserts raw data into the database.

recipe A recipe is a **program** executed on fab equipment. It corresponds to a test program executed on a tester.

reticle In its narrowest definition, a reticle refers to a diffusion mask, which typically specifies the size of multiple devices (on a wafer). This definition applies, for example, when doing reticle based zonal analysis on a wafer map.

Often however, the term is more loosely used to refer to a diffusion mask type, and thus a reticle set to a die type or **product**. Reticles and reticle sets (as in masks and mask sets) are maintained in **RETICLE** and **RETICLE_SET**.

serial key A unique key automatically generated by the Informix database engine. For tables that use them, the serial key is a unique identifier for the row and is generated for each row inserted in the table.

test **Esort** and final test (F/T) operation, executed on wafers or on packaged devices.

test stage Broad test category, such as **Esort** and Final Test (F/T). In some cases, a test stage also indicates the test temperature (such as WR for “wafer room”).

tester summary Summary produced by the test system. It contains counter and/or binning information.

traceability Forward traceability ensures that when a problem is discovered at an early processing step, all products affected by the problem can be isolated in the manufacturing chain or potentially at the receiving customer.

Backward traceability ensures that for bad products, all previous manufacturing steps can be clearly identified and thus analyzed. Backward traceability becomes more difficult with [lot merging](#).

30.0 Index

Symbols

..._key [441](#)
 ..._cnt [441](#)
 ..._val [441](#)
 \${col_string} [462](#), [463](#)

A

a_key [123](#)
 a_lg_key [396](#), [398](#), [415](#), [416](#)
 aatt_datatype [360](#)
 aatt_key [360](#)
 aatt_name [360](#)
 aatt_type [360](#)
 aatt_unit [360](#)
 accept_data [52](#)
 accept_percent [148](#)
 access_level [193](#)
 acl_class [493](#)
 ACL_OBJECT_IDENTITY [493](#)
 acl_object_identity [493](#)
 ACL_PERMISSION [493](#)
 acp_nv [390](#)
 active_flag [430](#), [480](#)
 adder_missing [315](#)
 additional_info [513](#)
 address1-3 [124](#), [125](#), [194](#)
 adl_flag [394](#)
 adl_key [405](#), [406](#)
 agent_time_updated [504](#)
 agg_level [380](#)
 aggregate [380](#), [412](#), [427](#)
 aging_days [490](#)
 ai_key [352](#), [356](#), [358](#), [360](#), [393](#)
 ai_key_i [353](#)
 ai_key_o [353](#)
 ai_name [353](#)
 all_cluster_flag [380](#)
 am_desc [355](#)
 am_flag [355](#)
 am_hier [355](#)
 am_purpose [355](#)
 am_title [355](#)

amap_key [354](#), [358](#), [414](#)
 amap_name [354](#)
 amapc_key [354](#), [414](#)
 amapc_name [354](#)
 amod_key [339](#), [355](#), [361](#), [401](#), [432](#)
 amod_name [355](#)
 analysis_type [315](#)
 anls_flag [355](#), [361](#), [363](#)
 annot_key [491](#)
 annot_type [491](#)
 app_command [486](#)
 app_desc [486](#)
 app_ins_num [486](#)
 app_key [485](#)
 app_name [485](#)
 app_path_key [485](#)
 app_server_key [486](#)
 app_server_name [486](#)
 app_version [485](#)
 apply_cluster_flag [380](#)
 apps_engr [93](#)
 architecture [504](#)
 archive_level [200](#), [201](#)
 ARCHIVE_LOG [200](#)
 ARCHIVE_PLOG [201](#)
 archive_time [200](#), [201](#)
 area_flag [148](#)
 arg1 [86](#)
 arg1...arg20 [211](#)
 arg2 [86](#)
 arg3 [86](#)
 arg4 [86](#)
 arg5 [86](#)
 array_value [214](#)
 ate_ver [228](#)
 att_type [371](#)
 attr_desc [489](#)
 attr_key [488](#)
 attr_name [488](#)
 attr_type [489](#)
 attr_value [488](#)
 auf [80](#)
 AUTHORITIES [492](#)
 authority [492](#)

avg [247](#)
 avg1 ... avg5 [246](#)

B

backward traceability [520](#)
 bad_die_pct [374](#)
 bad_yield [54](#), [130](#)
 base_flag [439](#)
 baseline [401](#)
 Batch [517](#)
 Bin [517](#)
 BIN_COLORS [436](#)
 bin_flag [88](#)
 BIN_GAL [436](#)
 bin_img [437](#)
 bin_index [54](#), [82](#), [83](#), [84](#), [181](#), [477](#)
 BIN_LOG [33](#), [79](#)
 bin_name [82](#), [83](#), [84](#), [459](#)
 bin_num [82](#), [83](#), [84](#), [149](#), [150](#), [151](#), [152](#), [436](#), [459](#), [460](#)
 bin_order [82](#), [83](#), [84](#), [459](#)
 bin_pg_cnt [460](#)
 bins [51](#)
 bitg_flag [263](#), [266](#)
 blob_data [498](#)
 block_cnt [288](#)
 block_cols [289](#), [291](#)
 block_name [289](#), [291](#)
 block_num [289](#)
 block_rows [289](#), [291](#)
 block_type [289](#), [291](#)
 blue [436](#)
 BM_ARGS [211](#)
 bmargs_key [211](#)
 bop_key [316](#), [318](#)
 bop_test_key [315](#), [318](#)
 border_width [266](#)
 branch_replace [453](#)
 bring_back [200](#), [201](#)
 burst [245](#)
 burst_mode [230](#)
 bw_flag [263](#)

C

c_chisq [152](#)

- [c_chisqc 152](#)
[c_df 152](#)
[c_status 152](#)
[c1... c16 150, 151](#)
[c1-c20 66](#)
[ca 389](#)
[ca_key 388, 389](#)
[calendar 500](#)
[calendar_name 496, 500](#)
[cat_type_flag 453](#)
[catt_datatype 346](#)
[catt_key 345, 346, 370, 386](#)
[catt_key1 361](#)
[catt_key2 361](#)
[catt_key3 361](#)
[catt_key4 361](#)
[catt_name 346](#)
[catt_type 346](#)
[catt_unit 346](#)
[cell_key 337, 339, 340, 341, 346, 349, 352, 362, 364, 368, 383, 384, 388, 390, 393, 396, 398, 400, 416](#)
[cell_name 338](#)
[cell_type 338](#)
[celli_block_name 342](#)
[celli_block_num 342](#)
[celli_key 339, 343, 348, 349, 353, 359, 393, 402, 419](#)
[celli_name 340](#)
[celli_zone_name 345](#)
[celli_zone_num 345](#)
[center_x 97, 99](#)
[center_x_estim 99](#)
[center_y 97, 99](#)
[center_y_estim 99](#)
[central_flag 196](#)
[central_schema 196](#)
[cf_flag 263](#)
[cfg1-2_s 466](#)
[cfg1-6_r 465](#)
[chain 245](#)
[channels 243](#)
[char_cmt 228](#)
[char_key 227, 233, 242, 244, 246, 247, 248, 249, 251, 252, 254](#)
[CHAR_LOG 226](#)
[char_title 227](#)
[Characterization Tables 225](#)
[chart_type 158](#)
[checkin_interval 502](#)
[chi-square 153](#)
[chi-square with confidence 153](#)
[ciatt_datatype 348](#)
[ciatt_key 348](#)
[ciatt_name 348](#)
[ciatt_type 348](#)
[ciatt_unit 348](#)
[ciblknm_key 342, 343](#)
[cift_flag 394](#)
[circles 147](#)
[city 124, 125, 194](#)
[ciznm_key 342, 345](#)
[cizns_key 342, 343, 344, 345](#)
[cizns_name 344](#)
[cl_merge 162](#)
[cl_min 162](#)
[cl_th 162](#)
[class_comment 128](#)
[class_key 450](#)
[class_level 128](#)
[class_name 451](#)
[class_option 448](#)
[cls_comment 117, 126](#)
[cls_key 128, 175, 325, 326, 327, 328, 433, 434, 477](#)
[CLS_METHOD 219](#)
[cluster 165](#)
[Cluster Factor
 calculation 80](#)
[cluster_alg_type 380](#)
[cluster_flag 374](#)
[cmc_key 268](#)
[cmmc_key 267](#)
[cnt 246, 248, 447](#)
[cnt_flag 89](#)
[code 93](#)
[coef_label 387](#)
[coef_order 375](#)
[coef_value 375](#)
[col_chisq 153](#)
[col_chisqc 153](#)
[col_df 153](#)
[col_flag 266](#)
[col_name 376](#)
[col_num 278, 287, 307, 308](#)
[col_offset 98](#)
[col_status 153](#)
[col_thickness 266](#)
[col_type 376, 465](#)
[col1... col24 150, 151](#)
[collection_num 267, 268](#)
[cols 147](#)
[comment 131](#)
[comments 474](#)
[comp_col_order 490](#)
[comp_flag 262, 265](#)
[comp_key 490](#)
[comp_options 490](#)
[comp_row_order 490](#)
[comp_type 371, 490](#)
[compare 245](#)
[compare_blocks 230](#)
[cond_index 85](#)
[cond_length 85](#)
[cond_name 85](#)
[cond_num 238](#)
[cond_type 85](#)
[COND_VALUE 238](#)
[cond0 69](#)
[CONDITION 85](#)
[conditions 52](#)
[condn 69](#)
[config_key 329, 335, 336, 338, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 352, 354, 355, 357, 359, 360, 361, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 377, 380, 381, 382, 383, 386, 388, 389, 390, 392, 399, 403, 404, 405, 407, 408, 409, 410, 411, 412, 413, 414, 415, 418, 419, 421, 422, 423, 424, 425, 426, 427, 429, 430, 431, 449, 451](#)
[config_name 335, 451](#)
[Configurable Statistics 87](#)

- [cont_sig_lvl 173](#)
[contact 124, 125](#)
[Contents 3](#)
[count 248](#)
[counter 517](#)
[country 124, 125, 194](#)
[cpar_end 235](#)
[cpar_key 227, 234, 238](#)
[cpar_name 234](#)
[cpar_orig_name 234](#)
[cpar_start 234](#)
[cpar_step 235](#)
[cpar_unit 234](#)
[cpk 444, 445](#)
[cpu_idle 505, 506](#)
[cpu_index 514](#)
[cpu_indicator 514](#)
[cpu_info 504](#)
[cpu_relative_strength 515](#)
[cpu_sys 505, 506](#)
[cpu_usage 509, 510](#)
[cpu_usr 505, 506](#)
[create_method 344](#)
[create_time 224, 430](#)
[crit 446](#)
[crit_bad_die_pct 380](#)
[critical 168](#)
[critical_flag 234](#)
[CRITICAL_PARAM 234](#)
[cron_expression 498](#)
[CT_CLASS 450](#)
[CT_CONFIG 451](#)
[ct_flag 263](#)
[CT_JOB 448](#)
[CT_MODIFY 453](#)
[CT_RULE_SET 450](#)
[CT_TREE 449](#)
[ctol_maj 180](#)
[ctol_min 180](#)
[ctype_key 338](#)
[current_heap_size 509, 510](#)
[current_mem 508](#)
[cust_key 61, 125, 127](#)
[cust_name 125](#)
[CUSTOMER 125](#)
[CV Tables 329](#)
[CV_AATTR 360](#)
[CV_AATTR_V 360](#)
[CV_ADL 405](#)
[CV_ADL_DEF 406](#)
[CV_AI 352](#)
[CV_AI_MAP 354](#)
[CV_AI_MAPC 354](#)
[CV_AI_MOD 355](#)
[CV_AI2AI 353](#)
[CV_BIN 391](#)
[CV_CA 388](#)
[CV_CA_NV 390](#)
[CV_CA_VAL 389](#)
[CV_CATEGORY 423](#)
[CV_CATTR 345](#)
[CV_CATTR_V 346](#)
[CV_CELL 337](#)
[CV_CELL_I 339](#)
[CV_CELL_TYPE 338](#)
[CV_CELL2AMOD 338](#)
[CV_CELLI_BLOCK_DEF 343](#)
[CV_CELLI_BLOCK_NUM 342](#)
[CV_CELLI_ZONE_NUM 345](#)
[CV_CELLI_ZONES 344](#)
[CV_CELLI2IDEF 341](#)
[CV_CIATTR 347](#)
[CV_CIATTR_V 348](#)
[CV_COEF_LABEL 386](#)
[CV_CONFIG 330](#)
[CV_DEF2TYPE_ 395](#)
[CV_DEFECT 393](#)
[cv_df_key 393](#)
[CV_DIE_RESULTS 409](#)
[CV_DOE 361](#)
[CV_DOE_VAL 362](#)
[CV_DOE2CELL 362](#)
[CV_E_IDEF2IDEF 415](#)
[CV_E2I_INHERIT 416](#)
[CV_EDEF2IDEF 396](#)
[CV_EDEF2IDEF_ALL 397](#)
[cv_exp_key 382](#)
[CV_EXPORT 382](#)
[CV_FAIL_LAYER 366](#)
[CV_FAIL_MODE 365](#)
[CV_FILTER_UFORM 425](#)
[CV_FINE_FTYPE 366](#)
[CV_FLTR 403](#)
[CV_FLTR_INFO 404](#)
[CV_FM_OUT_DATA_ 384](#)
[CV_FMD_TYPE 368](#)
[CV_FMD2LSPLT 371](#)
[CV_FMD2MDL 369](#)
[CV_FMD2PAR 370](#)
[CV_FORM 411](#)
[CV_FORM2LSPLT 412](#)
[CV_FORM2PARAM 411](#)
[CV_FRM_CLUST_DET 374](#)
[CV_FRM_COV 375](#)
[CV_FRM_DEF 376](#)
[CV_FRM_DIAG_ATTR 376](#)
[CV_FRM_DIAG_V 377](#)
[CV_FRM_MOD 373](#)
[CV_FRM_OUT_SUM 378](#)
[CV_FRM_PARAM 372](#)
[CV_FRM_PARM 372](#)
[CV_FRM_RUN 379](#)
[CV_FRM_RUN_DET 381](#)
[CV_FRM_USC_ 385](#)
[CV_FRN 367](#)
[cv_frs_def_key 430, 431, 432](#)
[CV_FRS_INP_SPLITS 432](#)
[CV_FRS_RUN 431](#)
[CV_FRS_SETUP 430](#)
[CV_FUNC2FORM 414](#)
[CV_FUNC2INP 415](#)
[CV_FUNCTION 413](#)
[CV_GATTR 410](#)
[CV_GATTR_V 410](#)
[CV_IDEF2IDEF 398](#)
[CV_INSP_SET 418](#)
[CV_INSP_V 419](#)
[CV_LIM2AI 356](#)
[CV_LIM2CELL 352](#)
[CV_LIMITS 350](#)
[CV_LIMITS_V 351](#)
[CV_LOG 417](#)
[CV_LOT2CFG 336](#)
[CV_MAP2IN 358](#)
[CV_MAP2OUT 358](#)
[CV_MATTR 346](#)
[CV_MATTR_V 347](#)
[CV_MEAS 349](#)
[CV_MEAS2PADGRP 422](#)

- CV_MEAS2PIN 350
 CV_MIC_DEF 420
 CV_MINOR_CONFIG 335
 CV_MODEL 363
 CV_MODEL_DEF 364
 CV_MODULE2CATTR 386
 CV_MODULE2CELL 383
 CV_P_VAL 389
 CV_PAD_GROUP 421
 CV_PAD_GRP_SUM 421
 CV_PAR2MEAS 413
 CV_PIN_RESULTS 407
 CV_PIN_SUM 408
 CV_PIN2PADGRP 423
 CV_PR_MOD 357
 CV_PRIO 401
 CV_SO_SET 418
 CV_STEP_ORDER 391
 CV_SUM_ZONE 400
 CV_TEST_PLAN 329
 CV_TEST_PROG 329
 CV_UCONSTANT 428, 429
 CV_UFORM 424
 CV_UFORM_DEF 424
 CV_UFORM2FILTER 426
 CV_UNQCI 359
 CV_UOPERATION 429
 CV_UPARAM 426
 CV_UPARAM_DEF 428
 CV_UPSTAT 399
 CV_USC_DEF 387
 CV_WF_CELLI 402
 CV_WF2MICFG 402
 cvb_key 391, 393
 cvcat_key 408, 422, 423
 cvcat_name 423
 cvlbl_key 378, 386
 cvlbl_key1 375
 cvlbl_key2 375
 cycle 242
 cycle_cnt 237
D
 d0 80
 d0_pgdw 80
 daemon_threads 509, 510
 Data Model 44
 data_desc 57, 228
 data_key 216
 DATA_SET 208
 DATA_TYPE 231
 data_type 57, 361
 DATABASES 220
 datalog 517
 datatype_key 227, 231
 date 196
 date_range 453
 datt_datatype 377
 datt_key 376, 377
 datt_name 377
 datt_type 377
 datt_unit 377
 DB_ENGINES 511
 db_host 512
 db_key 205, 220, 224, 511, 512
 db_name 220, 512
 db_password 512
 db_port 512
 db_schema 512
 db_server 220, 512
 db_time_updated 512
 db_user_name 512
 db_vendor 512
 DBA 517
 DBINFO 196
 DBTOOLS_MERGE_FAM 186
 DBTOOLS_MERGE_PROC 186
 DBTOOLS_MERGE_PROD 187
 DBTOOLS_MERGE_TECH 187
 dd 163
 def_cnt 163, 169, 179, 183, 406
 def_die 164
 def_die_cnt 173
 def_flag 71, 142, 156, 217, 271, 336, 344, 351, 390, 402, 420, 435, 477, 486
 def_img 171
 DEF... 65
 DEF2CLASS 169
 default_flag 484
 default_test 303
 DEFECT 164
 Defect Density
 calculation 80
 defectMAP Tables 159
 dep_key 193, 195
 dep_name 195
 DEPARTMENT 195
 descramb_key 320, 321
 description 156, 196, 218, 271, 336, 344, 418, 419, 477, 494, 496
 designer 93
 dest_fam 186
 dest_proc 186
 dest_prod 187
 dest_tech 187
 destruct 270
 dev_num 300
 Df 154
 df 401
 df_align 397, 398, 399, 416, 417
 DF_ALNIM 170
 df_area 165
 DF_BIN 181
 DF_CLASS 168
 df_clstr 176
 DF_CONFIG 182
 DF_CONTRIB 178
 DF_DIE 169
 DF_EXCLUDE 179
 DF_EXPORT 183
 DF_EXPORT_DEF 184
 DF_GALIMG 161
 DF_IMAGE 171
 DF_IMGSRC 173
 df_index 164, 171, 174, 394
 df_info 165, 394, 417
 df_key 164, 169, 171, 184, 395, 406
 DF_LOCK 181
 DF_LOG 159
 DF_LYRINFO 180
 DF_METHOD 167
 DF_RECIPE 174
 DF_SAMPLE 175

- [df_source 341, 394, 397, 398, 399, 416, 417](#)
[DF_SUM 172](#)
[DF_TEST 162](#)
[DF_TOLERANCE 177](#)
[DF_TRIGGER 184](#)
[df_type 165, 166, 394, 395](#)
[DF_TYPES 166](#)
[DF_UPSTAT 112](#)
[df1 446](#)
[df2 446](#)
[dfc_key 168, 169](#)
[dfc_name 168](#)
[dfcfg_key 181, 182](#)
[dfexp_key 183, 184](#)
[DFL2TEST 163](#)
[dfm_key 167, 168](#)
[dfm_name 167](#)
[die type 519](#)
[die_cnt 155](#)
[die_count 93](#)
[die_ht 97, 99](#)
[die_index 55](#)
[die_key 297, 298, 300, 304](#)
[die_type 100, 169](#)
[die_wd 97, 99](#)
[die_x 100, 103, 169, 228, 462](#)
[die_x_offset 98](#)
[die_y 100, 103, 169, 228, 462](#)
[die_y_offset 98](#)
[diex 121, 157, 164, 297, 298, 300, 304, 321, 322, 341, 359, 384, 394, 397, 398, 408, 409, 422, 455](#)
[diex_index 53](#)
[diey 121, 157, 164, 297, 298, 300, 304, 321, 322, 341, 359, 384, 394, 397, 398, 408, 409, 422, 455](#)
[diey_index 53](#)
[dim1 251, 254](#)
[dim2 251, 254](#)
[dim3 254](#)
[dimension 233](#)
[dir_path 217](#)
[DIR_PATH_DEF 216](#)
[direction 218](#)
[dist_type 68](#)
[distance 120](#)
[do_schedule 315](#)
[doe_hier 361](#)
[doe_index 387](#)
[doe_type 361](#)
[doef_key 361, 362, 432](#)
[doef_name 361](#)
[doef_sname 361](#)
[doef_val 362](#)
[doefv_key 362](#)
[dp_db 508](#)
[DP_FILE 481](#)
[DP_FOLDER 480](#)
[DP_FUNC 203](#)
[DP_LOG 204](#)
[DP_TEMP_RUN 211](#)
[DP_TEMP_RUN_KEYS 212](#)
[DP_TRIGGER_RUNS 214](#)
[DP_TRIGGER_RUNS_ARR
AY 214](#)
[dp_version 481](#)
[DPF_FIELD 483](#)
[DPF_GROUP 215](#)
[DPF_HIER 216](#)
[dpf_key 216, 481, 482](#)
[dpf_order 483](#)
[DPF_SUBGROUP 215](#)
[dpf_w_key 483](#)
[dpf_wr_key 483](#)
[DPF2FIELD 482](#)
[DPFILE2FILE 483](#)
[dpfld_key 482, 483](#)
[dpfldval_key 482, 483](#)
[dplg_key 43, 204](#)
[DPMON_ANNOT 491](#)
[DPMON_APP 485](#)
[DPMON_APP_SERVER 486](#)
[DPMON_DASH_COMP 490](#)
[DPMON_HOTLIST 484](#)
[dpmon_rep_desc 490](#)
[dpmon_rep_key 489, 491](#)
[dpmon_rep_name 489](#)
[DPMON_REPORT 489](#)
[DPMON_SCHEDULE_LOG
492](#)
[DPMON_TASK 487](#)
[DPMON_USER_ATTRIB 488](#)
[DPMON_USER_INFO 488](#)
[dpr_key 211, 212](#)
[dpsh_version 481](#)
[dpWEB Table 158](#)
[drcp_key 174, 175, 177, 178, 180](#)
[drcp_name 174](#)
[DRCP2PDL_YR 178](#)
[dsize 165](#)
[dt_format 206, 207](#)
[dt_key 208, 443, 448, 450](#)
[dt_name 208](#)
[dut_open_yield 409](#)
[dut_short_yield 409](#)
[dynamic 46](#)
E
[ec_key 190, 191](#)
[ec_name 191](#)
[em_key 37, 40, 42, 56, 142, 193, 204, 227, 303, 350, 363, 440, 480, 481, 484, 487, 488, 489, 490, 491, 492](#)
[e-mail 124, 125](#)
[email 193](#)
[end_time 37, 41, 43, 183, 185, 208, 228, 303, 380, 382, 405, 431, 441, 443, 449, 465, 482, 485, 496](#)
[entry_id 501](#)
[eq_key 40, 42, 59, 162, 188, 440](#)
[eq_level 189](#)
[eq_name 189](#)
[eqkey... 36](#)
[eqkey1...6 302](#)
[eqt_key 51, 135, 188, 190](#)
[eqt_name 190](#)
[equip_4 227](#)
[equip_ate 227](#)
[EQUIP_CLASS 191](#)
[equip_interface 227](#)
[equip_temp 227](#)
[EQUIP_TYPE 190](#)
[EQUIPMENT 188](#)
[equipment 518](#)

- err_code [185](#)
 err_msg [111](#), [112](#), [183](#), [185](#),
 [298](#), [318](#), [381](#), [383](#), [399](#),
 [406](#), [432](#), [456](#), [466](#)
 err_msg1 [381](#)
 err_msg2 [381](#)
 error_code [110](#), [112](#), [148](#), [149](#),
 [183](#), [205](#), [298](#), [318](#), [381](#),
 [383](#), [399](#), [406](#), [431](#), [456](#),
 [466](#)
 ERROR_MESSAGE [221](#)
 error_msg [148](#), [149](#)
 Esort [33](#), [518](#)
 esort_dlog [133](#)
 esort_summary [133](#)
 estep_key [341](#), [396](#), [397](#), [416](#)
 ETA_LOG [43](#)
 EV... [441](#)
 ev_key [73](#), [440](#)
 Event [518](#)
 EVENT_CONFIG [442](#)
 event_desc [258](#)
 EVENT_LOG [33](#), [440](#)
 event_name [257](#)
 event_time [258](#), [261](#)
 event_type [257](#)
 every_nLots [210](#)
 exclude [392](#)
 exp_dir [383](#), [430](#)
 exp_key [227](#), [231](#)
 EXPERIMENT_STEP [231](#)
 export_type [183](#), [405](#)
 extension [217](#), [224](#)

F
 f_val [401](#)
 F/T test [33](#)
 FAB [194](#)
 Fab [518](#)
 fab_key [140](#), [194](#), [227](#)
 fab_name [194](#)
 FAB_SKEW [232](#)
 FAB_STEPPING [232](#)
 FAB_TYPE [259](#)
 FAB2PROD [260](#)
 fabstep_desc [232](#)
 fabstep_key [227](#), [232](#)
 fabstep_name [232](#)

 fabt_desc [259](#)
 fabt_key [194](#), [259](#)
 fabt_name [259](#)
 Fail Bin [517](#)
 fail_adr_cnt [173](#)
 fail_bin [67](#)
 fail_bit_count [266](#)
 fail_channel [255](#)
 fail_clean_cnt [173](#)
 fail_cnt [385](#)
 fail_count [275](#), [276](#), [277](#), [278](#),
 [279](#), [281](#), [282](#), [283](#), [284](#),
 [285](#), [286](#), [287](#), [305](#), [306](#),
 [307](#), [308](#), [309](#), [310](#), [312](#),
 [313](#), [315](#)
 fail_die_cnt [172](#)
 fail_key [254](#), [369](#), [370](#), [371](#),
 [372](#), [413](#)
 fail_style [380](#), [401](#)
 fail_type [254](#), [353](#), [364](#), [365](#),
 [367](#), [368](#), [384](#), [388](#), [390](#),
 [394](#), [397](#), [398](#), [400](#), [401](#),
 [417](#)
 FAIL... [254](#)
 FAMILY [93](#), [105](#)
 fax [124](#), [125](#), [194](#)
 fci_count [400](#)
 fci_in_count [400](#)
 fco_flag [90](#)
 fcp_flag [90](#)
 fcs_flag [90](#)
 fcw_flag [90](#)
 feature [203](#)
 ff_key [414](#), [415](#)
 fft_flag [433](#)
 fft_key [353](#), [364](#), [367](#), [368](#),
 [384](#), [388](#), [390](#), [393](#)
 fgrp_desc [215](#)
 fgrp_key [215](#)
 fgrp_name [215](#)
 fhier_key [216](#)
 fhier_level [216](#)
 field_key [412](#), [426](#)
 field_name [483](#)
 field_type [483](#)
 field_value [482](#)
 file_blob [481](#)

 file_count [224](#)
 file_desc [482](#)
 file_descr [319](#)
 file_name [183](#), [219](#), [406](#), [481](#)
 file_type [481](#)
 FILE2MEMCONFIG [221](#)
 fileconf_key [318](#), [319](#)
 filt_cnt [405](#)
 filt_funct_desc [425](#)
 filter_flag [68](#)
 filter_key [490](#)
 filter_type [199](#)
 filtform_key [425](#), [426](#)
 filtform_name [425](#)
 final test [519](#)
 fine_fail_type [367](#)
 fire_time [492](#)
 fired_time [501](#)
 first_col [289](#), [292](#)
 first_fail [251](#), [252](#)
 first_row [289](#), [292](#)
 fitted [384](#)
 fixed [46](#)
 flag [111](#), [113](#), [298](#), [336](#), [382](#),
 [392](#), [399](#), [431](#), [456](#)
 flat [97](#), [99](#)
 flat_type [97](#), [119](#)
 fld_col [100](#)
 fld_cols [97](#), [102](#)
 fld_num [100](#)
 fld_row [100](#)
 fld_rows [97](#), [102](#)
 fld_x [100](#)
 fld_y [100](#)
 flow stage [518](#)
 flow_desc [139](#)
 flow_key [139](#)
 flow_name [139](#)
 flow_num [139](#)
 FLOW_STAGE [139](#)
 fltr_getmisval_flag [381](#)
 fltr_key [380](#), [382](#), [403](#), [404](#),
 [425](#), [432](#)
 fltr_name [403](#)
 fltr_txt [404](#)
 fltr_type [403](#)
 flyr_key [364](#), [366](#), [388](#)

[flyr_name 366](#)
[fm_key 58, 105](#)
[fm_name 105](#)
[fmd_key 364, 365, 369, 376, 378, 383, 386, 387](#)
[fmd_key1 375](#)
[fmd_key2 375](#)
[fmd_type 369](#)
[fmode_name 365](#)
[FMT_FILE 222](#)
[fmt_key 204, 222, 224](#)
[fmt_name 222](#)
[fmt_order 442](#)
[folder_key 480, 481, 484, 489](#)
[folder_name 480](#)
[folder_type 480](#)
[footer 158](#)
[foreign keys 27](#)
[form_cnt 414](#)
[form_id 411](#)
[form_key 411, 414](#)
[form_out_unit 411](#)
[form_type 411](#)
[formula_id 369](#)
[formula_id1 369](#)
[formula_id2 369](#)
[forward traceability 520](#)
[fresult1 378](#)
[fresult2 378](#)
[frgn_key 212](#)
[frm_mod_key 363, 373, 383, 386](#)
[frm_mod_name 373](#)
[frm_pm_desc 373](#)
[frm_pm_key 369, 372, 379, 387](#)
[frm_pm_name 373](#)
[frm_rn_desc 380](#)
[frm_rn_key 371, 374, 375, 377, 378, 379, 381, 384, 413](#)
[frm_rn_name 380](#)
[frml 459](#)
[from_fab 257](#)
[from_h 217](#)
[from_lot 108, 257, 261](#)
[from_lty_key 256](#)
[from_part_cnt 257](#)

[from_prod 257](#)
[from_s 217](#)
[from_v 217](#)
[from_waf_count 257](#)
[fs_bin 391, 394, 397, 398, 417](#)
[fs_free 505, 507](#)
[fs_index 515](#)
[fs_indicator 514](#)
[fs_info 504](#)
[fs_relative_strength 515](#)
[fsetup_key 318, 323, 324](#)
[fsgrp_desc 215](#)
[fsgrp_key 215](#)
[fsgrp_name 215](#)
[ftemp_end 229](#)
[ftemp_start 229](#)
[FTEST 236](#)
[ftest_key 227, 236](#)
[ftest_name 236](#)
[ftp_key 365, 368](#)
[full_name 237, 267](#)
[func_key 56, 203, 204](#)
[func_name 203](#)
[funct_key 414](#)
[fuse_id 228](#)

G

[gatt_datatype 410](#)
[gatt_key 410](#)
[gatt_name 410](#)
[gatt_type 410](#)
[gatt_unit 410](#)
[gds_x_offset 335](#)
[gds_y_offset 335](#)
[GENERAL_CONFIG 503](#)
[gevt_key 256, 257](#)
[gl_key 270, 271](#)
[gl_name 271](#)
[gp_key 270, 272](#)
[gp_name 270](#)
[green 436](#)
[GROUP_TYPE 198](#)
[grp_index 370, 372](#)
[grp_key 197, 202, 395, 443, 448, 450](#)
[grp_name 197](#)
[grpt_key 197, 198](#)
[grpt_name 198](#)

H

[h_flag 391](#)
[Hard Bin 517](#)
[header 158](#)
[HEADER_INFO 250](#)
[height 288](#)
[hfc_i_count 400](#)
[hfc_i_in_count 401](#)
[hi_lim 69](#)
[hide_flag 90](#)
[high_x 342](#)
[high_y 342](#)
[hire_date 193](#)
[HIST_BIN 82](#)
[HIST_BIN_EXTRAS 83](#)
[HIST_BIN_ZEROS 84](#)
[hist_count 65, 82, 83](#)
[hist_mean 65](#)
[hist_q1-3 66](#)
[hist_stdev 65](#)
[hist_yield 66](#)
[hole_flag 251, 253](#)
[holes 247](#)
[host_name 220](#)
[hotlist_desc 485](#)
[hotlist_key 484](#)
[hotlist_name 484](#)
[hotlist_type 484](#)

I

[i0 ... im-1 73](#)
[id 493](#)
[identifier_key 51, 142](#)
[idx_key 144](#)
[idx_name 144](#)
[idx_order 144](#)
[idx_type 145](#)
[image_level 470](#)
[img_com 171](#)
[img_dir 171](#)
[img_index 170, 171, 174](#)
[img_point 180](#)
[img_type 171](#)
[imp_tool 445](#)
[ims_key 171, 173](#)
[in_field 358, 415](#)
[in_x_offset 335](#)

[in_xsize 163](#)
[in_y_offset 335](#)
[in_ysize 163](#)
[include_wf 485](#)
[indexes 52](#)
[indidx 122](#)
[indiey 122](#)
[inp_cnt 414](#)
[inp_name 214](#)
[inp_ndx 432](#)
[inp_type1-10 214](#)
[ins_time 180](#)
[insert_time 37, 56, 64, 71, 128, 142, 156, 183, 185, 218, 219, 228, 271, 303, 320, 321, 336, 344, 373, 380, 382, 402, 405, 418, 419, 425, 430, 431, 441, 443, 449, 465, 474, 480, 482, 485, 486, 487, 489, 491](#)
[insp_area 163, 172](#)
[insp_die 163](#)
[insp_die_cnt 173](#)
[insp_flag 340, 419](#)
[insp_key 418, 419](#)
[insp_mod 162](#)
[insp_or 159](#)
[insp_prio 340](#)
[insp_set_name 418](#)
[instance_name 501, 502](#)
[Intensity 165](#)
[inter_flag 369](#)
[internal_index 289](#)
[invalid_flag 228](#)
[invert_fg 276, 282, 315](#)
[is_default 273](#)
[is_durable 494](#)
[is_new_flag 324](#)
[is_raw_bitmap 315](#)
[is_src 417](#)
[is_stateful 494, 501](#)
[is_volatile 494, 496, 501](#)
[is_wafer 108](#)
[isstub 449](#)
[istep_key 341, 396, 397, 416](#)
[item_index 324](#)

J

[jmx_port 508](#)
[job_class_name 494](#)
[job_data 494, 496](#)
[job_group 494, 495, 496, 501](#)
[job_key 448, 449](#)
[job_listener 495](#)
[job_name 494, 495, 496, 501](#)
[job_type 211](#)

K

[keep_alive 245](#)
[key_column 404](#)
[key_data 404](#)
[key_status 442](#)
[key_type 85](#)

L

[l_flag 391](#)
[l_value 351](#)
[last_checkin_time 502](#)
[last_pass 251, 252](#)
[last_picked 508, 514](#)
[last_update 224](#)
[lc_key 108](#)
[leff 228](#)
[leh_cat_stat 448](#)
[LEH_CONFIG 205](#)
[leh_key 445](#)
[LEH_LOG 40](#)
[leh_time_stat 448](#)
[len 228](#)
[lep 228](#)
[level_cnt 449](#)
[level_key 470](#)
[levels_block 229](#)
[lg_key 36, 39, 73, 74, 76, 79, 130, 149, 154, 159, 161, 163, 164, 169, 170, 171, 172, 178, 179, 180, 183, 184, 200, 201, 296, 298, 300, 302, 393, 395, 399, 405, 407, 408, 409, 417, 421, 437, 467](#)
[lgen_key 256, 258](#)
[li_flag 391](#)
[lib_wf_key 120](#)
[lim_date 71, 351](#)

[lim_flag 88](#)
[lim_key 70, 71, 350, 351, 352, 356](#)
[LIM_LOG 71](#)
[lim_name 71](#)
[lim_set_name 351](#)
[lim_type 71, 351, 352](#)
[LIM... 45, 70](#)
[limit_name 70](#)
[limits_type 67](#)
[LINE_DIST 246](#)
[LINE... 252](#)
[live_threads 509, 510](#)
[ll 445](#)
[llx 340](#)
[llx_org 340](#)
[lly 340](#)
[lly_org 340](#)
[lo_lim 69](#)
[load_level 208](#)
[location 195](#)
[lock_name 502](#)
[lock_time_ends_seconds 504](#)
[Log Tables 33](#)
[log_index 442](#)
[log_key 128, 324](#)
[log_level 79, 148, 155, 172](#)
[login 193, 211](#)
[lol_flag 90](#)
[loo 165, 394, 395](#)
[loo_step 179](#)
[loop 245](#)
[LOT 106](#)
[lot 518](#)
[lot merging 518](#)
[lot renaming 518](#)
[lot splitting 518](#)
[LOT_CLASS 126](#)
[lot_cls_desc 126](#)
[lot_cls_image 126](#)
[lot_cls_key 126](#)
[lot_cls_name 126](#)
[lot_cls_order 126](#)
[lot_code 257](#)
[lot_flag 88](#)
[LOT_GEN 256](#)
[LOT_GEVENT 257](#)

- [lot_id 108, 320](#)
[lot_index 205, 207](#)
[lot_key 36, 40, 42, 59, 61, 74, 76, 79, 94, 104, 108, 110, 112, 116, 127, 128, 130, 138, 148, 154, 172, 179, 200, 227, 256, 258, 260, 261, 296, 298, 300, 317, 336, 350, 374, 375, 381, 382, 384, 399, 400, 401, 431, 438, 439, 440, 456, 467](#)
[LOT_SPLIT 438](#)
[LOT_TYPE 259](#)
[lot_type 259](#)
[LOT... 45, 76](#)
[LOT2CUST 127](#)
[LOT2LOTGEN 261](#)
[LOT2TYPE 260](#)
[low_size 176](#)
[low_x 182, 342](#)
[low_y 182, 342](#)
[lower_bound 66](#)
[lspl_key 371, 438, 439](#)
[LTCLS2LOT 126](#)
[lty_key 256, 259, 260](#)
[LW_CLASS_LOG 128](#)
[lwc_lg_key 120, 128](#)
[lyr_img 161](#)
- M**
- [m 80](#)
[m_key 123](#)
[m_mc_key 286, 287, 308](#)
[m_mc_org_key 308](#)
[m_mr_key 284, 285, 306](#)
[m_mr_org_key 306](#)
[m_pgdw 80](#)
[machine_dsc 504, 507](#)
[machine_key 505, 506, 507](#)
[machine_name 504](#)
[machine_perf_coll_interval 503](#)
[MACHINE_STAT 505](#)
[MACHINE_STAT_ARCHIV E 506](#)
[machine_status 504](#)
[MACHINES 504](#)
- [mailbox 193](#)
[map_key 454, 455](#)
[MAP_SRV2DB 512](#)
[mask 493](#)
[mask set 519](#)
[matt_datatype 347](#)
[matt_key 346, 347](#)
[matt_name 347](#)
[matt_type 347](#)
[matt_unit 347](#)
[max 447](#)
[max_bins 196](#)
[max_cat_bins 451](#)
[max_col 275, 276, 277, 279, 281, 282, 283, 284, 285, 286, 305, 306, 307, 308, 309, 310, 312, 314, 315](#)
[max_col_count 266](#)
[max_def 174, 175](#)
[max_density 266](#)
[max_depth 452](#)
[max_flag 89](#)
[max_heap_size 509, 510](#)
[max_inter_col 281](#)
[max_inter_row 281](#)
[max_mem 508](#)
[max_reports 484](#)
[max_row 276, 277, 278, 279, 281, 282, 283, 284, 286, 287, 306, 307, 308, 309, 310, 312, 314, 315](#)
[max_row_count 266](#)
[max_terminals 452](#)
[max_val 248](#)
[max1 ... max5 246](#)
[max1_flag ... max5_flag 246](#)
[maxd_flag 262](#)
[MB_BIN 460](#)
[MB_FORMULA 458](#)
[MB_PROGRULE 460](#)
[MB_RULE 459](#)
[MB_SET 457](#)
[MB_SET2FORMULA 458](#)
[MB_UPSTAT 456](#)
[mbc_key 458](#)
[mbpr_key 460](#)
[mbr_key 459, 460](#)
- [mbset_key 456, 457, 458, 459](#)
[mbx_key 276, 309](#)
[mc_key 265, 268, 270, 272, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 299, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 317](#)
[mc_level 265](#)
[mc_name 265](#)
[mcd_key 278, 287, 307, 313](#)
[mcd_mmc_key 313](#)
[mcg_key 270, 272](#)
[mcg_name 272](#)
[mcl_key 269, 270, 273, 296, 298, 299, 302, 316, 318, 324](#)
[mcl_name 269](#)
[mcs_key 281, 282, 313](#)
[mcs_org_key 313](#)
[mcsc_key 282](#)
[mcst_key 277, 314](#)
[mdl_index 55](#)
[mdsn_blk_key 289](#)
[mdsn_key 273, 288, 289, 294](#)
[mean 447](#)
[mean_flag 89](#)
[mean_sq 401](#)
[meas_cnt 412](#)
[meas_key 347, 349, 350, 351, 352, 358, 413, 422](#)
[meas_name 349](#)
[meas_type 349](#)
[MEM_BIT_PATT 267](#)
[MEM_BLK_CFG 290](#)
[mem_blk_cnt 295](#)
[MEM_BOP_BOX 309](#)
[MEM_BOP_COLS 307](#)
[MEM_BOP_CONFIG 315](#)
[MEM_BOP_CROSS 313](#)
[MEM_BOP_CUSTOM 314](#)
[MEM_BOP_FOG 312](#)
[MEM_BOP_MISS 310](#)
[MEM_BOP_MULTI_COL 308](#)
[MEM_BOP_MULTI_ROW 306](#)

- MEM_BOP_ROWS 305
 MEM_BOP_SPOT 311
 MEM_BOP_TESTS 316
 MEM_BOP_UPSTAT 317
 MEM_BOP2PAT 316
 MEM_BOX 276
 mem_buswidth 288
 MEM_CELL_BLK 289
 MEM_CELL_CLASS 288
 MEM_CLASS 263
 MEM_CLASS_GROUP 272
 MEM_CLS2CLS 268
 mem_col_dir 295
 MEM_COLS 278
 mem_cols 288, 295
 MEM_CONFIG 293
 MEM_CROSS 281
 MEM_CROSS_BITS 282
 MEM_CUSTOM 277
 MEM_DEF2MEM 296
 MEM_DEFAULT_TEST 304
 mem_desc 288, 294
 mem_design 288
 MEM_DIE 304
 mem_dsgn_name 288
 MEM_FILE_CONFIG 319
 MEM_FILE_SETUP 318
 MEM_FILE2SETUP 323
 MEM_FOG 283
 MEM_GALIMG 303
 mem_img 303
 mem_info 504
 mem_key 296, 298, 300, 302, 317
 MEM_LAYER2PAT 299
 MEM_LIB2PROD 273
 MEM_LOG 301
 MEM_MC2MCGRP 272
 MEM_MCGRP 270
 MEM_MCGRPLIB 271
 MEM_MCL2MC 270
 MEM_MCLASS_COLL 266
 MEM_MCLIB 269
 MEM_MEMLOG2LAYER 299
 MEM_MEMORY 300
 MEM_META_CLASS 262
 MEM_META_RF 323
 MEM_MISS 279
 MEM_MULTI_COL 286
 MEM_MULTI_COL_BITS 287
 MEM_MULTI_ROW 284
 MEM_MULTI_ROW_BITS 285
 mem_name 294
 mem_num 294
 MEM_PAT_LOC 267
 MEM_PI 274
 MEM_RAW_FILE_DETAILS 320
 MEM_RAW_FILE_DETAILS_TMP 321
 mem_row_dir 295
 MEM_ROWS 275
 mem_rows 288, 295
 MEM_RUN_TEMPLATE 324
 MEM_SPOT 280
 mem_type 295
 MEM_UPSTAT 297
 MEM_WAFER 319
 MEM_WF_RF 322
 memblk_key 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 291, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314
 memcfg_key 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 291, 294, 296, 300, 302, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 316, 318
 memcfg_key 221
 memlog_key 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 296, 297, 300, 302, 303, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 318
 memory_index 514
 memory_indicator 514
 memory_real_free 505, 507
 memory_relative_strength 515
 memory_swap_free 505, 507
 memwf_key 319, 320, 321
 merge_time 109
 message 240
 meta_1 322
 meta_2 322
 META_DEF 144
 meta_desc 144, 222
 meta_flag 338
 META_IDX_DEF 144
 meta_key 43, 51, 71, 142, 144, 222
 meta_name 144, 222
 meta_type 71
 META_TYPES 222
 meta_value 43
 metarf_key 322, 323
 metarf1-4 323
 method_desc 220
 method_key 220
 method_name 220
 method_type 220
 mfg_key 283, 312
 mfg_org_key 312
 mi_key 275
 mi_rev 336
 micfg_key 335, 402, 420
 min 447
 min_child_method 452
 min_child_size 452
 min_clust 174
 min_col 275, 276, 277, 279, 280, 281, 282, 283, 284, 285, 286, 305, 306, 307, 308, 309, 310, 311, 312, 314
 min_density 265
 min_flag 89
 min_inter_col 281, 314
 min_inter_row 281
 min_prob 380

min_row 276, 277, 278, 279,
280, 281, 282, 283, 284,
286, 287, 306, 308, 309,
310, 311, 312, 314
min_val 247
min1 ... min5 245
min1_flag ... min5_flag 245
mind_flag 262
minimize_flag 490
misfire_instr 496
missing_factor_cut 452
mmc_key 262, 265, 267
mmc_name 262
mmd_key 279, 310
model_id 363
model_key 363, 364, 369, 373,
375, 376, 377, 378, 379,
387
monitor_flag 38
mrd_key 275, 285, 305, 313
mrd_mmc_key 313
msd_key 280, 311
msg_key 204, 221
mstep_key 146
MSTEP2PSTEP 146
mtemp 255
mtemp_end 229
mtemp_start 229

N

n 211, 368
n_grps 445
n_pars 444
n_steps 444
n_tools 445
name 182, 193, 211, 250, 443
name_alias 218
name1 218
name2 218
name3 218
name4 218
ndie_key 121, 320
nest 394
next_fire_time 496
next_operator 460
ni 38
nlo_flag 89
nlots 208

node_key 507, 509, 510, 514
node_name 507
node_num 453
node_perf_coll_interval 503
node_port 508
NODE_STAT 509
NODE_STAT_ARCHIVE 509
node_status 507
node_subtype 508
node_type 508
node_version 508
NODES 507
num_cat_srgts 452
num_charts 158
num_cizones 344
num_competitors 452
num_cont_srgts 452
num_days 484
num_die 368
num_lines 394
num_vias 394
num_zones 156
nuo_flag 89
NWMAP2DIE 122

O

object_identity 493
OP_LOG 33, 34
open_pass_cnt 407, 408, 409
open_test_cnt 407, 409
oper_hist_key 141, 474, 476
oper_level 474
oper_type 475
oper_type_key 141, 474, 475
operator 404, 477
Operator Bin 517
option_key 204, 224
ORG_WMAP 98
os_info 504
out_field 358
out_fname_template 324
out_groups 487
out_rep_name 485
out_report_name 487
out_spec 351
out_type 378
outliers 110
output_type 430

overlay_test_index 295
owner 71, 128, 156, 158, 182,
184, 195, 196, 197, 199,
208, 212, 217, 271, 336,
344, 373, 381, 383, 403,
406, 418, 419, 425, 429,
430, 443, 448, 450, 451,
474, 477

P

p_formula_id 369
p_formula_id1 369
p_formula_id2 369
p_key 368, 389, 390, 401, 430,
432
p_val 370, 390
p_value 444, 445, 446
p1 248
p1 ... p500 249
p1_flag 89
p10 248
p10_flag 89
p5 248
p5_flag 89
p90 248
p90_flag 89
p95_flag 89
p99 248
p99_flag 90
pad_value 408
pad_yield 408
pager 193
par 445
par_cnt 411, 425
PAR_GRP 197
par_id 412
par_key 370, 387, 411, 413
par_name 453
par_option_key 224
par_type 412
par_val 246
PAR2PARGRP 202
param_end 233
PARAM_INFO 233
param_name 233
param_name_1 ...10 508
param_start 233
param_step 233

- param_sweep 233
param_type 370
param_unit 233
param_val_ ...10 508
param_value 233
parent 188
parent_folder_key 480
parent_key 449
parent_object 493
part_cnt 108, 257
Pass Bin 517
pass_clean_cnt 173
pass_def_cnt 173
pass_fail 82, 83, 84, 459
pass_flag 407, 410
pat_file 236
pat_file_date 236
pat_index 242, 245
pat_key 237, 240, 241, 242, 244, 251, 252, 254
PAT_LIST 236
PAT_MODE 239
pat_name 237
pat_pwr_down 230
pat_set_rev 236
PAT_STATUS 240
PAT_TYPE 240
PAT2LIST 241
path_key 218, 219, 224
path_sequence 218
patlist_key 227, 236, 241
patlist_name 236
patmode_key 237, 239
patmode_name 239
patt_key 265, 267, 268
patt_name 267
PATTERN 237
pattern 436
patttype_key 237, 240
patttype_name 240
PCM 33, 518
pcm_flag 455
pd_desc 93
pd_key 37, 40, 42, 62, 93, 94, 95, 178, 182, 227, 256, 260, 273, 294, 298, 299, 302, 304, 317, 324, 399, 418, 441
pd_name 93
pdgrp_key 408, 421, 422, 423
pdgrp_name 421
PEOPLE 192
percent 246, 444
period_offset 209
period_type 209
period_value 208
pg_cnt 457
pg_grp_desc 56
pg_grp_key 51, 56
pg_grp_name 56
pg_key 36, 40, 42, 51, 57, 58, 59, 60, 61, 62, 63, 71, 79, 82, 83, 84, 85, 86, 110, 112, 120, 122, 128, 130, 144, 148, 149, 154, 155, 172, 181, 183, 190, 200, 201, 202, 205, 207, 227, 236, 254, 298, 302, 317, 319, 325, 326, 327, 328, 335, 399, 433, 434, 436, 440, 442, 445, 455, 460, 467, 468, 477, 480
pg_key1-10 457
pg_order 460
pgc_key 37, 51, 87, 95, 205, 216, 440, 464, 468
pgc_name 88
PGDW 80
phone 124, 125, 193, 194
pick_index 515
pick_relative_strength 515
pin 350, 407, 408, 423
PIN_PARETO 249
pin_type 350
pindef_key 227, 238
pindef_name 238
pixel_size 162
pk_key 63, 103, 104, 108, 227, 257, 260
pk_name 103
pk_type 103
PKG 93, 103
PKG2LOT 104
PKG2PROD 103
PKG2SRCLLOT 104
pkgtype_key 227, 239
pkgtype_name 239
plot_map 361
plot_type 195
pm_order 357
pm_type 357, 368, 390
pmod_flag 361
pmod_key 338, 341, 353, 357, 362, 374, 393, 397, 398, 400, 416, 432
pmod_name 357
por_level 196
pos_x 97, 99
pos_y 97, 99
post_code 229
postal_code 124, 125, 194
PPID 46, 51
ppid 457
pr_desc 137
pr_key 37, 40, 42, 58, 137, 138, 146, 441
pr_name 137
pre_code 229
pre_pat_key 227
precision 67
pretest 518
prev_desc 63
prev_fire_time 496
prev_key 63, 71
prio_per 401
priority 496, 501
prob_val 379, 401
proc_flag 160, 185
proc_index 205, 207
PROC_STEP 33, 133
proc_type 185
PROC2LOT 138
PROCESS 137
process 518
Process Related Tables 133
process step 518
process_id 508

[prod_engr 93](#)
[PROD2FAM 105](#)
[PROD2LOT 94](#)
[PROD2SRCLLOT 94, 138](#)
[PROD2WMAP 95](#)
[PRODUCT 91](#)
[Product 519](#)
[PROG_CLASS 87](#)
[PROG_DATATYPE 57](#)
[prog_desc 51](#)
[prog_load_date 228](#)
[PROG_REV 63](#)
[prog_type 46, 53](#)
[prog_wrap 55](#)
[PROG2CUST 61](#)
[PROG2EQUIP 59](#)
[PROG2FAM 58](#)
[PROG2LOT 59](#)
[PROG2PKG 63](#)
[PROG2PROC 58](#)
[PROG2PROD 62](#)
[PROG2SRCLLOT 61](#)
[PROG2STAGE 60](#)
[PROG2STEP 62](#)
[PROG2TECH 60](#)
[PROGRAM 44, 48](#)
[program 519](#)
 [definition 44](#)
[Program Class 87](#)
[PROGRAM_GROUP 56](#)
[pruning_conf 452](#)
[pstep_key 146](#)
[ptrn_flag 160](#)
[pts_above 67](#)
[pts_below 66](#)

Q

[q_chisq 152](#)
[q_chisqc 152](#)
[q_df 152](#)
[q_status 152](#)
[q1 247](#)
[q1 ...q40 195](#)
[q1_flag 89](#)
[q1... q4 150, 151](#)
[q1-q3 447](#)
[q2 247](#)
[q2_flag 89](#)
[q3 247](#)
[q3_flag 89](#)
[QRTZ_BLOB_TRIGGERS 498](#)
[QRTZ_CALENDARS 499](#)
[QRTZ_CRON_TRIGGERS 497](#)
[QRTZ_FIRED_TRIGGERS 500](#)
[QRTZ_JOB_DETAILS 494](#)
[QRTZ_JOB_LISTENERS 495](#)
[qrtz_locks 502](#)
[QRTZ_PAUSED_TRIGGER_GRP 500](#)
[QRTZ_SCHEDULER_STAT E 501](#)
[QRTZ_SIMPLE_TRIGGERS 497](#)
[QRTZ_TRIGGER_LISTENER RS 499](#)
[QRTZ_TRIGGERS 495](#)
[qu_name 195](#)
[quad_offset 147](#)
[quantile 379](#)
[QUERIES 195](#)
[qxpct 199](#)
[qypct 199](#)

R

[r_chisq 153](#)
[r_chisqc 153](#)
[r_df 153](#)
[r_flag 391](#)
[r_status 153](#)
[r1... r24 150, 151](#)
[radial_offset 147](#)
[radials 147](#)
[random_seed 453](#)
[rank 252](#)
[raw data 519](#)
[RAW_FILE 218, 219](#)
[rawfile_key 204, 219, 221, 320, 321, 323](#)
[rcp_key 37, 40, 42, 139](#)
[rcp_name 139, 162](#)
[RCP2STEP 140](#)
[reader 519](#)
[reasons 492](#)
[rec_order 270](#)
[rec_weighting 452](#)
[RECIPE 139](#)
[recipe 519](#)
[recipient 493](#)
[red 436](#)
[red_blk_cnt 295](#)
[release 93](#)
[release_date 63](#)
[reload_flag 43, 205](#)
[rep_flag 489](#)
[rep_key 199, 443, 445](#)
[rep_name 199](#)
[rep_type 199](#)
[repeat_count 497](#)
[repeat_interval 497](#)
[report_name_pattern 485](#)
[report_type 490](#)
[REPORTS 199](#)
[requests_recovery 494, 501](#)
[res_aging_days 53](#)
[res_flag 88](#)
[RES... 44, 72](#)
[residual 384](#)
[ret_align 335](#)
[ret_key 101](#)
[ret_name 101](#)
[RET2DIE 103](#)
[RETICLE 101](#)
[reticle 519](#)
[reticle set 519](#)
[RETICLE_SET 102](#)
[rev_date 136, 137](#)
[revision 63, 335](#)
[rework_action 53](#)
[rework_flag 38, 39, 41, 43, 303, 399, 437](#)
[rf_flag 263](#)
[rfail_key 242](#)
[RFX 202](#)
[rfx_filename 202](#)
[rm_flag 180](#)
[rma 242, 254](#)
[ROB_FAIL 242](#)
[ROBUST 244](#)
[Robustness 225](#)
[role 193](#)

[rootpath_key 219](#)
[row_chisq 153](#)
[row_chisqc 153](#)
[row_cnt 38, 248, 447](#)
[row_df 153](#)
[row_flag 266](#)
[row_key 492](#)
[row_num 275, 285, 305](#)
[row_offset 98](#)
[row_op 404](#)
[row_status 153](#)
[row_thickness 266](#)
[row_type 445](#)
[row1... row24 150, 151](#)
[rows 147](#)
[rowtype_index 205, 207](#)
[rpt_num 158](#)
[rs_desc 102](#)
[rs_key 93, 101, 102, 108](#)
[rs_name 102](#)
[rt_flag 263](#)
[rule 451](#)
[rule_cnt 174, 405](#)
[RULE_INFO 131](#)
[rule_name 325, 326, 327, 328, 433, 434, 435, 477](#)
[rule_order 459](#)
[rule_pg_cnt 459](#)
[rule_set_key 448, 450](#)
[rule_set_name 450](#)
[Run 517](#)
[run_descr 324](#)
[run_id 37](#)
[run_key 324](#)
[run_name 324](#)
[run_queue_length 505, 506](#)
[run_time 205](#)
[run_type 212](#)

S

[s_chisq 153](#)
[s_chisqc 153](#)
[s_df 153](#)
[s_flag 391](#)
[s_lg_key 341, 396, 398, 415, 416](#)
[s_status 153](#)
[s1... s20 150, 151](#)

[samp_pct 162](#)
[sampling_method 52](#)
[sampling_rate 52](#)
[sbin_num 70](#)
[sc_flag 391](#)
[scale_factor 68](#)
[scale_key 365](#)
[scale_type 366](#)
[sched_flag 324](#)
[schedule_type 485, 487](#)
[SCHEDULING_DECISION 513](#)
[scheduling_decision_key 513, 514](#)
[SCHEDULING_DETAILS 514](#)
[scheduling_details_key 513, 514](#)
[scrap_yield 55, 130](#)
[search_pattern 485](#)
[sec_key 184](#)
[sec_msg 184](#)
[sel_type 175](#)
[semi_dynamic 46](#)
[seq_num 39, 160](#)
[seq1 244](#)
[seq2 244](#)
[seq3 244](#)
[seq4 245](#)
[seq5 245](#)
[sequence 242](#)
[serial key 519](#)
[server_dsc 511](#)
[server_key 507, 511, 512](#)
[server_name 511](#)
[server_version 511](#)
[SERVERS 511](#)
[set_desc 142](#)
[set_key 420](#)
[set_name 182](#)
[set_type 420, 457](#)
[setup_name 319, 430](#)
[sfci_count 400](#)
[sfci_in_count 401](#)
[shift 193](#)
[SHM_STATS 247](#)
[Shmoo 225](#)

[SHMOO_CLASS 235](#)
[shmoo_class_key 234, 235](#)
[shmoo_class_name 235](#)
[SHMOO... 251](#)
[short_pass_cnt 407, 408, 409](#)
[short_test_cnt 407, 409](#)
[shrink 102](#)
[shutdown_command 508](#)
[sig_leh 444](#)
[sig_par 444](#)
[sig_pg 444](#)
[sig_step 444](#)
[sig_tool 444](#)
[sign 245, 478](#)
[SIGN_PARETO 248](#)
[signature 248](#)
[sim_curr 243](#)
[sim_next 243](#)
[sim_prev 243](#)
[site 455](#)
[site_index 53](#)
[SITE_MAP 454](#)
[SITE2DIE 454](#)
[size_type 176](#)
[skew_desc 232](#)
[skew_key 227, 232](#)
[skew_name 232](#)
[sleep_time 503](#)
[slot_flag 448](#)
[slot_index 205, 207](#)
[slot_num 39, 160](#)
[smpl_key 175](#)
[so_key 418](#)
[so_set_name 419](#)
[Soft Bin 517](#)
[sort_pg_key 455](#)
[sortcols 199](#)
[source lot 108](#)
[sp 446](#)
[SPLIT 438](#)
[split_date 438](#)
[split_desc 438](#)
[split_grp 439](#)
[split_grp_desc 438](#)
[split_key 438](#)
[split_key1-4 432](#)
[split_type 438, 453](#)

- [split_type1-4 432](#)
[split_value 453](#)
[splitter_method 452](#)
[sql_count 403](#)
[sql_order 404](#)
[sr_flag 391](#)
[src_fab 257](#)
[src_fam 186](#)
[src_lot 37, 40, 42, 108, 257, 300, 320, 399, 440](#)
[src_lot_id 417](#)
[src_proc 186](#)
[src_prod 187](#)
[src_tech 187](#)
[SSA_CLUSTER 471](#)
[SSA_CLUSTER_\\${pgc_key} 463](#)
[ssa_cluster_desc 471](#)
[ssa_cluster_key 131, 213, 462, 463, 467, 471, 472, 473](#)
[ssa_cluster_name 471](#)
[SSA_CLUSTER2WFCLS 472](#)
[SSA_DEF 468](#)
[SSA_DEF_DET 469](#)
[SSA_DIE_\\${pgc_key} 462](#)
[SSA_IMAGE 470](#)
[ssa_image 470](#)
[ssa_img_key 470](#)
[SSA_OPER 476](#)
[SSA_OPER_HIST 474](#)
[SSA_OPER_TYPES 475](#)
[ssa_rn_desc 464](#)
[ssa_rn_key 462, 463, 464, 467, 468, 469, 470, 471, 472, 473](#)
[ssa_rn_name 464](#)
[ssa_rn_type 464](#)
[ssa_rule_key 122, 184, 325, 326, 327, 433, 434, 476, 477](#)
[SSA_RUN 464](#)
[SSA_RUN_DET 467](#)
[SSA_RUN2ZONE 468](#)
[ssa_test_index 468, 469, 470](#)
[SSA_WF2CLUSTER 472, 473](#)
[SSA_YLD_RULE 476](#)
[SSA_YLD_RULE_DEF 477](#)
[SSA_ZONE_\\${pgc_key} 462](#)
[stack_name 217](#)
[STACKING_COLORS 217](#)
[stage_cond 206, 207](#)
[stage_desc 137](#)
[stage_key 37, 40, 42, 60, 136, 223, 302, 441](#)
[stage_name 137](#)
[stage_num 137](#)
[STAGE2STEP 223](#)
[STAGING_DIRS 223](#)
[start_col 318](#)
[start_row 318](#)
[start_time 37, 41, 43, 109, 205, 208, 228, 303, 441, 443, 449, 482, 485, 496](#)
[startup_command 508](#)
[stat 202, 412, 427, 454](#)
[stat_test 445, 446](#)
[stat_type 74, 76, 78, 469, 470, 506, 510](#)
[statcols 199](#)
[state 124, 125, 194, 501](#)
[statistics 87](#)
[stats_aging_days 53](#)
[status 154, 183, 240, 380, 382, 406, 431, 465, 503](#)
[status_key 52](#)
[stc_key 217](#)
[std 247](#)
[std_cfg 98](#)
[std_err 384](#)
[stddev 447](#)
[stdev_flag 89](#)
[stdir_key 204, 223](#)
[step 445](#)
[step_cond 206, 207](#)
[step_desc 135](#)
[step_key 36, 40, 42, 62, 112, 135, 143, 172, 178, 179, 183, 223, 257, 258, 297, 299, 302, 338, 392, 400, 405, 438, 440](#)
[step_name 135](#)
[step_num 135](#)
[step_order 392](#)
[STEP_ORDER_DETAILS 143](#)
[step_order_key 142, 143](#)
[step_order_name 142, 143](#)
[STEP_ORDER_SET 142](#)
[step_type 392, 395](#)
[stepper field 100](#)
[store_flag 68](#)
[str_key 224](#)
[STR_KEY... 221](#)
[str_value 224](#)
[STRING HOLDER 224](#)
[struct_cnt 385](#)
[STYPE 78](#)
[sub 242, 255](#)
[sub_operator 460](#)
[subsample_method 452](#)
[subsample_size 451](#)
[SUM_COND 86](#)
[sum_flag 89, 160](#)
[sum_key 444, 446, 447](#)
[sum_sq 401](#)
[sums_flag 89](#)
[supp_key 204](#)
[surrogate_imp 452](#)
[sweep_flag 245](#)
- ## T
- [t1 70](#)
[t1 ...tn 73, 75, 77](#)
[t1 ...tY 385](#)
[t1...tn 251, 253](#)
[T2T_REPORT 443](#)
[T2T_SUM 444](#)
[T2T_TEST 446](#)
[T2T_TOOL 447](#)
[t3, t6, t9, ... 251, 252](#)
[table 404](#)
[tag_action 54](#)
[tag_flag 109, 116, 129, 130](#)
[tag_level 131](#)
[TAG_LOG 130](#)
[tag_name 129](#)
[TAGS 129](#)
[target_value 66](#)
[target_yld 477](#)
[task_flag 487](#)
[task_key 487, 489](#)

task_name 487, 492
 task_status 492
 tci_count 400
 tci_in_count 400
 tech_desc 136
 tech_key 60, 136, 137, 140
 tech_name 136
 TECH_STAGE 136
 technical stage 518
 TECHNOLOGY 136
 temp 245
 temp_meas_flag 229
 test 519
 Test Numbers 517
 test stage 519
 test_accuracy 449
 test_cnt 457, 465
 test_command 508
 test_engr 93
 test_index 40, 42, 65, 202, 238, 298, 303, 304, 376, 408, 409, 422, 459, 469, 470
 test_key 162, 163, 320, 321, 324
 test_mode 37, 39
 test_name 40, 42, 459, 469, 470
 test_num 165, 459
 test_order 409
 test_plan 329
 test_prog 329
 test_proportion 451
 test_pwr_down 229
 test_sequence 316
 test_type 68
 test_unit 459
 tester summary 519
 Tester Summary Sheet Storage 46
 tester_cycle 254
 TESTER_PIN_DEF 238
 TESTER_PKG_TYPE 239
 tests 51
 text1_key 491
 text2_key 491
 text3_key 491
 text4_key 491
 tf_flag 262

thickness 275, 278, 285, 287, 305, 307
 ti_index 205, 207
 tiff_ptr 171
 time_range 506, 510
 time_updated 504, 505, 506, 508, 509, 510, 511, 513, 515
 time_zone_id 498
 times_triggered 497
 timing_block 229
 title 193
 tn 70
 to_h 217
 to_index 205, 207
 to_lot 108
 to_s 217
 to_v 218
 tol_maj 159
 tol_min 159
 tol_or 159
 tol_type 177
 tolerance 315, 392
 tool 447
 tool_date 196
 tool_type 217
 Tool-2-Tool Tables 443
 tot_adr_cnt 172
 total_index 515
 total_tested 80
 tpl_key 329
 tpl_revision 417
 tpr_key 329
 tpr_revision 417
 traceability 520
 train_accuracy 449
 transf 318
 transpose_flag 230
 tree_key 449, 453
 tree_name 449
 trgt_rn_key 141, 474
 trig_key 214, 487
 trig_r_key 489
 trig_type 214
 trigger_group 496, 497, 498, 499, 501
 trigger_groups 500

trigger_listener 499
 trigger_name 496, 497, 498, 499, 501
 trigger_state 496
 trigger_type 496
 trimpct 199
 twist_scheme 288
 twisted_flag 265
 type 182
 type_desc 166
 type_flag 167

U

u_flag 391
 u_value 351
 ucon_key 429
 ucon_val 429
 uform_equation1 425
 uform_equation2 425
 uform_id 425
 uform_key 424, 426
 uform_order 424
 uform_type 424
 uformd_key 424, 428
 ui_flag 391
 ul 445
 unique_flag 249
 unit_id 228
 unqci_key 341, 359, 393, 396, 398
 uol_flag 90
 uoper_desc 430
 uoper_key 424, 426, 429
 uoper_name 429
 uoper_out_unit 429
 up_size 176
 up_x 182
 up_y 182
 upar_key 426
 upar_name 428
 upar_type 428
 upard_key 426, 428
 UPDATE_STATS 110
 upper_bound 66
 urx 340
 urx_org 340
 ury 340
 ury_org 340

use_flag [453](#)

user1 [93](#)

user2 [93](#)

V

val [446](#)

val_column [404](#)

value [238](#), [305](#), [306](#), [307](#), [308](#),
[309](#), [310](#), [311](#), [312](#), [313](#),
[314](#), [346](#), [347](#), [348](#), [360](#),
[377](#), [394](#), [411](#)

value_data [404](#)

value1 [377](#)

value1-3 [101](#)

value2 [378](#)

var_src [401](#)

vector_cnt [237](#)

vector_rev [229](#)

ven_key [101](#), [119](#), [124](#), [190](#)

ven_name [124](#)

VENDOR [124](#)

version [196](#)

vrev [237](#)

W

w_flag [391](#)

waf_count [257](#)

waf_flag [88](#)

waf_orient [196](#)

WAF... [45](#)

WAF2LOTGEN [258](#)

WAFER [114](#)

WAFER_CLASS [117](#)

wafer_cnt [471](#)

WAFER_DISTANCE [120](#)

wafer_index [53](#)

WAFER_SPLIT [439](#)

WAFER_TYPE [119](#)

WAFER2LOT [121](#)

WEB_RPT_CFG [158](#)

WEH_CONFIG [207](#)

WEH_LOG [42](#)

weight [365](#)

WF_ALIAS [123](#)

wf_cls_desc [117](#)

wf_cls_image [117](#)

wf_cls_key [117](#), [118](#), [120](#), [472](#),
[474](#)

wf_cls_name [117](#)

wf_cls_order [117](#)

wf_cnt [155](#)

wf_desc [116](#)

wf_id [116](#), [320](#)

wf_index [207](#)

wf_key [37](#), [39](#), [74](#), [79](#), [110](#),
[112](#), [116](#), [118](#), [128](#), [130](#),
[155](#), [172](#), [179](#), [183](#), [227](#),
[258](#), [296](#), [298](#), [300](#), [304](#),
[317](#), [341](#), [368](#), [374](#), [381](#),
[382](#), [384](#), [392](#), [395](#), [396](#),
[398](#), [399](#), [400](#), [402](#), [405](#),
[407](#), [408](#), [409](#), [415](#), [416](#),
[422](#), [431](#), [437](#), [439](#), [456](#),
[463](#), [467](#), [472](#), [476](#)

WF_LOG [39](#)

wf_num [116](#), [320](#)

wf_size [96](#), [119](#)

wf_units [97](#), [119](#)

WFCLS2WF [118](#)

wfg_index [55](#)

wfrf_key [321](#), [322](#)

wft_desc [119](#)

wft_key [116](#), [119](#)

wft_name [119](#)

which_time [210](#)

width [209](#), [288](#)

win_perf_measure_interval
[503](#)

win_perf_wait_time [503](#)

window [195](#)

WMAP_CONFIG [96](#)

wmap_key [51](#), [93](#), [95](#), [96](#), [98](#),
[100](#), [103](#), [121](#), [147](#), [183](#),
[294](#), [335](#), [405](#), [455](#)

wmap_name [96](#)

WMAP2DIE [121](#)

WMAPS [100](#)

ww_day [158](#)

ww_offset [158](#)

X

x [268](#), [389](#)

x_count [163](#)

x_dim [454](#)

x_img [170](#)

x_offset [163](#)

x_origin [162](#)

x_pitch [163](#)

x_prod_index [205](#), [207](#)

x_pt [170](#)

x_shift [161](#)

x_size [165](#), [394](#)

x_tol [177](#)

x0 [365](#), [370](#)

x1 [289](#), [291](#), [294](#)

x2 [289](#), [291](#), [295](#)

xAbs [165](#), [394](#)

xpos [491](#)

Y

y [268](#)

y_count [163](#)

y_dim [454](#)

y_img [170](#)

y_offset [163](#)

y_origin [163](#)

y_pitch [163](#)

y_pt [170](#)

y_shift [161](#)

y_size [165](#), [394](#)

y_tol [177](#)

y1 [289](#), [291](#), [295](#)

y1 ...y10 [80](#)

y2 [289](#), [292](#), [295](#)

yAbs [165](#), [394](#)

yield [80](#)

yield_pgdw [80](#)

yield_type [353](#)

ylldo_flag [90](#)

yldp_flag [90](#)

ylds_flag [90](#)

yldw_flag [90](#)

ypos [491](#)

yr [80](#)

yr_pgdw [80](#)

Z

Z_CNT [150](#)

Z_CONFIG [149](#)

z_key [150](#), [151](#), [152](#), [154](#)

Z_LOG [154](#)

Z_STATS [152](#)

Z_UPDATE_LOT [148](#)

Z_UPDATE_WAF [149](#)

Z_WMAP_CONFIG [147](#)
Z_YLD [151](#)
zcnt_flag [90](#)
Zonal Bin Summary Tables
 [147](#)
ZONE_DEF [157](#)
zone_key [155](#), [156](#), [157](#), [380](#),
 [382](#), [431](#), [463](#), [468](#)
zone_name [156](#)
ZONE_NUM [156](#)
zone_num [156](#)
zone_type [156](#)
zoned_key [156](#), [157](#), [384](#), [400](#)
ZONES [155](#)
zstats_flag [89](#)
zstore_flag [150](#)